

第一章 引言

刘志荣

第 1 节 机器学习：无处不在

现在，机器学习是无处不在。

日常生活中，在很多学校、公司、博物馆、居民小区，“刷脸”（人脸识别）已经被广泛用于快速身份识别与人员管理。大家使用智能手机，也会利用“刷脸”和“指纹”来作为解锁和支付方式。这些技术的基础，都来源于机器学习。

新闻热点中也经常出现机器学习的身影。比如，最近几年一个经常被讨论的热门话题是 AI 换脸^a，即将照片或视频中一个人的脸换成另一个人的脸。它可以用于个人娱乐，例如，手机用户可通过上传自己的照片，使用换脸功能将视频中的演员换成自己的脸，以实现“和偶像同框”。它也会被坏人用来干坏事，例如利用换脸技术冒充领导、亲戚、朋友等身份实施网络诈骗。它也可以用于正当的商业行为，例如影视业所使用的 CG 数字替身特效就与换脸技术密切相关。《星球大战外传：侠盗一号》里的“星区总督塔金”以及“莉娅公主”就是使用这种技术让已经去世的明星重新登上大银幕的。

机器学习对于增强企业的核心竞争力具有重要助力。2020 年曾经有个新闻热门话题（很多人可能没能充分认识到其深刻含义），那就是“美国政府考虑 TikTok”。中国的制造业在国际上具有重要影响；但在软件、文体娱乐等偏“软”的产业在世界上的影响则要逊色得多。中国的新兴科技公司很少在美国等西方发达国家的市场取得巨大成功。字节跳动旗下的短视频社交平台 TikTok 是非常罕见的能够在欧美市场大杀四方的成功例子。究其原因，最重要的就是：算法，能够克服文化壁垒！正如某篇评论所指出的^b，“在某些行业和产品类别中，基于机器学习的算法有着极高的响应性和准确性，能够进一步穿透‘文化无知之幕’”。

而在过去 20 年里，与机器学习相关的最重要的新闻，是 2016 年 DeepMind 所研发的机器学习程序 AlphaGo（俗称“阿尔法狗”）击败围棋世界冠军李世石。它是机器学习成就的里程碑式事件，更是让人工智能的热潮随后引爆全球。

第 2 节 人工智能、机器学习，以及在化学中的应用

^a 近几年换脸技术泛滥的源头是 2017 年 Deepfakes（深度伪造）源代码公开并广泛扩散，被很多个人及公司所发展，后续产生了各种应用。

^b <https://www.huxiu.com/article/376933.html>，为什么 TikTok 能够横扫美国市场？

从广义上讲，人工智能是指通过计算机实现人的头脑思维所产生的效果，即能够从环境中获取感知并执行行动的智能体技术。凡是使用机器帮助、代替甚至部分超越人类实现认知、识别、分析、决策等功能的技术，都可以称为人工智能。人工智能研究的主要目标是使机器能够胜任一些通常需要人类智能才能完成的复杂工作。图灵在计算机刚出现时就讨论了机器与智能之间的关系^[1]，包括现在广为人知的图灵测试。1956 年，麦肯锡、明斯基、香农、Herbert Simon 等人在达特茅斯组织了一场研讨会，发表了关于人工智能的声明与提案，被视为人工智能学科正式诞生的标志。人工智能与化学学科的结合也相当早：人工智能的第一个专家系统 DENDRAL 就是由生物学家 Edward Feigenbaum（当时他负责一个太空生命探测项目，用质谱仪分析火星上采集来的数据）、计算机学家 Joshua Lederberg 与化学家 Carl Djerassi（他也是口服避孕药之父）在 1965-1968 年一起开发的，它能从质谱数据推断出样品中化学分子的组成^[2]。而在有机合成化学领域鼎鼎大名的 Corey 则在 1969 年开发了第一个计算机辅助有机合成程序 OCSS（Organic Chemical Synthesis Simulation）^[3]。

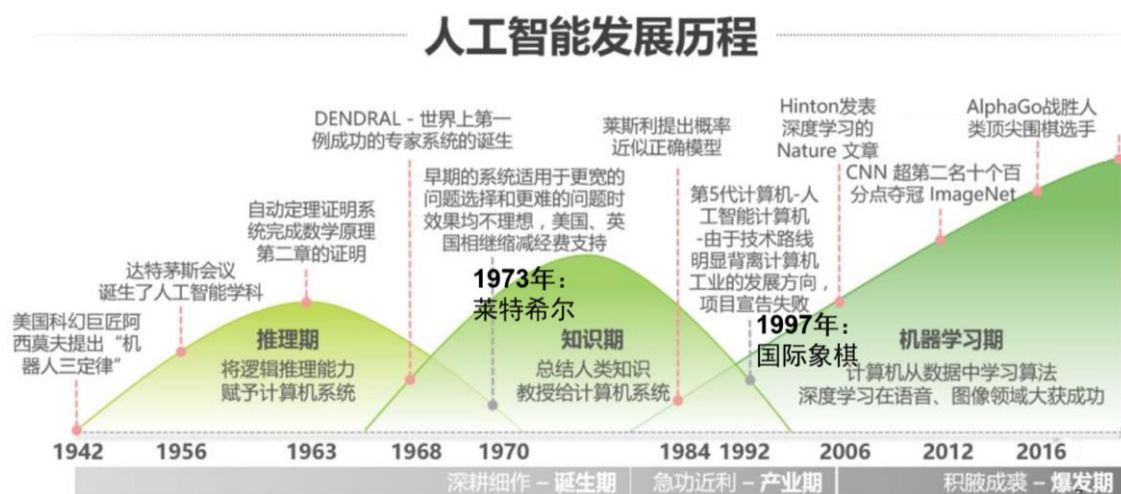


图 1.1. 人工智能发展的三起两落。

人工智能现在很火，但在历史上，它的发展其实非常坎坷，被总结为三起三落（或三起两落，因为目前的这一次繁荣还没有落下去）（图 1.1）^[4]。

- 人工智能学科在上世纪 50 年代诞生以后获得快速发展，也获得了质谱程序 DENDRAL、计算机跳棋程序、数学定理自动证明系统等代表性成果。那时候，科学家对人工智能学科抱有非常乐观的态度。例如 Herbert Simon（计算机图灵奖、诺贝尔经济学奖获得者）在 1965 年预言机器将在 20 年内能够做人类所能做的事情。但是人们很快就发现在当时的条件下机器能做的事情非常有

限。1973 年莱特希尔（应用数学家，他的莱特希尔方程是统计热力学教科书中关于化学平衡的内容之一^[5]）对当时的人工智能进行深入调研以后写了一个评估报告（后来世人称之为《莱特希尔报告》）^[6]，认为当时的人工智能研究不可能达到其所宣传的宏伟目标。美英等国政府大大缩减经费支持，人工智能学科进入第一次寒冬。

- 随着一些专家系统在企业实际生产中得到成功应用，以及日本提出“第五代计算机”（可以对话、翻译语言、解释图片、像人一样推理的计算机。形象地说，类似于《星球大战》里的机器人 R2-D2）的计划并被欧美各国跟进，人工智能学科在上世纪 80 年代进入第二个繁荣期。后来，“第五代计算机”计划失败，人工智能学科在上世纪 90 年代又一次进入寒冬。
- 对于专家系统，人们是把自己总结的知识教给计算机。后来，人们发现人类知识是无穷无尽，而且有些知识本身难以总结，于是就诞生了将知识学习能力赋予计算机本身的想法，即人工智能的机器学习方向。经过长时间的积累，机器学习方向的突破终于将人工智能带进最近的这一次繁荣期。开启这次繁荣期的标志是 2012 年 Hinton 在 ImageNet 图像识别挑战赛中极大地降低了机器学习的错误率^[7]。而 2016 年 DeepMind 研发的 AlphaGo 击败围棋冠军李世石^[8]，更是让人工智能的热潮引爆全球。

作为人工智能的一个方向，机器学习使机器的自动学习成为可能^[4]。也就是说，输入的是数据和想要的结果，输出则为算法模型，即把数据转换成结果的算法模型，进而提供相应的判断，达到某种人工智能的目的。得益于海量数据的出现与计算能力的提升，机器学习在近 10 年来取得了革命性的发展。机器学习也在日常生活中得到广泛应用，例如，刷脸与指纹识别。而且，与前两次人工智能发展热潮不同，中国在这一轮热潮中有深度的参与，有望取得“好风凭借力，送我上青天”的效果^[9]。

现在，机器学习的突破已经快速渗透到各个领域，其中就包括化学^[10-11]。在有机化学里，机器学习被用来预测有机反应能否进行^[12]（Nature 报道像化学家一样探索新反应的“AI”机器人^[13]），预测钨催化的 Buchwald-Hartwig 偶联反应的产率^[14]，预测对亚胺亲核加成反应的不对称催化对映体选择性^[15]，甚至进行卓有成效的逆向合成路线设计^[16-17]。在材料化学，机器学习被成功用于 MOFs 合成预测^[18]、超硬材料设计^[19]以及水光解产氢的催化剂配方优化^[20]（媒体报导：无情的科研机器来了：8 天不停，做了近 700 个实验^[21]）。在化学生物学，机器学习被用于磷酸转移酶的多肽底物的发现^[22]，以及抗癫痫药物的筛选^[23]。在理论与计算

化学方向^[24]，机器学习可以用于力场的发展，如早期 Parrinello 等人所做的尝试^[25]以及刘智攀^[26]与鄂维南^[27]等人最近的优秀工作，另外还可以利用神经网络等机器学习方法的万能函数特性来用于量子化学波函数的描述或密度泛函的近似^[28-31]。而最具代表性的突破则是 AlphaFold 程序在蛋白质结构预测的性能已经接近实验的精度^[32-35]。这个事情在社会上也造成了很大的轰动^[33, 36]，例如，有报导称“DeepMind 又推 AI 杰作：AlphaFold 蛋白结构预测击败人类”^[36]。

在本书中，我们将介绍机器学习的相关基础知识，并分析机器学习在化学前沿领域的典型应用例子，以便能够更好地面对机器学习给化学学科所带来的挑战与机会。

第 3 节 机器学习的主要内容

3.1 什么是机器学习？

那么，到底什么是机器学习呢？

机器学习的英文说法是 machine learning，注意其中 learning 是现在进行时。因此，简单地说，机器学习就是“会学习的机器”或“机器在学习”。

我们举一个简单的例子来体会一台会学习的机器是什么样子的。

（你正在通过举例子的方法训练一台机器）

你：机器呀，如果我问你（输入）“0？”，你要回答（输出）“1”。

机器：好啊，没问题。

你：如果我问你“1？”，你要回答（输出）“1.5”。

机器：好啊，没问题。

.....（又教了 1.6-2.4、3-4.5、4-6，如表格 1-1 所示）

你：机器呀，我们现在开始正式提问。

机器：好。

你：2？

机器：.....

如果这是一台不会思考的机器，那么它的回答是：

机器：（气愤）我不知道，你又没教过我这个问题。

如果这是一台会思考的机器，那么它大概会这样做：

（这个没见过的问题的答案会是什么呢？让我把前面已知的数据画到坐标系，看有什么规律没有。画好了，图 1.2。咦，好像这些数据都落在一条直线上耶。我知道了我知道了。）

机器：我猜，是 3！

这就是机器学习，你告诉机器一些你知道的东西，然后希望机器从中寻找规律（模式）并反过来告诉你一些你原本不知道的东西。那么，上述例子中机器的答案对不对呢？也许对，也许不对。因为我们在很多情况下都只知道零星数据，但并不知道从输入到输出的完整映射。机器学习本质上就是在猜测，而且是在不知道微观机制的情况下进行猜测。看似合理的规律也有可能是不对的，参见“题外：波尔文积分—规律失效的例子”。

表 1-1 机器学习的简单例子

输入（问题）	0	1	1.6	3	4	2
输出（答案）	0	1.5	2.4	4.5	6	???

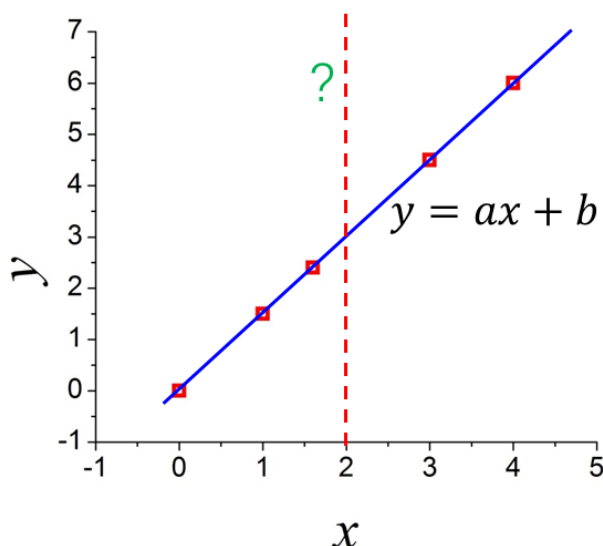


图 1.2. 机器学习的简单例子。红色小方块是已知的数据点（同表格 1-1），蓝色直线是对数据的直线拟合。虚线表示需要猜测的答案，即 $x = 2$ 时 $y = ?$

严格一些的机器学习的定义，可借用计算机图灵奖、诺贝尔经济学奖获得者 Herbert Simon 的观点表述如下：

如果一个机器（程序、算法、系统），能够通过执行某个过程，就此改进了它的性能，那么这就是机器学习。

而 Tom Mitchell 则给出了另一个被广泛引用的绕口令似的定义：

A computer program is said to *learn* from experience **E** with respect to some task **T** and some performance measure **P**, if its performance on **T**, as measured by **P**, improves with experience **E**.

这里的一个关键就是在学习后水平是有提高的。另外需要指出的是，“机器”更大程度上是算法或软件意义上的，而非硬件意义上的，可简单理解成一个算法程序，具有输入、输出、运算、存储等功能。

适合用机器学习去解决的问题，一般满足如下条件：有内在规律可以学习；规律复杂，很难通过解析或穷举的方法列清楚规则；有足够多能够学习到规律的数据。它尤其适用于那种机制不清（如人脸识别）、或者机制清但计算量太大（如围棋、量子力学计算），能够接受近似（误差）以换取速度的问题。

题外：波尔文积分—规律失效的例子

波尔文积分（Borwein integral）是波尔文父子发现的性质特殊的积分，经常被作为看似存在的规律最终失效的例子。

$$\int_0^{+\infty} \frac{\sin(x)}{x} dx = \frac{\pi}{2}$$

$$\int_0^{+\infty} \frac{\sin(x)}{x} \frac{\sin(x/3)}{x/3} dx = \frac{\pi}{2}$$

$$\int_0^{+\infty} \frac{\sin(x)}{x} \frac{\sin(x/3)}{x/3} \frac{\sin(x/5)}{x/5} dx = \frac{\pi}{2}$$

这个看似简单的规律一直持续到

$$\int_0^{+\infty} \frac{\sin(x)}{x} \frac{\sin(x/3)}{x/3} \dots \frac{\sin(x/13)}{x/13} dx = \frac{\pi}{2}$$

但在下一个数，这个规律就突然失效了：

$$\begin{aligned} \int_0^{+\infty} \frac{\sin(x)}{x} \frac{\sin(x/3)}{x/3} \dots \frac{\sin(x/15)}{x/15} dx \\ = \frac{467807924713440738696537864469}{935615849440640907310521750000} \pi \\ \approx \frac{\pi}{2} - 2.31 \times 10^{-11} \end{aligned}$$

3.2 监督学习

机器学习大致可以分成三大类：监督学习（supervised learning）、无监督学习（unsupervised learning）与强化学习（reinforcement learning）。前述的简单例子就属于监督学习的范畴。具体地，有如下定义：

监督学习：给学习算法一个包含了一系列问题与“正确”答案的数据集

$\{\mathbf{x}_n, y_n\} (n = 1, 2, \dots, N)$ ，让算法据此预测某一问题 $\mathbf{x}$ 的答案 y 。

这里 $\mathbf{x}$ 小写粗体无斜体代表矢量，有 D 个分量（数据维度），而大写粗体一般代表矩阵（后面会碰到）。 N 是数据的个数。在前述例子中， $D = 1$ ， $N = 5$ ，而且 $\mathbf{x}$ 只有一个分量，即 $\mathbf{x} = [x]$ 。而在更为复杂的图像识别的例子中，对于 640×480 分辨率的灰度图片， $\mathbf{x}$ 具有 640×480 个分量，彩色图片则有 $640 \times 480 \times 3$ 个分量（每个像素有 R、G、B 三个分量），即

$$\mathbf{x} = \begin{bmatrix} s_{1,1} \\ s_{1,2} \\ \dots \\ s_{1,480} \\ s_{2,1} \\ s_{2,2} \\ \dots \\ s_{2,480} \\ s_{3,1} \\ \dots \\ s_{640,480} \end{bmatrix} \quad \text{或} \quad \mathbf{x} = \begin{bmatrix} r_{1,1} \\ g_{1,1} \\ b_{1,1} \\ r_{1,2} \\ \dots \\ r_{1,480} \\ g_{1,480} \\ b_{1,480} \\ r_{2,1} \\ \dots \\ b_{640,480} \end{bmatrix} \quad (1.1)$$

进一步地，监督学习还可以分成两类：

回归 (Regression): y 的可能取值范围是连续的。例如曲线拟合，即上面所举的简单例子。

分类 (Classification): y 的可能取值范围是分立的。例如 y 的取值可以是 $\{0,1\}$ ，代表答案为“是”与“否”，类似于通常考试中的判断题。当然也可以有更多的取值数目，类似于通常考试中的选择题。

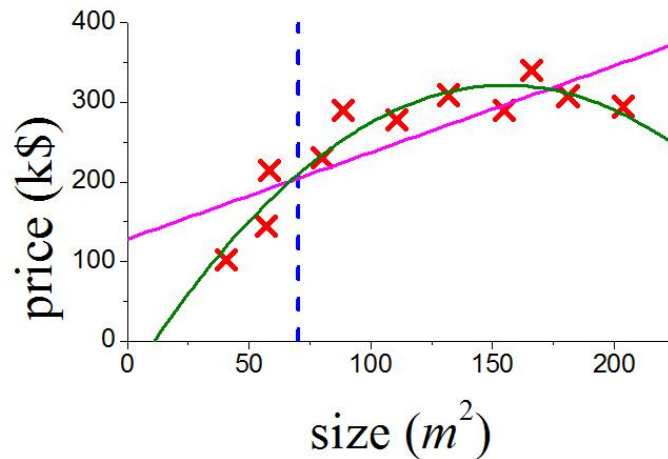


图 1.3. 房价预测的回归问题。离散点是房产中介积累的已知房价数据，虚线指出了需要估计价格的房子的面积。红色直线是对数据的直线拟合，绿色线是抛物线拟合。

上面所举的例子属于回归问题，但它过于简单。接下来我们再举一个现实一

些的回归问题。假设房产中介积累了某个区域最近的一些房价数据，如图 1-3 所示。现在想利用机器学习来预测某个面积（图中虚线所示）的房子大概能卖多少钱。我们（或会学习的机器）先试着用前一例子所用的直线拟合来拟合数据，就可给出猜测的结果。但我们会发现直线好像与已知数据点之间的误差有些大，经过更仔细的观察，发现用抛物线拟合效果更好，因此最后决定用抛物线来拟合数据并给出待评估房子的预测价格。

在这个例子里，决定房子真实价格的机制是未知的，影响因素其实很多，但我们仅有的输入只是房子面积，其它是不知道的。拟合的曲线与数据之间是有非零误差的。预测并不是百分百有把握的，但还是有参考意义，比一无所知要好。这些都是机器学习的典型特点。

回归的思路也能适用于更尖端的、更重大的科学问题。例如，开普勒关于行星轨道的工作（开普勒定律）就可以看做是个典型的回归问题。开普勒当时所面对的就是天文台观察得到的一些离散的带有误差的数据，而他的目标就是要从中猜测出行星的轨道。而他最后给出的猜测结果是椭圆轨道。注意在那个时候人们是不知道行星运动的微观机制的（牛顿定律）。

接下来我们简单介绍一下分类问题，并引入机器学习中的另一个重要概念——特征。

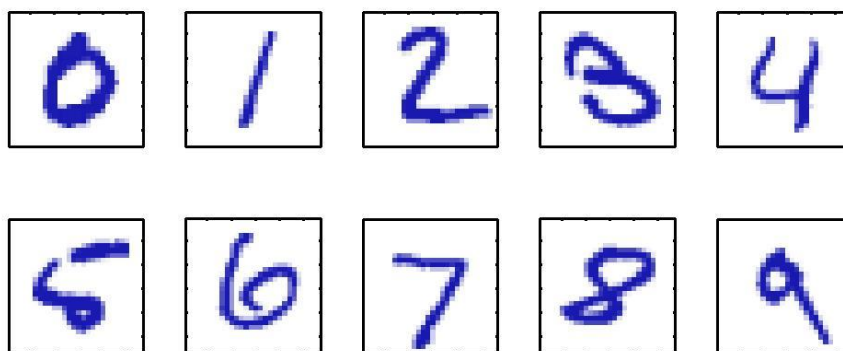


图 1.4. 手写数字识别问题。输入类似于图中所示的图像，输出则是 0-9 中的一个离散数值。

一个有实际应用的分类问题是手写数字识别，它在很早就应用于信件上的邮政编码自动识别（邮政信件的自动分拣系统）以及支票上的金额识别。此时，输入是类似于图 1.4 的图像，通常是通过扫描像素化变成固定大小，而输出（猜测/识别结果）则是 0-9 共十个离散数值中的一个。人脸识别的内容也是类似的，只不过是输出的离散数值更多（例如，一个学校所有人员的编号）。手写数字识别需要的算法比上面的直线拟合或抛物线拟合要复杂一些，例如可以使用后面章节所介绍的自动向量机或卷积神经网络，这里先不介绍。在机器学习的历史上，燕乐存（Yann LeCun）1989 年在 AT&T Bell 实验室利用卷积神经网络和反向传播

实现了很精准的手写数字识别，是机器学习的重要事件。

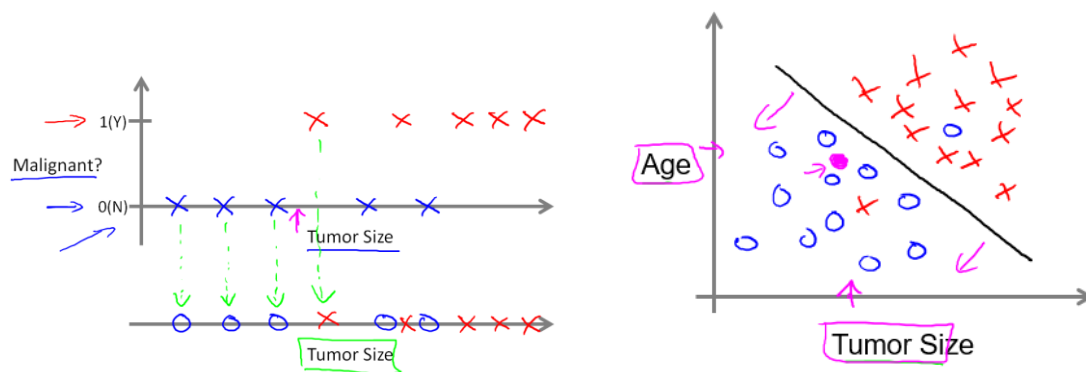


图 1.5. 肿瘤判断问题。(a) 横轴（输入）是肿瘤尺寸，纵轴输出是 0（肿瘤良性）或 1（肿瘤恶性）。(b) 横轴与纵轴表述输入的两个分量，即肿瘤尺寸与患者年龄，而良性肿瘤与恶性肿瘤分别用圆圈与叉表示。

再举一个简单一些的、更为直观的分类问题：恶性肿瘤判断。假设想通过病历数据来学习如何预测乳腺肿瘤良性与否。病例数据包含很多检查的结果（可作为输入），其中最重要的是肿瘤的大小。一组示意性的数据如图 1.5(a)所示，其中离散点是已知的训练数据，表示医院已经通过各种方法确诊了病人的肿瘤是良性的还是恶性的。现在我们需要让会学习的机器从中找出某种规律并实现根据输入就可以预测输出。图 1.5(a)的数据有个比较明显的规律是恶性肿瘤一般尺寸较大。因此一种方法是确定一个临界大小，尺寸大于临界值的都预测为恶性肿瘤，小于临界值的都预测为良性肿瘤，如图 1.5(a)中的虚线所示。这种算法会有一些错误的预测。为了提高预测的准确性，我们可以考虑更多的病例数据，例如患者的年龄、肿瘤的形状，等等。如果同时考虑肿瘤尺寸与患者年龄作为输入因素，数据如图 1.5(b)所示。可以看出，好像可以划一条直线把恶性肿瘤与良性肿瘤较好地分开（这种划直线的思路其实就是逻辑回归和支持向量机等重要机器学习算法的基础），而且同时考虑两个因素的预测效果明显好于只考虑一个因素（此时只能划竖直的线）。

这里就引出了一个很重要的概念——特征（feature）。在肿瘤判断的例子中，病历所包含的数据其实是非常多的，例如肿瘤拍片得到的图像其实包含了很多像素。为什么不是一股脑把所有数据喂给机器学习程序呢？原因是这些数据包含的信息繁杂，有些与肿瘤恶性与否有关，有些则不太相关，而且还包含了大量的噪声。如果简单地把所有数据提供给程序，效果不一定更好。如果能基于一些专家的认识把一些重要的数据（如上述的肿瘤尺寸与患者年龄）挑选出来再提供给程序，效果往往更好。这种把原始数据进行挑选以及变换（例如肿瘤尺寸可以认为是基于拍片图像进一步测量得到的）以后再提供给程序的输入，被称为特征，即能够更好体现所研究问题本质的内在特征。使用这种手段——读入原始输入数据、

进行变换以及挑选、然后把得到的特征喂给某个算法——被称为特征工程。特征工程是具体领域（如化学、物理、生物、材料等）专家能够在机器学习问题上起作用（而不是只需要计算机专家）的一个重要的原因。

思考：学生保研、公司招聘、青年人谈恋爱，是不是也可以描述成机器学习的分类问题，即基于众多输入预测一个输出？

3.3 无监督学习、强化学习

与监督学习类似，无监督学习也有已知的数据；但不同的是，其中并没有答案，即

无监督学习：给学习算法一个数据集 $\{\mathbf{x}_n\} (n = 1, 2, \dots, N)$ ，里面没有答案或标签，让算法据此寻找其中的规律与结构。

什么叫寻找其中的规律与结构呢？一个容易理解的例子见图 1.6 所示。比较容易看出好像这些离散数据大致可以分成两组，组与组之间有明显的距离，或者说，组内数据点相互比较接近，组间数据点之间比较远离。这在无监督学习被称为聚类。聚类在生活中有很多实际应用，例如手机套餐的设置、购物平台的频道划分、新闻组、讨论组、微信群等等，都可以应用聚类的思想。

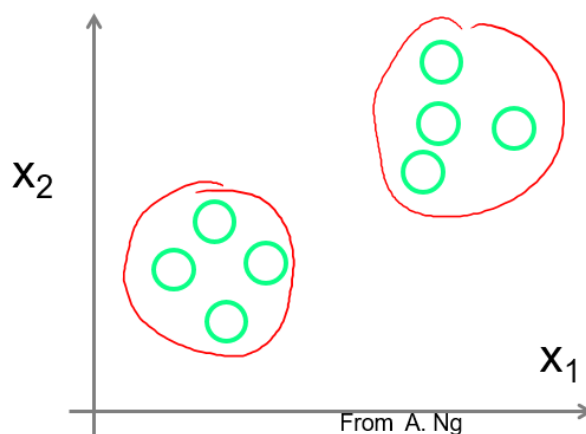


图 1.6. 无监督学习的例子：数据聚类。每个圆圈代表一个数据 $\mathbf{x}_n$ ，有两个分量 (x_1, x_2) 。

强化学习则与监督学习和无监督学习都不同，它没有现成数据，而是

强化学习：给学习算法（智能体 agent）一个规则和目标，让算法通过主动探索寻找达到目标的最佳策略。

它是通过与环境进行交互来获得数据。很多棋类游戏都可以利用强化学习来解决。例如，对于三连棋（图 1.7），只需要规则，就可以按一定策略（很糟糕的策略——如随机下子——也没问题）开始下，如果下输了，就知道这个策略不太好，下次避免（或减少）使用；如果下赢了，就知道这个策略比较好，下次多用；下了

很多盘以后，就会成为高手。在生活中，很多人玩电子游戏也是用的类似思路。

X	O	O
O	X	X
		X

图 1.7. 强化学习的例子：三连棋。对弈双方轮流在印有九格格的棋盘上划“X”或“O”字（或下棋子），谁先把三个己方记号（棋子）排成横线、直线、斜线，即是胜者。

同一个问题可以用不同类型的机器学习来解决。打败李世石的围棋人工智能程序 AlphaGo，既利用了监督学习（通过人类棋谱进行学习），也用了强化学习（试着下，左右手互搏）。

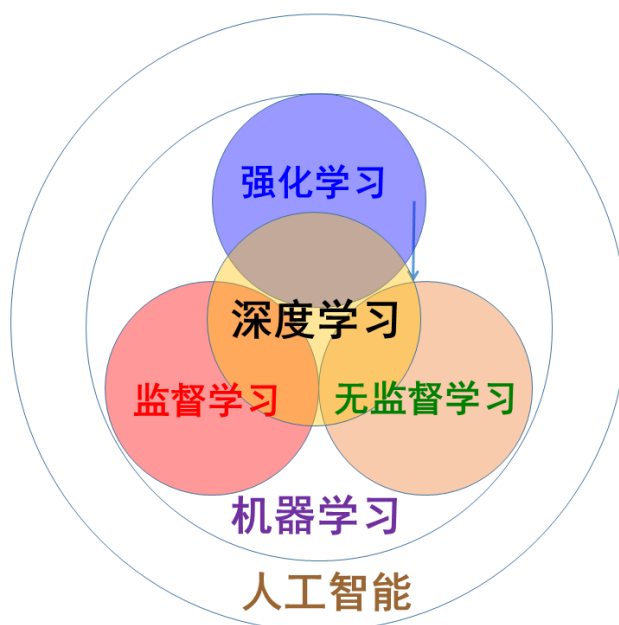


图 1.8. 机器学习主要分成三种类型（监督学习、无监督学习、机器学习）。而深度学习是一种影响深远的方法与架构，可用于三种类型的任务。人工智能的范围则比机器学习要更广一些，还包括一些硬件的内容（如探测器、执行器等），以及一些与机器从数据中自动学习所不同的类型（如专家系统、人工知识图谱）。

另外，还有一个热门的词，深度学习。它又是什么呢？

机器学习的主要任务是从数据中总结规律，以达到举一反三的目的。而数据有不同的形式，有些数据包含了答案，有些则没有，而另外一些则连现成的数据

都没有，需要主动去产生数据。根据数据形态与反馈信息的不同，机器学习可分为监督学习、无监督学习、强化学习三种类型。怎样从数据中总结规律，则可以有各种各样很不相同的方法/模型，例如后面章节中介绍的线性回归、逻辑回归、人工神经网络、近邻法、支持向量机、贝叶斯网络、主成分分析、动态规划、时序差分算法，等等。最近这波人工智能的热潮中，起主导作用的是一种称为深度神经网络的方法，采用这种方法的机器学习，也常被称为深度学习。它并不是与监督学习/无监督学习/强化学习并列，而是一种方法与体系架构，可以用于监督学习/无监督学习/强化学习。它们之间的关系总结在图 1.8 中。

在本书中，第 1-3 章主要介绍监督学习，第 4-6 章主要介绍无监督学习，第 7-9 章主要介绍强化学习，而在第 10 章则简要介绍深度学习。

3.4 机器学习与《物理化学》等课程的差别

《物理化学》课程所涉及的学科内容是非常完善成熟的，采用的是类似于热力学或牛顿力学的在同一框架之下的演绎法。其得出的结果与结论，都是确凿无疑、百分百可信的。

但机器学习很不一样，其内容还在快速发展/变化中。另外，它其实是某种归纳法，从例子中归纳（猜测）规则与原理。机器学习是在不知道微观机制的前提下做预测的，这也许就是为什么有那么多学习算法，而且缺乏如牛顿定律那样的统一框架的原因。这跟以往的理科思维有很大差别（见“一个机器学习的笑话”），很多学生刚接触时会不习惯。

题外：一个机器学习的笑话

考官：你的强项是什么？

我：我是机器学习专家。

考官：9+10 得多少？

我：3。

考官：差太远了，是 19。

我：16。

考官：不对，是 19。

我：18。

考官：还是不对，是 19。

我：19。

考官：恭喜你，你被录取了！

机器学习内容包含的一些经验与规律，不一定有坚实的理论基础。有人认为，机器学习尚处于类似于历史上的炼金术或炼丹术的阶段。不过从积极的角度讲，

炼金术可是现代化学的古代先驱呢，因此机器学习在未来肯定大有可为。

关于机器学习的地位，不同科学家有不同的看法。诺贝尔物理奖得主 Frank Wilczek 认为，“我们从天文学史获得的重要教训是，大数据本身是解释不了自己的。构建简化的数学模型，再将其与真实的物理世界联系起来，这才是从数据这块原始矿石中提炼出‘意义’这颗稀有宝石的可靠方法”。而计算机图灵奖得主 Yann LeCun 则认为，在科学技术史上，工程学上的进步几乎总是先于理论认识：望远镜诞生先于光学理论，蒸汽机先于热力学，飞机先于空气动力学，无线电和数据通信先于信息论，计算机先于计算机科学。

不管是哪种情况，机器学习都是值得我们付出时间进行学习的！

（待补充：数学基础、编程）

作业：

。。。

扩展阅读：

<https://www.iyiou.com/p/38486>

从 1308 年到 2016 年，人工智能在这 700 年时间里发生了什么？.mht

<http://baijiahao.baidu.com/s?id=1646913007012147051&wfr=spider&for=pc>

大数据不等于科学规律.mht

<https://www.huxiu.com/article/376933.html>

为什么 TikTok 能够横扫美国市场？.mht

<https://www.huxiu.com/article/325277.html>

人工智能还是人工智障.mht

<https://baijiahao.baidu.com/s?id=1596520892125761222>

这是一份文科生都能看懂的线性代数简介. mht

https://mp.weixin.qq.com/s?__biz=MzA3MzI4MjgzMw==&mid=2650731034&idx=1&sn=c700041fe10108ca1068c4d80aa0d05a&chksm=871b3664b06cbf726bd93d7b3db4f15125df77a6f69ddb0304b292b932071d2d78104ad5d4f0&scene=21#wechat_redirect

从贝叶斯定理到概率分布：综述概率论基本定义. mht

<https://www.jiqizhixin.com/articles/2021-01-11-3>

人工智能十年回顾：CNN、AlphaGo、GAN……它们曾这样改变世界. mht

<https://www.huxiu.com/article/428329.html>

未来科学的进步，将取决于人类还是 AI ？. mht

<https://www.huxiu.com/article/384223.html>

当 AI 遇到网文. mht

用 AI 技术来翻译网文，帮助中国文学出海。利用技术翻译一部网文，到最终全球上架，最快只需要 48 小时。

<https://www.huxiu.com/article/385803.html>

那些为 AI 研究做过贡献的人，29%在中国念的本科. mht

<https://www.jiqizhixin.com/articles/2019-01-25-13>

一文看懂机器学习 3 种类型的概念、根本差别及应用. mht

参考资料与文献：

第2章 线性回归：最简单的机器学习模型

刘志荣

在本章中，我们将介绍机器学习中最简单也最重要的内容——线性回归。说它简单，是因为它描述的基本就是前一章中所谈到的线性拟合的内容。说它重要，是因为它是统计学与机器学习领域里研究得最彻底的算法，包含了机器学习很多重要的思想，是进一步理解其它章节内容的基础。我们将。。

第1节 单变量线性回归

1.1 一般流程与实现方法

在上一章的简单例子（图 1.2）中，已知数据看起来可以用直线来拟合，即通过适当调整线性函数 $y = ax + b$ 中的参数 a 与 b ，可以使所有数据点都落在 $y = ax + b$ 所描述的直线上。作为一个一般性的机器学习过程，它的流程其实包含了下面三个步骤：

- （1）训练：利用部分已知数据（被称为训练集 **training set**）进行拟合，即确定参数 a 与 b 的数值；
- （2）测试：利用另一部分已知数据（被称为测试集 **test set**）来检验步骤（1）中拟合结果公式 $y = ax + b$ 的准确性；
- （3）预测：利用拟合所得到的函数 $y = ax + b$ 对未知数据进行预测。

之所以要留出一部分已知数据作为测试集，是因为我们希望能够独立评估拟合的效果，因此不能把所有已知数据用于训练（拟合），否则会造成“漏题”，评估不够准确。这有些类似于学校课程的作业（有参考答案）与考试的关系。如果考试题目来自于作业，那么学生可能只会死记硬背，而非真正掌握了相关知识，到不到课程的目的。

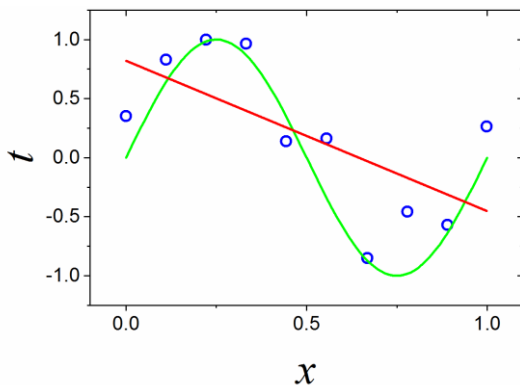


图 2.1. 线性回归的简单例子。离散的数据点是由正弦函数 $\sin x$ 加上高斯噪声得到的。绿色曲线是正弦函数 $y = \sin x$ 。红色直线是线性函数

$$y = ax + b。$$

上一章简单例子中所用的已知数据其实比较特殊，它们本身就是通过线性函数 $y = 1.5x$ 产生得到的，因此很容易找到一条直线通过所有数据点。但是这在一般情况下是不满足的。因此我们考虑更为普适性的数据。然后来看在这种情况下如何进行训练。

我们考虑的数据，是由正弦函数 $\sin x$ 加上高斯噪声所给出的，如图 2.1 所示。注意这里为了方便讨论与理解假设我们知道数据的真正规律（微观机制）为 $\sin x$ ，但在实际应用中大多数场合下其实是不知道数据的微观机制的。当然，机器学习程序（“学习的机器”）是不知道这个机制的，只会用线性拟合来试图揭示数据中存在的规律。

那么，程序是怎样实现上述步骤（1）的训练任务呢？我们可以调整 a 与 b 的数值，得到很多不同的直线（图 2.1），但不再有任何一条直线能够通过所有的数据点。应该取哪条直线作为拟合结果呢？方法就是计算每条直线与所有数据点之间的偏离程度，即误差函数（error function）或代价函数（cost function），然后挑出误差最小的一条直线作为拟合结果。误差函数有多种可能定义方式，其中最常用的是平方和误差函数(Sum-of-Squares Error Function)，也称二乘^c误差：

$$E(a, b) = \frac{1}{2} \sum_i (f(x_i; a, b) - y_i)^2 \quad (2.1)$$

其中 f 是拟合的函数 $f(x) = ax + b$ 。其中的 $1/2$ 是为了方便起见而采取的做法，在求极小值的过程中需要对 $E(a, b)$ 求导，求导后 $1/2$ 就会被消去，显得比较简洁。

利用高等数学知识，误差函数的极小值满足导数为零的条件，即

$$\begin{cases} \frac{\partial E}{\partial a} = \sum_i (ax_i + b - y_i)x_i = 0 \\ \frac{\partial E}{\partial b} = \sum_i (ax_i + b - y_i) = 0 \end{cases} \quad (2.2)$$

这是关于 a 与 b 的二元一次方程组，很容易得到如下的解：

$$\begin{cases} a = \frac{N \sum_i x_i y_i - \sum_i x_i \sum_i y_i}{N \sum_i x_i^2 - (\sum_i x_i)^2} = \frac{\langle xy \rangle - \langle x \rangle \langle y \rangle}{\langle x^2 \rangle - \langle x \rangle^2} \\ b = \frac{\sum_i x_i^2 \sum_i y_i - \sum_i x_i \sum_i x_i y_i}{N \sum_i x_i^2 - (\sum_i x_i)^2} = \frac{\langle x^2 \rangle \langle y \rangle - \langle x \rangle \langle xy \rangle}{\langle x^2 \rangle - \langle x \rangle^2} \end{cases} \quad (2.3)$$

其中 $\langle * \rangle$ 代表基于训练集数据所得到的平均值，即 $\langle xy \rangle = \sum_{i=1}^N x_i y_i / N$ 。利用公式（2.3），代入已知的训练集数据 $\{x_i, y_i\}$ ，就可以得到参数 a 与 b ，从而实现直线拟合的训练任务。

^c 所谓“二乘”，就是平方的意思。

练习：利用公式 (2.3) 编写你的第一个机器学习程序。

1.2 过拟合与解决方法

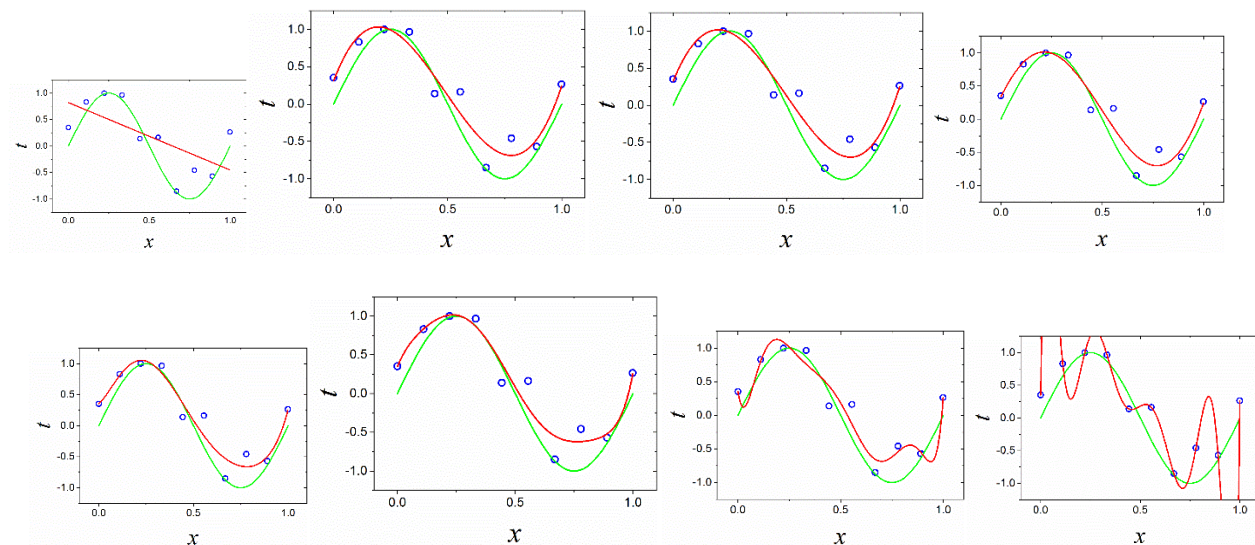


图 2.2. 不同阶数的多项式拟合的结果： $M = 1, 3, 4, 5, 6, 7, 8, 9$ 。数据点共有 10 个，用小圆圈表示。绿色曲线是正弦函数 $y = \sin x$ 。红色直线是多项式拟合结果。

利用上述方法对数据进行直线拟合所得到的结果如图 2.2(a) 所示。可以看出，拟合的效果不是很好，误差较大。不过，我们听过这种方法还是得到了一些信息的（例如，总体而言， $x < 0$ 部分的 y 要大于 $x > 0$ 部分的 y ），比一无所知或随机猜测要好一些。

为了提高拟合的效果，我们可以改用抛物线拟合，或者更普适地，多项式拟合：

$$f(x, \mathbf{w}) = w_0 + w_1x + w_2x^2 + \cdots + w_Mx^M = \sum_{j=0}^M w_jx^j \quad (2.4)$$

此处我们用 $w_0, w_1, w_2, \dots$ 代替 $a, b, \dots$ 以方便表示多个参数，而且按机器学习领域的惯例，成为权重（weight）而非参数。 $\mathbf{w}$ 被称为权重矢量。 M 是多项式的阶数。不同阶数的多项式的拟合结果如图 2.2 所示。从这里起我们一般用 t （target）来表示已知数据集中的（经常是带噪声的）答案 y_i ，以区别于预测值 $y = f(x)$ 或没有噪声影响的本征响应（ $y = \sin x$ ）。

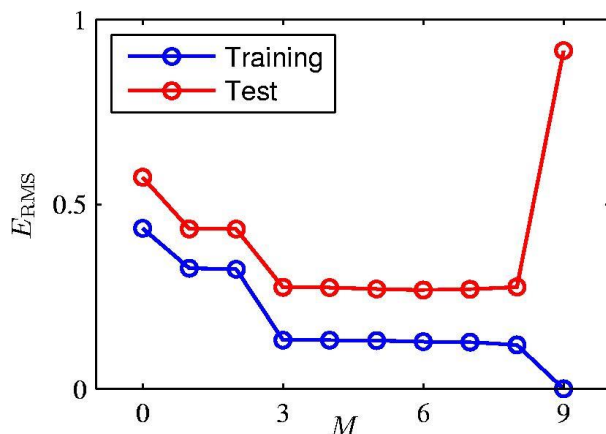


图 2.3. 多项式拟合的误差（方均根误差 $E_{\text{RMS}} = \sqrt{2E(\mathbf{w})/N}$ ）随阶数的变化。蓝色表示训练误差，红色表示测试误差。

从图 2.2 中可以看出，随着多项式阶数的增加，拟合的效果先是变好，但超过一定的阶数以后拟合效果反而变差。最明显的就是九阶多项式[图 2.2(h)]，与真实的正弦曲线偏离极大。注意此时九阶多项式的曲线穿过了所有的 10 个训练集数据点，训练误差为零（令 $f(x_i, \mathbf{w}) = t_i$ ，共 10 个方程，而 $\mathbf{w}$ 共有 10 个分量，因此这是十元一次方程组，一般有唯一解）。但训练数据点以外（例如测试集），就产生了很大的误差，见图。因此这种情况并不是我们所追求的效果。这种现象被称为过拟合，是机器学习中经常碰到的一个大麻烦：

过拟合 (overfitting): 机器学习模型太过迎合训练数据，导致虽然在训练集上表现非常优秀但是在测试集上却表现糟糕，不具泛化性，拿到新样本后不能给出准确的预测。

学习的目的是学到隐含在数据背后的规律，以便对具有同一规律的学习集以外的数据也能给出合适的输出，即对新样本也具有适应能力，称为泛化 (generalization)。也可以简单理解为泛化能力就是模型在测试集上的效果与真实效果的对比，拟合能力就是模型在训练集上与真实效果的对比。

过拟合在现实生活中有很多有趣的例子（参见“题外：生活中的过拟合”）。在科学上，即使是经验丰富的研究人员也可能犯错而造成过拟合，参见“题外：金星上可能有生命体存在”。

除了在测试集中性能变差以外，过拟合还有另一个特性：拟合系数变得太大。在表格 2-1 中，线性拟合的系数为?? 和??，三阶多项式拟合的系数最大也就几十，但发生过拟合的九阶多项式的拟合系数则高达几百万，明显不正常。这个特性也可以用来检查是否发生了过拟合。

题外：生活中的过拟合

- 在畅销书《算法之美：指导工作与生活的算法》中，作者指出，做拟合就是“想得太多”，并举了几个生活中的过拟合例子：
 - 警察的射击练习。警察在靶场进行射击练习时，会在把枪里的子弹打完后把地上的子弹壳捡起来放在口袋里带走。结果在某次与歹徒的实战结束后，一个警察发现口袋里装着自己刚刚捡的子弹壳——在实战中为什么要捡子弹壳呢？
 - 空手夺刀。还是警察的故事。警察刻苦地练习空手夺刀的技术，把陪练手上的刀夺过来，然后把刀再递给陪练，再来一次……。结果在一次与歹徒的搏斗中，这个警察很漂亮地夺过了歹徒的匕首，然后下一个动作是把匕首又塞回到歹徒手中！
 - 击剑的“甩尾”技巧。体育比赛的击剑来源于古代的实战。现在为了安全，击剑比赛中所用的剑实际很软；而为了方便计分，发展了灵敏的电子探测系统，即只要剑尖接触对方就可得分。最后运动员就发展处理类似于“甩尾”的技巧，即通过甩动使剑发生弯曲击中对方——这在实战中是不会出现的。
- 过拟合就是过于看重所设置的指标，反而会妨碍达到真正的目的。我们可以找出生活中更多的过拟合例子：
 - 学生为了冲绩点而选“水课”，学不到东西但可以得高分。这明显是与学习的目的相悖的。
 - SCI 论文与影响因子：教授为了追求论文数目与影响因子而投高影响因子杂志而非与所研究方向最契合的杂志，或故意把一个大的工作拆成几个小工作发表。
 - 大学排名：如果大学排名包含国际留学生比例的指标，那么大学就有动力招收更多留学生而忽视留学生的真实水平或学生毕业后能对社会的贡献；如果大学排名包含了录取分数线指标，那么大学就有动力设置更多提前录取或特殊渠道录取名额（这些不统计到学校的录取分数线），尽量减少正常批的名额以造成供不应求的假象。
- 在经济上，有古德哈特定律（Goodhart's law）：一个指标一旦成为目标，它就不是一个好指标。也就是说，一项经济指标一旦成为一个既定目标，并以此来指引宏观政策的制定，那么该指标就已经失去了本来的信息价值。
- 英文谚语 “The best is the enemy of good”，即 “过于追求完美反而得不到好结果”。
- 眼镜蛇效应（cobra effect）。在英国殖民统治印度期间，为了消灭毒蛇，规定每杀死一条蛇，都会给予报酬。这一政策最初效果不错，但当地人开始饲养蛇，以确保能继续领钱。后来当局不得不停止这一计划，人们就把蛇放掉，这样该地区的眼镜蛇比以前更多了。

题外：金星上可能有生命体存在

2020 年有个鼓舞人心的新闻：金星上可能有生命体存在。其根源是 Nature 子刊发表了英国科学家的一篇论文，宣称他们通过射电望远镜，在金星的大气中检测到了微量磷化氢。他们利用的是磷化氢会吸收频率为 267GHz 的电磁波的事实，得到的吸收谱曲线如下图左小图所示，中间的凹陷就是 267GHz 处的吸收。但很快，有人就指出，他们的这种图其实是过拟合的结果，即不恰当地使用了高达 12 阶的多项式来扣掉基线(baseline)，如下图有小图所示(其中实线为 12 阶多项式基线)；如果按照这种处理方法，那么想在哪个地方找到吸收峰都能实现(加图?)。

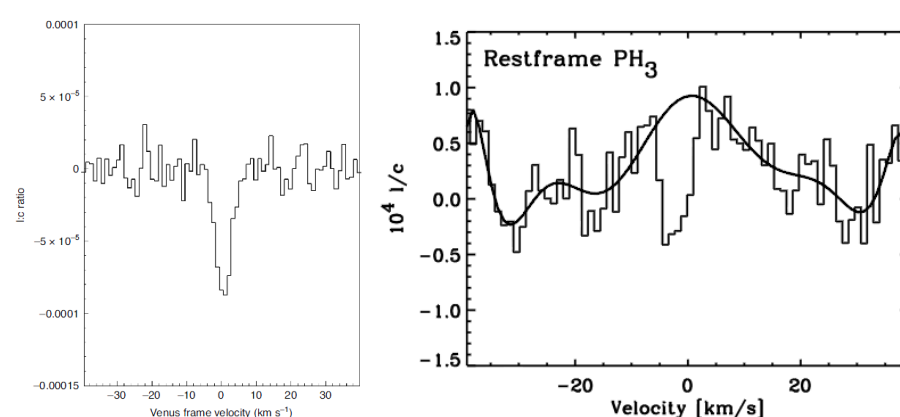


表 2-1 多项式拟合的系数

	$M = 0$	$M = 1$	$M = 3$	$M = 9$
w_0^*	0.19	0.82	0.31	0.35
w_1^*		-1.27	7.99	232.37
w_2^*			-25.43	-5321.83
w_3^*			17.37	48568.31
w_4^*				-231639.30
w_5^*				640042.26
w_6^*				-1061800.52
w_7^*				1042400.18
w_8^*				-557682.99
w_9^*				125201.43

那么，怎样解决过拟合这个问题呢？这里先介绍两个方法，更多方法会在后续章节中介绍。

第一个方法是增加数据。对于形式固定为九阶多项式的拟合函数，如果训练集只有 10 个数据，则如上所示发生过拟合；但如果将数据增加到 15 个，则过拟合程度明显减少（图 2.4(a)）；如果增加到非常多（100 个），则过拟合完全消失，九阶多项式的拟合结果与真实的曲线 $\sin x$ 非常接近（图 2.4(b)）。

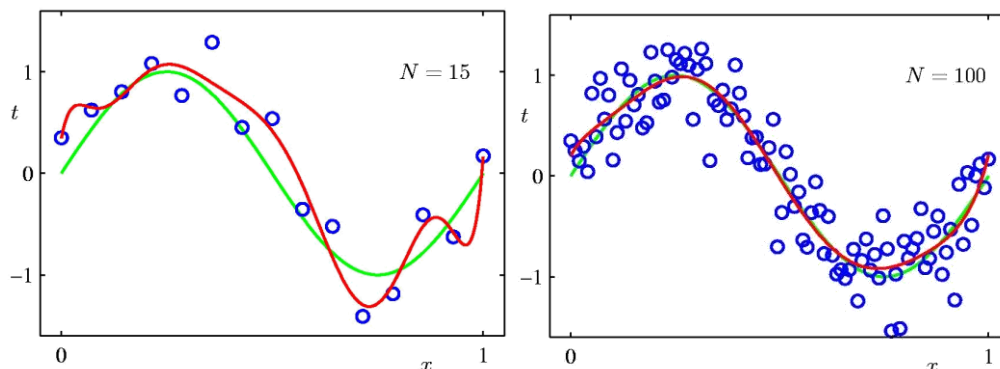


图 2.4. 增加数据对拟合结果的影响。拟合函数固定为九阶多项式，但数据的数目增加到 (a) 15 或 (b) 100。

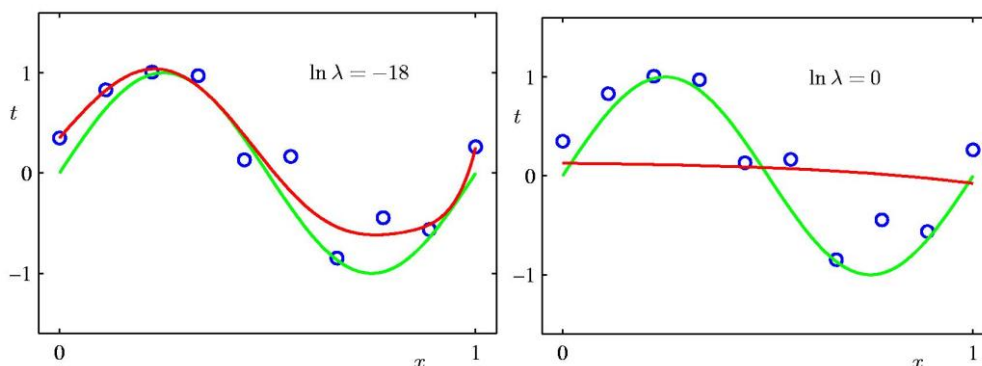


图 2.5. 正则化对拟合结果的影响。(a) $\ln \lambda = -18$; (b) $\ln \lambda = 0$ 。

另一个方法是采用正则化 (Regularization)^d，即“不要想太多”。简单地说，就是鼓励使用简单的模型，惩罚复杂性！在多项式拟合的例子中，项数越多（阶数越高），越容易发生拟合。因此，为了防止过拟合，可以直接规定项数不能过高，这是一种硬约束；也可以允许使用较多的项，但对使用的项增加某种惩罚（类似于税收），这是某种软约束。正则化就是一种软约束的思想。例如，我们可以在原来的误差函数 $E(\mathbf{w})$ 基础上多加一项惩罚项，得到下面的结果：

$$\tilde{E}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N [f(x_n; \mathbf{w}) - y_n]^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2 \quad (2.5)$$

其中 $\|\mathbf{w}\|^2 = \sum_{j=1}^M w_j^2$ 。 $\|\mathbf{w}\|$ 在泛函分析中被称为范数，类似于通常的向量长度。上式等号右边第二项就是正则化项，起到惩罚的作用，其强度由超参数 (hyperparameter) λ 所控制。通过对极小化 $\tilde{E}(\mathbf{w})$ 来确定 $\mathbf{w}$ 的值，实现模型的训练。如果 $\lambda = 0$ ，则没有任何惩罚。如果 $\lambda > 0$ 越大，则惩罚越强，此时为了减小误差，模型将倾向于不要使用太大的拟合系数 w_j 。 λ 非常大时， w_j 将接近零，拟合结果是

^d 注意区分：正则 (canonical) 系综。两者的英文是不同的单词。

接近横轴的平坦的线（图 2.5）。为了确定 λ 的优化值，一般做法是在 λ 的一系列不同取值下进行训练与测试，考察训练误差与测试误差随 λ 的变化。测试误差通常随 λ 呈现“两头翘”的变化（图 2.6），即太弱与太强的正则化都会使效果变差。 λ 优化值的选取应该使测试误差较小，并在可能的情况下使测试误差与训练误差接近（这涉及过拟合与欠拟合的性质，后面第 7 章会详细介绍）。

思考：为什么太弱与太强的正则化都会使效果变差？为什么训练集误差单调增长？

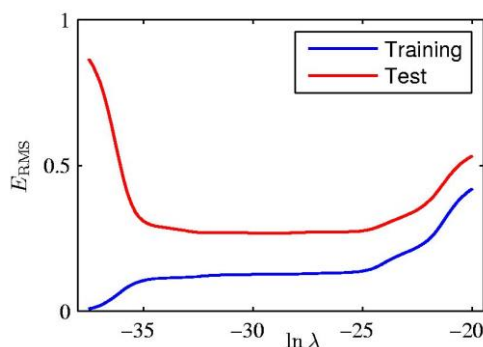


图 2.6. 误差随正则化超参数 λ 的变化关系。注意 λ 采用了对数坐标。基于 $\tilde{E}$ 的最小化进行训练，但图中的训练集误差则是用 $E_{\text{RMS}} = \sqrt{2E(\mathbf{w})/N}$ 计算的。

1.3 维度灾难：机器学习中的另一个大麻烦！

除了过拟合以外，机器学习中还有另一个大麻烦：维度灾难（Curse of Dimensionality，又称维度诅咒）。

维数灾难是最早由理查德·贝尔曼（Richard E. Bellman）在考虑优化问题时首次提出来的术语，用来描述当空间维度增加时（通常有成百上千维），因体积指数增加而遇到各种问题的情形。这样的难题在低维空间中不会遇到。

维度灾难指的是当输入数据 $\mathbf{x}$ 的分量个数 D 很大时（即 $\mathbf{x}$ 是很高维度空间中的点），机器学习的任务将变得非常困难。例如，前面所举例子中 $\mathbf{x}$ 是一维的，一维变量的曲线是很容易用多项式等方法进行拟合的（想象一条光滑的曲线，如果知道曲线上的 100 个点，那一般情况下我们对这条曲线长什么样就很有把握了）。但如果类似的方法用到高维数据（如人脸识别），问题就出现了：参数太多！对于 D 维数据，如果用 M 阶多项式进行拟合，则参数数目正比于 D^M 。对于通常的分辨率为 640×480 的图像，使用线性函数进行拟合需要 1 个零阶参数与， $640 \times 480 = 307200$ 个一阶参数，如果使用二阶多项式函数，则还需要 471874560 个参数。此时很难找到足够数量的数据来做训练。

从另一个方面讲，维度灾难是在高维空间中我们已知的数据相对太少。一个一维变量的函数（二维空间中的一维曲线）取 100 个点可以很好描述；如果按每

个维度上按这个密度取值，则组合后 D 维变量的函数($D + 1$ 维空间中的 D 维曲面)的数据有 100^D 个；如果每个维度只取 10 个点(详细一条曲线上只知道 10 个点，这对了解这条曲线已经不太足够了)，则需 10^D 个数据；即使每个维度只取两个点(这已经明显不够了)，那 100 维变量的函数也有约 10^{30} 个数据点，已经是天文数字了。

在前一小节的分析中我们知道，数据量多，或参数少，才能避免过拟合。维度灾难其实与此密切相关。当数据维数提高时，可能状态空间的体积提高太快(指数上升)，导致可用数据变得很稀疏，很难得到可靠的结果与认识。以分类为例，由于稀疏性，我们更容易找到一个超平面来实现分类，训练样本错误率的可能性变得无限小，模型泛化能力变差。

题外：AI 笑话大全

在训练 AI 的时候，研究者们常用的方法是：制定一个目标（误差函数），让 AI 自己去试错（调整参数，尽量减小误差），从而得出最优结果。但是这种方法有个问题：程序经常会作弊，搞出超展开的解法。

- 检测 X 光片有无肺炎：程序实际检测的不是 X 光片的内容而是拍摄它使用的机器，因为它“发现”病重的病人更可能在特定的医院使用特定的机器拍片。
- 扫地机器人防撞：给扫地机器人编了个程序，鼓励它加速，但不鼓励它撞到东西触发撞击感受器。于是它学会了倒退行驶，因为后面没有撞击感受器。
- 检测皮肤癌：程序发现照片里皮肤病变的边上如果摆了一把尺子，那么这个病变就更可能是恶性的。
- 自动修复 bug：美国点评网站 Yelp 的工程师们训练了一个用来消除 bug 的神经网络，万万没想到，该网络删除一切，从根上彻底实现了“bug-free”。

我们生活在低维世界里，很多高维空间的性质对我们来说是反直觉的。例如，高维球面的面积绝大部分集中于赤道附近。高维不容易想象。因此，维度增加以后，AI 更容易作弊，或者说，即使机器学习程序在高维数据中没有真正学到东西，我们也不容易发现。例如，我们给机器学习程序一堆猫与狗的照片（以及标签，即告诉机器哪些是猫的照片哪些是狗的照片），希望程序能够学会根据照片区分猫狗。但由于照片（图像）是高维的，有很多很多特征，由于数据相对不够，可能不相关的特征也可以用于在训练中成功区分猫与狗，例如，照片的平均亮度、照片里植物的多少、绿色像素的数量，等等。但这些并不是我们真正想让程序学

习的。AI 领域的很多笑话都与此相关（参见“题外：AI 笑话大全”）。

第 2 节 基于概率论角度的分析

2.1 关于误差函数的疑虑

上一节所介绍的机器学习流程中的训练步骤使用的其实是最小二乘法。最小二乘法就有悠久的历史，最早是法国数学家勒让德于 1805 年在他的著作《计算彗星轨道的新方法》中提出的。它既可用于线性拟合，也可用于非线性拟合。

最小二乘法有个关键，就是它是以预测数据与实际数据之间误差的平方和（二乘误差）作为误差大小的总体衡量，将这个误差最小化就可以得到最优参数。但是，为什么非得采用这种形式呢？其它形式就不行吗？例如，为什么不是使用误差绝对值的和 $\sum |f(x_n; \mathbf{w}) - y_n|$ ？或者误差四次方的和 $\sum [f(x_n; \mathbf{w}) - y_n]^4$ ？

这其实涉及到怎样把多个指标综合成单一指标的普适问题，是非常复杂的。举个生活中的例子，某个学校通过考试录取学生，考两个科目，学生 A 的得分是 90 分与 80 分，学生 B 的得分是 100 分与 70 分，学生 C 是 85 分与 85 分，那么最应该录取那个学生呢？采用不同的综合标准（例如，简单总分，单科最高分^e，丢分平方和），结论就会不一样。又如，大学招聘教师，会从应聘者的科研、教学、教育背景（毕业学习、导师）、研究方向、获取项目经费能力（包括与工业界的联系）、甚至是性别等方面考虑，这些方面的结果应该怎样综合成一个单一指标以决定录取谁呢？还有体育运动中的射击、冰壶球等项目，都涉及到类似问题。

数学王子高斯（1777—1855）对最小二乘法也像我们一样心存怀疑。高斯换了一个思考框架，通过概率统计的思路来进行分析，终于阐明了最小二乘法的理论基础，从而奠定了最小二乘法在科学上的重要地位。

接下来我们从概率论的角度来分析最小二乘法的合理性。

2.2 观察值的概率

假设观察值 t 可写成

$$t(x) = f(x, \mathbf{w}) + \epsilon = y(x, \mathbf{w}) + \epsilon \quad (2.6)$$

其中 $f(x, \mathbf{w})$ 是一个确定的函数，而 ϵ 是个随机值（噪声），满足一定的分布（例如，高斯分布，参见“数学背景：高斯分布”）。我们在前一节中所介绍的 $\sin x$ 例子的数据就是这样产生的。注意从这里起我们一般用 t （target）来表示已知数据集中的（经常是带噪声的）答案 y_i ，以区别于预测值 $y = f(x)$ 或没有噪声影响的本征

^e 钱钟书的入学考试成绩是数学 15 分，国学与英语 100 分，结果被录取，显然更接近于“单科最高分”的标准。

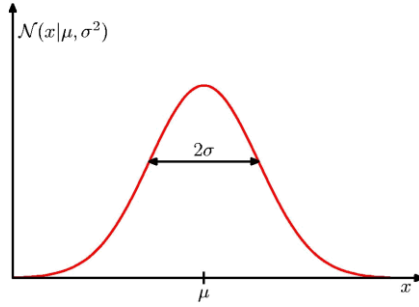
响应（例如 $y = \sin x$ ）。

数学背景：高斯分布（正态分布）

高斯分布为

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(x-\mu)^2} \equiv \mathcal{N}(x|\mu, \sigma^2)$$

其中 μ 是 x 的平均值（mean）或期望值（expectation）， σ^2 是方差（variance）。" $|$ "表示右边的值是参数，左边是变量。后面将把 $\mathcal{N}(x|\mu, \sigma^2)$ 重新诠释为条件概率。典型曲线如下图：



由于 ϵ 的影响，即使是对某一固定的 x ，观察值 $t(x)$ 将是个随机值，在 $f(x, \mathbf{w})$ 附近涨落。如果在某一 x_n 处观察到 t_n ，则在这一点上的噪声为

$$\epsilon_n = t_n - y(x_n, \mathbf{w}) \quad (2.7)$$

我们假设噪声服从高斯分布，因此其概率为

$$p(t_n|x_n) = \mathcal{N}(\epsilon_n|0, \sigma^2) = \mathcal{N}(t_n - y(x_n, \mathbf{w})|0, \sigma^2) = \mathcal{N}(t_n|y(x_n, \mathbf{w}), \sigma^2) \quad (2.8)$$

如果在一系列 $\{x_n\}$ 处观察到 $\{t_n\}$ ，则总的概率为：

$$p(\mathbf{t}|\mathbf{x}, \mathbf{w}, \beta) = \prod_{n=1}^N \mathcal{N}(t_n|y(x_n, \mathbf{w}), \beta^{-1}) \quad (2.9)$$

这里 Π 是连乘号，公式右边表示将 N 个数据点的概率乘起来。这种相乘表明我们此时是假设不同点的噪声是独立的。我们还假设所有点的噪声的分布也是相同的，即它们都有相同的方差 σ^2 ，并按机器学习领域的惯例记 $\sigma^2 \equiv 1/\beta$ 。也就是说，噪声是零均值独立同分布。另外，黑体 $\mathbf{t}$ 与 $\mathbf{x}$ 表示它们包含了不同数据点（训练集）的值，即 $\mathbf{t} = \{t_n\}$ ， $\mathbf{x} = \{x_n\}$ ，并不表示单个数据点的 x_n 或是 $\mathbf{x}$ 包含多个分量的矢量。后面在不容易引起混淆的情况下，将有时候用黑体表示多个数据，有时候用黑体表示矢量。

2.3 概率最大化（最大似然法）

在机器学习的训练步骤中，我们知道的是训练集 $\{x_n, t_n\}$ ，不知道（需要求解）的是参数 $\mathbf{w}$ 和 β 。根据公式（2.9），任一具体的 $\mathbf{w}$ 和 β 数值下，我们都可以计算得到非零的我们能观察到数据 $\{x_n, t_n\}$ 的概率 $p(\mathbf{t}|\mathbf{x}, \mathbf{w}, \beta)$ 。也就是说，一切皆有可能，

即使是图 2.7 中那条明显偏离数据点的棕色曲线 $y(x_n, \mathbf{w})$ ，也有可能在加上噪声后产生图里的那些数据点，只是可能性/概率很小（但大于零）而已。而那条蓝色曲线加上噪声后产生图中数据点的可能性则要大得多。那么，如果我们只能观测 $\{x_n, t_n\}$ 而永远不能直接观测 $y(x_n, \mathbf{w})$ ，但我们想尽可能合理地猜测（推断） $y(x_n, \mathbf{w})$ ，我们会选择哪一条曲线来作为我们最后的猜测结果呢？很显然，蓝色曲线会比棕色曲线更好，因为它的概率更大。这就引出了最大似然法^f，即通过使公式（2.9）的概率最大化，从而确定参数 $\mathbf{w}, \beta$ 。

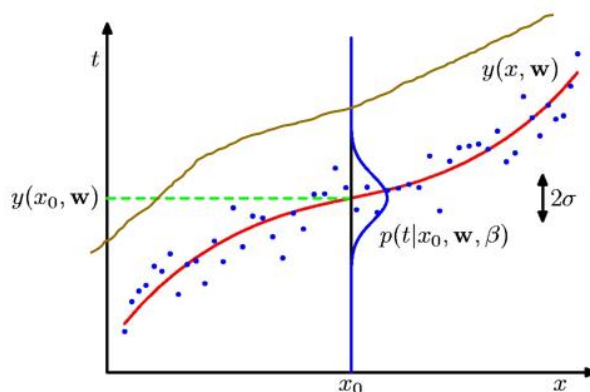


图 2.7. 基于已知的 $\{x_n, t_n\}$ ，各种 $y(x_n, \mathbf{w})$ （例如图中的红色与棕色曲线）都是可能的，只是概率不同而已。

与最大化概率公式（2.9）等价，但数学上更方便的做法，是最大化其对数值 $\ln p(\mathbf{t}|\mathbf{x}, \mathbf{w}, \beta)$ （这将使其中的连乘变成相加），即

$$\ln p(\mathbf{t}|\mathbf{x}, \mathbf{w}, \beta) = -\frac{\beta}{2} \sum_{n=1}^N [y(x_n; \mathbf{w}) - t_n]^2 + \frac{N}{2} \ln \beta - \frac{N}{2} \ln(2\pi) \quad (2.10)$$

注意公式等号右边第一项等于 $-\beta E(\mathbf{w})$ ，即与二乘误差 $E(\mathbf{w})$ 成正比，需要基于数据集 $\{x_n, t_n\}$ 进行计算；而其余两项与数据点 $\{x_n, t_n\}$ 并没有直接关系。根据极值点的数学性质， $\ln p(\mathbf{t}|\mathbf{x}, \mathbf{w}, \beta)$ 对 $\mathbf{w}$ 的偏导数等于零，即

$$\frac{\partial}{\partial \mathbf{w}} \ln p(\mathbf{t}|\mathbf{x}, \mathbf{w}, \beta) = -\beta \frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} = 0 \quad (2.11)$$

β 不等于零，可消去，因此方程（2.11）正好与前面最小二乘法的方程（2.1）与（2.5）等价。这就从概率论的角度论证了最小二乘法的合理性！可以解出参数 $\mathbf{w}$ 的值 $\mathbf{w}_{\text{ML}}$ （下标 ML 特别指出这是机器学习解出来的最佳参数 $\mathbf{w}$ 值，而不是一般性的未知变量 $\mathbf{w}$ ）。另外，最大似然法还可以进一步求出噪声的大小，即基于 $\frac{\partial}{\partial \beta} \ln p(\mathbf{t}|\mathbf{x}, \mathbf{w}, \beta) = 0$ 得到：

^f 在下一节中我们将会介绍“似然”一词的含义，以及为什么这种最大化概率的方法会被称为最大似然法。

$$\frac{1}{\beta_{\text{ML}}} = \sigma^2 = \frac{1}{N} \sum_{n=1}^N \{y(x_n, \mathbf{w}_{\text{ML}}) - t_n\}^2 \quad (2.12)$$

这个信息在最小二乘法中是得不到的。

小结一下，我们可将观察到的数据理解为由某个随机过程产生的，具有一定的概率分布。我们力图透过有随机性的数据看清其下的确定性规律。具体做法通过最大化观察值的概率，求出 $f(x, \mathbf{w})$ 的参数 $\mathbf{w}$ ，以及噪声 ϵ 的强度。这是一种重要思路(概率论的角度)，与直接定义误差在有本质性的不同(凭什么这么定义?)。

第3节 一个更深刻的角度：贝叶斯理论

“贝叶斯统计之所以困难，是因为思考是困难的”

——Don Berry

“贝叶斯定理堪称一座知识宝库，从神经科学到人工智能，无所不计。

这是一个带来启示与革新，改变认知和预测方式，
颠覆固有思维的奇妙定理。”

——黄黎原

机器学习是在不知道数据背后微观机制的情况下，直接寻找(猜测)数据中可能的模式。AI 系统接收输入并产生输出，其性能可以通过测试误差等指标来衡量，但其深层次的运作逻辑与理论根据在很多时候并不是很清晰，因此经常被看做是黑盒子。但是，需要指出的是，机器学习并不一定要是黑盒子，它也可以有自己的理论基础。对于什么是机器学习最重要的理论基础，不同人持有不同的观点。我的个人观点是，机器学习最重要的基础是贝叶斯定理。

3.1 贝叶斯公式

我们从一个抛硬币的故事开始。

(你有一个朋友，掏出一个硬币，想和你玩个游戏)

朋友：我们玩个抛硬币的游戏吧？每抛一次，正面朝上的话你给我一块钱，反面朝上的话我给你一块钱。

你：(这题我会，读小学的时候就知道了硬币概率了) 好。

朋友：(抛，结果正面朝上) 我赢了

你：好，给你一块钱。

朋友：(再抛，正面朝上) 我又赢了

你：可以理解。给你一块钱。

朋友：(又抛，还是正面朝上) 你输了

你：(概率论告诉我们，每一次抛硬币的结果都与之前的结果无关。所以，这也没什么不可以的。) 给，一块钱。

..... (连抛了十次, 每次都是正面朝上)

朋友: 你又输了

你: *!\$@&*#. (你是否还是觉得没问题?)

在这个故事里, 你会不会怀疑你的朋友做了弊, 比如, 使用了一枚两面都是正面的特殊硬币? 中国古代就有类似故事, 参见“题外: 狄青的铜钱”。我们以前熟悉的是这样的题目: 已知一枚硬币正面朝上的概率是 50%, 求抛 6 次出现 4 次朝上的概率? 或已知前面已经抛出了 5 次朝上, 那再抛第六次朝上的概率是多少? 这是知道原因 (硬币概率 p , 取决于硬币的构造), 来推断结果 (抛, 朝上/朝下的可能性)。但上述的例子其实是知道结果 (抛了多次, 每次的朝上/朝下), 想推断原因 (概率 p)。基于原因推断结果, 是人类擅长的思维; 但从结果推断原因, 却是一般人所不擅长的。例如, 你觉得你朋友作弊的概率是多少?

题外: 狄青的铜钱

狄青带军出征, 敌众我寡, 士兵士气低落。

狄青将士兵带到一座灵庙, 掏出一百枚铜钱, 向神灵问卦: “我把这些铜钱撒在地上, 如果全都是正面朝上的话, 那么我们将大获全胜; 否则, 我们将死无葬身之地”。

结果, 撒出的百枚铜钱全部正面朝上, 遂士气大振。

狄青令人用铁钉将铜钱钉在地上, 承诺战后回来谢神。

果然大胜!

回来后, 谢神, 收回铜钱, 交给士兵查看: 原来是特制的铜钱, 两面都是正面!

(后来, 这百枚铜钱被称为狄青钱, 又称无背钱, 被认为有神奇的效果, 可以挡灾护命。小说《哑舍》有个故事“无背钱”就是讲述其中一枚狄青钱的后续故事。)

贝叶斯 (Thomas Bayes, 1702-1761) 考虑的不是铜钱问题, 而是彩票问题。例如, 考虑下面的问题:

如果你的邻居买了 10 张新出的彩票, 其中有 5 张中奖, 那么, 你估算这期彩票的中奖概率是多少? 你是否也应该去买几张?

一般人会估计中奖概率是 $p = 5/10$, 即再买一张会有 50% 的概率中奖。那么, 再考虑下面这个问题:

如果你的邻居买了一张新出的彩票, 中奖了, 那么, 你估算这期彩票的中奖概率是多少? 你是否应该把房子卖了全部去买彩票?

按上面的思路, 中奖概率好像是 $p = 1/1 = 100\%$ 。你真的相信这个结果而卖了

房子去买彩票吗（中奖概率可是百分之百耶）？贝叶斯对此进行了深入研究，并得出了一个深刻的见解：试图根据我们看到的中奖和未中奖彩票结果来分析整体彩票池的方法，本质上是倒推。贝叶斯写了一篇名为“An essay towards solving a problem in the doctrine of chances”（机遇理论中一个问题的解）来解答这个问题，其中所给出的重要公式后来被称为贝叶斯公式或贝叶斯定理。

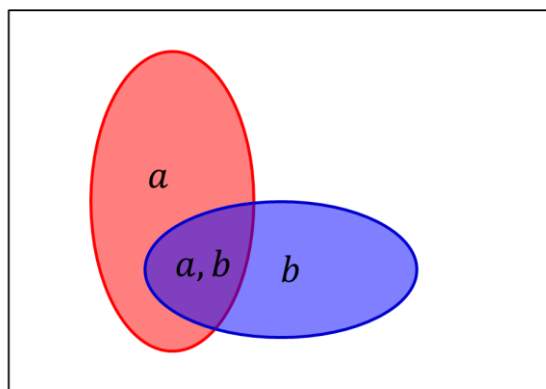


图 2.8. 条件概率示意图。随机变量 a 与 b 的可能取值是 $\{0,1\}$ 。方框面积表示所有可能的概率。两个椭圆的面积分别给出 $P_A(a=1)$ 与 $P_B(b=1)$ 的值，而它们的重合面积则给出 $P(a=1, b=1)$ 。因此在 $b=1$ 条件下 $a=1$ 的条件概率为 $P(a=1|b=1) = \frac{P(a=1, b=1)}{P_B(b=1)}$ 。其它取值下也有

$$P(a|b) = \frac{P(a,b)}{P_B(b)}。$$

贝叶斯公式并不难推导。考虑两个随机变量 a, b （比如在摸球游戏中，摸出的球可以有不同颜色 a ，上面还印有不同数值 b ）的联合概率 $P(a, b)$ 。如果只关心其中一个变量的概率，例如 a 的概率 $P_A(a)$ （此处用 P_A 以特别表示这是关于 a 的单变量概率），则有 $P_A(a) = \sum_b P(a, b)$ 。在 b 已经发生的条件下， a 发生的概率被称为 a 在 b 条件下的条件概率，记为 $P(a|b)$ ，它可以由下式计算（参见图 2.8）：

$$P(a|b) = \frac{P(a,b)}{P_B(b)} \quad (2.13)$$

或者有

$$P(a, b) = P(a|b)P_B(b) \quad (2.14)$$

也就是说， a, b 同时发生的概率，等于 b 发生的概率乘上在 b 已经发生的条件下 a 发生的概率。这在概率论里被称为乘法规则。类似地，有

$$P(a, b) = P(b|a)P_A(a) \quad (2.15)$$

结合（2.14）与（2.15），有

题外：贝叶斯公式的影响

在现代的概率论中，主要有两个学派，一个是频率学派，另一个是贝叶斯学派。在早年间，主流是频率学派。但现在，贝叶斯学派的影响越来越大。例如，数学家杰恩斯（E. T. Jaynes）在其影响颇大的教科书《概率论沉思录》（英文原名 *Probability Theory: The Logic of Science*，因此更准确的翻译应该是《概率论：科学的逻辑》）从贝叶斯公式出发，将概率和统计推断融合在一起，系统阐述了贝叶斯概率论作为多个科学学科的数学基础的可能性。

黄黎原在科普畅销书《贝叶斯的博弈：数学、思维与人工智能》（英文原名 *The Equation of Knowledge: From Bayes' Rule to a Unified Philosophy of Science*，直接翻译应该是《知识的公式：从贝叶斯公式到科学的统一哲学》）中系统阐述了贝叶斯定理背后的知识、思考方法和哲理。作者从数学、哲学、计算机科学、神经科学和人工智能等研究角度，列举众多日常生活中与人工智能领域有关的案例，一步步再现了贝叶斯的思维框架，直击机器学习的核心方法，深入探索贝叶斯定理的威力。其广告词写得尤其精彩：有人生前波澜不惊，死后却名声大振，贝叶斯就是其中之一；以他命名的“贝叶斯定理”堪称一座知识宝库，从神经科学到人工智能，无所不计。

$$P(a|b)P_B(b) = P(b|a)P_A(a) \quad (2.16)$$

把左边的 $P_B(b)$ 移到右边，得到

$$P(a|b) = \frac{P(b|a)P_A(a)}{P_B(b)} \quad (2.17)$$

这就是鼎鼎大名的贝叶斯公式（贝叶斯定理，Bayes' Theorem）。这个公式形式上平淡无奇，内涵上却博大精深，需要反复琢磨才能领会。严格讲，此处没有因果的概念。但我们可以把它应用到因果推断上，把 a 理解成原因（例如，是正常铜钱还是狄青钱），把 b 理解成结果（例如，把铜钱抛了若干次所出现的正面/反面结果）。从原因比较容易计算结果的概率，即 $P(b|a)$ 比较容易计算；从结果比较难反推原因，即 $P(a|b)$ 难于直接计算。而贝叶斯公式（2.17）就提供了一种从 $P(b|a)$ 计算 $P(a|b)$ 的方法，可用于从结果反推原因。注意此时 b 是已经发生的，相当于取了固定值，公式右边分母的 $P_B(b)$ 只起到归一化常数的作用，可以扔掉不管，只取分子 $P(b|a)P_A(a)$ 来计算 a 不同取值的相对概率大小，或者根据相对概率大小最后再把这个归一化系数求出来（ $P_B(b) = \sum_a P(b|a)P_A(a)$ ）。这有些类似于玻尔兹曼分布的情况， $p(\{\mathbf{r}\}) = \frac{1}{Q} \exp\left[-\frac{E(\{\mathbf{r}\})}{k_B T}\right]$ ，其中配分函数 $Q = \int \exp\left[-\frac{E(\{\mathbf{r}\})}{k_B T}\right] d\mathbf{r}$ 起到归一化系数的作用。

题外：贝叶斯公式在沉船搜索中的应用

1968 年，美国的天蝎号核潜艇在大西洋亚速海海域突然爆炸失踪。由于失事时潜艇航行的速度与方向、爆炸冲击力的的大小、爆炸时潜艇方向舵的指向都是未知的，即使知道潜艇在哪里爆炸，也很难确定潜艇残骸最后被海水冲到哪里。为了搜寻天蝎号，美国政府调集了包括多位专家的搜索部队。其中，数学家 John Craven 提出的方案使用了贝叶斯公式。他把各位专家的看法综合到一起，得到了一张天蝎号核潜艇可能沉睡的 20 英里海域的估计概率图。整个海域被划分成了很多个小格子，每个小格子有两个概率值 p 和 q ， p 是潜艇躺在这个格子里的概率， q 是如果潜艇在这个格子里，它被搜索到的概率。如果一个格子被搜索后，没有发现潜艇的踪迹，那么按照贝叶斯公式，这个格子潜艇存在的概率就会降低。由于所有格子概率的总和是 1，这时其它格子潜艇存在的概率值就会上升。每次寻找时，先挑选整个区域内潜艇存在概率值最高的一个格子进行搜索；如果没有发现，根据贝叶斯公式更新概率分布图，搜寻船只就会驶向新的“最可疑格子”进行搜索；这样一直下去，直到找到天蝎号为止。

最初开始搜救时，海军人员对 Craven 的建议嗤之以鼻，他们凭经验估计潜艇是在爆炸点的东侧海底。但几个月的搜索一无所获，他们才不得不听从了 Craven 的建议，按照概率图在爆炸点的西侧寻找。经过几次搜索，潜艇果然在爆炸点西南方的海底被找到了。

由于这种基于贝叶斯公式的方法在后来多次搜救实践中被成功应用，现在已经成为海难空难搜救的通行做法。

类似做法还可以用于矿藏与石油的勘探。

贝叶斯公式揭示了一个非常重要的思想：信念，随着证据的出现而变化。例如，在前述的抛硬币例子中，即使你最开始很相信你的朋友不会作弊（例如，99%，即作弊概率只有 $P_A(a) = 1\%$ ），但是，随着每次都抛出正面朝上的结果（ b ），你相信你朋友不会作弊的信念会一点点变弱，或者说，他作弊的可能性 $P(a|b)$ 会不断上升。这个思想非常重要。它是把概率定义为信念的程度，后来发展成了概率论的现代贝叶斯学派。硬币只有一枚，事实被掩盖（你不能直接测量/检测 a 的值），你只能根据你看到的来做判断。这其实就是机器学习的做法，从数据中尽可能地猜测真相。贝叶斯公式在很多领域中有广泛应用（参见“题外：贝叶斯公式的影响”）海难搜救方面也有重要应用（参见“题外：贝叶斯公式在沉船搜索中的应用”）。

1774 年，拉普拉斯（Pierre-Simon Laplace, 1749-1827）（参见“题外：拉普拉斯与拉普拉斯兽”）发表著作《事件原因的概率论》，独立地再次发现了贝叶斯公式，并对彩票问题给出后来广为人知的如下解答。

如果中奖概率为 x ，则抽出 $m+n$ 张彩票有 m 张中奖的概率为

$$p(m, n|x) = C_{m+n}^m x^m (1-x)^n \quad (2.18)$$

其中 C_{m+n}^m 是从 $m+n$ 个不同元素中取出 m 个元素的组合数。如果我们知道邻居买了 $m+n$ 张彩票有 m 张中奖，则可以利用贝叶斯公式(2.17)反算彩票池的中奖率为 x 的概率

$$p(x|m, n) = \frac{p(m, n|x)p(x)}{p(m, n)} = \frac{p(m, n|x)p(x)}{\int p(m, n|x)p(x)dx} \quad (2.19)$$

拉普拉斯假设 x 的先验分布（即还没有买任何彩票时对中奖率的认识） $p(x)$ 是均匀分布， $p(x) \propto \text{constant}$ ，则

$$p(x|m, n) = \frac{C_{m+n}^m x^m (1-x)^n}{\int_0^1 C_{m+n}^m x^m (1-x)^n dx} \quad (2.20)$$

因此，再买一张彩票中奖的概率是

$$\langle x \rangle = E(x|m, n) = \int_0^1 x p(x|m, n) dx = \frac{\int_0^1 C_{m+n}^m x^{m+1} (1-x)^n}{\int_0^1 C_{m+n}^m x^m (1-x)^n dx} = \dots = \frac{m+1}{m+n+2} \quad (2.21)$$

其中 E 代表期望值。这是现代贝叶斯学派引以为荣的一个结论。如果买一张彩票中了，则奖池平均中奖比例是 $2/3$ ，而不是 100% 。如果买三张彩票中了一张，则奖池中奖平均中奖比例比例为 $2/5$ ；如果三张全中，则结果为 $4/5$ ，还是小于 100% 。

题外：拉普拉斯与拉普拉斯兽

拉普拉斯是个很奇怪的人，人品上综合了伟大和卑微，科学上综合了决定论和概率论。拉普拉斯对概率的理解极深，被认为是贝叶斯定律的“养父”。但是，他同时认为骰子所表现出来的随机性其实是可以“决定”的：假设世界上有一个神兽，它知道宇宙中每个原子确切的位置和动量，那么就能够使用牛顿定律来展现宇宙所有事件的完整过程，包括过去以及未来（因此也包括骰子哪一面朝上）。对一个无所不知的“妖”而言，随机性根本不存在。这种设想后来被称为拉普拉斯妖或拉普拉斯兽，能够推演万物未卜先知。它与缩地成寸永远追不上的“芝诺龟”、逆转时空起死回生的“麦克斯韦妖”和超越因果亦生亦死的“薛定谔猫”被戏称为物理学“四大神兽”。东野圭吾的小说《拉普拉斯的魔女》就是从这里获得的灵感。

3.2 贝叶斯公式在线性回归中的应用

针对机器学习的问题，我们可以把贝叶斯公式重新写成

$$p(\mathbf{w}|\mathcal{D}) = \frac{p(\mathcal{D}|\mathbf{w})p(\mathbf{w})}{p(\mathcal{D})} \quad (2.22)$$

其中 $\mathcal{D}$ 代表我们测量到的数据，或者说是已知的训练集 $\{\mathbf{x}_n, t_n\}$ ；而 $\mathbf{w}$ 则代表我们想通过推断得到的模型参数。这里我们把 $\mathbf{w}$ 也看做是某种随机变量，在没有看到任何数据之前服从概率分布 $p(\mathbf{w})$ ，依赖于某些先验知识。 $p(\mathbf{w})$ 被称为先验(prior)概率。 $p(\mathcal{D}|\mathbf{w})$ 是在 $\mathbf{w}$ 给定条件下 $\mathcal{D}$ 的分布概率，类似于公式前面公式(2.9)给出的结果，即从原因($\mathbf{w}$)推断到结果($\mathcal{D}$)。在机器学习问题中， $\mathcal{D}$ 是知道的、固定的， $\mathbf{w}$ 是未知的、需要推断的，因此 $p(\mathcal{D}|\mathbf{w})$ 也可以看做是关于变量 $\mathbf{w}$ 的函数，称为似然(likelihood)^g函数。注意 $p(\mathcal{D}|\mathbf{w})$ 是 $\mathcal{D}$ 的分布概率，但不是 $\mathbf{w}$ 的分布概率。 $p(\mathbf{w}|\mathcal{D})$ 被称为后验(posterior)概率，是在 $\mathcal{D}$ 给定条件下 $\mathbf{w}$ 的分布概率，即从结果($\mathcal{D}$)反推原因($\mathbf{w}$)。这就是贝叶斯定理在机器学习中的应用。公式中的 $p(\mathcal{D})$ 可看做归一化常数，通常不重要。因此，先验概率、似然函数与后验概率之间满足如下关系

$$\text{后验概率} \propto \text{似然函数} \times \text{先验概率} \quad (2.23)$$

公式(2.22)是贝叶斯定理用于机器学习的基础。当它用于求解前述的线性回归问题时，有多种不同的用法。

首先，直接应用公式(2.22)需要先验概率 $p(\mathbf{w})$ 的信息，这依赖于某些先验认识，并不存在于现成数据 $\mathcal{D}$ 中，一般不太容易获得（这是贝叶斯公式不容易理解的原因之一）。因此，一种近似的处理方法是将 $p(\mathbf{w})$ 丢弃（其实是假设它是个常数）。这样 $\mathbf{w}$ 的概率分布 $p(\mathbf{w}|\mathcal{D})$ 就等于似然函数 $p(\mathcal{D}|\mathbf{w})$ （除了一个无关紧要的归一化系数以外），也即前面公式(2.9)所给出的 $p(\mathbf{t}|\mathbf{x}, \mathbf{w}, \beta)$ 。如果我们取最可能的（概率最大的） $\mathbf{w}$ 作为我们问题的解，那其实就是前面2.3节的概率最大化方法。它是使似然函数最大，因此被称为最大似然函数法，或简称最大似然法。

其次，我们也可以对先验概率做出某些假设。例如，我们猜测 $\mathbf{w}$ 越大越不可能，因此假设它们具有某种高斯分布：

$$p(\mathbf{w}|\alpha) = \mathcal{N}(\mathbf{w}|\mathbf{0}, \alpha^{-1}\mathbf{I}) = \left(\frac{\alpha}{2\pi}\right)^{(M+1)/2} \exp\left\{-\frac{\alpha}{2}\mathbf{w}^T\mathbf{w}\right\} \quad (2.24)$$

其中是 α 是模型参数 $\mathbf{w}$ 的分布的参数，即“参数的参数”，被称为超参数。上标“T”表示对矢量或矩阵进行转置(transposition)操作，即行-列互换($(A^T)_{ij} = A_{ji}$)。结合似然函数的公式(2.9)，可得到后验概率：

$$p(\mathbf{w}|\mathbf{x}, \mathbf{t}, \alpha, \beta) \propto \left(\frac{\alpha}{2\pi}\right)^{(M+1)/2} \exp\left\{-\frac{\alpha}{2}\mathbf{w}^T\mathbf{w}\right\} \prod_{n=1}^N \mathcal{N}(t_n|y(x_n, \mathbf{w}), \beta^{-1}) \quad (2.25)$$

^g “似然”是对 likelihood 的贴近文言文的翻译，用现代的中文来说就是“可能性”：likely 可能的 + hood 性质 → 可能性。

这给出了参数 $\mathbf{w}$ 的更准确的概率分布。如果我们还是取概率最大的 $\mathbf{w}$ 作为我们问题的解，对其（先取对数以方便运算）求最大化， $\mathbf{w}$ 的解由下式的最小化给出：

$$\beta \tilde{E}(\mathbf{w}) = \frac{\beta}{2} \sum_{n=1}^N [y(x_n; \mathbf{w}) - t_n]^2 + \frac{\alpha}{2} \mathbf{w}^T \mathbf{w} \quad (2.26)$$

这种方法被称为最大后验概率法。仔细比较上式与前面正则化最小二乘法的公式（2.5），可以发现这里的 $\tilde{E}(\mathbf{w})$ 就是公式（2.5）的结果，参数间满足关系 $\lambda = \alpha/\beta$ 。因此，它们的求解结果是相同的，或者说，最大后验概率法等价于正则化的最小二乘法，为后者提供了理论基础。

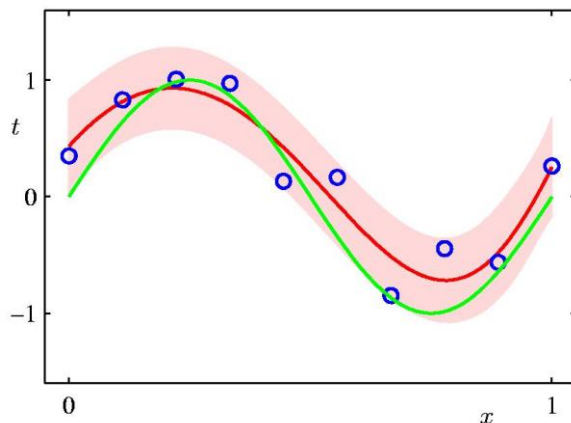


图 2.9. 贝叶斯曲线拟合结果。红色曲线代表预测结果 $t(x)$ 的平均值，阴影代表其不确定度（标准偏差）。

最后，在假设先验概率服从高斯分布公式（2.24）的条件下，也可以不要象最大后验概率法那样解出 $\mathbf{w}$ 的一组值，而是把 $\mathbf{w}$ 看做是在 $\mathcal{D}$ 下具有一定的概率分布（2.25）的随机变量。这样，由于 $\mathbf{w}$ 是随机变量，任一 x 下的函数 $y(x; \mathbf{w})$ 也是随机变量，可推导其概率分布，并进而利用公式（2.6）给出任一 x 的预测值 t 的分布（具体推导略，感兴趣的读者可参考 Bishop 的教科书）：

$$p(t|x, \mathbf{x}, \mathbf{t}) = \int p(t|x, \mathbf{w}) p(\mathbf{w}|\mathbf{x}, \mathbf{t}) d\mathbf{w} = \mathcal{N}(t|m(x), s^2(x)) \quad (2.27)$$

其中

$$m(x) = \beta \phi(x)^T \mathbf{S} \sum_{n=1}^N \phi(x_n) t_n \quad (2.28)$$

$$s^2(x) = \beta^{-1} + \phi(x)^T \mathbf{S} \phi(x) \quad (2.29)$$

$$\mathbf{S}^{-1} = \alpha \mathbf{I} + \beta \sum_{n=1}^N \phi(x_n) \phi(x_n)^T \quad (2.30)$$

$$\phi(x) = \begin{pmatrix} 1 \\ x \\ x^2 \\ \dots \\ x^M \end{pmatrix} \quad (2.31)$$

这种方法被称为贝叶斯曲线拟合。注意虽然 $\mathbf{w}$ 是随机的，服从简单的先验概率（高斯分布（2.24）），但由于数据 $\mathcal{D}$ 的出现，把 $\mathcal{D}$ 作为证据，导致了对 $\mathbf{w}$ 的信念的改变，因此能实现拟合的效果。对正弦曲线数据集的拟合结果如图 2.9 所示。此时预测值 $t(x)$ 本身是个随机变量，满足公式（2.27）所示的高斯分布，不但可以给出其平均值（图中红色曲线），还可以给出其置信范围（图中阴影）。注意置信范围（分布宽度）不但有噪声 ϵ 的贡献（公式（2.29）中的 β^{-1} ），也有 $\mathbf{w}$ 本身不确定的贡献（公式（2.29）中的 $\phi(x)^T \mathbf{S} \phi(x)$ ）。

3.3* 先验概率在物化中的例子：NPT 系综

我们一般对先验概率的概念不熟。因此这里来看一个物理化学中的例子。

在统计热力学中，我们比较熟悉的是体积 V 固定的 NVT 系综，此时体系状态满足玻尔兹曼分布：

$$p(\mathbf{p}^N, \mathbf{r}^N | V) = \frac{1}{Q} e^{-\frac{E(\mathbf{p}^N, \mathbf{r}^N)}{k_B T}} \quad (2.32)$$

其中 $\mathbf{r}^N$ 代表 N 个粒子的位置矢量， $\mathbf{p}^N$ 是它们的动量矢量。 $k_B \approx 1.38 \times 10^{-23}$ J/K 是玻尔兹曼常数， T 是热力学温度。 Q 是归一化系数。在这个式子中，体积 V 是个固定的参数，因此 $p(\mathbf{p}^N, \mathbf{r}^N | V)$ 是条件概率的含义。

如果体系不是体积固定，而是压强 P 固定，则体积 V 会涨落。例如，可想象如下情形：气缸中气体的体积由无摩擦活塞的位置所决定，活塞上施加恒外压，则有限体系时气体分子撞击活塞，导致活塞位置涨落。此时体系性质可用 NPT 系综描述。体积涨落，结果可描述成某种分布 $p(V)$ 。而对于其中具有某个指定 V 值的所有状态，很显然满足的就是体积固定的分布（玻尔兹曼分布，公式（2.32））。因此， NPT 系综的分布可写成

$$p(\mathbf{p}^N, \mathbf{r}^N, V) = p(\mathbf{p}^N, \mathbf{r}^N | V) p(V) \quad (2.33)$$

统计热力学中能够给出 NpT 系综分布的具体结果：

$$p(\mathbf{p}^N, \mathbf{r}^N, V) = \frac{1}{\Delta} e^{-\frac{E(\mathbf{p}^N, \mathbf{r}^N) + PV}{k_B T}} \quad (2.34)$$

联合（2.32-2.34），原则上可求出恒压体系的先验概率 $p(V)$ 。对于理想气体，最后的结果比较简单：

$$p(V) = \int p(\mathbf{p}^N, \mathbf{r}^N, V) d\mathbf{p}^N d\mathbf{r}^N \propto V^N e^{-\frac{PV}{k_B T}} \quad (2.35)$$

这就是在根据数据所遵循的微观机制严格推导得到的先验概率的表达式。可以看出它不一定是高斯函数（不过，在 N 很大的情况下它可以很好地近似成高斯分布）。

利用贝叶斯公式，有（动量的行为比较简单，此处略去）

$$p(V|\mathbf{r}^N) \propto p(\mathbf{r}^N|V)p(V) \quad (2.36)$$

假设有个很灵敏的单分子实验能观察到 $\mathbf{r}^N$ 但观察不到 V ，那利用上式可以基于 $\mathbf{r}^N$ 推断 V 。

第 4 节 更普适的线性回归模型：基函数的使用

4.1 带基函数的线性模型

带基函数的线性模型是多项式拟合想法的简单推广。它假设拟合函数可以写成一些固定基函数的线性组合：

$$\begin{aligned} f(\mathbf{x}; \mathbf{w}) &= w_0\phi_0(x_1, x_2, \dots, x_D) + w_1\phi_1(x_1, x_2, \dots, x_D) + w_2\phi_2(x_1, x_2, \dots) + \dots \\ &= \sum_{j=0}^M w_j\phi_j(\mathbf{x}) = \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}) \end{aligned} \quad (2.37)$$

其中的 $\phi_j(\mathbf{x})$ 被称为基函数，通常取 $\phi_0(\mathbf{x}) = 1$ 。基函数 $\phi_j(\mathbf{x})$ 本身是固定的，不包含任意未知参数。位置参数 $\mathbf{w}$ 是以线性的形式出现在函数 $f(\mathbf{x}; \mathbf{w})$ 中，或者说， $f(\mathbf{x}; \mathbf{w})$ 是 $\mathbf{w}$ 的线性函数。因此，公式（2.37）所示的模型被称为线性模型。它也可以看做是先从 $\mathbf{x}$ 做变换到 $\boldsymbol{\phi}$ （特征），然后以特征为自变量再做通常的线性拟合。多项式拟合也是线性模型，理论分析上相对简单。

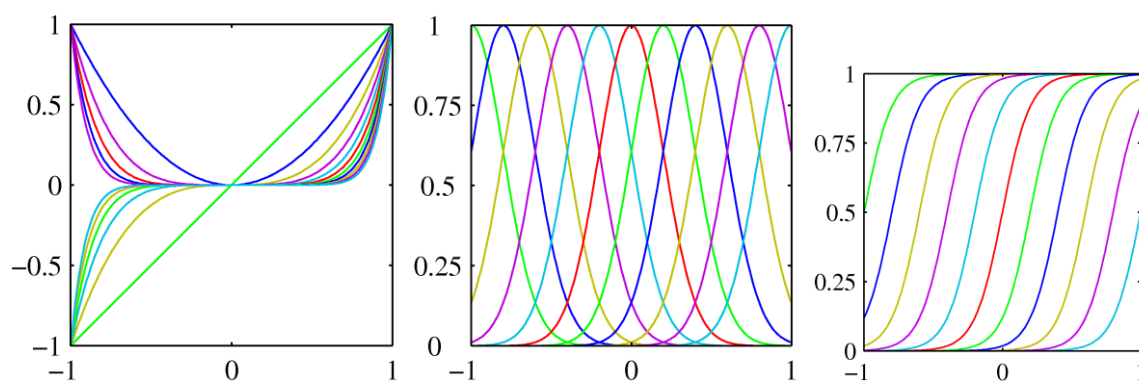


图 2.10. 不同类型的基函数：(a) 多项式基函数；(b) 高斯基函数；(c) 逻辑函数。

基函数可以根据需要灵活选取（一些例子如图 2.10 所示）。在前面的例子里，我们使用的其实是多项式基函数：

$$\mathbf{x} = x, \quad \phi_j(\mathbf{x}) = x^j \quad (2.38)$$

它们具有全局性（global）， x 的微小变化影响会所有基函数[图 2.10（a）]。另一种常用的基函数是高斯基函数，即

$$\phi_j(x) = \exp\left[-\frac{(x-\mu_j)^2}{2s^2}\right] \quad (2.39)$$

这里 μ_j 和 s 控制峰状曲线的位置和宽度（标度）。注意 μ_j 和 s 选定以后在拟合过程中是固定的，并不随参数 $\mathbf{w}$ 或数据 $\mathcal{D}$ 而变化（否则就不属于线性模型了）。高斯基函数具有局域性（local）， x 的微小变化只影响附近的基函数。最后介绍的一种基函数是 S 形曲线（sigmoidal function）：

$$\phi_j(x) = \sigma\left(\frac{x-\mu_j}{s}\right) \quad (2.40)$$

其中

$$\sigma(a) = \frac{1}{1+\exp(-a)} \quad (2.41)$$

也被称为称逻辑（logistic）函数，除了用于线性模型的基函数外，也经常被用作逻辑回归与神经网络的激活函数（后面章节会介绍）。逻辑函数在 x 很小趋向于 0，在 x 很大趋向于 1，而在 μ_j 附近则从约等于 0 光滑地上升到约等于 1，呈 S 形状，变化区域的宽度约为 s 量级。左右互换一下（ $x \leftrightarrow -x$ ），它就与统计热力学中的费米-狄拉克分布（在数学上）等价。

4.2 线性基函数模型的求解

在上一节中介绍过的多项式拟合的求解方法，只需要适当推广后就可应用于线性基函数模型的求解。

假设观察值 t 可写成公式(2.37)所示的确定性函数 $f(\mathbf{x}; \mathbf{w})$ 与随机噪声 ϵ 之和：

$$t(\mathbf{x}) = f(\mathbf{x}; \mathbf{w}) + \epsilon \quad (2.42)$$

噪声 ϵ 满足高斯分布 $p(\epsilon|\beta) = \mathcal{N}(\epsilon|0, \beta^{-1})$ 。因此， t 的分布为

$$p(t|\mathbf{x}, \mathbf{w}, \beta) = \mathcal{N}(t|f(\mathbf{x}; \mathbf{w}), \beta^{-1}) \quad (2.43)$$

如果已知一组数据 $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$ 与 $\mathbf{t} = [t_1, t_2, \dots, t_N]^T$ ， $\mathbf{X}$ 是个矩阵而 $\mathbf{t}$ 是个矢量，则在这些数据下的似然函数为

$$p(\mathbf{t}|\mathbf{X}, \mathbf{w}, \beta) = \prod_{n=1}^N \mathcal{N}(t_n|\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n), \beta^{-1}) \quad (2.44)$$

为了方便分析，取其对数值：

$$\ln p(\mathbf{t}|\mathbf{X}, \mathbf{w}, \beta) = \frac{N}{2} \ln \beta - \frac{N}{2} \ln(2\pi) - \beta E_D(\mathbf{w}) \quad (2.45)$$

其中 $E_D(\mathbf{w})$ 是平方和误差函数（二乘误差）：

$$E_D(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N [t_n - \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n)]^2 \quad (2.46)$$

4.2.1 最大似然函数法

我们可以通过最大化似然函数（即最小化误差函数）来求解参数 $\mathbf{w}$ 与 β 。此时，似然函数的对数公式（2.45）对 $\mathbf{w}$ 求偏导数等于零，即

$$\frac{\partial}{\partial \mathbf{w}} \ln p(\mathbf{t}|\mathbf{X}, \mathbf{w}, \beta) \equiv \nabla_{\mathbf{w}} \ln p(\mathbf{t}|\mathbf{X}, \mathbf{w}, \beta) = \beta \sum_{n=1}^N \{t_n - \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n)\} \boldsymbol{\phi}(\mathbf{x}_n)^T = 0 \quad (2.47)$$

其中记梯度 $\nabla_{\mathbf{w}} f \equiv \begin{pmatrix} \frac{\partial f}{\partial w_0} \\ \frac{\partial f}{\partial w_1} \\ \dots \end{pmatrix}$ 。上式是一个关于 $\mathbf{w}$ 的线性方程组，在线性代数里有非

常成熟的解法，很多编程语言的函数库也提供高效的数值求解函数可以直接调用。线性代数求解结果（过程略）为

$$\mathbf{w}_{ML} = (\boldsymbol{\Phi}^T \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^T \mathbf{t} \quad (2.48)$$

这被称为最小二乘法问题的正规方程（normal equations）。其中 $\boldsymbol{\Phi}$ 是个矩阵，被称为设计矩阵（design matrix），其矩阵元为

$$\Phi_{nj} = \phi_j(\mathbf{x}_n) \quad (2.49)$$

公式（2.49）中的 $(\boldsymbol{\Phi}^T \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^T$ 起到类似于 $\boldsymbol{\Phi}$ 的逆矩阵的作用（注意 $\boldsymbol{\Phi}$ 并不一定是方阵，也不一定行列式非零），有时候被称为 $\boldsymbol{\Phi}$ 的 Moore-Penrose 伪逆（pseudo-inverse），并记为 $\boldsymbol{\Phi}^\dagger$ 。进一步地，还可以由 $\frac{\partial}{\partial \beta} \ln p(\mathbf{t}|\mathbf{X}, \mathbf{w}, \beta) = 0$ 求出噪声的大小：

$$\frac{1}{\beta_{ML}} = \sigma^2 = \frac{1}{N} \sum_{n=1}^N \{t_n - \mathbf{w}_{ML}^T \boldsymbol{\phi}(\mathbf{x}_n)\}^2 \quad (2.50)$$

当训练集中数据太多时，利用正规方程（2.48）的求解过程可能会变得比较耗时^h。此时也可以使用其它一些替代方法，例如循序学习（Sequential Learning）。它的主要想法是基于误差函数公式（2.46）应用梯度下降法来迭代调整 $\mathbf{w}$ 使误差不断下降，而且在迭代过程中不需使用所有数据点来计算梯度，而是每次只使用

^h 正规方程需要计算矩阵乘积及求逆。因为矩阵逆的计算时间复杂度为 M^3 ，因此如果特征数量 M 较大则运算代价大。通常来说当 M 小于 1000 时还是可以接受的。它的好处是不需要选择学习率，不需要迭代。而梯度下降法则需要选择学习率，需要多次迭代，但好处是当特征数量很大时也能较好适用，而且适用于各种类型的模型（包括非线性模型，如神经网络）。

一个数据：

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla_{\mathbf{w}} E_n = \mathbf{w}^{(\tau)} - \eta [t_n - \mathbf{w}^{(\tau)T} \boldsymbol{\phi}(\mathbf{x}_n)] \boldsymbol{\phi}(\mathbf{x}_n) \quad (2.51)$$

其中 η 被称为学习率（learning rate），一般取很小的值。 E_n 是第 n 个数据对误差函数的贡献， $\mathbf{w}^{(\tau)}$ 是第 τ 次迭代时 $\mathbf{w}$ 的值。这种方法也称为在线学习，在神经网络与深度学习中则被称为随机梯度下降。

4.2.2 最大后验概率法：正则化下的最小二乘法

贝叶斯定理框架下的最大后验概率法，就等价于正则化下的最小二乘法。在二乘误差公式（2.46）基础上加上与 $\mathbf{w}^T \mathbf{w}$ 成正比的正则化项，得到

$$\tilde{E}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N [t_n - \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n)]^2 + \frac{\lambda}{2} \mathbf{w}^T \mathbf{w} \quad (2.52)$$

利用线性代数的知识，对上式最小化可解出

$$\mathbf{w}_{\text{ML}} = (\lambda \mathbf{I} + \boldsymbol{\Phi}^T \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^T \mathbf{t} \quad (2.53)$$

这种方法也被称为岭回归（ridge regression）。其中“岭”一词的来源，是式中正则化贡献 $\lambda \mathbf{I}$ 的可视化形象是在0构成的平面上有一条高度为 λ 的“岭”贯穿整个对角线。

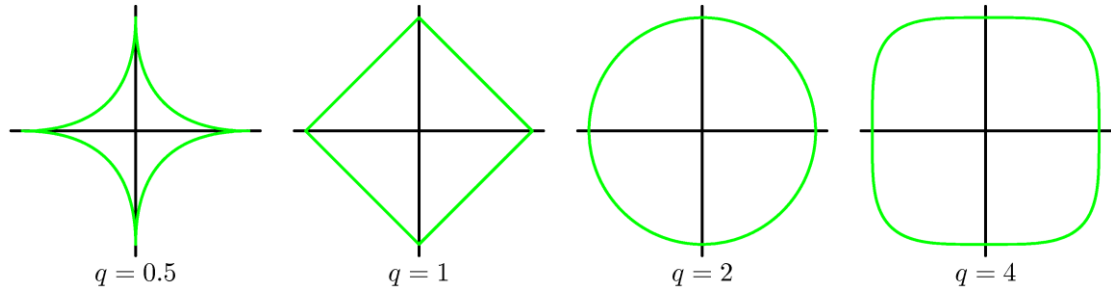


图 2.11. 不同的正则化方案的效果：不同 q 值下 $\sum_{j=0}^M |w_j|^q$ 的等高线。

岭回归中所使用的正则化惩罚项与 $\mathbf{w}^T \mathbf{w}$ 成正比，正如前面的分析所表明的，这其实是假设 $\mathbf{w}$ 的先验概率可用高斯分布公式（2.24）描述。因此，如果 $\mathbf{w}$ 服从的是其它形式的先验概率分布，那么表现出来的正则化项就不一定是 $\mathbf{w}^T \mathbf{w}$ 形式。当然，也可以直接假设不同形式的正则化项，例如，

$$\tilde{E}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N [t_n - \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n)]^2 + \frac{\lambda}{2} \sum_{j=0}^M |w_j|^q \quad (2.54)$$

使用 $\mathbf{w}$ 分量绝对值的 q 次方的和作为惩罚项。 $q = 2$ 对应的就是以前的 $\mathbf{w}^T \mathbf{w}$ 形式，也称为L2正则化项。不同 q 值对参数的限制效果是不同的（图2.11）。 $q = 1$ 时也

称为 L1 正则化项，对应的先验概率是拉普拉斯分布 $p(\mathbf{w}|\alpha) \propto \prod \exp(-\alpha|w_j|)$ ，最后所得到的 L1 正则化下的线性回归方法被称为 Lasso 回归。与岭回归相比，Lasso 回归经常能使一些 $\mathbf{w}$ 参数缩减为 0，有利于减少特征数目（图 2.12），达到简化模型的目的，但求解更复杂。 $q = 0$ 则是另一种重要的方法，被称为压缩感知，陶哲轩在这个主题上有重要贡献。

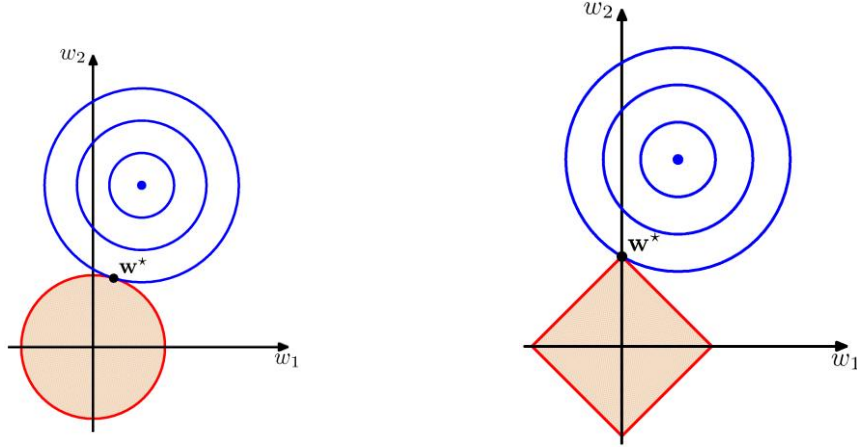


图 2.12. 岭回归与 Lasso 回归的正则化效果比较示意图。蓝色圆圈是二乘误差 $E_D(\mathbf{w})$ 的等高线。红色曲线则是正则化项 $E_w(\mathbf{w}) = \frac{\lambda}{2} \sum_{j=0}^M |w_j|^q$ 的等高线。交点 $\mathbf{w}^*$ 是对 $\tilde{E}(\mathbf{w})$ 求最小化的结果。

4.2.3 贝叶斯线性回归

从比较彻底的贝叶斯学派的观点来看，我们应该使用 $\mathbf{w}$ 的后验概率分布而非 $\mathbf{w}$ 的单一最佳点估计值（最大似然函数法或最大后验概率法的做法）来构建线性回归。假设 $\mathbf{w}$ 的先验概率具有某种高斯分布：

$$p(\mathbf{w}) = \mathcal{N}(\mathbf{w}|\mathbf{m}_0, \mathbf{S}_0) \quad (2.55)$$

其中 $\mathbf{m}_0$ 是均值（矢量）， $\mathbf{S}_0$ 是方差矩阵。下标“0”表示这是在已知 0 个数据（即没有数据）下 $\mathbf{w}$ 的分布参数。在噪声的影响下，某一组 $\mathbf{w}$ 值下得出观察数据的似然函数由公式（2.44）描述。应用贝叶斯公式，可得后验概率满足如下的高斯分布（过程略）

$$p(\mathbf{w}|\mathbf{t}) = \mathcal{N}(\mathbf{w}|\mathbf{m}_N, \mathbf{S}_N) \quad (2.56)$$

其中

$$\begin{cases} \mathbf{m}_N = \mathbf{S}_N(\mathbf{S}_0^{-1}\mathbf{m}_0 + \beta\Phi^T\mathbf{t}) \\ \mathbf{S}_N^{-1} = \mathbf{S}_0^{-1} + \beta\Phi^T\Phi \end{cases} \quad (2.57)$$

在很多应用中，会进一步简化假设 $\mathbf{w}$ 的分量间是独立的，各自满足期望值为 0，

方差为 α^{-1} 的高斯分布，即前面的公式 (2.24)。此时，结果可以简化成：

$$\begin{cases} \mathbf{m}_N = \beta \mathbf{S}_N \Phi^T \mathbf{t} \\ \mathbf{S}_N^{-1} = \alpha \mathbf{I} + \beta \Phi^T \Phi \end{cases} \quad (2.58)$$

贝叶斯线性回归所给出的任一 x 的预测值 t ，并不是单个值，而是来自于满足后验概率分布的各种 $\mathbf{w}$ 的贡献。对 $\mathbf{w}$ 积分，可以得到预测 t 的概率：

$$p(t|\mathbf{t}, \alpha, \beta) = \int p(t|\mathbf{w}, \beta) p(\mathbf{w}|\mathbf{t}, \alpha, \beta) d\mathbf{w} = \mathcal{N}(t | \mathbf{m}_N^T \Phi(\mathbf{x}), \sigma_N^2(\mathbf{x})) \quad (2.59)$$

其中

$$\sigma_N^2(\mathbf{x}) = \frac{1}{\beta} + \Phi(\mathbf{x})^T \mathbf{S}_N \Phi(\mathbf{x}) \quad (2.60)$$

既来自于噪声，也来自于 $\mathbf{w}$ 的不确定性。贝叶斯线性回归有助于避免过拟合的出现。

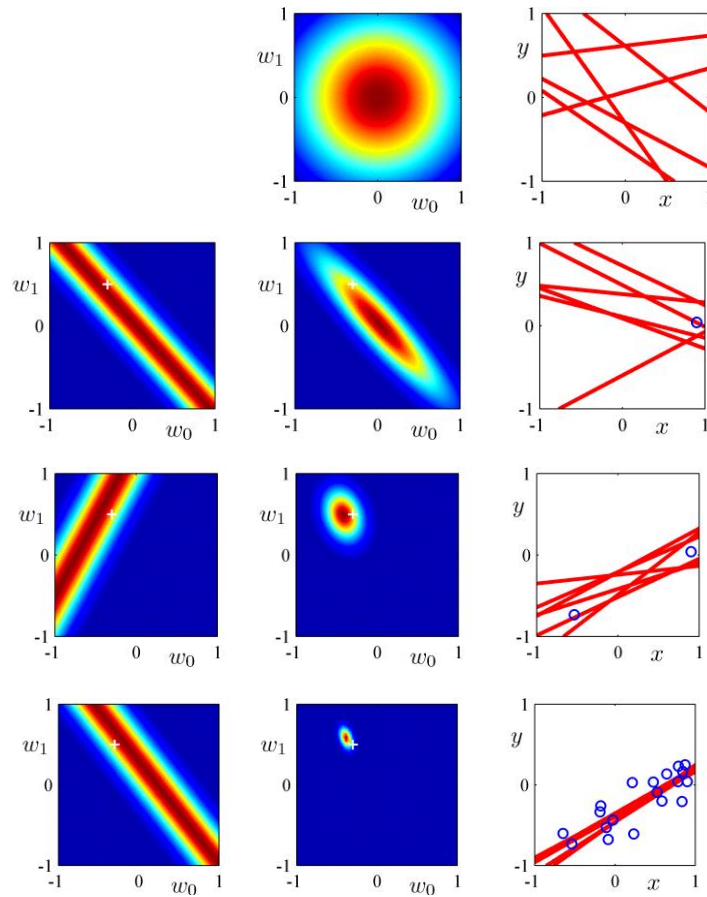


图 2.13. 贝叶斯线性回归中数据点对参数 $\mathbf{w}$ 分布的影响。(左) 新增点带来的似然函数；(中) 后验概率分布；(右) 数据空间（已知数据点用蓝色小圆圈代表，根据后验概率抽样得到参数 $\mathbf{w}$ 所画的拟合函数用红色直线表示）。(从上到下) 数据点个数分别为 0, 1, 2, 20。拟合函数为 $y = w_0 + w_1 x$ 。真实数据是一条直线，其 $\mathbf{w}$ 取值在作图与中图中用十

字符标记。

图 2.13 中提供了贝叶斯线性回归的具体例子，可以观察 $\mathbf{w}$ 的分布。这里假设真实数据是一条直线，其 (w_0, w_1) 取值在图 2.13 的左列与中列中用十字符号标出。最上一行是还没有任何观察数据时的拟合结果。此时 $\mathbf{w}$ 的分布完全依赖于先验知识，即后验概率完全等同于先验概率（最上一行中间列）。随着观察数据的加入， $\mathbf{w}$ 的分布（中间一列）越来越窄，而且很接近真实结果。所抽样得到的 $\mathbf{w}$ 所画的曲线（直线）也越来越聚在一起（右列）。

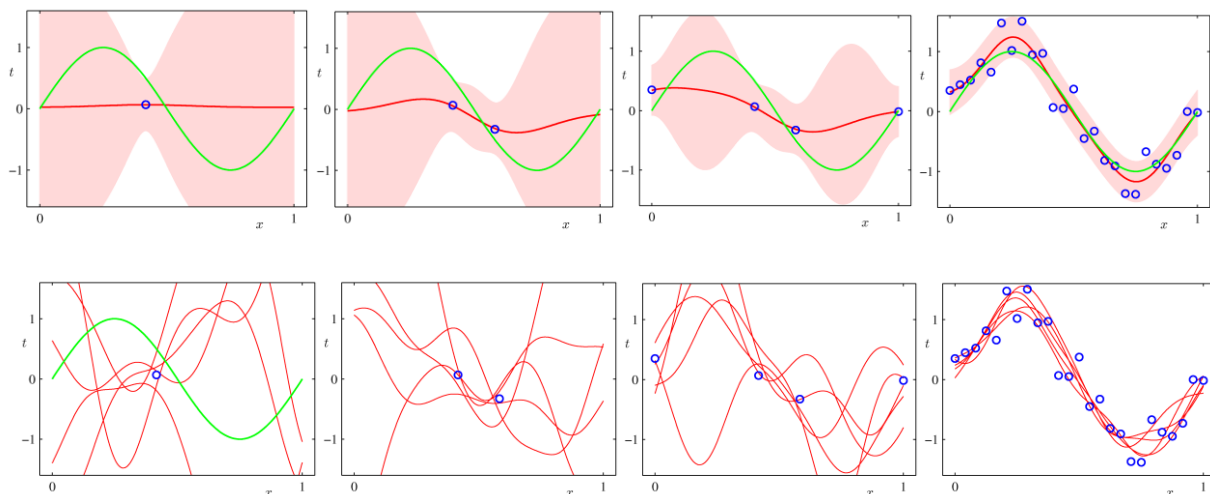


图 2.14. 贝叶斯线性回归中数据点对预测值分布的影响。（从左到右）数据点个数分别为 1, 2, 4, 25。真实曲线 $\sin x$ 用绿色曲线表示。红色曲线上一行代表预测 $t(x)$ 的平均值；在下一行中则代表用几组抽样得到参数 $\mathbf{w}$ 所画的拟合函数。拟合函数用了用了 9 个高斯基函数。

图 2.14 则给出了更复杂的模型（9 个高斯基函数）在拟合真实曲线 $\sin x$ 时随数据点的变化。当只有一个数据点时，多达 9 个基函数也能给出拟合结果（最左边一列），此时在已知数据点附近预测值的分布宽度小一些。而随着数据点的增多，各处的预测值分布宽度也逐渐变小，总的效果就是在数据点少的地方分布宽度就会比较大，即预测比较没有把握。但数据很多（25 个，最右边一列），拟合曲线的分布已经很一致了，各处的预测值的分布宽度都很窄。

第 5 节 线性回归方法的应用例子

线性回归在机器学习中算是很简单的方法。不过，如果选择的问题比较合适，线性回归方法也可以取得很好的效果。下面介绍文献中的两个例子。

5.1 高效减水剂的设计

第一个例子是属于胶体/高分子化学领域的：

A. Menon, C. Gupta, K. M. Perkins, B. L. DeCost, N. Budwal, R. T. Rios, K. Zhang,

B. Poczos, and N. R. Washburn.

Elucidating multi-physics interactions in suspensions for the design of polymeric dispersants: a hierarchical machine learning approach.

Mol. Syst. Des. Eng. **2**, 263-273 (2017).

悬浮液是固体分散相与液体的不均匀混合物，生活中的很多材料，如油漆、涂料、化妆品等，都属于悬浮液。对于悬浮液，有个特性很重要，那就是黏度。比如，黏度是影响酸奶口感的一个重要因素；黏度也会影响油漆性能，比如是否容易抹匀，是否容易附着在墙上而不是流淌下来。第一个例子中研究的主题则是高效减水剂对水泥黏度的影响。

混凝土（Concrete）是指由胶凝材料（水泥）将集料（砂、石）胶结成整体的工程复合材料。混凝土在施工过程中必须具有一定的流动性（通过水分含量来控制），以便于在管道内顺畅传输，同时也能够保证施工时的均匀性和密实度。但施工后又需要让水分蒸发、快速凝固以便进行下一工序。因此，如果能在减少水分含量的同时保持混凝土的黏度，那就非常有利了。高效减水剂就是一种很有用的高分子添加剂，对水泥有强烈分散作用，能大大提高水泥拌合物流动性和混凝土坍落度，同时大幅度降低用水量，显著改善混凝土工作性。第一代高效减水剂主要是萘基和密胺树脂基的高分子，又被称为超塑化剂。第二代高效减水剂是氨基磺酸盐。第三代高效减水剂须臾聚羧酸系，其中既有磺酸基又有羧酸基的接枝共聚物是最重要的代表。

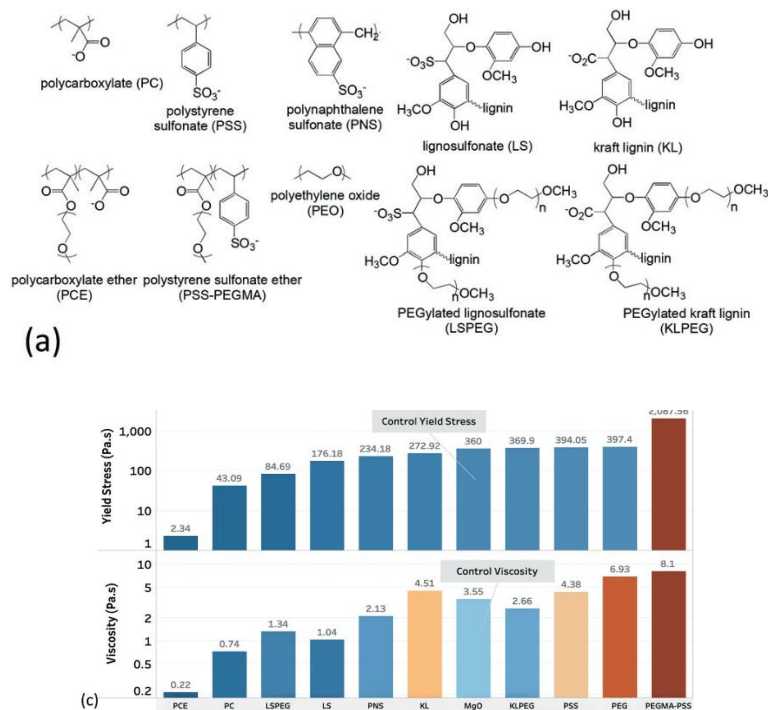


图 2.15. 实验体系及其性能测量结果。其中聚羧酸乙醚（PCE）是市场上的商用高效减水剂。图片来自文献???

例子工作的目的是利用机器学习方法设计新的高效减水剂，以规避现有商用高效减水剂的专利限制。机器学习需要（训练）数据。例子作者通过实验制备了图 2.15 中的 10 种不同化学组分的高分子，并测量了它们在水泥混凝土中的减水性能，主要是两个指标（这也是机器学习问题中的输出 t ）：slurry yield stress（泥浆屈服应力） τ_y 与 slurry viscosity（泥浆黏度） η_s 。市场上性能最好的商用高效减水剂是图 2.15 左下角所示的聚羧酸乙醚（PCE）。可以看出，PCE 体系的黏度在所有 10 个体系中是最小的。

输出有了，那输入呢？按道理，这里的机器学习问题是希望输入高分子的信息，然后机器学习模型输出泥浆屈服应力 τ_y 与泥浆黏度 η_s 的值。但是如何输入一个化学分子的信息，使计算机能够识别与有效利用，是很复杂的问题。比如，我们可以输入原子的类型以及原子间的键连信息，但这样输入 $\mathbf{x}$ 的分量数目就会很多，输入与输出之间的联系比较复杂，而且再考虑到可用的数据集很小（只有 10 个数据），就会很容易发生过拟合。这时候就用上了前面介绍过的特征工程了。作者挑选了高分子-MgO 溶液的 5 个比较重要的特征（表 2-1） $\theta(\mathbf{x}, c_0)$ ， η_{pol} ， π_{pol} ， ζ_{pol} ， s_{pol} （这些特征/所涉及的体系比泥浆体系的 τ_y 、 η_s 简单一些，在论文中被称为 single-physics interactions），并利用领域的专业模型知识把它们表示成一些更简单的量 $U(\mathbf{x})$ ， $\eta_{\text{pol}}(\mathbf{x})$ ， $A_2(\mathbf{x})$ ， $\zeta_{\text{pol}}(\mathbf{x})$ ， $\eta_{\text{pol}}(\mathbf{x})$ 的函数（参见表 2-1）。

表 2-1 水泥高效减水剂问题中的特征选择及其模型

特征变量	含义	专业模型
$\theta(\mathbf{x}, c_0)$	高分子在 MgO 颗粒表面的覆盖率。Coverage of polymer on particle surfaces	$\theta(\mathbf{x}, c_0) = \frac{c_0 e^{-U(\mathbf{x})/k_B T}}{1 + c_0 e^{-U(\mathbf{x})/k_B T}}$
η_{pol}	高分子-MgO 溶液的黏度。Viscosity of pore solution with dissolved polymer	$\eta_{\text{pol}} = c_0 [1 - \theta(\mathbf{x}, c_0)] \eta_{\text{pol}}(\mathbf{x})$
π_{pol}	高分子-MgO 溶液的渗透压。Osmotic pressure of pore solution with dissolved polymer	$\pi_{\text{pol}} = RT \left\{ \frac{c_0 [1 - \theta(\mathbf{x}, c_0)]}{M} + [c_0 [1 - \theta(\mathbf{x}, c_0)]]^2 A_2(\mathbf{x}) \right\}$
ζ_{pol}	高分子-MgO 溶液的 Zeta 电势。Zeta potential of MgO particle with adsorbed polymer	$\zeta_{\text{pol}} = c_0 \theta(\mathbf{x}, c_0) \zeta_{\text{pol}}(\mathbf{x})$
s_{pol}	高分子-MgO 溶液的沉降系数。Sedimentation coefficient of MgO particle with adsorbed polymer	$s_{\text{pol}} = \frac{c_0 \theta(\mathbf{x}, c_0) [e s_{\text{pol}}(\mathbf{x})]}{\eta_{\text{H}_2\text{O}} + c_0 [1 - \theta(\mathbf{x}, c_0)] \eta_{\text{pol}}(\mathbf{x})}$

这些关于高分子-MgO 溶液的性质 $\theta(\mathbf{x}, c_0)$, η_{pol} , π_{pol} , ζ_{pol} , s_{pol} 也通过实验测量确定了它们的值。接着, 假设 $U(\mathbf{x})$, $\eta_{\text{pol}}(\mathbf{x})$, $A_2(\mathbf{x})$, $\zeta_{\text{pol}}(\mathbf{x})$, $\eta_{\text{pol}}(\mathbf{x})$ 是高分子所含各种化学基团 (磺基、羧基、烷基、芳香基、PEG、PEO 等) 含量 (即原始输入 $\mathbf{x}$) 的线性函数, 就可以利用线性回归 (使用了简单的最小二乘法) 确定其线性系数。最后, 才是把 $\theta(\mathbf{x}, c_0)$, η_{pol} , π_{pol} , ζ_{pol} , s_{pol} 看做最后机器学习模型的特征, 假设混凝土中的减水性能 τ_y 与 η_s 是这些特征的多项式 (即使用多项式基, 最多考虑二阶), 利用 Lasso 回归 (这有助于减少非零系数的数目, 降低过拟合的风险) 得到最终的预测模型:

$$\Delta\tau_y = -0.26\zeta_{\text{pol}}^2 + 1.93\eta_{\text{pol}} + 0.13\pi_{\text{pol}}^2 - 1.00\eta_{\text{pol}}\pi_{\text{pol}} + 1.40\eta_{\text{pol}}s_{\text{pol}} + 0.03\pi_{\text{pol}}\zeta_{\text{pol}} \quad (2.61)$$

$$\Delta\eta_s = -0.02\zeta_{\text{pol}}s_{\text{pol}} + 0.16\pi_{\text{pol}}^2 + 0.39\eta_{\text{pol}} - 0.09\pi_{\text{pol}} + 0.84\pi_{\text{pol}}s_{\text{pol}} + 0.42\pi_{\text{pol}}\zeta_{\text{pol}} \quad (2.62)$$

有了这个机器学习模型, 就可以用于分子设计 (虚拟筛选): 对各种各样的可能高分子的性能进行预测, 并从中挑出性能较好的分子作为设计结果, 并进行实验合成与性质测量。作者设计了一个基于聚羧酸接枝木质素磺酸盐 (LSPC, 参见图 2.16) 的高分子。最后实验的结果表明 LSPC 的泥浆屈服应力 τ_y 小于 5 Pa.s, 性能几乎与商业 PCE 相同。

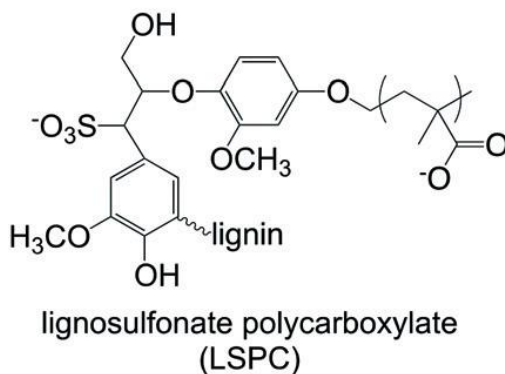


图 2.16. 所设计的 LSPC 高分子。

5.2 不对称催化的对映选择性预测

第二个例子来自有机化学领域:

J. P. Reid, M. S. Sigman.

Holistic prediction of enantioselectivity in asymmetric catalysis.

Nature **571**, 343-348 (2019).

当化学家们面对一个新反应时，通常会把文献中报道的相似反应的底物、催化剂、溶剂及添加剂和反应条件用于新反应。但是很多时候，底物或条件的一些微不足道的区别就会导致反应不能成功进行。于是，系统考察反应条件是大多数合成化学工作者都会遇到的情况，这种试错过程相当耗时耗资源。因此，如何把人们对已知反应的认识迁移到新反应中是一个很重要的问题。通常，有机化学家们通过几十年的训练培养出一种化学直觉来达到这个目的。

上述工作基于文献中的化学反应数据，利用机器学习方法来预测新的反应，以帮助化学家缩小需要探索的条件范围。具体地，是预测 1,1'-二-2-萘酚 (BINOL) 衍生的手性磷酸催化的对亚胺亲核加成反应的对映选择性。

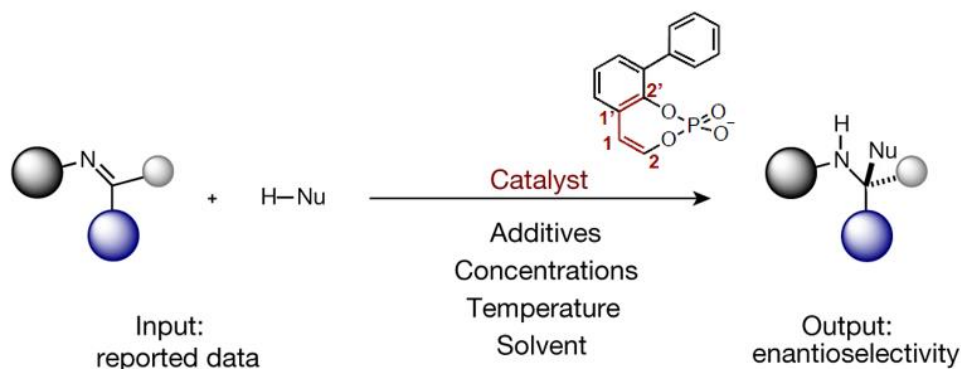


图 2.17. BINOL 衍生的手性磷酸催化的对亚胺亲核加成反应。

反应如图 2.17 所示，反应的产物具有手性（几何构象 E 与 Z）。实验产物中两种对映体含量之比（对映体比例 e.r.）可进一步折算成过渡态自由能差 $\Delta\Delta G^\ddagger = -RT\ln(\text{e.r.})$ ，作为反应对映选择性的衡量，也是机器学习的输出信号。而所使用的底物、亲核试剂、催化剂、溶剂等条件都会影响反应的产率与对映选择性，它们都是机器学习问题的输入。

论文作者收集了来自 17 篇文献的 381 条反应的条件和对映选择性 $\Delta\Delta G^\ddagger$ ，经过数据整理最终得到 367 条有效数据，其中 Z 型 156 条，E 型 211 条。这些反应涉及溶剂 12 种，催化剂 18 种，亲核试剂 54 种，亚胺 180 种。

为了给机器学习模型提供易于理解的特征，作者利用定量构效关系 (QSAR)、分子力学 (MM)、密度泛函理论 (DFT) 计算等途径得到的大量的描述分子的特征，例如键长、键角、分子振动频率和强度、原子的部分电荷与极化率、最高已占分子轨道 (HOMO) 和最低未占分子轨道 (LUMO) 能量，以及用于描述分子形状的拓扑学和连接性二维描述符。其他反应变量如浓度、分子筛也被包含在内。最后得到 133 个关于溶剂的特征，58 个关于催化剂的，14 个关于亲核试剂，以

及 21 个描述亚胺性质的特征。这些特征被作为机器学习模型的输入分量。

这么多输入分量，容易导致过拟合。为了一致过拟合以及找出影响对映选择性的关键因素，作者采用了前向逐步线性回归 (Forward stepwise linear regression) 的方法 (大致就是每次加上一个最能提高预测性能的特征)，并把数据按 50:50 的比例分给验证步骤与测试步骤 (验证步骤的内容将在后面第 7 章介绍)。

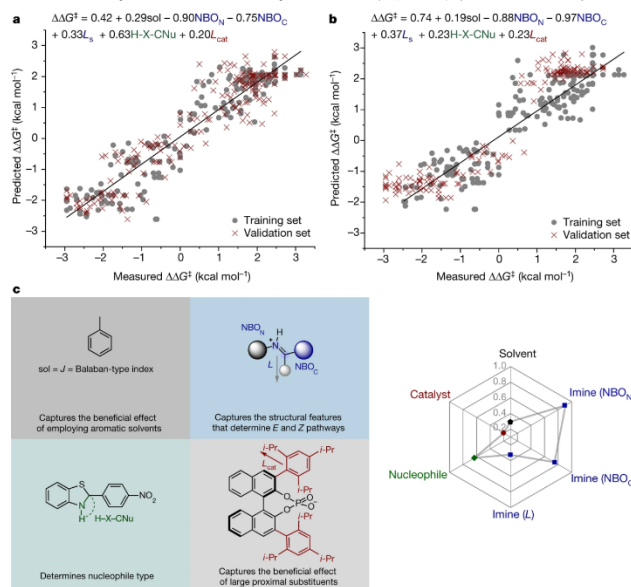


图 2.18. 对映选择性的学习结果。(a,b) 拟合的效果，其中的式子给出了最后的线性回归表达式。(c) 影响对映选择性的几个关键因素。

模型训练与分析的效果如图 2.18 所示。最后的模型只保留了 6 个影响最大的特征，与溶剂、亲核试剂与和催化剂有关的各有一个，与亚胺有关的有 3 个，其中影响最大的是亚胺的 N 和 C 原子的自然键轨道 (natural bond orbital, NBO) 参数。预测值与实验值符合很好，相关系数为 $R^2 \approx 0.87$ 。

最后，作者还将模型应用于不在训练数据集中的底物，以评估所得机理原则从一种反应到另一种反应的转移能力。共评估了亲核试剂未包含在训练集的 15 个反应，结果表明平均绝对 $\Delta\Delta G^\ddagger$ 误差为 0.37 kcal mol $^{-1}$ ，并且正确地预测了所有反应的优势对映体。

作业：

扩展阅读：

- <https://blog.csdn.net/fnqtyr45/article/details/81024328>
算法之美-贝叶斯法则：预测未来.pdf
- 算法之美-7 过渡拟合：不要想太多.pdf
- 线性回归的故事.pdf
- https://blog.csdn.net/laputa_ml/article/details/80072570
线性回归中“回归”的含义.mht
- https://www.sohu.com/a/147459515_46874
谈谈机器学习中的维度灾难.mht
- <https://www.visiondummy.com/2014/04/curse-dimensionality-affect-classification/>
The Curse of Dimensionality in classification.mht
- https://mp.weixin.qq.com/s/h9fX_vvyajsbKQMsJjWMEg
机器学习的本质：理解泛化的新观点 .mht
- <https://www.huxiu.com/article/308769.html>
AI 笑话大全.pdf
- https://www.sohu.com/a/419676894_105067
（过拟合？）绩点为王：中国顶尖高校年轻人的囚徒困境.mht
- <https://www.huxiu.com/article/392267.html>
天文学家连发 3 篇论文质疑，金星有生命迹象是场大乌龙？.pdf
- <https://www.jiqizhixin.com/articles/2019-05-06-4>
一文读懂机器学习中的贝叶斯统计学.mht
- <https://baijia.baidu.com/s?id=1598070443227371560>
Python 代码和贝叶斯理论告诉你，谁是最好的棒球选手.mht
- 刘志荣《统计热力学》内容摘录
概率论：贝叶斯学派的角度.pdf
- <https://finance.sina.com.cn/tech/2021-09-08/doc-iktzscyx2962545.shtml>
你吃过的每一枚鸡蛋，都能在这个数学公式里找到.mht
相当于是重要的先验知识。可思考其与先验概率的关系以及在监督学习中的应用

- <https://www.huxiu.com/article/445192.html>
[这可能是十年来最酷的神经科学发现.mht](#)
独特的感觉输入，加上大脑根据你个人经验做出的最好预测，这才创造出情绪。其实就是贝叶斯预测。
- <https://www.huxiu.com/article/398840.html>
[哪怕你清华北大毕业，也没法让你孩子保持精英.mht](#)
承认吧，你家孩子不如你优秀是大概率事件
- <https://36kr.com/p/1124806560681224>
[如何和这个“势利”的世界相处？.mht](#)
“势利”在数学上还可以用概率论中著名的贝叶斯公式来表达，即根据条件的变化修正最后概率的结果。
这个世界既残酷又温柔。
一起做金子，不做烂泥，共勉！
- https://baijia.baidu.com/s?id=1594514807888322634&wfr=pc&fr=ch_lst
[机器学习萌新必学的 Top10 算法.mht](#)
- <https://www.x-mol.com/news/18656>
[Nature：预测不对称催化的对映选择性？来开个“上帝视角”.mht](#)

参考资料与文献：

第三章 分类的线性方法：逻辑回归

刘志荣

在本章中，我们将介绍一种机器学习的分类方法——逻辑回归。它属于很简单的线性方法，但同时又非常强大，甚至被誉为“世界上使用最广泛的一种分类算法”ⁱ。

第1节 简单的思路：线性拟合与感知器

在第一章中，我们介绍过，机器学习的分类问题（classification）就是根据输入数据 $\mathbf{x}$ 预测离散化的输出值 t ，例如 $\mathbf{x}$ 所属的类别（class）：

$$t(\mathbf{x}) = \begin{cases} 1, & \text{属于第一类} \\ 0, & \text{属于第二类} \end{cases} \quad (3.1)$$

既然离散值也属于连续值的一部分，一个简单的思路是把这里的 t 看做连续值，直接应用前一章详细介绍过的线性回归方法，即利用（这里定义 $x_0 \equiv 1$ ）：

$$f(\mathbf{x}; \mathbf{w}) = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_D x_D = \mathbf{w}^T \mathbf{x} \quad (3.2)$$

但这样计算得到的 f 一般并不落在分类问题所需要的离散值 $\{0,1\}$ 范围内。因此，我们可以进一步利用下式将其变成符合要求的值：

$$\text{预测值} = \begin{cases} 0 & (\text{if } f(\mathbf{x}; \mathbf{w}) \leq 0.5) \\ 1 & (\text{if } f(\mathbf{x}; \mathbf{w}) > 0.5) \end{cases} \quad (3.3)$$

也就是说，我们利用公式（3.2）在训练集 $\{\mathbf{x}_n, t_n\}$ 上运用线性回归方法进行拟合以确定参数 $\mathbf{w}$ 的值，然后在需要做预测时进一步利用公式（3.3）进行预测。这样做的好处是可以直接套用线性回归的很多分析结果。这相当于是通过套壳的方法，将线性回归改造成一个分类方法。

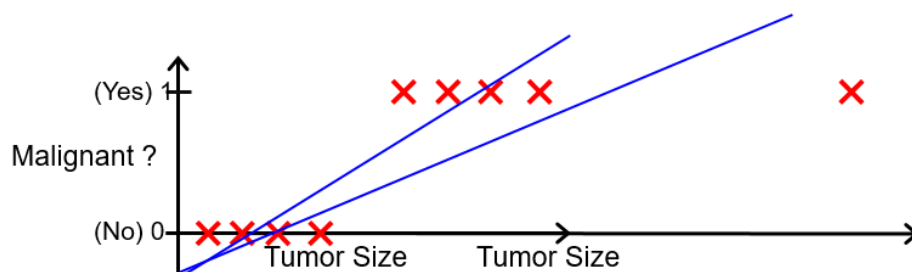


图 3.1. 线性回归用于简单的分类问题——肿瘤判断。最右边的星号表示新加入的数据点。紫色与蓝色分别代表星号数据点加入前后的模型训练结果，实线代表线性回归的结果，虚线代表预测时的临界 x

ⁱ 吴恩达（Andrew Ng）网络公开课《机器学习》。

值，即虚线左边会被预测为 0，右边预测为 1。

这个方法在肿瘤判断问题上的示意性结果如图 3.1 所示。在这个简单的训练集中，两类 ($t_n = \{0,1\}$) 数据之间具有明显的边界，在 $(x=??)$ 附近。对于两类数据的数目与 x_n 分布宽度都比较接近的情况（叉形数据），效果还是比较好的，可以把两类数据分开（图中紫色线）。但如果在此基础上再加上一个远离边界的数据（图中星号），效果反而变差。究其原因，是这种基于线性回归的方法对远离边界的点过于“正确”，硬是要把其 $f(\mathbf{x}; \mathbf{w})$ 拉到 1 或 0 附近（否则在线性回归中的二乘误差太大），导致预测的边界反而偏离正确位置。对于 $\mathbf{x}$ 是二维数据的情况，性质也是类似的（图 3.2），即本来应该比较容易判断的远离边界的数据点反而会导致结果变差。

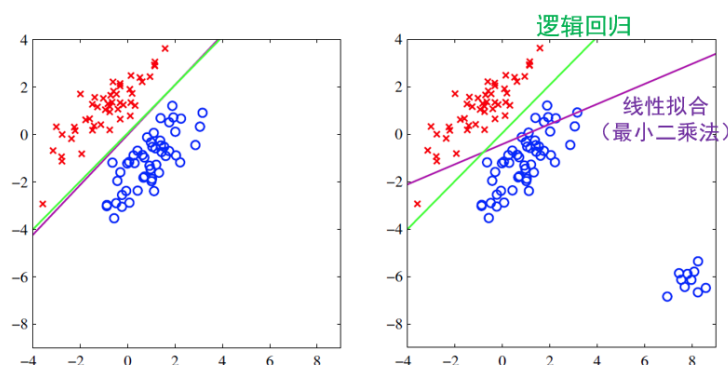


图 3.2. 线性回归用于二维 $\mathbf{x}$ 输入的分类问题。横轴与纵轴表示输入 $\mathbf{x}$ 的两个分量，而两种类别 ($t_n = \{0,1\}$) 的数据分别用圆圈与叉表示。紫色直线是线性回归的分类边界，而绿色是逻辑回归的结果。

将线性回归方法直接应用于分类问题时效果不好，一个重要原因是训练时使用了公式 (3.2)，其函数值可以一直增大或减少，而不是局限在训练集的取值范围 $t_n = \{0,1\}$ 。与此不同，公式 (3.3) 则能保证取值范围为 $\{0,1\}$ 。如果直接利用公式 (3.3) 来训练，就得到了机器学习另一个重要的分类模型——感知器 (perceptron) 模型，可重新表述成：

$$y(\mathbf{x}; \mathbf{w}) = f(\mathbf{w}^T \mathbf{x}) \quad (3.4)$$

其中 f 是阶跃函数^j：

$$f(z) = \begin{cases} 0 & (\text{if } z \leq 0) \\ 1 & (\text{if } z > 0) \end{cases} \quad (3.5)$$

直接用公式 (3.4) (3.5) 所示的 $y(\mathbf{x})$ 去拟合训练集数据 $\{\mathbf{x}_n, t_n\}$ ，就不会受到远离边界的点的不良影响（图 3.3）。感知器模型是 Frank Rosenblatt 在 1962 年提出来

^j 在有些文献中，激活函数 $f(z)$ 被写成 $f(a)$ 。我们这里采用前者做法以与后面的神经网络等内容保持一致。

的，在人工智能早期历史上具有重要地位。它甚至有专门的硬件来实现，可以用来识别数字与字母（图 3.3(b)）。例如，把一个数字图案放到摄像装置前面，利用探测器得到像素化的 $\mathbf{x}$ ，然后通过专门的硬件处理后就得到输出并以亮灯的形式展示给观众，给人一种“能够像人类一样识别数字与文字的神奇机器”的直观效果，很受新闻媒体欢迎。

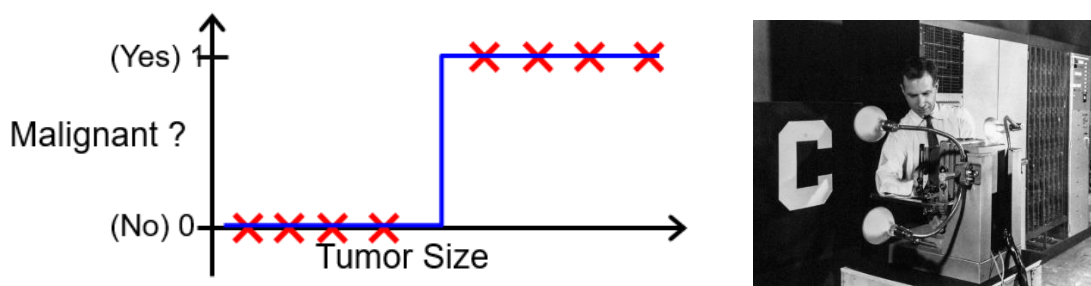


图 3.3. 感知器模型。(a) 感知器用于简单的分类问题。实线代表感知器模型的公式 (3-?)。(b) Rosenblatt 与感知器硬件实现装置。

感知器模型相当于是在线性回归的基础上加了个形式为阶跃函数的激活函数。阶跃函数的引入虽然解决了输出取值范围的问题，但它不容易计算导数（其导数在阶跃处会产生一个无穷高尖峰，在数值计算上不容易处理）以及进行模型的训练与优化求解。如果将阶跃函数替换成光滑的函数，就得到接下来介绍的重点内容——逻辑回归。

第 2 节 逻辑回归

2.1 逻辑回归模型

逻辑回归将感知器模型中的阶跃函数改造成光滑的函数，即

$$y(\mathbf{x}; \mathbf{w}) = f(\mathbf{w}^T \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x}) \quad (3.6)$$

其中所采用的激活函数（activation function）如下所示^k：

$$\sigma(z) = \frac{1}{1+e^{-z}} \quad (3.7)$$

被称为 S 型（sigmoid）函数或逻辑（logistic）函数。 $\sigma(z)$ 的性质如图 3.4 所示。它的函数值落在(0,1)区间，随着自变量 z 的增加会在 $z = 0$ 附近光滑地从接近 0 的值变成接近 1 的值。分类问题输出的合法取值范围为{0,1}，严格讲并不能取(0,1)区间的连续数值。因此，在逻辑回归中，进一步把 $\sigma(\mathbf{w}^T \mathbf{x})$ 解读为数据点 $\mathbf{x}$ 属于第一类（ $t = 1$ ）的概率^l，即模型（会学习的机器）有大小为 $\sigma(\mathbf{w}^T \mathbf{x})$ 的把握（信念）

^k 有些教科书里用 a 来表示激活函数的自变量，即 $\sigma(a)$ 。我们这里用 $\sigma(z)$ 以便与后面的神经网络模型中的写法一致。

^l 这有点类似于类似于贝叶斯彩票问题中的彩票池中奖率 x 。在其上还有 x 的概率问题。两者不要混淆。

认为 $\mathbf{x}$ 属于第一类。这是一种概率模型。逻辑回归的直观理解参见“阅读框：逻辑回归的直观图像”。逻辑函数在描述增长模型与游戏对战匹配方面也有应用(参见“题外：逻辑函数/模型的前世今生”“题外：国际象棋与电子游戏中的等级与对战匹配…”)。

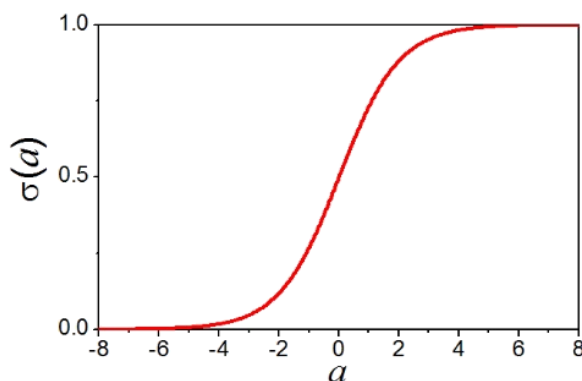


图 3.4. 逻辑函数曲线。

阅读框：逻辑回归的直观图像

逻辑回归模型 (3.6) 包含了两步：第一步是线性计分，即对输入 $\mathbf{x}$ 做线性变换 $z = \mathbf{w}^T \mathbf{x}$ ；第二步是对上一步的计分结果 z 做一个非线性激活（评价）变换 $y = \sigma(z)$ 。

我们可以用生活里的例子来理解这个过程。例如，某私立小学为了评估学生是否优秀，采取如下制度：（1）上课讲话每次扣 1 分，不交作业每次扣 2 分，比赛得奖每次加 8 分，也即总评分 $z = w_1 x_1 + w_2 x_2 + w_3 x_3$ ，其中 x_1, x_2, x_3 代表“上课讲话”、“不交作业”和“比赛得奖”的次数，而 $w_1 = -1, w_2 = -2, w_3 = 8$ 代表每种行为在“优秀”这一类别下的权重；（2）根据最终评分来判断每位同学的表现好坏。如果将某一阈值以上都视为优秀，则这是一种感知器。但这会导致分数恰好在阈值之下学生的不满，会有学生来申诉说他某次作业其实是被课代表漏收了，以期找回一分进入优秀的行列。如果采用逻辑回归的做法，则是将分数通过 $\sigma(\cdot)$ 函数变换到 0 与 1 之间，其含义是这个学生属于“优秀”的概率。这样其实比一刀切的阈值方法更为合理，学生也不会有强烈动机来找分。

思考：大学里的成绩等级制是否应该改成这种概率方法？

题外：逻辑函数/模型的前世今生

逻辑函数可用来描述一定环境中种群的增长情况。

类似于“鸡生蛋，蛋生鸡”的故事或马尔萨斯的人口论，简单的种群增长模型认为种群的增长率与当前的种群数量 $y(t)$ 成正比：

$$\frac{dy(t)}{dt} = ay(t)$$

其中 a 是增长率。它的解就是指数增长：

$$y(t) = y_0 e^{at}$$

在 1838 年，Verhulst 提出一定环境中种群的增长存在一个上限，原因是由于环境资源是有限的，因此增长率 a 会随种群 y 增加而降低，例如， $a(t) = a_0 - ry(t)$ ，则演化方程变成

$$\frac{dy(t)}{dt} = [a_0 - ry(t)]y(t)$$

它的解为

$$y(t) = \frac{a_0/r}{1 + \left(\frac{a_0}{ry_0} - 1\right)e^{-rt}} = \frac{y_{\max}}{1 + \left(\frac{y_{\max}}{y_0} - 1\right)e^{-rt}}$$

它所描述的结果其实就是逻辑函数（经过平移与标度变化以后），即种群先是经历缓慢的增长，然后到一定阶段后快速增长，最后增长又变得缓慢，进入近乎不变的阶段。这被称为 Pearl-Verhulst 模型（Pearl 的工作在很长时间内被人所忽略，直到 Verhulst 在 1920 年重新提出了这个模型）。

其实，不光是物种数量的建模，很多社会、经济现象都可以借助于生长模型来解释。一般来说，事物总是经过发生、发展、成熟三个阶段，而每一个阶段的发展速度各不相同。通常在发生阶段，变化速度较为缓慢；在发展阶段，变化速度加快；在成熟阶段，变化速度又趋缓慢。比如某互联网公司推出一款爆款产品以后，其增长可能也会遵循类似规律：刚开始时用户呈指数级增长；达到一定规模以后，由于竞争对手出现、以及潜在用户总数是有限的，这个产品的增长率就会逐渐放缓。

这种规律对于预测未来大势是非常有用的。如果你在 2001 年能利用当时的 PC 与笔记本电脑的数据预测 20 年后的大致情况，那么你就能制定有利的长远发展策略。而在 2023 年，你认为 Facebook 能一直保持增长势头吗？

题外：国际象棋与电子游戏中的等级与对战匹配...

逻辑函数可用于国际象棋、围棋、足球、篮球等运动以及各种电子游戏中的等级调整与对战匹配。

具体地，是采用阿帕德·埃洛（Arpad Elo）所提出的衡量各类对弈活动水平的评价方法，被称为 ELO 算法。这个算法假设选手的实际临场表现 t 在其实力（真实等级分数 x ）附近一定范围内波动（噪声），符合正态分布：

$$p(t|x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(t-x)^2}$$

因此如果一个选手的真实等级分数比对方高 Δx ，则获胜概率为

$$P(\Delta x) = \iint_{t' > t} p(t'|x') p(t|x) dt dt' = \dots = \int_{-\Delta x}^{+\infty} \frac{1}{\sqrt{4\pi\sigma^2}} e^{-\frac{1}{4\sigma^2}(\Delta t)^2} d\Delta t$$

这是一个高斯误差函数 $\text{erf}(x)$ ，计算比较复杂，在 ELO 算法中近似成如下简单公式：

$$P(\Delta x) = \frac{1}{1 + e^{-\alpha\Delta x}}$$

其中 α 是个参数。这就是个逻辑函数，给出了真实水平相差为 Δx 时的对战获胜概率，类似于逻辑回归模型的概率诠释。在现实中，真实水平是未知的，但对战结果是可以知道的。因此可根据上式应用贝叶斯公式来从结果推断原因（真实水平）。通常采用类似于梯度下降的方法在一定的学习率下进行等级分数的调整。

在电子游戏中，当一个新玩家加入时，如何为其匹配对手？此时可考虑匹配不同对手时，对战结果对减少新玩家的真实水平估计误差有什么贡献（影响），并给其匹配贡献最大的对手。

ELO 算法在 1960 年开始用于国际象棋协会的比赛和积分计算。后来成为对弈水平评估的公认的权威方法，被广泛用于围棋、足球、篮球等运动。DOTA2、守望先锋、LOL、王者荣耀等各种电子游戏也使用 ELO 算法设计其玩家匹配机制。

当 $\sigma(\mathbf{w}^T \mathbf{x}) > 0.5$ 时，预测数据点属于第一类的概率大于第二类的概率；而当 $\sigma(\mathbf{w}^T \mathbf{x}) < 0.5$ 时，则预测第二类的概率大于第一类。因此，所预测的第一类与第二类的边界（决策面，decision surface 或 decision boundary），由下式所描述

$$\sigma(\mathbf{w}^T \mathbf{x}) = 0.5 \quad (3.8)$$

即

$$\mathbf{w}^T \mathbf{x} = 0 \quad (3.9)$$

这是 $\mathbf{x}$ 空间中的一个线性平面。进一步地，预测概率值的等高线（面）也是一些 $\mathbf{x}$ 空间中的（互相平行的）线性平面。这就是逻辑回归被分类为线性方法的原因。不过，由于非线性激活函数 $\sigma(\cdot)$ 的存在， y 对 $\mathbf{w}$ 不再是线性函数。因此，优化求解比线性回归复杂（但比感知器容易）。

2.2 逻辑回归的求解

逻辑回归模型的求解，就是在已知一系列 $\{\mathbf{x}_n, t_n\}$ 数据的条件下，求 $y(\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x})$ 中参数 $\mathbf{w}$ 的最佳值。

在逻辑回归模型的概率诠释下^m，当 $t_n = 1$ （第一类）时，模型预测结果与 t_n 一致的概率（即预测正确的概率）是 $y_n \equiv y(\mathbf{x}_n) = \sigma(\mathbf{w}^T \mathbf{x}_n)$ ；当 $t_n = 0$ （第二类）时，模型预测正确的概率是 $1 - y_n$ 。在 $t_n = 1$ 时，我们应该追求 y_n 尽可能大；而在 $t_n = 0$ 时，我们应该追求 y_n 尽可能小。综合起来，不管 t_n 取值如何，模型预测正确的概率可写成

$$y_n^{t_n} (1 - y_n)^{1-t_n} \quad (3.10)$$

假设数据点之间是独立的，则似然函数为

$$p(\mathbf{t}|\mathbf{w}) = \prod_{n=1}^N y_n^{t_n} (1 - y_n)^{1-t_n} \quad (3.11)$$

接下来可以利用前一章中所介绍的贝叶斯公式的思路来求解。如果采用最简单的最大似然函数法，可以定义（交叉熵ⁿ）误差函数

$$E(\mathbf{w}) = -\ln p(\mathbf{t}|\mathbf{w}) = -\sum_{n=1}^N [t_n \ln y_n + (1 - t_n) \ln (1 - y_n)] \quad (3.12)$$

相当于是先取 $p(\mathbf{t}|\mathbf{w})$ 的对数（对数函数是递增函数，因此这不改变函数的极值的位置），再乘上一个负号（把最大值变成最小值，符合对误差的通常理解）。这个交叉熵误差函数衡量的其实是实验观察到的概率 t_n 与模型所预测的概率 y_n 之间

^m 逻辑回归的概率诠释的理由其实不如线性回归充足。线性回归数据是在线的周围，因此偏差近似为高斯分布算是比较合理。而逻辑回归可把两类样本看做有能量与分布的含义，在决策面附近做展开到一阶，但这种展开显然对远离决策面的点是不成立的，误差很大。

ⁿ 它与熵的联系，可比较物理化学课程中规则溶液的混合熵 $\Delta S_{\text{mixing}} = -Nk_B [x \ln x + (1 - x) \ln (1 - x)]$ 。

差别程度，即 KL 散度（参考“扩展知识：两个概率分布的相似性与 KL 散度”）。这种基于对数函数所定义的误差函数有个额外的好处，就是它在数学上属于凸 (convex) 函数（形象地讲，就是任何一段都是开口向上的，或都是开口向下的，如图 3.5 (a) 所示，而不会出现图 3.5 (b) 所示的有些地方朝上有些地方朝下的情况。严格地讲，是对任意两点 x_1, x_2 满足 $f(\lambda x_1 + (1 - \lambda)x_2) \leq \lambda f(x_1) + (1 - \lambda)f(x_2)$ ，其中 $0 < \lambda < 1$ ），具有良好的数学性质，容易求解最大/最小值。例如，凸函数的最小值存在且唯一；沿着某种能够持续下降的方向搜索（例如采用下述的梯度下降法）总能找到最小点。而按通常的最小二乘法定义得到的误差函数则是非凸(non-convex)函数，数值求解更为困难。

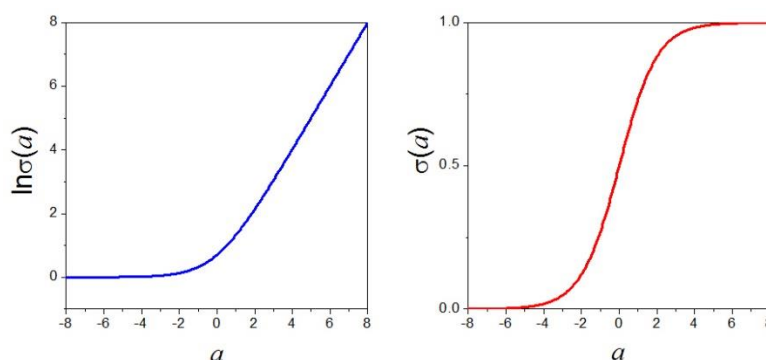


图 3.5. 交叉熵误差函数的凸函数性质。(a) $\ln \sigma(z)$ ，是凸函数。(b) $\sigma(z)$ ，是非凸函数。

为了利用梯度下降法求解逻辑回归模型中参数 $\mathbf{w}$ 的最佳值（使误差函数尽可能小），我们先来推导误差函数的梯度表达式。根据激活函数 $\sigma(z)$ 的表达式 (3.7)，其导数为

$$\frac{d\sigma(z)}{dz} = \frac{-e^{-z}}{-(1+e^{-z})^2} = \sigma(z)[1 - \sigma(z)] \quad (3.13)$$

利用这个式子，再根据交叉熵误差函数的表达式 (3.12)，可得到误差的梯度

$$\begin{aligned} \frac{dE(\mathbf{w})}{d\mathbf{w}} &= -\sum_{n=1}^N \left[\frac{t_n}{y_n} - \frac{(1-t_n)}{1-y_n} \right] \frac{dy_n}{d\mathbf{w}} = -\sum_{n=1}^N \left[\frac{t_n}{y_n} - \frac{(1-t_n)}{1-y_n} \right] \frac{d\sigma(\mathbf{w}^T \mathbf{x}_n)}{d\mathbf{w}} = -\sum_{n=1}^N \left[\frac{t_n}{y_n} - \right. \\ &\quad \left. \frac{(1-t_n)}{1-y_n} \right] y_n(1-y_n) \mathbf{x}_n = \sum_{n=1}^N (y_n - t_n) \mathbf{x}_n \end{aligned} \quad (3.14)$$

这是个很简单的结果。它是个矢量，有时也记为 $\nabla_{\mathbf{w}} E(\mathbf{w})$ ，给出了 $E(\mathbf{w})$ 随 $\mathbf{w}$ 增加最快的方向与增加速度。如果乘上 -1 ，就可得到下降最快的方向，可用于梯度下降法的迭代求解：

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla_{\mathbf{w}} E(\mathbf{w}) \quad (3.15)$$

扩展知识：两个概率分布的相似性与 KL 散度

假设我们有两个概率分布函数 $p(x)$ 与 $q(x)$ ，我们能够怎样衡量它们之间的相似程度（或不相似程度）？

解决这个问题的方案有很多种，其中一种常用的方案就是 KL 散度（Kullback-Leibler divergence），它利用了香农的信息熵的概念。

在信息论中，考虑一个随机值 x 及其概率分布 $p(x)$ ，要把 x 的值传输出去（例如通过无线电波），在最佳编码方案下所需要的平均传输长度与下式成正比：

$$H[p(x)] = - \int p(x) \ln p(x) dx$$

这被称为信息熵，即一个 x 值所包含的平均信息长度。例如，假设 $x = \{\text{是}, \text{否}\}$ ，对应概率 $p(x) = \{0.5, 0.5\}$ ，则最佳编码方式是 $\{0, 1\}$ ，传输一个结果需要的二进制电报平均长度为 $0.5 \times 1 + 0.5 \times 1 = 1$ 。又如，假设某个足球赛事夺冠的可能球队是 $x = \{\text{巴西}, \text{德国}, \text{西班牙}, \text{阿根廷}\}$ ，对应概率 $p(x) = \{\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{8}\}$ ，则最佳编码方案是 $\{0, 10, 110, 111\}$ ，即用长度为 1 的代码来编码出现几率最大的“巴西”而用长度为 3 的代码编码“西班牙”或“阿根廷”，这样要把结果传输出去的二进制电报平均长度为 $\frac{1}{2} + \frac{1}{4} \times 2 + 2 \times \frac{1}{8} \times 3 = -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{4} \log_2 \frac{1}{4} - \frac{1}{8} \log_2 \frac{1}{8} - \frac{1}{8} \log_2 \frac{1}{8} = 1.75$ ，即上述的信息熵 $H[p(x)]$ （ $-\ln p(x)$ 代表某一个可能取值的编码长度，乘上其出现概率 $p(x)$ ，就得到平均传输长度）（相差一个所用二进制所导致的常数因子）。数学上可以证明，其它的编码方式都不能得到更短的平均传输长度。

KL 散度的想法大致如下：如果 x 的真实发生概率是 $p(x)$ ，但我们使用另一个概率函数 $q(x)$ 来设计编码方案，那么结果肯定不会比 $p(x)$ 的最佳编码方案更好，或者说，在平均传输长度上存在差距。这个差距就可以用来定量衡量 $p(x)$ 与 $q(x)$ 之间的差别大小，被称为 KL 散度：

$$KL(p||q) = - \int p(x) \ln p(x) dx - \left(- \int p(x) \ln q(x) dx \right) = - \int p(x) \ln \left[\frac{q(x)}{p(x)} \right] dx$$

可以证明， $KL(p||q) \geq 0$ ，而且只有当 $p(x) \equiv q(x)$ 时等号才成立。需要注意的是，KL 散度并不是对称的，即 $KL(p||q) \neq KL(q||p)$ 。

其中 τ 是迭代的轮数， η 是步长或学习率，可取固定值或（通过一维方法搜索不同 η 值所得到的）优化值。这种梯度下降法在机器学习的优化中被广泛使用。它其实是极小化算法，但在逻辑回归中，由于误差函数是凸函数，只存在一个极小值点，因此所找到的极小值点也就是全局的最小值点，非常方便。

更普适地，逻辑回归模型的求解就是误差函数的极小化，因此，除了使用最常见的梯度下降法以外，其它的极小化算法，例如共轭梯度法与质牛顿法（BFGS方法等），也可以使用。

当数据个数太多时，梯度计算公式（3.14）中的求和号计算需要耗费大量的计算资源。考虑到迭代过程（3.15）本来就是一个逐步逼近最佳值点的过程，不能一蹴而就，每一步的移动方向不一定要非常精确，因此也可以利用部分数据而非全部数据来计算梯度。在极端的做法下，迭代时每次只利用一个训练数据点，即

$$\mathbf{w}^{(\tau+1)} = \mathbf{w}^{(\tau)} - \eta \nabla_{\mathbf{w}} E_n(\mathbf{w}) \quad (3.16)$$

其中 $E_n(\mathbf{w})$ 代表只使用数据点 n 所计算的误差。这被称为随机梯度下降法（Stochastic gradient descent，简称SGD），其迭代过程的示意图见图3.6。只使用一个数据点怎么能计算得准呢？这可以利用贝叶斯公式的观点来理解：假设你有了一个新的数据，参数应该如何调整？新证据的出现将改变信念，随机梯度下降法的迭代公式就代表了这种信念改变的结果。这种随机梯度下降法中每一步中数据点的选择是随机的，因此迭代过程具有随机性。在误差函数曲面形状比较复杂的情况下（例如后面章节中介绍的神经网络方法），这种随机性反而有利于逃离鞍点（通常使用所有数据点的梯度下降法可能会在迭代过程中错误收敛到复杂曲面中无处不在的鞍点）。

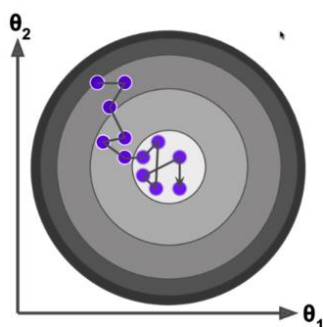


图 3.6. 随机梯度下降法的迭代过程示意图。圆圈代表误差函数的等高线，圆心是最佳值。

2.3 基函数与正则化的使用

在线性回归中的很多重要思路，例如基函数与正则化的使用，都可以很容易推广应用到逻辑回归中。

引入一些固定基函数 $\phi_j(\mathbf{x})$ ，则线性回归公式变成

$$y(\mathbf{x}; \mathbf{w}) = \sigma\left(\sum_{j=0}^M w_j \phi_j(\mathbf{x})\right) = \sigma(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x})) \quad (3.17)$$

其中 $\boldsymbol{\phi}(\mathbf{x})$ 代表矢量 $[\phi_0(\mathbf{x}), \phi_1(\mathbf{x}), \dots, \phi_M(\mathbf{x})]^T$ 。此时 $\mathbf{w}$ 的数目不再与 $\mathbf{x}$ 的分量数目相关，而是等于基函数的数目。似然函数与交叉熵误差函数仍然可用公式(3.11)与(3.12)描述，只不过其中的 y_n 需要改用带基函数的(3.17)计算。误差的梯度则变成

$$\frac{dE(\mathbf{w})}{d\mathbf{w}} = -\sum_{n=1}^N \left[\frac{t_n}{y_n} - \frac{(1-t_n)}{1-y_n} \right] \frac{d\sigma}{d\mathbf{w}} = \sum_{n=1}^N (y_n - t_n) \boldsymbol{\phi}(\mathbf{x}_n) \quad (3.18)$$

在没有引入基函数时，逻辑回归的决策面（预测类别之间的边界）是 $\mathbf{x}$ 空间中的简单线性平面，描述能力有限。例如，对于图 3.7 (a) 中的数据，不管怎样调整 $\mathbf{w}$ ，原始逻辑回归模型的决策面只能是 (x_1, x_2) 平面内的直线，没有方法把两类数据分开。但如果引入基函数，虽然决策面在 $(\phi_0(\mathbf{x}), \phi_1(\mathbf{x}), \dots, \phi_M(\mathbf{x}))$ 空间中依然是线性平面，但折算回 $\mathbf{x}$ 空间以后就可以呈现出复杂的决策面（边界），从而大大增强模型的描述能力。例如，还是对于图 3.7 (a) 中的数据，如果我们引入一些多项式基函数：

$$1, x_1, x_2, x_1^2, x_1 x_2, x_2^2 \quad (3.19)$$

则这种带基函数的逻辑回归模型可以把两类数据完全分开（图 3.7 (b)）。这个例子表明，通过基函数，低维线性不可分的数据投影（变换）到高维空间后很可能变得线性可分，从而可用线性方法处理。

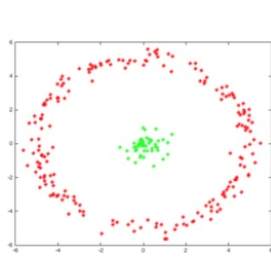


Figure 1: Original Data

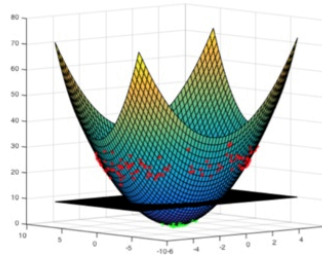


Figure 2: Data on feature space

图 3.7. 基函数对逻辑回归的影响。(a) 训练数据集。 $\mathbf{x}$ 具有两个分量 x_1, x_2 。两个类别分别用红色与绿色表示。(b) 引入基函数 $(\phi_1(\mathbf{x}) = x_1, \phi_2(\mathbf{x}) = x_2, \phi_3(\mathbf{x}) = x_1^2 + x_2^2)$ 后的结果。

为了防止模型过拟合，我们可以引入正则化惩罚项，例如，把误差函数变成

$$E(\mathbf{w}) = -\sum_{n=1}^N [t_n \ln y_n + (1 - t_n) \ln(1 - y_n)] + \frac{\lambda}{2} \mathbf{w}^T \mathbf{w} \quad (3.20)$$

其中 λ 是超参数，控制着正则化惩罚的强度。与线性回归中的情况类似，这种正则化的逻辑回归相当于贝叶斯公式框架下的最大化后验概率方法。另外，还可以采用类似于贝叶斯拟合的完整方法，这里不再赘述。

2.4 多类别分类

前面介绍的逻辑回归解决方法所针对的是两分类问题，即数据点只属于两个类别。有时候，我们还会碰到多类别分类的问题。例如，第一章中提到过的手写数字识别就是一个多类别分类问题，有 10 个类别，分别代表数字 0 至 9。又如，我们要开发一个 email 的自动标签/整理功能，希望能够将每封电子邮件标注为“工作 Work”、“朋友 Friends”、“家庭 Family”、“爱好 Hobby”，共有 4 个类别。再如，我们要根据卫星云图预测第二天的天气是“晴”、“多云”还是“雨”，共有 3 个类别。

将两分类方法推广运用于多类别分类问题，常见的是下面两种主要思路。

第一种被称为“一对余”(one-versus-the-rest)方法，每次考虑一个类别与其它类别的分类，从而将多类别分类转化为多个二分类的问题。例如，数据属于三个类别 A、B、C，那我们可以将 B 和 C 合成一类（称为“B+C”或“非 A”），训练一个逻辑回归模型来区分 A 与“非 A”，即预测某个数据点属于类别 A 的概率；然后再把 A 和 C 合成一类（“非 B”），训练一个模型来区分 B 与“非 B”；再训练一个模型来区分 C 与“非 C”。这种方法得到的各类概率之和不一定等于 1，因此对于要求各类概率和等于 1 的场合会有困难。但它可以应用于类间不互相排斥的场合，例如一个数据点可以既属于 A 又属于 B（比如电影的类型标签），此时并不要求概率和等于 1。

另一种被称为 softmax 方法，在实际场合中应用更多。在这种方法中，对每个类别 i ，定义

$$z_i(\mathbf{x}) = \mathbf{w}_{(i)}^T \mathbf{x} \quad (3.21)$$

再利用如下 softmax 函数来计算数据点 $\mathbf{x}$ 属于类别 i 的概率

$$y^{(i)}(\mathbf{x}) = \frac{e^{z_i}}{\sum_i e^{z_i}} = \frac{e^{\mathbf{w}_{(i)}^T \mathbf{x}}}{\sum_i e^{\mathbf{w}_{(i)}^T \mathbf{x}}} \quad (3.22)$$

它满足各类别概率之和等于 1 的要求，因此特别适用于类别互斥的情形。它的形式有些类似于玻尔兹曼分布（把 $-\mathbf{w}_{(i)}^T \mathbf{x}$ 看做是不同状态 i 的能量，则 $y^{(i)}$ 就是状态

i 的玻尔兹曼分布概率)。

如果你在开发一个音乐分类的应用，需要对 k 种类型的音乐进行识别，那么是选择使用“一对余”方法呢，还是使用 softmax 方法？这一选择取决于你的类别之间是否互斥。

第 3 节 逻辑回归的应用例子

3.1 点击率预测

第一个例子是网络点击率的预测，包括了谷歌公司解释其所用模型的两篇论文：

- (1) H. Brendan McMahan, Gary Holt, D. Sculley, Michael Young, Dietmar Ebner, Julian Grady, Lan Nie, Todd Phillips, Eugene Davydov, Daniel Golovin, Sharat Chikkerur, Dan Liu, Martin Wattenberg, Arnar Mar Hrafnkelsson, Tom Boulos, Jeremy Kubica.

Ad Click Prediction: A View from the Trenches.

Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 1222-1230 (2013).

- (2) H. B. McMahan.

Follow-the-regularized-leader and mirror descent: Equivalence theorems and L1 regularization.

Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics, PMLR 15, 525-533 (2011).

互联网广告、个性化推荐，甚至一般的互联网产品，一个重要指标就是点击率。点击率是指互联网网页面上某一内容被点击的次数与被显示次数之比。它反映了内容受关注的程度，经常被用来衡量广告的吸引程度，计算广告价格，并把更有价值的条目放到更重要的位置上。点击率预测是各大互联网公司几乎都要做的重要问题，是新旧巨头们挣钱的核心模块之一。百度通过点击率预测，筛选出变现能力更强的广告；淘宝通过点击率预测，筛选出更有可能让你“大出血”的商品；抖音通过点击率预测，让你时刻忍不住刷一刷。社交公司会收集用户偏好，推送适合的广告，提高广告点击率和转化率，从而提高公司的营收水平，这也许就是 facebook 值钱的原因（？）。

点击率预测是一个分类问题。它的输出是用户“点击”还是“不点击”，而它的输入是能影响用户点击行为的所有因素。例如，所展示内容在页面上的位置、内容本身（比如其标题与简介语）、其临近位置的内容；还有就是用户本身相关数据，如用户的以往习惯、用户在网站上所搜索的内容，等等。而其中的任何一项，又包含了很多的数据，例如，怎样量化一个标题的信息，一种方法是统计出

其所包含的单词，这本身已经包含了数量巨大的输入分量。

点击率预测所涉及的数据很庞大，而且由于线上系统往往对实时性要求很高，训练好的模型在线上进行预测时必须能够快速响应。因此，虽然分类问题有很多更加高级、更加复杂的算法，但逻辑回归模型由于其足够简单却又足够有效，仍然顽强坚守在点击率预测的第一线，发挥着不可替代的作用。

谷歌的点击率预测算法名为 Follow-the-regularized-leader-Proximal，简称 FTRL-Proximal。它是基于逻辑回归进行适当改造而成的。标准的逻辑回归是使用公式(3.12)所示的交叉熵误差函数 $E(\mathbf{w})$ ，计算其对权重 $\mathbf{w}$ 的梯度 $\mathbf{g} = \nabla_{\mathbf{w}} E(\mathbf{w})$ ，并采用迭代方式的梯度下降法来求解，例如，

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta_t \mathbf{g}_t \quad (3.23)$$

这里 $\mathbf{g}_t$ 只使用一条或少数几条数据进行计算，即随机梯度下降的做法，在点击率预测中广泛被采用。FTRL-Proximal 把上述迭代公式改成

$$\mathbf{w}_{t+1} = \arg \min_{\mathbf{w}} \left(\mathbf{g}_{1:t} \cdot \mathbf{w} + \frac{1}{2} \sum_{s=1}^t \sigma_s |\mathbf{w} - \mathbf{w}_s|^2 + \lambda_1 |\mathbf{w}| \right) \quad (3.23)$$

其中 $\arg \min_{\mathbf{w}}$ 表示使括号中的目标函数取最小值时的变量值 $\mathbf{w}$ ，而 σ_t 则是学习率 η_t 的倒数。最后一项就是在上一章中介绍过的用于 Lasso 回归的 L1 正则化项，它有助于减少非零参数的数目，达到增加模型稀疏性的目的。如果只取目标函数中的前两项，并只考虑第二项中的求和里的最后一项（ $s = t$ 的项），则结果与通常的（3.23）等同。把（3.23）中的待优化函数整理后重新写成

$$(\mathbf{g}_{1:t} - \sum_{s=1}^t \sigma_s \mathbf{w}_s) \cdot \mathbf{w} + \frac{1}{\eta_t} |\mathbf{w}|^2 + \lambda_1 |\mathbf{w}| + \text{const} \quad (3.24)$$

引入辅助变量 $\mathbf{z}_{t-1} = \mathbf{g}_{1:t-1} - \sum_{s=1}^{t-1} \sigma_s \mathbf{w}_s$ ，并在第 t 步先计算 $\mathbf{z}_t = \mathbf{z}_{t-1} + \mathbf{g}_t + \left(\frac{1}{\eta_t} - \frac{1}{\eta_{t-1}}\right) \mathbf{w}_t$ ，然后再解出 $\mathbf{w}_{t+1}$ 的值：

$$w_{t+1,i} = \begin{cases} 0 & (\text{if } |z_{t,i}| \leq \lambda_1) \\ -\eta_t (z_{t,i} - \text{sgn}(z_{t,i}) \lambda_1) & (\text{otherwise}) \end{cases} \quad (3.25)$$

详细的算法内容参见上述论文。

思考：对于在线学习，每次预测后等用户结果出来还可以将其作为训练数据，进一步改善预测模型。因此在线学习系统应该是越用越流畅。这也是手机操作系统的理想境界。

3.2 MOFs 中金属离子的价态

第二个例子是金属有机框架（MOF）材料中的金属离子的价态预测：Kevin Maik Jablonka, Daniele Ongari, Seyed Mohamad Moosavi and Berend Smit. Using collective knowledge to assign oxidation states of metal cations in metal-organic frameworks. *Nature Chemistry* **13**, 771-777 (2021).

元素的氧化态（价态），也称为氧化数，近似描述了一个原子必须获得或失去多少电子才能与另一个原子形成化学键。氧化态对于化学家解释（氧化还原）反应性以及光谱特性等至关重要。例如，Mn(II)对应于 $3d^5$ 、Mn(IV)对应 $3d^3$ ，而 Mn(VII)对应 $3d^0$ 形式的电子构型。高锰酸盐（Mn(VII)）化合物具有很强的氧化性，通常呈紫色，而 Mn(II)化合物的反应性较低，通常是无色的。因此，有些人认为氧化态应该被表示为元素周期表的第三维。

尽管弄清楚单个元素的氧化态非常简单，但当涉及由多种元素组成的化合物时，事情就变得复杂了。目前，剑桥结构数据库（Cambridge Structural Database, CSD）中的氧化态以材料的名称给出，但数据库非常混乱，有很多错误，混合了实验、专家猜测和价键理论的不同变体，用于指定氧化态。而从理论上讲，氧化态只是个近似的概念。对于复杂的材料，实际上不可能根据第一性原理预测氧化态。事实上，很多量子程序都需要金属的氧化态作为输入。

目前预测氧化态的方法一般是基于 20 世纪初提出的价键理论，该理论根据原子之间的距离来估计化合物的氧化态。但这并不总是有效的，尤其是晶体材料。在晶体材料中，不仅原子间距离很重要，金属配合物的几何形状也有很大影响。

上述工作训练了一组机器学习模型（包括逻辑回归），对 MOFs 中金属离子的氧化态进行预测。作者使用 CSD 的 Python 应用程序编程接口（API）检索 MOF 子集结构的数据，最后得到四万多个有用数据用于机器学习的训练与分析。采用的特征结合了领域专家的知识，包括三个方面：金属类型（包括其在周期表中行、列、原子序数、价电子数等）、配位环境的几何形状和化学环境（例如金属原子与相邻配位原子的电负性之差）。作者采用了四个基本的机器学习分类模型，包括逻辑回归、K 近邻算法（KNN）、决策树、梯度提升树，并利用四个模型进行“投票”（即委员会方法，参见后面第 7 章）来进行最终的预测。

机器学习对金属价态的预测效果如图 3.8 所示。对于元素周期表的 *s* 区元素（Li、Na 和 Ca 等），所有价态都预测正确。*p* 区（Al、Pb 和 Bi 等）的准确率也在 98% 以上。即使对于更具挑战性的 *d* 区（例如 Fe 和 Cu）和 *f* 区元素（例如 Ce、Eu 和 Ho），也能获得至少 90% 的准确率。

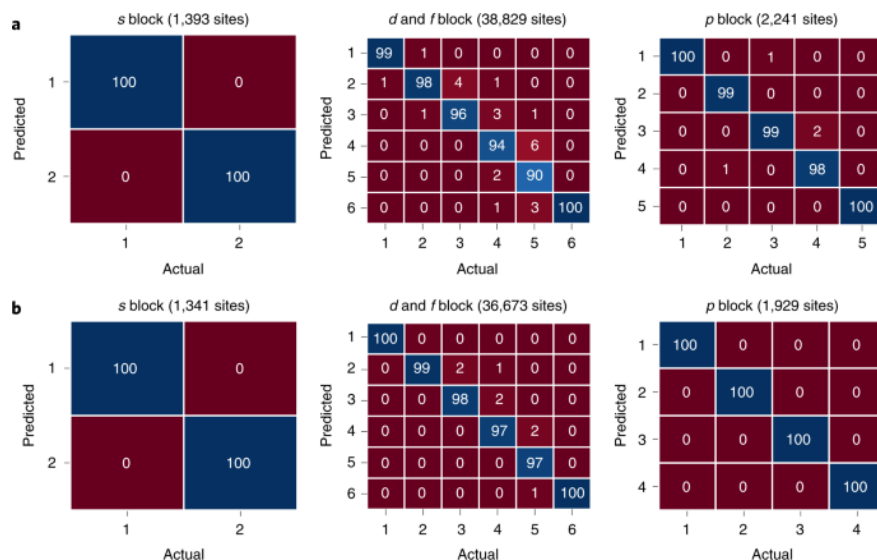


图 3.8. 金属离子价态的预测效果。矩阵中的数字是百分比，并相对于列进行了归一化。

作业：

3.1 证明逻辑函数 $\sigma(a)$ 满足对称性质 $\sigma(-a) = 1 - \sigma(a)$ ，其逆函数为 $\sigma^{-1}(y) = \ln \frac{y}{1-y}$ 。

3.2 利用逻辑回归研究某个分类问题，输入变量有两个分量 ($x = (x_1, x_2)$)。学习到的参数为 $w_0 = 2, w_1 = -2, w_2 = 1$ 。(1) 请计算 $x = (0.5, -1)$ 与 $x = (1, -1)$ 时逻辑函数的值 $y(x) = \sigma$ 。(2) 在 (x_1, x_2) 平面上的决策面 (线) 是什么？

3.3 证明逻辑回归误差函数对 w 的二阶导数 (这可用于牛顿法求解 w) 为 $\frac{d^2 E(w)}{dw dw} = \sum_{n=1}^N y_n (1 - y_n) x_n x_n^T$

扩展阅读：

- <https://www.jiqizhixin.com/articles/2018-10-19-15>
达观数据王子豪：这 5 个例子，小学生都能秒懂分类算法.mht
- <https://www.meihua.info/a/65329>
关于点击率模型，你知道这三点就够了.mht

- <https://blog.csdn.net/guoziqing506/article/details/81328402>
[逻辑回归 \(logistic regression\) 原理详解.mht](#)
- <https://www.pinlue.com/article/2017/08/2723/594294870044.html>
[逻辑回归模型的前世今生.mht](#)
它是一个简单的模型，却支撑着互联网新旧巨头最挣钱的核心模块；
它是一个经典的模型 行业内却很少有人能讲清它的前世今生
- <https://www.huxiu.com/article/400368.html>
[在游戏里虐菜鸟，已经成为一种奢侈？.pdf](#)
目前，游戏（例如 DOTA2）的玩家匹配机制都在使用一个叫 ELO 的算法，这一算法从象棋而来。
- <https://zhuanlan.zhihu.com/p/57480433>
[ELO 算法的原理及应用.pdf](#)
- https://bbs.gameres.com/thread_228018_1_1.html
[ELO 算法教程.mht](#)
可体会逻辑函数的概率含义，以及学习率的使用与学习结果的收敛。
- <https://www.jiqizhixin.com/articles/2021-09-17-9>
[元素周期表应加上氧化态？机器学习破解晶体结构的氧化态.pdf](#)

参考资料与文献：

第四章 模型选择与评估

刘志荣

在前面的章节中，我们重点介绍了机器学习的两种重要模型：用于回归问题的线性回归，以及用于分类问题的逻辑回归。在后面的各章中，我们还将介绍更多的、更复杂的模型。而在本章中，我们不会介绍具体的机器学习模型（方法），而是先来讨论各种模型所面临的一些共同议题，例如怎样准备数据，怎样对模型进行有效的训练，怎样在不同模型之间进行选择，怎样对模型的表现进行评估，等等。

第 1 节 模型的训练、选择与交叉检验

1.1 训练集、验证集、测试集

机器学习模型中存在参数，如线性回归与逻辑回归中的线性参数 $\mathbf{w}$ 。通过对训练集上的误差函数进行最小化，可以确定这些参数的值。但是，模型中还存在超参数，例如线性回归与逻辑回归中的正则化强度 λ ，以及下一章将介绍的神经网络模型中的隐藏层节点数目，等等。我们应该怎样确定它们的最优值？更进一步地，如果我们有不同模型可以选择，我们应该选择哪一个模型？线性回归还是神经网络？

在线性回归中，我们是通过比较不同 λ 值下模型在测试集上的误差来确定超参数的最优取值的。但是，严格地讲，这种做法是不严谨的。我们不能用测试集误差来选择超参数与模型，否则测试集的信息会“泄露”到模型选择中，从而导致测试集误差不能用来独立衡量模型的泛化能力。也就是说，机器学习模型学习的过程就是利用一些已知数据来确定参数 $\mathbf{w}$ 与超参数 λ 的值。我们应该用一些模型“从没见过”的数据来衡量它的学习效果（泛化能力），但如果这些数据已经被用于超参数的确定，那么它们就是模型“已经见过”而非“从没见过”的^o。

因此，较好的做法是把已知数据分成训练集（training set）、验证集（validation set）与测试集（test set），分别用于模型的训练（training）、选择（selection）以及对最终所选中的最优模型所进行的测试评估（assessment）。这里所说的选择包括超参数的确定，即从某种角度上把具有不同超参数数值的模型看做不同的模型。

以监督学习为例，记已知数据集为 $\{\mathbf{x}_n, t_n\}$ ，机器学习模型的预测函数记为 $\hat{f}(\mathbf{x}; \mathbf{w})$ ，其中 $\mathbf{x}$ 是自变量， $\mathbf{w}$ 是参数。此处我们在 f 上加上三角帽子“ $\hat{}$ ”（即 $\hat{f}$ ）

^o 在 AI 竞赛中，即使把已知数据分成训练集、验证集与测试集分别用于模型的训练、选择与测试评估，获胜模型的测试误差也有可能不能忠实反映模型的泛化能力。原因是“获胜”本身就是一种模型选择，根据测试误差挑选获胜者这一行为造成了信息的“泄露”。获胜的模型也不一定是最好的模型，冠军获胜可能只是因为运气好。

以特别指出这是模型的预测函数,以便用不带帽子的 f 来表示内在的真实函数(例如,在回归问题中真实数据满足 $t = f(\mathbf{x}) + \epsilon$,但其中是可测量的, f 与 ϵ 是不可测量的),严格区分两者。在不需要严格区分两者的情况下,则用 f 表示(例如第二章第二节)。将单个数据点上的损失函数(误差)记为 $L(t, \hat{f}(\mathbf{x}; \mathbf{w}))$ 。例如,在回归问题中我们采用二乘误差损失函数

$$L(t, \hat{f}(\mathbf{x}; \mathbf{w})) = \frac{1}{2} [t - \hat{f}(\mathbf{x}; \mathbf{w})]^2 \quad (4.1)$$

其中的因子 $\frac{1}{2}$ 有时候忽略不要,不影响结论。而在分类问题的逻辑回归中我们采用交叉熵损失函数

$$L(t, \hat{f}(\mathbf{x}; \mathbf{w})) = -t \ln \hat{f}(\mathbf{x}; \mathbf{w}) - (1 - t) \ln [1 - \hat{f}(\mathbf{x}; \mathbf{w})] \quad (4.2)$$

因此在任一数据集 $\mathcal{D}$ 上的误差可一般性写成

$$E(\mathbf{w}, \mathcal{D}) = \sum_{(\mathbf{x}, t) \in \mathcal{D}} L(t, \hat{f}(\mathbf{x}; \mathbf{w})) \quad (4.3)$$

在有些场合里,会在求和号前乘上 $\frac{1}{N}$ (其中 N 是 $\mathcal{D}$ 中的数据点总数目)以使结果与 N 近似无关。在模型的训练中,我们在误差函数 $E(\mathbf{w})$ 上增加正则化惩罚项,例如:

$$\tilde{E}(\mathbf{w}, \lambda, \mathcal{D}) = E(\mathbf{w}, \mathcal{D}) + \frac{\lambda}{2} \|\mathbf{w}\|^2 \quad (4.4)$$

模型训练的内容就是基于训练集 $\mathcal{D}_{\text{train}}$ 变化 $\mathbf{w}$ 使上式取最小值,求解得到 $\mathbf{w}$ 的值

$$\mathbf{w}_{\text{ML}}(\lambda) = \arg \min_{\mathbf{w}} \tilde{E}(\mathbf{w}, \lambda, \mathcal{D}_{\text{train}}) \quad (4.5)$$

这里特意指出它是依赖于超参数 λ 。此时在训练集上的误差被称为训练误差:

$$E_{\text{train}}(\lambda) = E(\mathbf{w}_{\text{ML}}(\lambda), \mathcal{D}_{\text{train}}) \quad (4.6)$$

注意这里计算的是 E 而不是 $\tilde{E}$,并不包含正则化项的值。在验证集 $\mathcal{D}_{\text{validation}}$ 上的误差则是验证误差

$$E_{\text{validation}}(\lambda) = E(\mathbf{w}_{\text{ML}}(\lambda), \mathcal{D}_{\text{validation}}) \quad (4.7)$$

而模型选择(超参数的确定)的内容就是基于验证集 $\mathcal{D}_{\text{validation}}$ 变化超参数 λ 使上式取最小值,即

$$\lambda_{\text{ML}} = \arg \min_{\lambda} E_{\text{validation}}(\lambda) = \arg \min_{\lambda} E(\mathbf{w}_{\text{ML}}(\lambda), \mathcal{D}_{\text{validation}}) \quad (4.8)$$

有时候也把得到的最小值 $E_{\text{validation}} = \min_{\lambda} E_{\text{validation}}(\lambda) = E(\mathbf{w}_{\text{ML}}(\lambda_{\text{ML}}), \mathcal{D}_{\text{validation}})$ 叫做验证误差, 请读者自行分辨。最后, 在最优参数与超参数取值下对测试集 $\mathcal{D}_{\text{test}}$ 计算测试误差:

$$E_{\text{test}} = E(\mathbf{w}_{\text{ML}}(\lambda_{\text{ML}}), \mathcal{D}_{\text{test}}) \quad (4.9)$$

它被用来衡量模型的最终效果, 即泛化能力。

关于如何划分训练集、验证集与测试集, 不同的人有不同的看法。Trevor Hastie 等人所著的《The Elements of Statistical Learning》所建议的比例是 50% : 25% : 25%。吴恩达在网络公开课《机器学习》中的建议是 60% : 20% : 20%。而在数据不是很足够的情况下, 人们往往会减少测试集的数据量, 并使用交叉验证的方法来统筹使用训练集与验证集, 以减少数据的浪费。

1.2 误差的偏差-方差分解: 欠拟合与过拟合

在图 2.3 与图 2.6 中, 测试误差随多项式的项数或正则化强度的变化呈现两头翘的形状。这是为什么呢? 事实上, 两头翘起的误差增大的来源是不同的。通过对误差来源的进一步分析, 可以获得更深入的认识。常用的一种方法是偏差-方差分解。

假设观察值 t 可表示成 $t(\mathbf{x}) = f(\mathbf{x}) + \epsilon$, 其中 f 是内在的真实函数, ϵ 是均值为零的噪声。基于数据集 $\mathcal{D}$ (训练集与验证集) 训练、选择得到的机器学习函数为 $\hat{f}(\mathbf{x}|\mathcal{D})$, 即上述的 $\hat{f}(\mathbf{x}; \mathbf{w}_{\text{ML}}(\lambda_{\text{ML}}))$ 。则在某一个测试数据 $(\{\mathbf{x}_0, t(\mathbf{x}_0)\})$ 下的测试误差 (二乘误差形式) 为^p

$$E_{\text{test}} = [f(\mathbf{x}_0) + \epsilon - \hat{f}(\mathbf{x}_0|\mathcal{D})]^2 \quad (4.10)$$

显然, 学习到的模型 $\hat{f}(\mathbf{x}_0|\mathcal{D})$ 依赖于数据集 $\mathcal{D}$ 。当数据集 $\mathcal{D}$ 变化时 (例如从总的已知数据集中随机挑选出一些作为训练集 $\mathcal{D}$, 此时 $\mathcal{D}$ 具有随机性; 或对某个 $\mathbf{x}_n$ 下的 t_n 进行测量, 由于噪声的存在, t_n 也具有随机性), $\hat{f}(\mathbf{x}_0|\mathcal{D})$ 也会变化, 具有一定的分布。另一方面, 即使 $\mathcal{D}$ 不变, 有些机器学习模型的训练过程本身具有随机性 (例如神经网络的随机梯度下降法), 也会导致所得到的 $\hat{f}(\mathbf{x}_0|\mathcal{D})$ 的随机性。总之, 可以把 $\hat{f}(\mathbf{x}_0|\mathcal{D})$ 看做是具有随机性的值, 并将其期望值 (平均值) 记为 $\langle \hat{f}(\mathbf{x}_0|\mathcal{D}) \rangle$ 。

^p 如果 $\mathcal{D}$ 代表训练集, $\hat{f}(\mathbf{x}|\mathcal{D})$ 代表训练步骤得到的函数 $\hat{f}(\mathbf{x}; \mathbf{w}_{\text{ML}}(\lambda))$, $\{\mathbf{x}_0, t(\mathbf{x}_0)\}$ 代表一个验证数据, 则 (4.10) 代表的是验证误差。结论是类似的。

在公式 (4.10) 中加上 $\langle \hat{f}(\mathbf{x}_0|\mathcal{D}) \rangle - \langle \hat{f}(\mathbf{x}_0|\mathcal{D}) \rangle$ (这个等于零, 因此不改变公式的结果), 并稍加整理, 得到误差

$$E_{\text{test}} = \{[f(\mathbf{x}_0) - \langle \hat{f}(\mathbf{x}_0|\mathcal{D}) \rangle] - [\hat{f}(\mathbf{x}_0|\mathcal{D}) - \langle \hat{f}(\mathbf{x}_0|\mathcal{D}) \rangle] + \varepsilon\}^2 \quad (4.11)$$

它包含了三项, 第一项 $[f(\mathbf{x}_0) - \langle \hat{f}(\mathbf{x}_0|\mathcal{D}) \rangle]$ 是没有随机性的常数, 因此误差的期望值为

$$\begin{aligned} \langle E_{\text{test}} \rangle = \langle \varepsilon^2 \rangle + [f(\mathbf{x}_0) - \langle \hat{f}(\mathbf{x}_0|\mathcal{D}) \rangle]^2 + \left\langle [\hat{f}(\mathbf{x}_0|\mathcal{D}) - \langle \hat{f}(\mathbf{x}_0|\mathcal{D}) \rangle]^2 \right\rangle - 2 \\ \langle [\hat{f}(\mathbf{x}_0|\mathcal{D}) - \langle \hat{f}(\mathbf{x}_0|\mathcal{D}) \rangle] \varepsilon \rangle \end{aligned} \quad (4.12)$$

其中利用了常数 $[f(\mathbf{x}_0) - \langle \hat{f}(\mathbf{x}_0|\mathcal{D}) \rangle]$ 可以提到期望值操作 $\langle \cdot \rangle$ 外面的性质。最后一项中的 $[\hat{f}(\mathbf{x}_0|\mathcal{D}) - \langle \hat{f}(\mathbf{x}_0|\mathcal{D}) \rangle]$ 与 ε 两部分的随机性分别来源于训练集 $\mathcal{D}$ 与测试集, 是独立的, 因此最后一项结果为 0, 得到

$$\langle E_{\text{test}} \rangle = \langle \varepsilon^2 \rangle + [f(\mathbf{x}_0) - \langle \hat{f}(\mathbf{x}_0|\mathcal{D}) \rangle]^2 + \left\langle [\hat{f}(\mathbf{x}_0|\mathcal{D}) - \langle \hat{f}(\mathbf{x}_0|\mathcal{D}) \rangle]^2 \right\rangle \quad (4.13)$$

这被称为偏差-方差分解 (bias-variance decomposition)。它表明误差有那个来源, 第一个是 $\langle \varepsilon^2 \rangle$, 完全来自于噪声, 不管我们的 $\hat{f}$ 训练得多准确 (例如 $\hat{f}$ 与 f 一模一样) 都是没法避免的; 第二个是 $[f(\mathbf{x}_0) - \langle \hat{f}(\mathbf{x}_0|\mathcal{D}) \rangle]^2$, 反映了预测平均值 $\langle \hat{f}(\mathbf{x}_0|\mathcal{D}) \rangle$ 与真实值 $f(\mathbf{x}_0)$ 之间的偏差, 与模型的容量有关; 最后一个是 $\left\langle [\hat{f}(\mathbf{x}_0|\mathcal{D}) - \langle \hat{f}(\mathbf{x}_0|\mathcal{D}) \rangle]^2 \right\rangle$, 反映了 (由于 $\mathcal{D}$ 的随机性或训练过程的随机性而导致的) 每次预测的值 $\hat{f}(\mathbf{x}_0|\mathcal{D})$ 与预测平均值 $\langle \hat{f}(\mathbf{x}_0|\mathcal{D}) \rangle$ 之间的涨落, 它与数据的多少有关。以多项式拟合的线性回归为例 (图 4.1): 如果取直线作为 $\hat{f}$, 则它的拟合结果不太受随机选取的 10 个训练数据影响, 每次拟合的结果都差不多, 方差很小, 但它的容量很小 (能描述的曲线种类有限), 只能是一条直线, 因此偏差很大, 此时被称为欠拟合; 如果取九阶多项式作为 $\hat{f}$, 则它的拟合结果非常依赖于随机选取的 10 个训练数据, 每次拟合的结果相差很大, 因此方差很大, 发生了过拟合, 但它能描述各种形状的曲线, 容量很大, 拟合的平均值可以很接近于真实曲线, 即偏差很小。这种偏差-方差分解还可以用飞镖游戏来类比 (图 4.2): 如

果射出的飞镖落点很集中但落点偏离靶心，就是高偏差、低方差；如果射出的飞镖落点分散但围绕着靶心，则是低偏差、高方差。

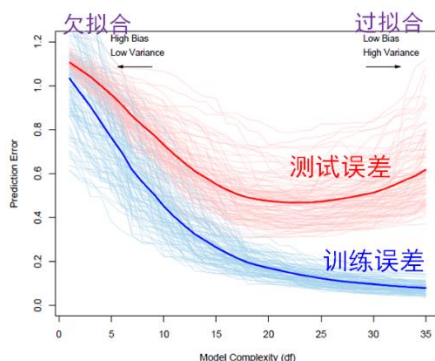


图 4.1. 偏差与方差对误差的影响。浅蓝色曲线表示训练误差随着模型复杂性的增加的变化情况，而浅红色曲线表示测试误差。共使用了 100 个大小为 50 的训练集的结果。

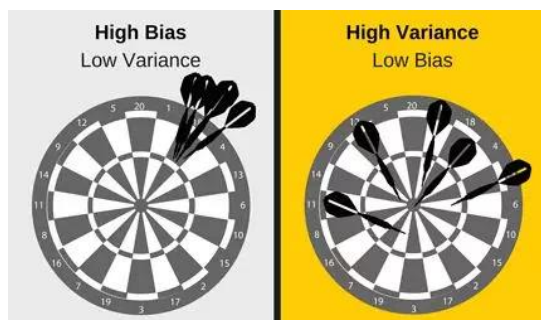


图 4.2. 偏差与方差对误差的影响：(a) 高偏差、低方差；(b) 低偏差、高方差。

偏差和方差是机器学习中预测误差的两种主要来源。当一个模型过于简单时，或者过于正则化，将使得它在学习数据集时不够灵活，不能充分学习/利用数据集中所包含的信息，这被称为欠拟合（under-fitting）。欠拟合往往有较小的方差和较大的偏差。相反地，如果算法过于复杂或灵活，它将学到很多虚假的“认识”，最终可能“死记硬背”而不是找到真正的规律，发生过拟合。过拟合往往有较小的偏差和较大的方差。

对偏差和方差的分析有助于我们改进机器学习模型，减少误差。对于偏差过大（欠拟合）的情况，我们应该增加模型的灵活性与容量。而对于方差过大（过拟合）的情况，我们应该通过正则化等方式来限制模型的灵活性，另外也可以采用组合模型（将在第 7 章介绍）的方法来减小方差，还有就是增加数据量也能减小方差。不过，减少偏差时，可能会导致方差增加；反之亦然。如何在偏差与方差之间取得最佳平衡，是机器学习中的关键问题，被称为偏差-方差权衡（bias-

variance tradeoff)^q。

这种偏差-方差分解及其解决方法，在生活中也有应用。参见“题外：《思考，快与慢》之后，丹尼尔·卡尼曼的《噪音》之旅”。

思考：在有很多数据时，即使增加模型容量也不会导致过拟合，因此可以同时维持很小的方差与偏差。

题外：《思考，快与慢》之后，丹尼尔·卡尼曼的《噪音》之旅

有两个人打枪，一个人总是打歪，而且歪的角度，歪的程度都差不多。另一个人也打歪，但是歪上歪下都有，偶尔不歪了还能命中一枪。

如果真要让你选择一个，你会选哪一个？

《思考快与慢》的作者丹尼尔卡尼曼，在 87 岁的高龄又出了一本神书《噪声》，在人类的决策和判断领域又填上了一块拼图。

如果说《思考快与慢》说的是人在做决策时容易陷入的各种认知偏误（即机器学习中的偏差），那么《噪声》说的就是人在做判断时候的各种噪声（其实包含了机器学习中的方差）。

错误=偏差+方差+噪声。这是人类决策和判断时的误差公式。

1.3 学习曲线

在构建机器学习模型时，我们希望将误差保持在尽可能低的水平。误差的两个主要来源是偏差和方差。如果我们能够设法减少这两个，那么我们就可以建立更准确的模型。要对误差进行偏差-方差分解以及判断欠拟合/过拟合，除了可以考察图 4.1 所示的误差随模型复杂性的变化情况以外，通常的做法是考察学习曲线。

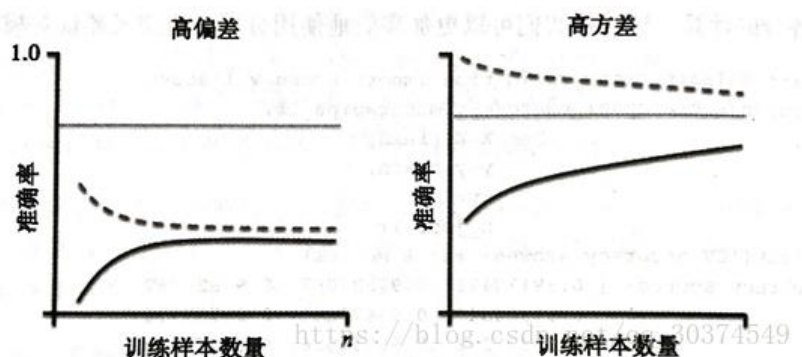


图 4.3. 学习曲线。(a) 欠拟合；(b) 过拟合。虚线是训练误差，实线

^q 方差可通过对多个模型的平均来减小。后面第十章组合模型有具体介绍。

是验证误差，细实线是内在的噪声。

所谓学习曲线，就是训练误差和验证误差（或精度）随训练集数据量的变化曲线（图 4.3）。通常，训练误差随训练集的增加而增加，而验证误差随训练集的增加而减小，而且验证误差一般大于训练误差。如果只有一个训练数据，很容易画一条直线（不使用基函数的线性回归）通过这个数据点，训练误差为 0，但这条直线是围绕单个点（训练集数据）构建的，显然无法准确地“猜测”以前未见过的数据（验证集数据），验证误差很大。随着我们不断增加训练集的大小，模型将不再能够完美地适合越来越多的训练数据点，因此训练误差变大；但是，由于模型接受了更多数据的训练，对数据所遵循的规律有越来越多认识，因此可以更好地描述验证集，验证误差变小。需要注意的是，在这个过程中，验证集数据是保持不变的。

对于欠拟合（图 4.3 (a)），模型过于简单，不需要太多的训练数据就能使拟合结果稳定下来，增加更多训练数据也不会使结果变化。例如，如果只允许画一条直线，那么把训练数据从 100 个增加到 200 个，得到的直线都是差不多的。此时，误差曲线会很快饱和，而且饱和以后训练误差与验证误差之间的差别比较小。对于过拟合（图 4.3 (b)），模型过于灵活，能够很好贴合训练数据，训练误差很小，但验证误差很大，两者之间保持比较大的差距。这个差距会随训练集的增加而减小，但即使使用了已有的所有训练数据也没能使它们的差距变得很小（即还是过拟合）。学习曲线中训练误差与验证误差之间的差距可以作为方差的大致估计。

在理想的情况下，有足够多的训练数据，采用足够灵活的模型（偏差小）也不会过拟合（方差小），偏差与方差都很小，此时误差只剩下没法减少的噪声（图 4.3 中的细直线）。

1.4 K 折交叉验证

将已知数据集分成训练集、验证集与测试集三部分，虽然避免了信息泄露的问题，但却大大减少了可用于模型训练的样本数量，并且得到的结果也受数据集拆分时的随机性所影响。为了减少数据的浪费和拆分数据的随机性，可采用交叉验证（cross-validation）的方法^r。这是一种简单的、应用非常广泛的估计误差的方法。

此时，将已知数据集留下一部分作为测试集，其余的数据用于如下的交叉验证（图 4.4）：将数据划分为 K 份，每次拿出 1 份作为验证集而其余 $K - 1$ 份作为训练集，重复 K 次，最后将得到的性能/误差进行平均作为超参数优化与模型选择的指标。模型选定后，再用所有数据对其重新训练。这也被称为 K 折交叉验证（K-

^r 另外一种有效的方法是 Bootstrap，将在后面第十章中介绍。

fold cross-validation)。常用的 K 值为 5 或 10，此时被称为五折交叉验证与十折交叉验证。在最极端的情况，将 K 值取为数据的总数，即每次取出 1 个数据作为验证集而其余数据作为训练集，此时被称为留一法(Leave one out cross validation)。交叉验证虽然计算代价很高，但是它不会浪费太多的数据。

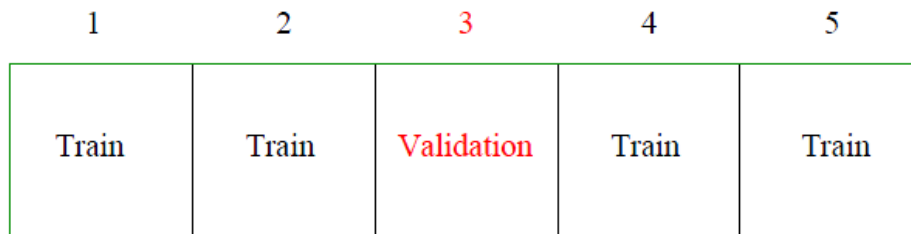


图 4.4. K 折交叉验证示意图。

第 2 节 数据预处理与模型保存

在机器学习中，在拿到原始的数据后，一般不能直接进行训练，而是需要先进行数据的预处理，比如适当的特征标准化、非数值数据的转化、异常值的剔除，等等。而在训练、模型选择与测试评估完成后，需要将得到的模型保存好，以方便下次使用。

2.1 数据标准化

机器学习输入数据（特征）的各个分量可能具有不同的变化范围，甚至是具有不同量纲单位的量。例如，在房价预测问题中，房子面积与房间数目是影响房价的两个特征，但它们的量纲是不同的，而且面积也可以采用不同单位，如平方米或平方英尺，这会导致它们的数值相差很多。这样的情况有时会影响到模型训练的效率与结果。

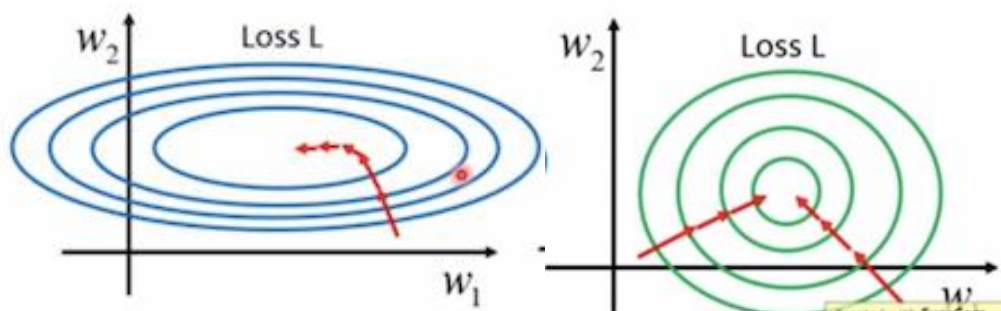


图 4.5. 输入数据的分量变化范围对误差函数 $E(w_1, w_2)$ 的影响。(a) x_1 和 x_2 数值相差过大，例如， x_1 和的取值为 1,2,..., x_2 和的取值为 100,200,...。(b) x_1 和 x_2 数值相差较小，例如， x_1 和的取值为 1,2,..., x_2 和的取值为 1,2,...。图中的椭圆或圆表示 $E(w_1, w_2)$ 的等高线；红色箭头表示 $E(w_1, w_2)$ 的梯度下降方向（与等高线垂直），也表示梯度下降法的迭代过程。

假设我们有一组样本数据，并使用简单的线性回归来拟合数据：

$$f(\mathbf{x}) = w_1x_1 + w_2x_2 + b \quad (4.14)$$

其中 x_1 和 x_2 是样本的两个特征。很显然，在最优解下，如果我们将某个分量 x_1 的数值增大 10 倍（例如，房子面积的单位从平方米改成平方英尺），则其对应最佳权重 w_1 的数值缩小为原来的 1/10，最终模型所给出预测值 $f(\mathbf{x})$ 是不变的。但是，由于在训练完成前我们是不知道最佳权重的，实际权重偏离最佳权重，此时的误差函数随 (w_1, w_2) 的变化关系（敏感程度）以及数值迭代求解的效率是会受分量 x_1 和 x_2 相对大小差异的影响的（图 4.5）。当 x_2 的数值远大于 x_1 时， w_2 的微调会对 f 值产生巨大影响（这也可以从梯度表达式 $\frac{dE(\mathbf{w})}{d\mathbf{w}} = \sum_{n=1}^N (y_n - t_n)\mathbf{x}_n$ 看出，数值大的输入分量所对应的梯度分量也大），误差函数的轮廓（等高线）是扁长的椭圆（图 4.5 (a)），梯度下降方向并不是直接指向 (w_1, w_2) 的最优点（椭圆圆心），迭代求解过程曲折且耗时。当 x_1 和 x_2 的数值相差较小时（图 4.5 (b)），误差函数的轮廓更接近圆形，梯度下降过程更加笔直，收敛速度更快。

因此，为了消除输入分量不同变化范围的不利影响，需要进行数据的标准化（standardization）处理，有时候也称为归一化（normalization）。原始数据经过数据标准化处理后，各分量处于同一数量级，更容易训练，也更适合进行综合对比评价^s。

常见的数据标准化方法有两种。第一种方法是将特征缩放到给定的最小值和最大值之间，通常在 0 和 1 之间，被称为最小-最大缩放（Min-Max scaling）：

$$x'_i = \frac{x_i - \min(\{x_i\})}{\max(\{x_i\}) - \min(\{x_i\})} \quad (4.15)$$

其中 x_i 代表第 i 个数据的指定分量。或者也可以将每个特征的最大绝对值转换至单位大小。第二种方法被称为 Z-score normalization，是通过线性变换使变换后的数据的均值为 0，方差为 1：

$$x'_i = \frac{x_i - \mu(\{x_i\})}{\sigma(\{x_i\})} \quad (4.16)$$

其中 $\mu(\{x_i\})$ 与 $\sigma(\{x_i\})$ 分别是原数据 $\{x_i\}$ 的均值与方差。在回归、分类、聚类算法中，需要使用距离来度量相似性的时候，或者使用 PCA 技术进行降维的时候，第二种方法表现更好。在不涉及距离度量、协方差计算的时候，可以使用第一种方法，比如图像处理中将 RGB 图像转换为灰度图像后将其值限定在[0,255]的范

^s在机器学习中很多算法的目标函数（例如线性模型的 L1 和 L2 正则化）的基础都是假设所有的特征都是零均值并且具有同一阶数上的方差。如果某个特征的方差比其他特征大几个数量级，那么它就会在学习算法中占据主导位置，导致学习器并不能从其他特征中学习。

围。

概率模型一般不需要进行数据标准化，因为它们不关心变量的值，而是关心变量的分布和变量之间的条件概率。决策树方法就属于这种情况。而线性回归、逻辑回归、神经网络、支持向量机等最优化问题一般需要进行数据标准化。

2.2 非数值型（类别）数据的编码

数据按其属性可大致分成三大类：（1）标称属性（nominal attribute），又称分类属性或类别属性（categorical attribute），其值是一些符号或实物的名称，每个值代表某种类别或状态，这些值不必具有次序的含义，也不是定量的。例如，{男、女}，{技术岗、管理岗、销售岗、售后岗}，{对、错}。二元属性（binary attribute）属于标称属性，只有两个类别或状态，常用 0 与 1 来表示。（2）序数属性（ordinal attribute），可能的取值之间具有顺序的含义，但相邻值之间的差别大小是未知的。例如，学生的成绩属性可以分为{优、良、中、差}四个等级，餐饮店的饮料杯通常具有{大、中、小}或{超大、大、中}三个值，但具体“大”比“中”大多少是未知的。（3）数值属性（numerical attribute），是一种定量的属性，给出了可度量的大小或数量，它可以用整数或实数值表示。

在机器学习中，一般不太区分标称属性和序数属性的数据，而且不太严格地把它们看做是“类别”数据或“取值分立/离散”的数据，而把数值属性的数据看做“取值连续”的数据。例如，监督学习的分类与回归就是这么区分的。

在机器学习方法中，例如我们前面介绍过的线性回归与逻辑回归，一般只能处理以“数”的形式出现的数据 $\{\mathbf{x}_n, y_n\}$ ，但不能处理“文字”“名称”等形式的数据。因此，在预处理中，需要把标称属性和序数属性的数据转换（编码）成数值，通常是整数。

对于具有序数属性的数据，由于数据本身具有顺序意义，可采用普通数值转换的方法按顺序将它们编码成连续的整数就可以，例如{优、良、中、差} $\rightarrow\{0,1,2,3\}$ 或{3,2,1,0}，以及{大、中、小} $\rightarrow\{3,2,1\}$ 。

对于具有标称属性的数据，有时候为了编码紧凑也会采用类似序数属性的普通数值转换方法，例如{男、女} $\rightarrow\{0,1\}$ ，{技术岗、管理岗、销售岗、售后岗} $\rightarrow\{0,1,2,3\}$ 。但当有多个（两个以上）的状态取值时，会让机器学习模型误以为状态之间具有类似序数属性的内在联系，效果可能不好。效果更好的方法是采用独热编码（one-hot encoding），把具有 K 个可能取值的类别数据编码成 K 个 0-1 分量，且只能有一个分量为 1。如果数据的取值是第 i 个状态，则第 i 个分量为 1，其它分量为 0。例如，{男、女}编码成 2 分量形式的 $\left\{\begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix}\right\}$ ，{技术岗、管理

岗、销售岗、售后岗}则可编码成 4 分量形式的 $\left\{ \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \right\}$ 。独热编码的

模型容量大，但忽略了分量之间的可能内在联系。

在一般情况下，数据的属性情况可能比较复杂，不是纯粹的相互之间没有联系的类别属性，但其内在联系又不是很容易表示成序数属性。例如，{铁、铝、铜、锌、钙}，它们之间其实有很多内在联系，但并不容易采用普通数值转换方法表示成单分量整数。

2.3 异常值检测

离群点或异常值（outlier）是指与其它数据点相距甚远的异常点，与其它数据点并不服从同一分布。例如，对于一维数据{20, 3, 1, 27, 26, 6, 22, 19, 4200, 3, 24, 27, 20, 15, 28}，其中的 4200 显然是个异常点。而对于图 4.6 所示的二维例子，也可以轻易看出其中的异常点。

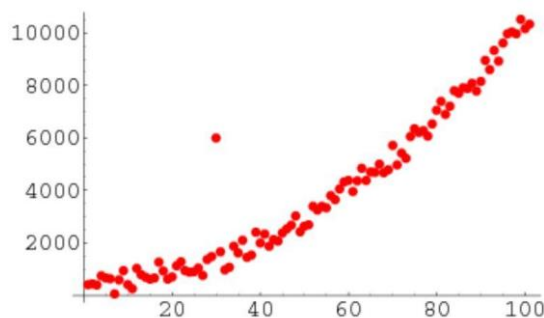


图 4.6. 离群点/异常值示意图。

我们关心异常值的检测，有两方面的原因。在一方面，异常值可能是错误的。比如，人工抄写数据时抄错了^t，或者仪器产生与传输实验数据时出现了未知的错误。此时，这些错误数据应该被丢弃（或校正——如果可能的话），否则会对机器学习的结果产生负面影响。例如，对图 4.6 中的数据进行拟合（训练）时很明显不应该包含那个离群点。这是在数据预处理时需要关注的。在另一方面，这些异常值可能是真实的，只是其产生原因与其它点不同而已。例如，现在可以通过智能手表和手环每几分钟检测一次心率，发生心脏疾病的数据相比于占绝大多数比例的正常数据就是异常值或离群点。此时这些异常数据是很有意义的，甚至是我们进行机器学习的目标所在。

^t 意大利作家/化学家普里莫·莱维在小说《元素周期表》记载了一则趣事：某化工厂使用多年的检测方法中有一个步骤，“滴入盐酸 23 滴”，但真正的步骤是“滴入盐酸 2~3 滴”——在多次的技术文档整理过程中，有一次中间的“~”被抄漏了。

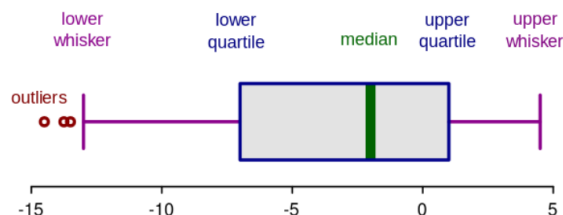


图 4.7. 箱形图示意图。

异常值检测有多种方法。当数据是一维时，辨别离群点相对比较容易，常用的方法有均方差法（Z-score 方法）与箱形图法。在统计学中，如果数据集服从高斯分布，那么大约 68% 的数据值会在均值的一个标准差范围内，大约 95% 会在两个标准差范围内，大约 99.7% 会在三个标准差范围内。因此，如果你有任何数据点超过标准差的 3 倍，那么这些点很可能是异常值或离群点。这可以通过计算如下的 Z-score 来判断：

$$z_i = \frac{x_i - \mu(\{x_i\})}{\sigma(\{x_i\})} \quad (4.17)$$

如果 Z-score 超过某一临界值（常用有 2.5, 3.0, 3.5 等），就预测为异常值。而箱形图法则进一步考虑到数据集不一定满足高斯分布，改用箱形图（Box-plot）来衡量数据分散情况以及可能的离群点。箱线图的绘制方法是（图 4.7）：先找出一组数据的两个四分位数（下四分位数 Q_1 与上四分位数 Q_3 ，即把数据集从小到大排列时在排在 1/4 和 3/4 位置的数），将其作为两端边的位置画一个矩形盒；中位数 X_m 位置用粗线画在箱体中间；在 $Q_3 + 1.5IQR$ 和 $Q_3 - 1.5IQR$ 处画出上下触须（whisker）并与箱体相连接，其中四分位距 $IQR = Q_3 - Q_1$ ；处于上下触须范围之外的数据点被认为是离群点或异常值，用小圆圈画出。箱形图在识别异常值方面比 Z-score 方法更有优越性。

当数据是高维的，异常值与离群点的检测则更为困难，需使用专门的机器学习算法来实现，例如 DBScan 聚类、孤立森林等。这里不再介绍。

2.4 训练模型的保存与载入

在机器学习实践中，经常需要对模型进行保存与重新载入。例如，在做模型训练的时候，尤其是在做交叉验证时，通常想要将模型保存下来，然后放到独立的测试集上测试。而训练好的模型，也需要保存下来，在以后需要做预测应用时再重新载入，或通过网络发送出去供其他人使用。

Python 中训练模型的保存和载入主要有两种方法。第一种是使用通用的 pickle 模块。它可以将 Python 程序中的对象序列化成字符串并保存到磁盘上的文件里，以及将保存的文件或字符串反序列化为原来的对象。另一种方法是使用 Python 的常用机器学习工具库 scikit-learn（简称 sklearn）中自带的模块 joblib，

它针对机器学习做了很多的优化。对于大数据而言，`joblib` 比 `pickle` 更加高效，但是 `joblib` 只能将对象存储在磁盘文件中，不能转化为字符串。

上述方法只能在 `Python` 内部实现模型的保存与载入，即需要相同的软件框架环境。但有时候我们希望能够在不同的机器学习框架与软件之间实现迁移。例如，`sklearn` 主要适合中小型的机器学习项目，而很多优秀的视觉模型是用 `Caffe` 写的，很多新的研究论文则是用 `PyTorch` 写的，另外还有很多模型用 `TensorFlow` 写成。怎样将一个框架下的模型迁移到另外一个框架下呢？这就需要一些框架间的通用格式。

`Open Neural Network Exchange`（ONNX，开放神经网络交换）就是一种用于表示深度学习模型的标准，可使基于神经网络的模型在不同框架之间进行迁移。ONNX 定义了一种可扩展的计算图模型、一系列内置的运算单元（OP）和标准数据类型。每一个计算流图都定义为由节点组成的列表，并构建有向无环图。其中每一个节点都有一个或多个输入与输出，每一个节点称之为一个 OP。这相当于一种通用的计算图，不同深度学习框架构建的计算图都能转化为它。

`Predictive Model Markup Language`（PMML，预测模型标记语言）则利用 XML 描述和存储机器学习模型，是一个已经被 W3C 所接受的标准。PMML 支持回归模型、聚类模型、决策树模型、神经网络模型、朴素贝叶斯模型等。PMML 是跨平台的利器，但它为了满足跨平台，也牺牲了很多平台独有的优化。例如，PMML 模型文件往往比使用平台自有格式保存的文件大很多，加载也慢得多，另外转化后的模型有时候会有一些小的偏差。

第 3 节 数据不平衡及分类模型的评价指标

3.1 不平衡数据

顾名思义，不平衡数据指的是数据集中不同类别的样本数量差别巨大。例如，在上一章介绍过的点击率预测的例子中，网页上展示的各项内容往往只有很小的比例会被点击。其它例子包括电子商务领域的商品推荐（推荐的商品被购买的比例很低）、信用卡欺诈检测（发生欺诈的比例很低，否则用户就不爱用信用卡了，银行也不愿意发行信用卡了）、疾病筛查（新冠筛查中，大部分数据都是阴性的，远多于阳性数据）、网络攻击识别、恐怖分子识别，等等。

数据不平衡给机器学习模型的训练与评价增加了额外的难度。举一个虚拟的例子。有个学生毕业后进了一家生产企业，老板让他发展一个机器学习模型来根据预测产品是否有缺陷。这个学生利用逻辑回归在数据上进行训练后，准确率达到了 96.2%。老板很高兴，在生产线上布置了这个模型。几个星期后，老板生气地冲进入办公室，拍着桌子说这个模型完全没用，一个有缺陷的产品都没发现。

这怎么可能呢？经过一番调查，发现企业的产品中大约有 3.8% 的存在缺陷，而机器学习模型总是回答“没有缺陷”，因此准确率达到 96.2%。也就是说，准确率确实高达 96.2%，但另一方面，这个模型也确实没什么用！这就是对不平衡数据直接套用通常的机器学习训练方法所得到的结果。为了从不平衡数据中学习到有用的信息，我们必须使用一些有针对性的方法。

3.2 评价指标

在分类问题中，混淆矩阵(confusion matrix)很好地概述了模型的运行情况，可作为任何分类模型评估的基础与起点。混淆矩阵用 n 行 n 列的矩阵形式(n 是类别的总数)来表示，其中第 i 行第 j 列的矩阵元等于实际归属第 i 类而被预测为第 j 类的数据点的个数(样本的数量)。例如，在图 4.8 (a) 例子中，矩阵元“43”说明有 43 个属于第一类的样本被正确预测为第一类，而“5”说明有 5 个属于第二类的样本被错误预测为了第一类。因此，在混淆矩阵中，对角线是预测对的样本数，非对角是预测错的样本数。每一行之和表示该类别的真实样本数量，而每一列之和表示被预测为该类别的样本数量。

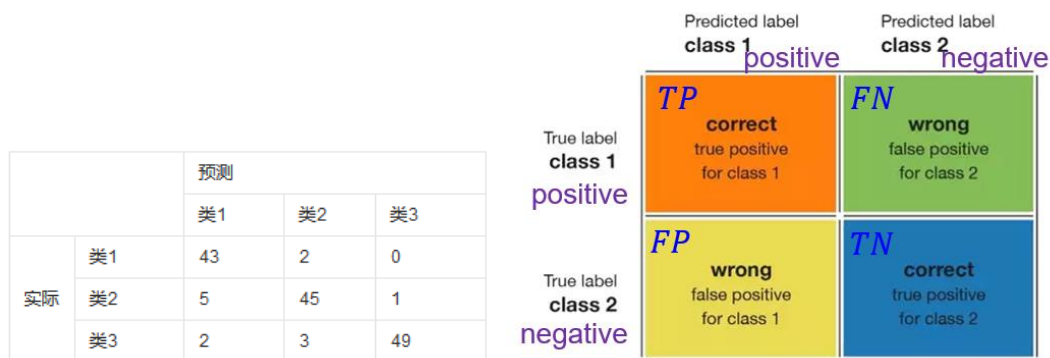


图 4.8. 混淆矩阵。(a) 一个三分类的例子。(b) “阳性-阴性”二分类下混淆矩阵的矩阵元名称。 P 与 N 分别代表预测的阳性(Positive)与阴性(Negative)， T 与 F 分别代表预测是正确的(True)与错误的(False)。

在医学与生物等应用场合中，经常用阳性与阴性来代表二分类下的两个类别，例如在检查结果中某种性质的存在与否(微生物感染、抗体反应、结构异常，等等)。此时，混淆矩阵的四个矩阵元分别代表“真阳性” TP (True Positive)(预测为阳性且预测对了)、“假阳性” FP (False Positive)(预测为阳性但预测错了)、“真阴性” TN (True Negative)、“假阴性” FN (False Negative)的样本数，参见图 4.8 (b)。需要特别指出的是，这里用词里的“阳性”与“阴性”是表示预测的类别标签而不是实际的类别标签。

基于混淆矩阵，就可以进一步计算其它各种评价指标，例如：

- 准确率 (accuracy): $\frac{TP+TN}{TP+FP+TN+FN}$ 。它是在所有的预测结果中正确预测所占的比例, 表征了一个分类器的区分能力。

需要注意的是, 这里的区分能力没有偏向于是正例还是负例, 这也是准确率作为性能指标最大的问题所在。例如, 在前面所举的产品缺陷的例子中, 虽然准确率达到了 96.2%, 但所有预测正确的例子都发生在无缺陷类中, 在更重要的缺陷类中预测都是错误的。又比如, 考虑一个地震的预测模型器, 0 表示没有地震, 1 表示地震; 由于地震概率非常小 (假设是 10^{-6}), 因此, 只要将所有情况都预测为 0, 就能获得高达 0.999999 的准确率, 看起来是很好的分类器, 但其实不然。对于地震, 我们最关心的是能否把所有的正例全部预测出来, 至于在没有地震时我们预测为 1, 只要在可接受范围内, 那么, 这就是一个好的预测模型。

从这两个例子可以看出, 对于数据不平衡或是当某类数据预测错误时会产生很大代价的时候, 我们需要更有效的指标作为补充。

- (类别) 精度 (precision, 又称查准率)^u: $\frac{TP}{TP+FP}$ 。它代表的是: 在所有被预测为某一类的样本中, 真正是这一类的比例。它表征了当模型预测一个点属于该类, 预测结果的可信程度。

这个指标常常被应用于推荐系统中, 例如, “网易云音乐每日推荐的 20 首歌曲中, 我确实喜欢的歌曲所占的比例有多高”。此时, “网易云音乐没有推荐的歌曲, 我是不是确实不喜欢” 并不太重要。

- (类别) 召回率 (recall, 又称查全率)^v: $\frac{TP}{TP+FN}$ 。它代表的是: 在所有实际属于某一类的样本中, 被正确预测为这一类的样本比例。简单说就是 “这一类总共有这么多, 你预测出了多少? ”。它所表征的是表示模型能够检测到该类的比率。

正样本 (实际是阳性的样本, 也称阳性样本) 的召回率在医学上常常被称为敏感度 (Sensitivity)。从 1 减去敏感度, 即 $\frac{FN}{TP+FN}$, 在医学上被称为漏诊率。

漏诊可能导致病人没有及时治疗, 代价非常高, 因此医学上把敏感度或漏诊率作为评价的主要指标。前面所举的产品缺陷与地震预测的例子也是类似的, 注重缺陷类或地震发生类的召回率。

^u 此处假设类别是阳性。对于阴性类别, 精度的表达式则为 $\frac{TN}{TN+FN}$ 。

^v 此处假设类别是阳性。对于阴性类别, 召回率的表达式则为 $\frac{TN}{TN+FP}$ 。

- Specificity(特异性): $\frac{TN}{FP+TN}$ 。即负样本的召回率。从 1 减去特异性, 即 $\frac{FP}{FP+TN}$,

在医学上被称为误诊率。它也是医疗中需要考虑的指标。

举个极端例子, 一个预测模型把所有的样本都预测成患病, 此时敏感度为 1, 没有任何漏诊, 但是特异性却很低, 误诊率实在太高, 这也是不行的。因此, 在医学领域, 特异性和敏感度是需要同时考量的 (虽然前者相对更重要)。

- (类别) F_1 分数: $F_1 = 2 \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$ 。即精度和召回率的调和平均值。它能够将一个类的精度和召回率结合在同一个指标当中。

精度和召回率的不同组合, 就会给出对于一个给定类别的不同预测效果:

(1) 高精度+高召回率: 模型能够很好地检测该类。这相当于, 对“好学生”这个类别, 同时实现了“说你好, 你确实好”与“你好, 肯定说你好”。

(2) 高精度+低召回率: 模型不能很好地检测该类, 但是在它检测到这个类时, 判断结果是高度可信的。

(3) 低精度+高召回率: 模型能够很好地检测该类, 但检测结果中也包含其他类的点。

(4) 低精度+低召回率: 模型不能很好地检测该类。

以前面讨论过的产品缺陷检测为例, 假设模型对 10000 个产品 (样本) 的混淆矩阵如图 4.9 所示。则准确率为 $\frac{9630+0}{9630+0+370+0} = 96.3\%$; 无缺陷类的精度为 $\frac{9630}{9630+370} =$

96.3%, 召回率为 $\frac{9630}{9630+0} = 100\%$ (这很好, 所有无缺陷的产品都会被检测出来);

有缺陷类的精度为 $\frac{0}{0+0} = ?$, 不可计算, 召回率是 $\frac{0}{370+0} = 0$ (这很糟糕, 所有有缺陷的产品都没有检测出来)。因此, 这个模型对有缺陷类是效果很差的。

	Predicted label not defective	Predicted label defective
True label not defective	9630	0
True label defective	370	0

图 4.9. 产品缺陷检测例子的混淆矩阵。

3.3 ROC 曲线与 AUC 指标

另外一种判别分类器好坏程度的方法是考察 ROC 曲线 (Receiver Operating

Characteristic curve, 受试者特征曲线)。它的定义和给定类别 C 相关(为了方便理解, 以下假设 C 是阳性)。

基于模型(例如逻辑回归)输出的类别概率 $P(C|\mathbf{x})$, 我们可以设置决策阈值 P_{cri} (不一定为 0.5), 即当且仅当 $P(C|\mathbf{x}) \geq P_{\text{cri}}$ 时, 才判定(预测) $\mathbf{x}$ 属于类别 C 。如果 $P_{\text{cri}} = 1$, 则仅当模型 100%可信时, 才将该点预测为类别 C ; 而如果 $P_{\text{cri}} = 0$, 则每个点不管其 $\mathbf{x}$ 值如何都被预测为类别 C 。通过调整决策阈值 P_{cri} , 就可以获得不同的性能组合结果。例如, 如果 $P_{\text{cri}} = 1$, 则类别 C 的召回率为 100%, 漏诊率为 0, 但误诊率为 100%, 即其它类别的召回率为 0; 而如果 $P_{\text{cri}} = 0$, 则误诊率为 0, 漏诊率为 100%。所谓的 ROC 曲线, 就是当 P_{cri} 从 1 变化到 0 时, 以正样本的真阳性率($\frac{TP}{TP+FN}$)为 y 值(纵轴), 负样本的假阳性率($\frac{FP}{FP+TN}$)为 x 值(横轴), 所画出来的曲线(图 4.10)。其中负样本的假阳性率(横轴)代表的是负样本中没有正确查出来的概率, 形象地说就是“总共有这么多 0, 分类器没有查出来多少”, 而没有查出来的, 自然就都被错误分类为 1 了, 那么这些 0 就被误诊了, 即横轴代表了“误诊率”, 或虚报概率(参考“题外: ROC 曲线的起源与水果忍者”)。它在数值上等于 1 减去特异性。

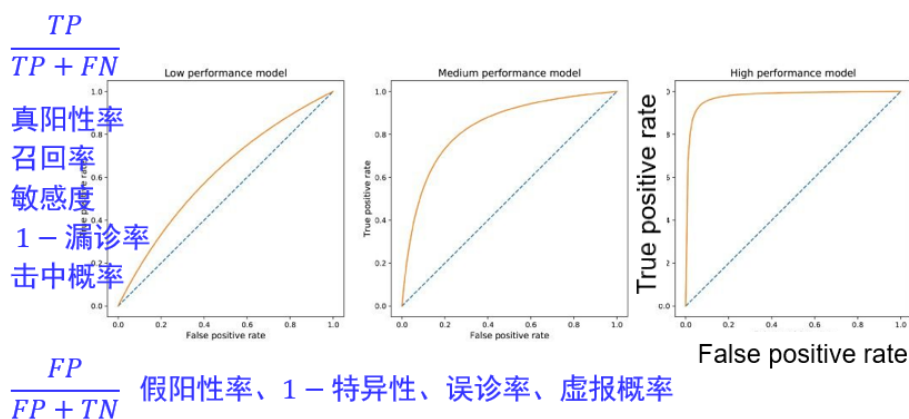


图 4.10. ROC 曲线示意图。横轴代表。纵轴是负样本的假阳性率, 等价于误诊率、1 减去特异性、虚报概率。纵轴是正样本的真阳性率, 等价于召回率、敏感度、1-漏诊率、击中概率。左侧模型性能很差, 必须牺牲很多精度才能获得高召回率; 右侧模型非常好, 可以在保持高精度的同时达到高召回率; 中间的模型效果介乎两者之间。对角的虚线代表随机猜测的结果, 即以概率 P_{cri} 随机猜测样本属于阳性。

ROC 曲线从点(0,0)开始, 在点(1,1)处结束, 而且单调增加。好模型的 ROC 曲线会快速从 0 增加到 1。这意味着只需要牺牲一点特异性就能获得高召回率(敏感性)。随机猜测的模型, 即对于任一样本, 以概率 P_{cri} 预测其是阳性, 以概率 $1 - P_{\text{cri}}$ 预测其是阴性, 则所得到的 ROC 曲线是沿对角方向的直线(图 4.10 中虚线所示)。通常模型的 ROC 曲线在对角线的上方。

ROC 曲线的应用场景有很多。根据上述的定义，其最直观的应用就是能反映模型在选取不同阈值的时候其召回率（敏感度）和其特异性（误诊率）的变化。ROC 曲线的横轴与纵轴取值是分别对正负样本计算的，因此当正负样本的比例发生变化时（例如为了提高训练效果采用上采样或下采样的方法，从而改变了正负样本的比例），ROC 曲线能够基本保持不变，因此降低不同测试集带来的干扰，更加客观地衡量模型本身的性能。这是一个很大的优点。

基于 ROC 曲线，可以进一步构建另一个常用的评价指标：AUC (Area Under the Curve)，即 ROC 曲线下的面积。AUC 在最佳情况下（正负样本在模型下完全可分，100%准确）将趋近于 1.0，而在最坏（随机猜测）情况下将趋向于 0.5。通常 AUC 的取值在 0.5~1 之间，不会小于 0.5：如果 AUC 小于 0.5，则只需要把原来的预测颠倒一下，把原来预测为阳性（阴性）的重新预测为阴性（阳性），则 AUC 等于 1 减去原来的 AUC，将大于 0.5。AUC 的值越大，说明该模型的性能越好。AUC 还与曲线的与基尼系数相关^w。

题外：ROC 曲线的起源与水果忍者

ROC 曲线起源于二战时期的雷达信号判断问题。当时雷达技术还没有那么先进，存在很多噪声（比如一只大鸟飞过），所以每当有信号出现在雷达屏幕上的时候，雷达兵的任务就是去解析雷达的信号。有的雷达兵比较谨慎，凡是有信号过来，他都会倾向于解析成是敌军轰炸机；有的雷达兵又比较疏忽，会倾向于解析成飞鸟。这个时候，雷达兵的上司就纠结了，需要一套评估指标来汇总每一个雷达兵的预测信息，以及来评估这台雷达的可靠性。于是，最早的 ROC 曲线分析方法就诞生了：每个雷达兵有不同的触发阈值，以其击中概率与虚报概率为坐标可画出图中的一个点，不同雷达兵的点就构成一条 ROC 曲线，可作为评估雷达可靠性的依据。后来，ROC 曲线就被推广运用于医学以及机器学习领域。例如，放射科医生也面临着在复杂背景下识别病变组织的任务。

在手机游戏《水果忍者》中，玩家需要切开飞来的水果并躲避炸弹，因此也面临类似的内在制约局面：想多切水果（高召回率），就必须冒错误切到炸弹（假阳性、误诊）的危险；想谨慎躲过所有炸弹，就不得不漏过一些水果不去切。

AUC 是分类模型（特别是二分类模型）使用的主要评测指标之一。除了上述的“曲线下面积”的解释，还可以从排序能力角度来理解 AUC。例如 $AUC=0.7$

^w 基尼系数 (Gini coefficient) 是指曲线与对角线（绝对公平线）所围成的面积与对角线以下面积的比值，数值上等于 $2 \times AUC - 1$ 。在决策树算法中，使用基尼系数来描述信息增益比，代表了模型的不纯度，基尼系数越小，则不纯度越低，特征越好。

可以大概理解为：给定一个正样本和一个负样本，在 70% 的情况下，模型对正样本的打分高于对负样本的打分。在这个解释下，我们关心的正负样本之间的分数高低，而具体的分值则无关紧要。因此，从本质上讲，AUC 不关注具体得分，只关注排序结果，这使得它特别适用于排序问题的效果评估，例如推荐排序的评估。

最后需要指出的是，数据极端不平衡时，ROC 与 AUC 可能不是好的评价方法：ROC 曲线基本不随正负样本的比例而变化，因此很容易因为 AUC 值高而忽略算法实际对少数类样本的效果不理想的情况。此时建议使用 PR (Precision-Recall) 曲线，以了解算法对于数据的敏感程度，以及帮助确定最优的决策阈值。

对分类模型评价指标的认识，还有助于提高对新闻报道内容的理解与分辨能力，参见“亚马逊 AI 人脸识别系统遭质疑”与“新冠病毒核酸检测”。

题外：美国参众两院议员中有 28 名罪犯？亚马逊 AI 人脸识别系统遭质疑

当出现新的技术进步时，我们需要有所认识，以分辨到底什么是真的，什么不是。

2018 年，美国某民权组织使用亚马逊的人脸识别系统，对照 25,000 张公开的罪犯照片扫描了所有 535 名美国国会议员的脸部照片，结果产生了 28 个错误匹配，并以“美国参众两院议员中有 28 名罪犯？”为题进行了新闻炒作。

然而，根据正文中的介绍，我们知道模型的评价指标与决策阈值有关，可以通过调整阈值来匹配特定的应用场合。亚马逊在回应中也指出了这一点：“人脸识别 API 的默认置信阈值为 80%，这对很多通用案例来说很实用（如在社交媒体上识别名人或者在照片应用中识别长相相似的家庭成员），但这并不适用于公共安全领域。在需要高度精确的面部相似性匹配案例中，我们建议使用 99% 的置信阈值。”

另外，需要指出的是，几乎所有的算法都会有误差，不能做到 100% 准确。而且，现在的机器学习算法实际上比人工进行人脸-照片匹配的准确率更高。在现实世界的公共安全和执法场景中，人脸识别系统可以用来帮助缩小范围，后续还需要其它环节的进一步确认，并不是完全由计算机决定。这些用途包括寻找走失儿童、打击人口贩卖或预防犯罪。但在其它应用案例——如社交媒体中，置信阈值稍微低一点也没关系。

题外：新冠病毒核酸检测：不要无脑炒作

2020 年，瑞典方面曾经通过多家西方媒体炒作说中国公司的新冠病毒核酸检测试剂“不准确”，导致该国 9 个地区的受检者中出现了 3700 个“假阳性”结果。

然而，《环球时报》通过分析后发现，瑞典方面其实是在误导普通民众，而科学真相则是：在疫情期间，最需要的是找出那些真正的“阳性”病例，即追求高召回率，所以就得调低阈值。中国的检测试剂非常灵敏，不会错过人群中的任何病例；这会不可避免导致一些假阳性结果，但这是在衡量“漏检新冠病人”与“误检健康人”的不同严重后果后采取的理性选择。

而且，根据日本传染病学会的评估，当时国际上新冠抗体检测试剂盒的假阳性概率大约在 2%-11% 之间。试剂盒并不是确诊的唯一依据，还需要后续的一系列检查。

3.4 评价指标的使用

上述所介绍的性能指标（准确率、召回率、敏感度，等等）往往是我们发展机器学习模型的最终目标。但是这些指标通常是不可微分的，不容易计算导数，无法直接作为模型训练时的损失函数。因此在训练阶段通常采用二乘误差、交叉熵等可微的损失函数来作为模型训练阶段的直接目标，并期望在这些损失函数降低的时候，能够提高相关的性能指标。而在模型的选择与测试评估的阶段中，则可直接采用这些不可微的性能指标作为依据和标准。

第 4 节 数据不平衡下的训练方法

4.1 不平衡例子与理论最小误差

我们先考虑一个非常简单的数据不平衡的例子。假设我们有两个类： C_0 和 C_1 ，其中 C_0 的数据点服从均值为 0、方差为 4 的一维高斯分布，即 $P(x|C_0) = \mathcal{N}(x|0, 2^2)$ ； C_1 的数据点服从均值为 2、方差为 1 的高斯分布，即 $P(x|C_1) = \mathcal{N}(x|2, 1^2)$ 。再假设数据集中 90% 的点来自 C_0 ，其余 10% 来自 C_1 。则总体数据中两类的分布如图 4.11(a) 所示，可以看到 C_0 的曲线总是在 C_1 曲线之上，因此对于任意给定点 x ，它来自 C_0 类的概率总是大于出自 C_1 类的概率。用贝叶斯公式来表示，即：

$$P(C_0|x) = \frac{P(x|C_0)P(C_0)}{P(x)} > \frac{P(x|C_1)P(C_1)}{P(x)} = P(C_1|x) \quad (4.18)$$

此时，从理论上讲，只有当分类器每次都把数据预测为 C_0 类时准确率才会最大。或者说，如果目标就是获得最大准确率，那么确实应该总是把数据点预测为 C_0 类。

这里体现了一个很重要的角度，即从生成模型的角度来理解机器学习。它是考虑 $(\mathbf{x}, y)$ 数据是怎样产生的，而不只是考虑在 $\mathbf{x}$ 固定的条件下 y 的分布如何。

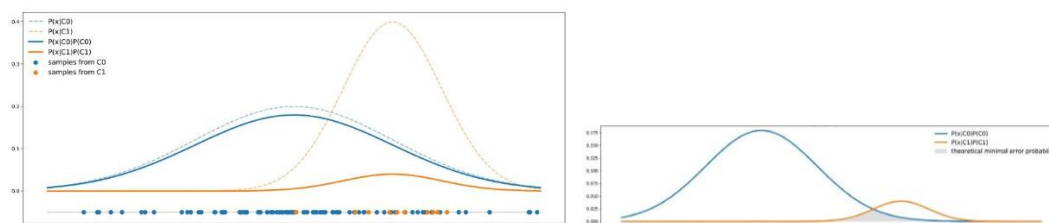


图 4.11. 总体数据中两类的分布（蓝色代表 C_0 ，红色代表 C_1 ）。(a)

$P(x|C_0) = \mathcal{N}(x|0, 2^2)$, $P(x|C_1) = \mathcal{N}(x|2, 1^2)$, $P(C_0) = 0.9$,
 $P(C_1) = 0.1$ 。(b) ???。

另外，从这个简单例子中，我们还可以理解为什么分类器具有理论意义上的最小误差概率。对于任一 $\mathbf{x}$ ，理论上最好的分类器将从两个类中选择概率值 $P(C|\mathbf{x})$ 最大的类别 C 作为预测结果，此时概率值较小的类所对应的数据点自然就会预测错误。例如，如果 $P(C_0|x) > P(C_1|x)$ ，那么会预测为 C_0 类， (x, C_0) 的数据点就预测对了，而 (x, C_1) 的数据点就预测错了。因此，对于图 4.11，理论最小误差概率就是由两条曲线最小值下的面积给出的。这个概率通常不为零，也就是说预测很多情况下都不可能百分百准确。

4.2 重新处理数据集：过采样、欠采样、生成合成数据

面对不平衡数据集，如果没办法通过实验直接收集更多的少数类样本，最传统的思路是重新处理现有数据集的方法，如过采样、欠采样与生成合成数据等，以重新平衡数据集。

过采样（oversampling，又称上采样，或过抽样、上抽样），就是通过复制一些少数类样本，以增加其数目。过采样的缺点是数据集中会反复出现一些样本，训练出来的模型会有过拟合的风险。因此，可以在每次复制数据点时加入轻微的随机扰动，以减少过拟合。

欠采样（undersampling，又称下采样，或欠抽样、下抽样），是删除部分多数类样本，或者说，从多数类样本中进行再抽取，仅保留这些样本的一部分。欠采样的缺点是最终的训练集丢失了不分数据，模型可能只学到了数据中隐藏规律的一部分。为减少信息的损失，可采用模型融合的方法（如 EasyEnsemble），通过有放回采样产生多个不同的训练集，进而训练多个不同的分类器，通过组合多个

分类器的结果得到最终的结果^x；或利用增量训练的思想（Boosting），先通过一次下采样产生训练集，训练一个分类器，对于那些分类正确的多数类样本不放回，然后对这个缩小了的多数类再次进行下采样产生训练集，训练第二个分类器，以此类推，最终组合所有分类器的结果得到最终结果，典型方法如 BalanceCascade。另外，还可以利用 K 近邻算法（KNN）^y 挑选那些最具代表性的多数类样本，例如 NearMiss 方法。

生成合成数据（人工样本），则是根据少数类样本创建新的合成数据点，以增加其数目。例如，数据增强算法 SMOTE（Synthetic Minority Over-sampling）利用少数类样本在特征空间的相似性来生成新样本：对于少数类样本 $\mathbf{x}_i$ ，从它属于少数类的 K 近邻中随机选取一个样本点 $\mathbf{x}'$ ，生成一个新的少数类样本 $\mathbf{x}_{\text{new}} = \mathbf{x}_i + \xi(\mathbf{x}' - \mathbf{x}_i)$ ，其中 $\xi \in [0,1]$ 是随机数。

需要指出的是，以上这些方法的直接目的是重新平衡（部分或全部）数据集，它们背后隐含的可能依据有两个。其中一个认为是认为不平衡数据并没有反映现实，或者说，实际数据其实是平衡的，但由于我们采集数据时的问题造成两类数据比例相差过大。另外一个认为是认为虽然现实的数据确实是不平衡，但重要性并不是简单地与样本数目成正比，少数类的重要性在总体上并不亚于多数类，例如前面举过的产品缺陷检测的例子。在这两种情况下，我们必须重新平衡数据集，才能通过训练得到有用的模型。

相反地，如果不平衡数据是真实的，单个的少数类样本也并没有比单个的多数类样本更重要，则需要谨慎使用这些改变类别样本比例的方法：使用重采样方法时，我们其实是在训练过程中向分类器传递了两个类的错误比例信息，“扭曲”了事实，这可能会误导机器学习模型，导致学得分类器在未来实际应用的表现甚至比未改变数据集的分类器更差。实际上，类的真实比例对于分类新的样本点非常重要，而这一信息在重新采样数据集时被丢失了。

另外，数据不平衡并不代表两个类无法很好地分离。例如，在图 4.11（b）中，虽然总体上 90% 的数据点来自 C_0 ，来自 C_1 的数据只占 10%，但 C_0 曲线并不总是高于 C_1 曲线，因此有些点出自 C_1 类的概率就会高于出自 C_0 的概率。在这种情况下，两个类分离得足够开，足以补偿不平衡，不需要重采样，直接对不平衡数据进行训练也能得到令人满意的结果。如果两个类是不平衡、在现有特征下是不可分离的，还可以尝试添加新的特征使两个类变得可分离。与重采样的方法相比，这相当于是使用更多来自现实的信息丰富数据，而不是改变数据的现实性。

^x 参考第十章 组合模型。

^y 参考第 7 章 核方法

4.3 重新考虑训练与验证目标

面对不平衡数据集，另外一种思路是不改变数据集，但重新考虑训练的目标或损失函数。

在通常的机器学习模型中，其实是假设假阳性与假阴性具有相同的成本（cost），即错误的代价是对称的。然而实际情况往往不是这样。比如，在产品缺陷预测例子中，将有缺陷的产品预测为无缺陷所导致的代价/成本（损失）远大于将无缺陷产品预测为有缺陷；在癌症检测中，将癌症样本检测为正常所导致的代价也远大于将正常样本检测为癌症。在这些例子中，错误的代价是不对称的。此时，不能再以最佳准确率为目标，而是应该追求最低的错误预测总代价（成本）。

	cancer	normal
cancer	0	1000
normal	1	0

图 4.12. 癌症检测的代价。

将类别为第 k 类的样本预测为第 k' 类时的代价（损失）记为 $L_{kk'}$ 。一个癌症检测的例子如图 4.12 所示。将模型把样本 $\mathbf{x}$ 预测为第 k' 类的概率记为 $P_{k'}(\mathbf{x})$ 。则对于能够输出预测类别概率的模型，可将训练的目标函数设置为期望预测成本（代价）：

$$\sum_{n,k,k'} L_{kk'} P_{k'}(\mathbf{x}_n) \quad (4.19)$$

对于一些其它模型（例如贝叶斯分类器），可以使用重采样方法来偏置类的比例，以便通过类比例的调整来反映预测的成本误差信息。例如，在图 4.12 的例子中， $L_{01} > L_{10}$ ，我们可以对少数类按照 L_{01}/L_{10} 比例进行过采样，或者对多数类按照 L_{10}/L_{01} 的比例进行欠采样。这是一种类重新加权的思路，即在分类器训练期间直接考虑成本误差的不对称性，这使每个类的输出概率都嵌入成本误差信息。然后这个概率后续将用于定义具有 0.5 阈值的分类规则。

另外一种思路是在正常训练后再考虑成本的影响，也即按照基本的方法训练分类器，计算每一类别的预测概率，然后根据误差成本来调整分类预测的阈值。在图 4.12 的例子中，如果满足

$$L_{01}P_1(\mathbf{x}_n) < L_{10}P_0(\mathbf{x}_n) \quad (4.20)$$

则预测 $\mathbf{x}_n$ 所属的类为 C_0 ，否则预测为 C_1 。在一般情况下，则预测 $\mathbf{x}_n$ 的类别为 $\arg \min_{k'} \sum_k L_{kk'} P_{k'}(\mathbf{x}_n)$ 。

4.4 其它方法：一分类

对于不平衡数据集，还可以改变思路，不再把它当做通常的分类问题，而是看出是一分类（one class learning）或异常检测（novelty detection）问题。此时，重点并不是捕捉类间的差别，而是把样本作为一个整体分析它们的主流共性，然后再找出其中有哪些（少数）样本与主流不一样。例如，我们要判断一张照片里的人脸是男性还是女性，这是个二分类问题。通常的方法是在大量已标注了男女性别的人脸照片中寻找能够区分男性/女性的特征差异（比方说女性一般有长头发，男性一般有胡子等等），就可以进行预测了。但是，如果男女照片的比例不平衡，只有一两张女性照片，那么就很难从中可靠地总结出女性的特征。此时，可以转而寻找哪几张照片与大部分照片很不一样，即属于异常数据，那么，既然我们知道样本只有两类而且大部分是男性照片，我们就可以得出这些异常照片不是男性照片（也就是女性照片）的判断。

在这种思路下，可以应用前面介绍过的异常检测算法，或 One-class SVM 等方法。

4.5 天下没有免费午餐定理

从前面的介绍中我们了解到的重要一点是：要恰当地评价一个机器学习模型，需要清楚自己的问题的实质与目的是什么。与此相关的，机器学习领域中有个颇具哲学意味的定理：天下没有免费午餐定理（No free lunch theorem）^z。这个定理指出，对所有可能的目标函数求平均，得到的所有学习算法（包括随机猜测）的非训练集误差的期望值都是相同的。

这个定理咋看上去非常奇怪，怎么可能所有的学习算法的效果都是相同的呢？其实这里的关键是“对所有可能的目标函数求平均”。例如，对于有 N 个 $\mathbf{x}$ 数据点的二分类问题，每个 $\mathbf{x}$ 都有两种可能的类别取值， $t = 0$ 和 $t = 1$ ，所有 $\mathbf{x}$ 的可能 t 取值组合起来，一共有 2^N 种可能的目标函数。对任一 $\mathbf{x}_n$ ，总有一半目标函数认为其值为真（1），一半为假（0）。不管你的算法预测这个变量为真还是为假，平均总是 50%正确，即使是随机猜测也是如此。这个结果与已知的训练集无关。

这个定理似乎是一个让人非常失望的消息。我们真的没法学到任何东西吗？其实不是的。在真实世界中，我们所关心的、要学习的函数并非均匀地来自所有可能的函数！比如，光滑的函数就比完全随机跳动的函数更有可能发生；越复杂的函数也许越不容易发生；相邻的点总是有些关联。而这些其实就是机器学习的依据。机器学习是一种归纳，基于数据就可以产生有用的结果，但它也不是完全在对世界一无所知的情况下工作，还是有少量的已知知识的（贝叶斯概率论框架下的先验概率）。

^z D. H. Wolpert and W. G. Macready. No free lunch theorems for optimization. *IEEE Trans. Evol. Comput.* **1**, 67 (1997).

天下没有免费午餐定理最重要意义就是提醒我们，在脱离实际意义情况下，空泛地谈论哪种算法好毫无意义，要谈论算法优劣必须针对具体学习问题，否则就会犯前面的缺陷产品检测的那种错误。不要奢望能找到一种算法对所有问题都适用。我们面对的总是一个一个特定的问题，而不是所有问题。这个定理对于盲目的“算法崇拜”与教条主义有毁灭性的打击。例如，现在深度学习如日中天，那是不是深度学习就比其余任何算法都要好？其实也未必，我们需要加深对问题的理解，研究不同问题的假设和数据，以寻找合适的算法^{aa}。就如 George Box 所说，所有的模型都是错的，但有些是有用的（All models are wrong, but some are useful）。

扩展阅读：

- <https://www.jiqizhixin.com/articles/2018-10-19-15>
达观数据王子豪：这 5 个例子，小学生都能秒懂分类算法
 - <https://developer.aliyun.com/article/228637>
如何保存和恢复 scikit-learn 训练的模型
 - <https://www.jiqizhixin.com/articles/2018-11-30-6>
开源一年多的模型交换格式 ONNX，已经一统框架江湖了？.mht
 - <https://www.cnblogs.com/princecoding/p/6714216.html>
机器学习常用性能指标总结
 - <https://www.cnblogs.com/xuexuefirst/p/8858274.html>
衡量机器学习模型的三大指标：准确率、精度和召回率
 - 作者：佩德罗·多明戈斯（Pedro Domingos）；译者：刘知远。机器学习那些事。中国计算机学会通讯，第 8 卷，第 11 期，74-86 页（2012 年）
 - <https://www.jiqizhixin.com/articles/2023-08-21-11>
预测热门歌曲成功率 97%？这份清单前来「打假」
- 本文的关键错误就在于，这种训练 - 测试分离是在数据已经过采样之后进行的。

^{aa} 一位面试官的经验：比如深度学习是否比别的模型好，我们就期待面试者能说“分情况讨论”，若是能提到“没有免费的午餐定理”更是加分了。

- 模型存在机器学习中最常见的缺陷之一：数据泄漏。
- <https://baijiahao.baidu.com/s?id=1625963857192376606>
[什么是机器学习数据预处理？如何进行数据预处理？这里统统都有](#)
- <https://www.huxiu.com/article/318865.html>
[揭穿 AI 竞赛的真实面目](#)
- 这相当于是把测试集当成了验证集
- Epi101 让你抛 10 次硬币。如果你得到 8 个及以上的正面，就证明了硬币是有魔力的。(虽然这个断言显然是无稽之谈，但你要继续下去，因为你知道 8/10 个正面相当于一枚均匀硬币的 p 值 <0.05 ，所以它一定是合理的)。在你不知道的情况下，Epi101 对另外 99 个人也做了同样的事情，他们都认为只有自己在测试硬币。你希望发生什么？
- <https://blog.csdn.net/cumei1658/article/details/107362588/>
[学习曲线 机器学习_机器学习的学习曲线](#)
- <https://www.pybloggers.com/2018/01/learning-curves-for-machine-learning/>
[Learning Curves for Machine Learning](#)
- <https://baike.baidu.com/item/%E4%BA%A4%E5%8F%89%E9%AA%8C%E8%AF%81/8543100?fr=aladdin>
[交叉验证](#)
- https://blog.csdn.net/zk_ken/article/details/82013965
[机器学习中不平衡数据的处理方式](#)
- <https://www.cnblogs.com/charlotte77/p/10455900.html>
[【机器学习】如何解决数据不平衡问题](#)
- <https://www.jiqizhixin.com/articles/2019-05-30-7>
[一文教你如何处理不平衡数据集（附代码）](#)
- <https://blog.csdn.net/pipisorry/article/details/52574156>
[机器学习：分类、多分类、回归模型的评估](#)
- <https://www.jianshu.com/p/66dc6ba84ff9>
[机器学习中的混淆矩阵](#)
- <https://www.jianshu.com/p/be343414dd24>
[机器学习：如何解决机器学习中数据不平衡问题](#)
 - 讲得非常好
 - 英文原文：<https://towardsdatascience.com/handling-imbalanced-datasets-in-machine-learning-7a0e84220f28>
- <https://sklearn.apachecn.org/master/40/>

5.3 预处理数据

- <https://www.jiqizhixin.com/articles/2018-12-26-15>
实现特征缩放和特征归一化的方法有哪些？
- <https://www.cnblogs.com/feiyumo/p/9866818.html>
机器学习中的归一化方法
- <https://blog.csdn.net/u010315668/article/details/80374711>
机器学习之特征归一化（normalization）
- <https://www.jianshu.com/p/5cdb140aded2>
机器学习之归一化
- <https://www.jiqizhixin.com/articles/2018-12-26-15>
实现特征缩放和特征归一化的方法有哪些？
- <https://www.jiqizhixin.com/articles/2019-07-09-5>
深度学习中的 Normalization 模型（附实例&公式）。中间层也要归一化。
- <https://www.jiqizhixin.com/articles/2020-09-16-6>
one-hot encoding 不是万能的，这些分类变量编码方法你值得拥有
- https://blog.csdn.net/qq_30374549/article/details/81805613
机器学习之离散值处理
- <https://blog.csdn.net/msgla/article/details/98111623>
机器学习实践/数据预处理/离散特征处理（1）
- <https://www.jianshu.com/p/893413b20866>
【机器学习】离散和连续数据（Discrete & Continuous Data）
- <https://baijiahao.baidu.com/s?id=1625963857192376606>
什么是机器学习数据预处理？如何进行数据预处理？这里统统都有
- <https://baike.baidu.com/item/%E7%8B%AC%E7%83%AD%E7%BC%96%E7%A0%81/9717350>
独热编码
- <https://www.cnblogs.com/zongfa/p/9305657.html>
数据预处理：独热编码（One-Hot Encoding）和 LabelEncoder 标签编码
- <https://www.2cto.com/kf/201901/790999.html>
Python 逻辑回归模型原理及实际案例应用
- <https://www.jiqizhixin.com/articles/2019-07-05-2>
学会五种常用异常值检测方法，亡羊补牢不如积谷防饥
- <https://baijiahao.baidu.com/s?id=1619431536284756645>
四种检测异常值的常用技术简述

- https://scikit-learn.org/stable/modules/outlier_detection.html
2.7. Novelty and Outlier Detection
- <https://sklearn.apachecn.org/docs/0.21.3/26.html>
2.7. 新奇点和离群点检测
- <https://baike.baidu.com/item/%E7%AE%B1%E5%BD%A2%E5%9B%BE/10671164>
箱形图
- <https://blog.csdn.net/helloxiaozhe/article/details/80658438>
机器学习-训练模型的保存与恢复 (sklearn)
- <https://yq.aliyun.com/articles/228637>
深度学习小技巧 (二): 如何保存和恢复 scikit-learn 训练的模型
- <https://www.jiqizhixin.com/articles/2018-11-30-6>
开源一年多的模型交换格式 ONNX, 已经一统框架江湖了?
- <https://www.jianshu.com/p/65cfb475584a>
ONNX 简介
- <https://www.cnblogs.com/pinard/p/9220199.html>
用 PMML 实现机器学习模型的跨平台上线
- https://mp.weixin.qq.com/s/x48Ctb0_Eu1kcSGTYLt5BQ
机器学习中如何处理不平衡数据?
- <https://baike.baidu.com/item/%E6%B7%B7%E6%B7%86%E7%9F%A9%E9%98%B5/10087822>
混淆矩阵
- <https://baike.baidu.com/item/%E6%8E%A5%E5%8F%97%E8%80%85%E6%93%8D%E4%BD%9C%E7%89%B9%E5%BE%81%E6%9B%B2%E7%BA%BF/2075302?fromtitle=ROC%E6%9B%B2%E7%BA%BF>
接受者操作特征曲线
- <https://blog.csdn.net/pipisorry/article/details/51788927>
分类模型评估之 ROC-AUC 曲线和 PRC 曲线
- https://blog.csdn.net/weixin_44192389/article/details/125027633
ROC 曲线理解
- <https://link.springer.com/article/10.1186/s40537-019-0197-0>
深度学习领域的增强数据
- <https://www.jiqizhixin.com/articles/2018-12-20-14>
如何优雅而时髦的解决不平衡分类问题
- <https://zhuanlan.zhihu.com/p/655464162>

统计学中最反直觉的悖论—斯坦悖论，用中国茶叶的价格来更准确地预测墨尔本的降雨概率？

- https://blog.csdn.net/sinat_26917383/article/details/76647272
无监督 | 异常、离群点检测 一分类——OneClassSVM
- <https://www.zhihu.com/question/22365729>
什么是一类支持向量机（one class SVM），是指分两类的支持向量机吗？
- <https://baike.baidu.com/item/%E6%B2%A1%E6%9C%89%E5%85%8D%E8%B4%B9%E5%8D%88%E9%A4%90%E5%AE%9A%E7%90%86/8848514>
没有免费午餐定理
- https://blog.csdn.net/qq_28739605/article/details/80607330
奥卡姆剃刀和没有免费的午餐定理
- <https://www.zhihu.com/question/267135168/answer/329318812>
机器学习包含哪些学习思想？

第五章 神经网络

刘志荣

在本章中，我们将简要介绍目前机器学习领域中最重要、影响最大的模型与方法——神经网络（neural network，或人工神经网络 artificial neural network）。人工智能的发展历史可以总结为三起两落（参见第一章第2节），其目前正在经历的最新繁荣期，正是由神经网络所驱动的。本章仅介绍神经网络的初步知识，后面的第7章中将进一步介绍神经网络的更多知识。

第1节 神经网络的想法、数学描述与性质

1.1 思路：适应数据的可变基

前面章节主要介绍了适用于回归问题的线性回归方法以及适用于分类问题的逻辑回归方法。它们都属于线性方法，在数学上可以统一描述为

$$y(\mathbf{x}) = f(\mathbf{w} \cdot \mathbf{x}) \quad (5.1)$$

或

$$y(\mathbf{x}) = f\left(\sum_{j=0}^M w_j \phi_j(\mathbf{x})\right) = f(\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x})) \quad (5.2)$$

其中 f 是已知的、固定的函数。对于线性回归， f 是恒等函数： $f(a) = a$ ；对于逻辑回归， f 是逻辑函数： $f(a) = \sigma(a) = \frac{1}{1+e^{-a}}$ 。而 $\phi_j(\mathbf{x})$ 是固定的基函数， $\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x})$ 是把这些基函数线性组合起来以描述目标函数 $y(\mathbf{x})$ （严格讲是 $f^{-1}(y(\mathbf{x}))$ ），其中的组合系数 $\mathbf{w}$ 可以比较容易地求解出来。

这类线性方法简单易行，但有个很大的缺点，就是（由于固定基的使用而导致）不够灵活。当输入的特征较多（高维数据 $\mathbf{x}$ ）时会导致基函数数目过于庞大，即前面介绍过的维度灾难问题。例如，如果输入 $\mathbf{x}$ 是分辨率为 640×480 的图像，使用线性函数作为基函数，则需要 $640 \times 480 = 307200$ 个基函数，而如果使用二阶多项式函数，则还需要近5亿个参数，很难求解。

关于固定基的缺点，可以用化学里的光谱曲线拟合作为例子来理解。谱线通常以多个峰叠加的形式出现。如果是使用固定基的线性组合来描述实验测得的光谱数据（即线性回归方法），则可以用一系列不同位置的单峰作为基函数，它们的组合就会给出多峰结构，权重越大，所对应的峰就越高。由于基函数峰的位置是事先固定的，不随训练数据而变化，因此，为了准确地描述实验的各种可能谱线，可能就需要准备很多个基函数，例如，每个波数位置都准备一个基函数。这样不管实验中的谱峰出现在哪个位置，都有合适的基函数可以描述它。不过，由

于每个样品的光谱曲线通常只有有限数量的峰，因此每次拟合都有大量的基函数派不上用场，导致这种固定基的方法很低效。事实上，实际的光谱拟合使用的并不是这种方案，而是可变基的方案：假设谱线有若干个峰，它们的位置、宽度、高度都是可以变化的（事先未知的）；然后根据实验的数据，来调整这些位置、宽度、高度，使得它们所描述的峰的叠加结果（曲线）尽量与实验数据相符。这其实就是神经网络方法背后的普适想法：适应基（adapted basis）的引入。

适应基的主要想法，就是让基函数本身是可变的，随训练数据而变化，或者说，让基函数去“适应”训练数据。具体做法上，是使基函数 $\phi_j(\mathbf{x})$ 本身带有可调参数（ $\mathbf{w}'$ ）：

$$y(\mathbf{x}) = f\left(\sum_{j=0}^M w_j \phi_j(\mathbf{x}; \mathbf{w}')\right) \quad (5.3)$$

具体方案可以是千变万化的。在神经网络模型中，采用的是如下方案：

$$\phi = \sigma(\mathbf{w}' \cdot \mathbf{x}) \quad (5.4)$$

即 $y(\mathbf{x}) = \sigma(\mathbf{w}^T \sigma(\mathbf{w}' \cdot \mathbf{x}))$ ，其中 σ 是逻辑函数，因此它是用串联的一系列（逻辑）函数变换来实现带有可调参数的非线性函数，相当于是串联的多层逻辑回归。神经网络的这种形式比支持向量机^{bb}（后续章节介绍）更紧凑，预测更快（但训练更慢），不过其代价是其似然函数不是凸的，理论分析更为困难。

1.2 数学描述

神经网络模型的一般结构如图 5.1 所示。其中的圆圈（正圆形或椭圆形）被称为节点（node）或神经元，从左到右分成几层，最左边被称为输入层，最右边被称为输出层，中间的被称为隐藏层（可以有多个隐藏层，最简单的只有一层）。信号从左边输入（ $\mathbf{x} = \{x_1, x_2, x_3\}$ ），从左到右一层层地进行变换，最后从右边输出（ $y = \{a_1^3, a_2^3\}$ ），从而提供一个复杂的映射函数 $y(\mathbf{x})$ 。具体的变换由节点间的多变量线性变换（图中的带箭头直线）与节点内部的单变量非线性变换（图中的内含竖杠的椭圆形）组成：

$$z_i^{(k+1)} = \sum_{j=0} w_{ij}^{(k)} a_j^{(k)} \quad (5.5)$$

$$a_i^{(k+1)} = h\left(z_i^{(k+1)}\right) \quad (5.6)$$

其中 k 是层数的编号。 $a_j^{(k)}$ 代表第 k 层第 j 个节点的激活（activation）信号，它们通

^{bb} 支持向量机是先定义一些中心位于数据点上的基，再从中选择需要留下的基作为最后的结果，其它基扔掉。它的基函数数目通常比数据点少得多，但可能仍然很多（相比神经网络方法）。

过线性组合[公式 (5.5)]给下一层 (第 $k+1$ 层) 的节点 i 提供刺激 $z_i^{(k+1)}$, $w_{ij}^{(k)}$ 是未知的组合系数, 是需要学习的参数; 然后在节点 i 上通过激活函数 $h()$ (有时也被称为 hidden function) 把刺激 $z_i^{(k+1)}$ (节点的输入) 变成激活 (节点的输出) 信号 $a_i^{(k+1)}$ [公式 (5.6)]^{cc}。通常激活函数 $h()$ 是已知的, 不包含未知参数; 输出层的 $h()$ 可与隐藏层不同。在输入层, 有简单的关系: $a_i^{(1)} = x_i$, 这样就可以从输入数据开始, 逐层应用公式 (5.5) (5.6) 进行计算, 得到最后的输出。另外, $a_0^{(k)} \equiv 1$ 是个常数, 为线性运算提供截距项(bias), 有时候也可以根据实际需要固定为 0, 即线性运算不包含 bias。

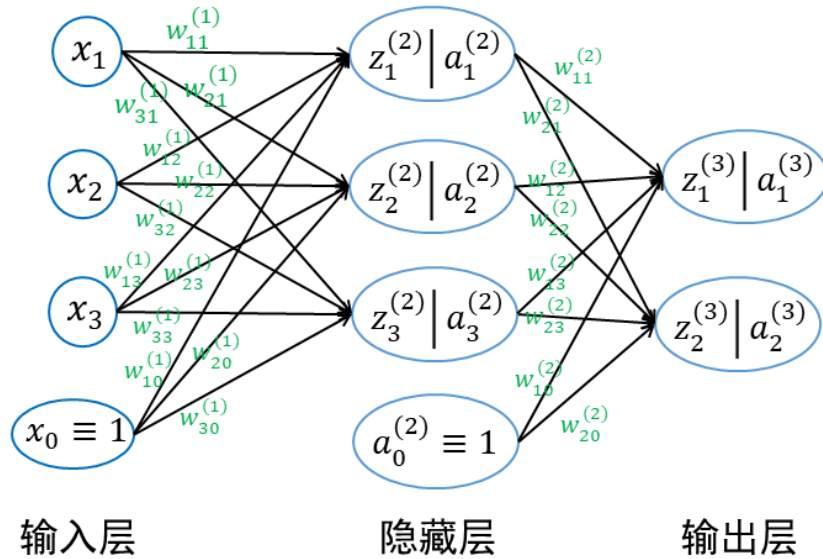


图 5.1. 神经网络示意图。信号从左边输入 ($\mathbf{x} = \{x_1, x_2, x_3\}$, $x_0 \equiv 1$ 是个常数, 为线性组合提供截距项, 有时候不看成输入的组成部分), 从右边输出 ($\mathbf{y} = \{a_1^3, a_2^3\}$)。带箭头的直线表示线性组合运算; 椭圆代表单变量的非线性激活运算。

激活函数 h 有很多种形式。对于隐藏层, 常用的 $h()$ 包括 (图 5.2):

- 逻辑函数 (sigmoid 函数) $\sigma(z) = \frac{1}{1+e^{-z}}$ 。它简单地沿用了逻辑回归中的激活函数。
- 双曲正切函数 $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$ 。它可看做是对逻辑函数 $\sigma(z)$ 进行平移和拉伸的结果, 是以 0 为中心的中心对称函数 (具有零对称性), 克服了逻辑函数输

^{cc} 注意有些教科书里符号 a 与 z 的含义与这里相反, 即 $z_i^{(k+1)} = h(a_i^{(k+1)})$ 。

出总为正值的缺点。当 z 的值很小的时候， $\tanh(z)$ 的值也很小。

- 线性整流函数（Rectified Linear Unit）： $\text{ReLU}(z) = \max(0, z)$ 。对于 $\sigma(z)$ 和 $\tanh(z)$ 激活函数而言，左边与右边都存在饱和区；当参数与信号落入饱和区时，梯度很小（这在深度神经网络中被称为梯度消失问题），将导致学习不容易进行。 $\text{ReLU}(z)$ 则在正 z 区域总是保持一定大小的梯度，从而改善了效果。

不同的激活函数对学习效果有一定的影响，因此这方面有很多研究，提出了很多五花八门的方案（*扩展阅读：五花八门的激活函数*）。值得指出的是，隐藏层的激活函数不能用线性函数。其原因是两个线性函数的复合效果仍然是线性函数，因此，如果隐藏层的激活函数采用线性函数，那它的作用完全可以反映到其前面的线性组合运算[公式（5.5）]中，导致激活函数及其下一层的线性组合并没有真正的作用。

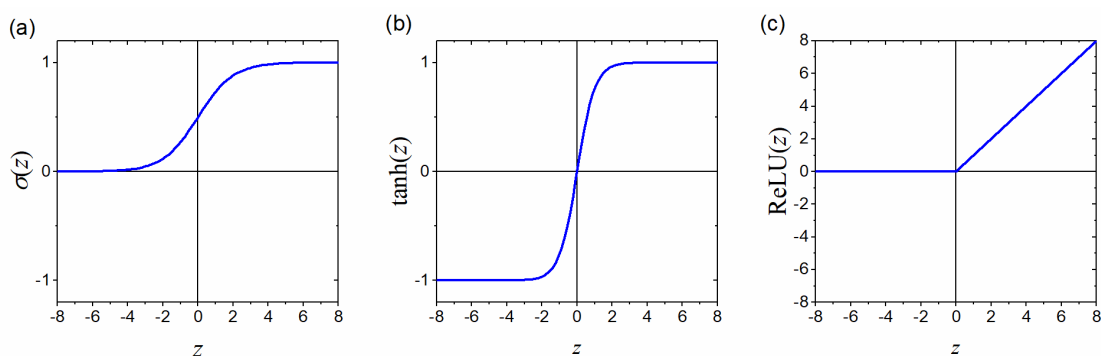


图 5.2. 神经网络模型中常见的激活函数。(a) 逻辑函数；(b) 双曲正切函数；(c) 线性整流函数 ReLU。

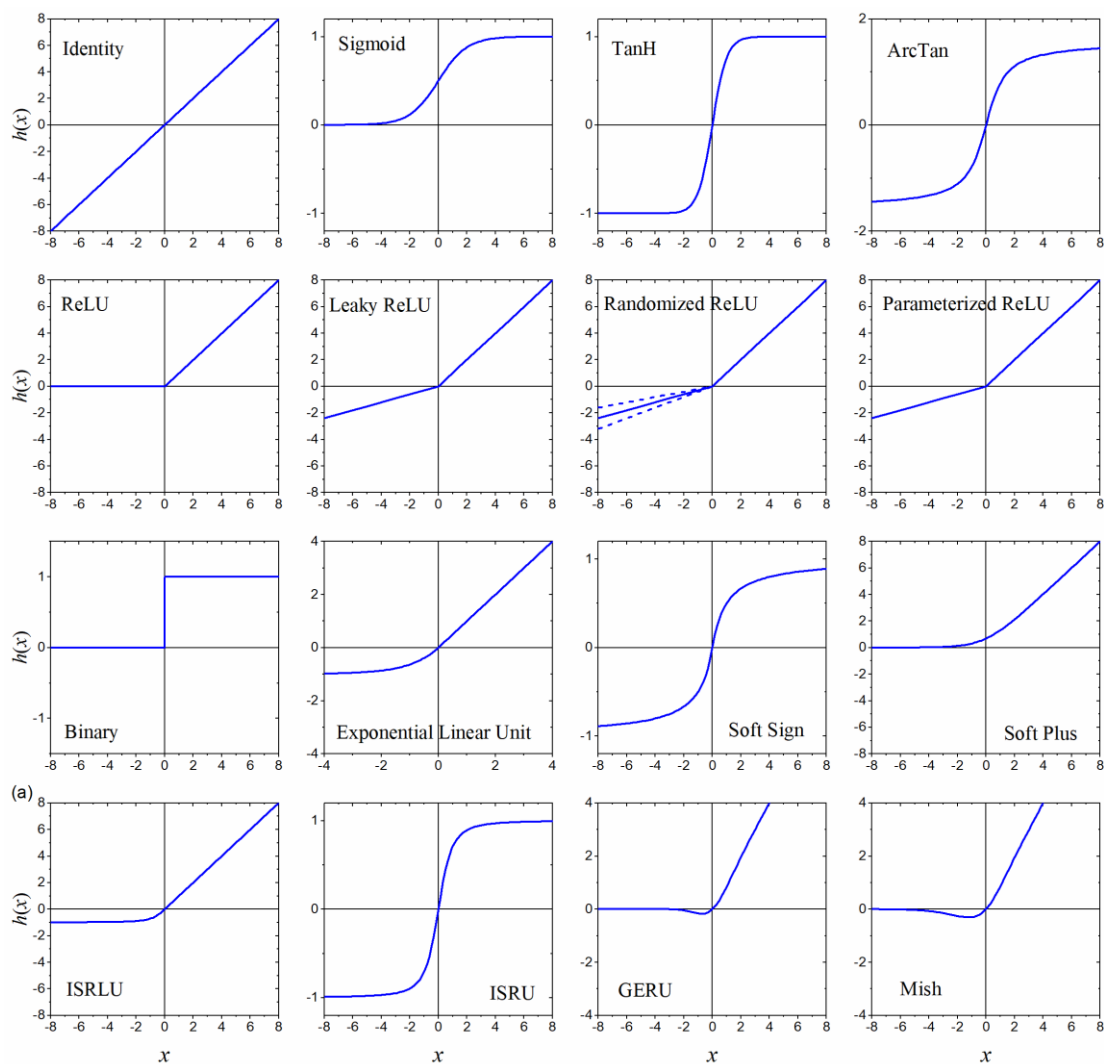
对于输出层，激活函数需要针对不同的任务类型进行选择：

- 对于回归问题，采用恒等变换： $y = h(z) = z$ 。
- 对于分类问题，通常采用逻辑函数： $y = \sigma(z) = \frac{1}{1+e^{-z}}$ 。
- 对于多分类问题，通常采用 softmax 函数： $y_i = a_i = \frac{e^{z_i}}{\sum_i e^{z_i}}$ 。

输出层的这些选择与前面章节（线性回归与逻辑回归）中的做法基本一致。

扩展阅读：五花八门的激活函数

通常认为，好的激活函数应该满足：零对称性；无饱和区域；梯度计算高效。在文献中，存在着各种各样的方案：



- 恒等函数（Identity）: $\text{Identity}(x) = x$
- 逻辑函数（Sigmoid）: $\text{sigmoid}(x) = \sigma(x) = \frac{1}{1+e^{-x}}$
- 双曲正切函数（Tanh）: $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
- 反正切函数（ArcTan）: $\arctan(x)$
- 线性整流函数（ReLU）: $\text{ReLU}(x) = \max(0, x)$
- 渗漏型整流线性单元（Leaky ReLU）: $\text{LReLU}(x) = \begin{cases} \alpha x & (\text{if } x \leq 0) \\ x & (\text{if } x > 0) \end{cases}$ 。它的优点是不会饱和（两边都不会）、梯度计算高效（收敛快）、不会出现失效（dead）神经元。
- 随机修正型整流线性单元（Randomized ReLU）: $\text{RReLU}(x) = \begin{cases} \alpha x & (\text{if } x \leq 0) \\ x & (\text{if } x > 0) \end{cases}$ 。其中训练时 α 在 $[l, u]$

区间均匀随机取值，而测试与预测时取 $\alpha = \frac{l+u}{2}$ 。通常 $l = \frac{1}{8}$, $u = \frac{1}{3}$ 。其想法与深度学习中的 dropout 类似。

- 参数化整流线性单元 (Parametric ReLU): $\text{PReLU}(x) = \begin{cases} \alpha x & (\text{if } x \leq 0) \\ x & (\text{if } x > 0) \end{cases}$ 。其中 α 是可学习参数。
- 二值化 (Binary): $\text{Binary}(x) = \begin{cases} 0 & (\text{if } x \leq 0) \\ 1 & (\text{if } x > 0) \end{cases}$
- 指数化线性单元 (Exponential Linear Unit): $\text{ELU}(x) = \begin{cases} \alpha(e^x - 1) & (\text{if } x \leq 0) \\ x & (\text{if } x > 0) \end{cases}$ 。缺省值 $\alpha = 1$ 。
- SoftSign: $\text{SoftSign}(x) = \frac{x}{1+|x|}$
- Inverse Square Root Unit (ISRU): $\text{ISRU}(x) = \frac{x}{\sqrt{1+\alpha x^2}}$ 。其中 $\alpha = 1.0$ or 3.0 。
- Inverse Square Root Linear Unit (ISRLU): $\text{ISRLU}(x) = \begin{cases} \frac{x}{\sqrt{1+\alpha x^2}} & (\text{if } x \leq 0) \\ x & (\text{if } x > 0) \end{cases}$ 。其中 $\alpha = 1.0$ or 3.0 。
- Softplus: $\text{Softplus}(x) = \frac{1}{\beta} \ln(1 + e^{\beta x})$ 。缺省值 $\beta = 1$
- Gaussian Error Linear Units (GELU): $\text{GELU}(x) = x\Phi(x)$ ，其中 $\Phi(x)$ 是高斯分布的累计分布函数。通常采用近似: $\text{GELU}(x) = \frac{1}{2}x \left[1 + \tanh\left(\sqrt{\frac{2}{\pi}}(x + 0.044715x^3)\right) \right]$
- Mish: $\text{Mish}(x) = x \tanh[\text{Softplus}(x)] = x \tanh\left[\frac{1}{\beta} \ln(1 + e^{\beta x})\right]$

如果只看神经网络某一层节点（激活变换）以及其左边连着的直线（线性变换），把其它部分遮盖掉，则这就是一个标准的逻辑回归模型。因此，神经网络可看做多层串联的逻辑回归。每层的参数（ $w_{ij}^{(k)}$ ）是互相独立的，并不相同。输入信号是已知的，输出信号则可以与目标（ t_n ）进行直接比较，也有明确意义。但隐藏层的信号是变换过程中的辅助变量（信号），在实际的实验中是不可测（不能直接测量）的，这也是“隐藏”一词的含义。每个隐藏层的节点数目，被称为神经网络的宽度（width）。串联的逻辑回归层的层数，或者说，带有激活功能的节点层的层数，被称为神经网络的深度（depth）。因此，图 5.1 给出的是一个 2 层神经网络^{dd}，宽度为 4。有很多隐藏层的神经网络，即很“深”的神经网络，被称为深度神经网络（deep neural network）。例如，何恺明等人提出的用于图像分类识别的残差网络 ResNet^{ee}具有 152 层！人工智能目前最为火热的研究方

^{dd} 有少数文献把输入层也算到深度的计数，认为图 5.1 是一个 3 层神经网络。不过这种做法并不流行。

^{ee} K. He, X. Zhang, S. Ren, J. Sun. Deep Residual Learning for Image Recognition, in 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, June 01, 2016; p 1.

向--深度学习（deep learning），主要指的就是深度神经网络。

神经网络的深度与宽度，以及不同激活函数的选取，都是模型的超参数，需利用验证集来进行选择（参见前一章），以达到更好的效果。

上述神经网络模型的这种架构，其实是受到了人脑的神经网络运作方式的启发（参见题外：神经网络的历史）。

题外：神经网络的历史

神经网络方法的缘起，是最初希望通过直接模拟人脑的神经网络的运作方式，以实现人工智能的目的。

人脑中的神经网络是一个非常复杂的组织，由约 1000 亿个神经元组成。一个神经元通常具有多个树突（多个输入），用来接受传入信号；同时具有一条轴突（一个输出），用来发出信号。轴突尾端有许多分叉的末梢，跟其它神经元的树突建立连接，从而可以给其它多个神经元传递信息（提供输入）。当一个神经元的输入信号的总和（加权总和，权重取决于其树突与其它神经元的轴突末梢的连接强度）超过某一个阈值时，它（的轴突）就输出高电位（1）；否则处于低电位（0）。这种运行机制，与图 5-1 所示的神经网络模型完全类似。或者更准确地说，图 5-1 的模型就是对人脑神经网络的模仿。（现在逐渐认识到并不需要受此限制，就像设计飞机不需要像鸟。）

多层神经网络可以实现复杂的函数拟合，在语音识别、图像识别、自动驾驶等领域有重要应用，在上世纪 80 年代末曾风靡一时，连娱乐界都受到了影响。例如，在《终结者 1》中施瓦辛格有一句台词：“我的 CPU 是一个神经网络处理器，一个会学习的计算机”。

上世纪 90 年代中期，支持向量机算法在多个方面体现出了相对于神经网络的优势：无需调参、高效、全局最优解，迅速打败神经网络算法成为机器学习主流。在本世纪初，神经网络卷土重来。目前，深度神经网络在人工智能界占据统治地位。

1.3 神经网络隐藏层的作用

与逻辑回归相比，神经网络多了一个或多个中间层（隐藏层），可以更有效地提取新的辅助特征，从而增强模型的表达能力与性能。

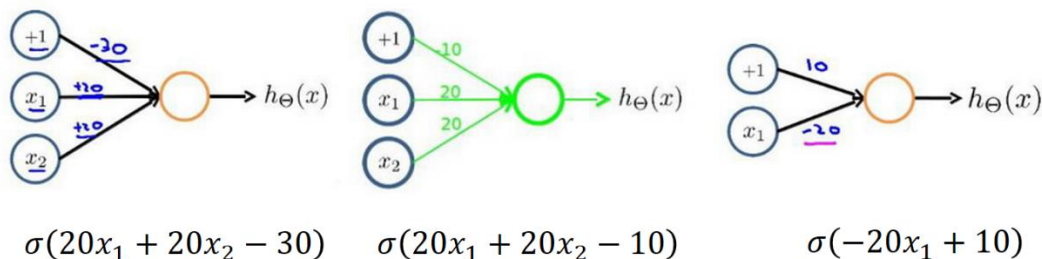


图 5.3. 神经网络中对逻辑运算 and, or, not 的实现。连线上的数字表示

相应线性组合的权重 $w_{ij}^{(k)}$ 。激活函数采用逻辑函数 $a = \sigma(z)$ 。

以逻辑运算（布尔运算）为例，我们利用一个中间节点，可以很容易实现两个输入信号（ x_1 , x_2 ）的“与”“或”“非”的操作（图 5.3）。对于图 5.3(a)所给出的参数， $a = \sigma(20x_1 + 20x_2 - 30)$ ，只有当 x_1 与 x_2 取值都为 1（真）时输出才为（很接近）1（真），而只要 x_1 与 x_2 中有任一个取值为 0（假）时输出就为（很接近）0（假），即实现了“与”运算。容易验证图 5.3(b,c)分别给出了“或”和“非”操作。一个大的神经网络可以把这些模块进一步以并联或串联的方式组合起来，即“与”“或”“非”运算的复合。数学上已经证明，“与”“或”“非”运算是完备的，即只要有足够多的“与”“或”“非”运算，就可以组合起来实现任何的布尔逻辑表达式。因此，神经网络可以实现任何布尔逻辑函数^{ff}。

这种信息与特征的分层处理，可能是神经网络有效性的重要原因。例如，对于手写数字识别，如果输入像素是二值（黑白）的，一张输入图片就对应于几百个布尔值的 x 分量（输入节点），输出则为 10 个布尔值（输出节点），分别代表这张图片“是否包含数字 0？”、“是否包含 1？”、“...是否包含 9？”。这就是一个多变量（几百个自变量）的布尔逻辑函数。对于这个问题，也许“直接寻找输入与输出之间的关系”并不是好的解决方法。好的方法可能是：先尝试辨别一些简单的模式，建立一些简单的概念，如“这里有一条横线”“这里有一条竖线”；然后再把这些简单概念组合起来构建稍微复杂一些的概念，如“这里有一条折线”（基于横线与竖线的认识）；然后再组合成更复杂的概念；经过很多层以后，也许就可以达到“这里有一个 0”之类的终极目标。人类的认知与思维很可能就是分层次的，而神经网络（特别是深度神经网络）很好地模拟了这种方式（扩展阅读：神经网络的分层信息）。

^{ff} 图 5.3 也可以看做是逻辑回归模型。因此逻辑回归模型也能实现“与”“或”“非”运算。但由于逻辑回归不能把模块串联起来，因此对于布尔逻辑运算并不是完备的。例如，逻辑回归并不能实现“异或”运算（xor）：当 $x_1 \neq x_2$ 时，结果为真；否则结果为假。这也是早期感知器与逻辑回归模型倍受攻击的原因。

神经网络的分层信息

有人对一个超大型卷积神经网络 AlexNet（共有 9 层，65 万个神经元，6 千万个参数。用于图片分类，输入是图片，输出是 1000 个类，比如小虫子、美洲豹、救生船等）进行分析，发现不同的隐藏层响应了不同层次的特征：

- 第一层神经元主要负责识别颜色和简单纹理。
- 第二层的一些神经元可以识别更加细化的纹理，比如布纹、刻度、叶纹。
- 第三层的一些神经元负责感受黑夜里的黄色烛光、鸡蛋黄、高光。
- 第四层的一些神经元负责识别萌狗的脸、七星瓢虫和一堆圆形物体的存在。
- 第五层的一些神经元可以识别出花、圆形屋顶、键盘、鸟、黑眼圈动物。

Ref.: [1] M. D. Zeiler, R. Fergus. Visualizing and Understanding Convolutional Networks. (2013). arXiv:1311.2901.

[2] <https://www.zhihu.com/question/22553761>

1.4 神经网络是一种万能近似函数

神经网络具有很强的表现能力。事实上，数学上已经证明它是一种万能近似函数（universal approximator，又称万能逼近器、通用近似器）：

万能近似定理（universal approximation theorem） 一个神经网络如果具有线性输出层和至少一层具有任何某种“挤压”（squashing）属性的激活函数（例如逻辑函数）的隐藏层，只要拥有足够数量的隐藏层节点，它可以以任意高的精度来近似任何函数⁸⁸。

简单地说，就是只要有任意数量的隐藏层节点以及合适的参数值 $w_{ij}^{(k)}$ ，那神经网络所给出的结果 $y(\mathbf{x})$ 与目标函数的差值可以小于任意事先指定的（正）值（精度）。

⁸⁸ 数学上严格讲，是可以近似“从一个有限维空间到另一个有限维空间的 Borel 可测函数”。常见的函数（曲线、曲面）都被涵盖在这个范围内。文献参见：K. Hornik, M. Stinchcombe, and H. White, *Neural Networks*, 2, 359 (1989); *ibid*, 3, 551 (1990); 以及 G. Cybenko, *Mathematics of Control, Signals, and Systems*, 2, 303 (1989). 万能近似定理中所谓“挤压”性质的函数主要是指类似于逻辑函数的非常数、有界、单调递增的连续函数。但神经网络的通用近似性质也被证明对于其它类型的激活函数，比如 ReLU，也都是适用的。

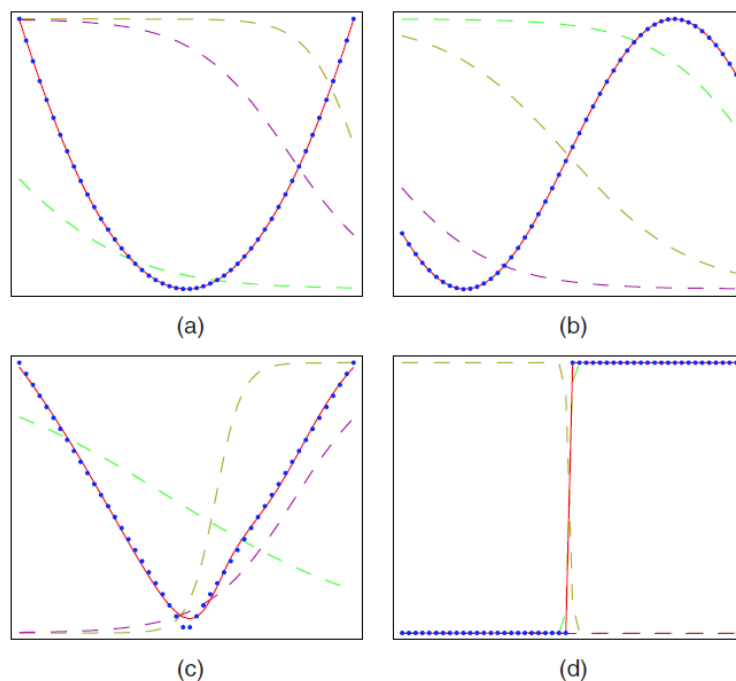


图 5.4. 神经网络的万能近似能力。圆圈代表目标曲线，分别是：(a) 抛物线；(b) 正弦曲线；(c) V 形折线；(d) 阶跃曲线。实线是神经网络的结果。所用神经网络包含一个输入节点、一个中间层（3 个节点，采用 sigmoid 激活函数）、一个输出节点（采用恒等激活函数）。虚线是 3 个中间层节点对输出的（权重加和）贡献分量。

图 5.4 给出了几个直观的一维例子。不管是全程连续、可微的曲线 (a, b)，还是包含有限个不可微 (c) 或不连续 (d) 点的曲线，仅包含 3 个隐藏节点的神经网络都可以很好地进行描述。

那么，怎样理解神经网络的这种万能近似能力呢？以上述的一维曲线为例来解释：包含 3 个隐藏节点的神经网络其实就是把 3 个经过（水平以及竖直方向）适当伸缩后的 sigmoid 曲线叠加起来；由于每个伸缩过的 sigmoid 函数可以提供一条局部上升或下降的曲线（即整条曲线只有一个上升变形或下降变形，而且变形大致是在有限区域里完成的，曲线在此区域以外基本是平的），在最后的合成曲线可以贡献一个升降，如果把很多个升降组合起来，就可以形成各种形状的曲线。在图 5.4 中，每条曲线最多包含 3 个升/降，因此大致可以用 3 个隐藏节点来描述。节点越多，描述就可越精确。对于多分量的 $\mathbf{x}$ 也可以类似分析^{hh}。

万能近似函数听起来很神奇，但实际上这种能逼近任何连续函数的能力并不是神经网络所独有的，像多项式函数（泰勒展开）、三角函数（傅里叶变换）ⁱⁱ都

^{hh} 对多变量 $\mathbf{x}$ ，很容易将其线性组合成任意方向的单变量 $\mathbf{w} \cdot \mathbf{x}$ ，并用上述方法实现任意函数 $f(\mathbf{w} \cdot \mathbf{x})$ 。利用傅里叶变换或类似技巧，可把任意多变量函数分解成一些单方向的波，类似 $f(\mathbf{w} \cdot \mathbf{x})$ ，每一个都可用上述方法实现。

ⁱⁱ 傅里叶变换就可看作是使用了某种具有“挤压”与平移性质的激活函数 $\sin(x)$ 。

具有这种能力。另外，万能近似定理只保证函数是“可表示的”，但不能保证它是“可以被学习的”。也就是说，它证明了存在合适的参数值 $w_{ij}^{(k)}$ ，使得神经网络可以逼近任何函数，但它不能保证这些值能够（经由模型的训练）被找到。用化学的语言来类别，就是虽然某个分子在热力学上是稳定的，但如何在实验上合成它，需要考虑动力学的因素，有可能是非常困难的任务。

第2节 神经网络的梯度与训练

前面介绍了神经网络的基本形式与性质，现在我们来看神经网络的训练，即如何确定参数 $w_{ij}^{(k)}$ 。基本思路与线性回归和逻辑回归中的套路类似：考虑一个合适的误差函数；计算误差函数相对于待确定参数的（偏）导数；利用这些导数，使用梯度下降法等优化算法不断调整参数，使误差函数下降，最后得到神经网络的最优参数。

2.1 误差函数

误差函数主要取决于所要解决的问题，而不太依赖于所使用的模型。因此，对于回归问题，与线性回归类似，在神经网络中可以采用均方（二乘）误差：

$$E(\mathbf{w}) = \sum_{n=1}^N E_n = \frac{1}{2} \sum_{n=1}^N (y_n - t_n)^2 = \frac{1}{2} \sum_{n=1}^N [y(\mathbf{x}_n; \mathbf{w}) - t_n]^2 \quad (5.7)$$

其中 $\{\mathbf{x}_n, t_n\}$ 是已知的训练集； $y_n = y(\mathbf{x}_n; \mathbf{w})$ 是神经网络模型以 $\mathbf{x}_n$ 为输入、以 $\mathbf{w}$ 为参数时的输出（预测）结果。对于对分类问题，与逻辑回归类似，在神经网络中可以采用交叉熵误差函数：

$$E(\mathbf{w}) = \sum_{n=1}^N E_n = - \sum_{n=1}^N [t_n \ln y(\mathbf{x}_n; \mathbf{w}) + (1 - t_n) \ln (1 - y(\mathbf{x}_n; \mathbf{w}))] \quad (5.8)$$

此时神经网络（输出层激活函数采用逻辑函数）的输出 $y_n = y(\mathbf{x}_n; \mathbf{w})$ 预测的不是分类结果 t_n （可能取值是1和0），而是预测了 $t_n = 1$ 的概率，与逻辑回归中的想法一致。

2.2 神经网络的导数计算：误差反向传播算法

神经网络由公式（5.5）（5.6）描述，是一个嵌套函数的形式，因此，误差对各层参数的导数（梯度）可以用链式法则来计算。通常定义一个很有用的辅助变量 $\delta_j^{(k)} = \frac{\partial E_n}{\partial z_j^{(k)}}$ ，经过简单的推导，可得到如下结果：

$$\delta_j^{(k)} = \sum_i \frac{\partial E_n}{\partial z_i^{(k+1)}} \cdot \frac{\partial z_i^{(k+1)}}{\partial z_j^{(k)}} = \sum_i \frac{\partial E_n}{\partial z_i^{(k+1)}} \cdot \frac{\partial z_i^{(k+1)}}{\partial a_j^{(k)}} \cdot \frac{\partial a_j^{(k)}}{\partial z_j^{(k)}} = h'(z_j^{(k)}) \sum_i \delta_i^{(k+1)} w_{ij}^{(k)}$$

(5.9)

$$\frac{\partial E_n}{\partial w_{ij}^{(k)}} = \frac{\partial E_n}{\partial z_i^{(k+1)}} \cdot \frac{\partial z_i^{(k+1)}}{\partial w_{ij}^{(k)}} = \delta_i^{(k+1)} a_j^{(k)} \quad (5.10)$$

可看到，利用公式（5.9），每一层的辅助变量 $\delta_j^{(k)}$ 可由下一层的 $\delta_i^{(k+1)}$ 计算得到。

在最后一层（记其层数编号为 L ）， $y = a_j^{(L)}$ （此时 j 只有一个取值，可略去），对于回归问题和分类问题，基于公式（5.7）（5.8）可给出统一的简单结果：

$$\delta_j^{(L)} = y_n - t_n = a_j^{(L)} - t_n \quad (5.11)$$

因此，神经网络计算函数值 y 时，是从左到右（正向）逐层迭代计算激活信号 $a_j^{(k)}$ ；

而在计算导数时则是从右到左（反向）逐层迭代计算 $\delta_j^{(k)}$ ，即“正向计算激活，反向计算导数”。这种计算导数的方法也被称为“误差反向传播算法”（error backpropagation），是 Geoffrey Hinton 等人在 1986 年首先提出来的^{jj}。

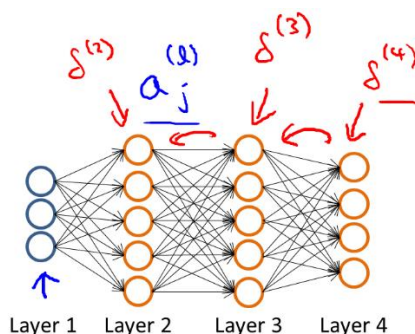


图 5.5. 误差反向传播算法示意图。利用公式（5.5）（5.6）正向计算激

活 $a_j^{(k)}$ ，利用公式（5.9）（5.10）反向计算导数 $\delta_j^{(k)}$ 。

值得指出的是，误差反向传播算法具有非常高的效率。通常，计算单个函数导数的计算量与计算函数本身的计算量在量级上大致相当。对于具有 W 个参数（ $w_{ij}^{(k)}$ ）的神经网络，如果想计算所有的 W 个偏导数，是不是需要的计算量是单个偏导数（或函数值本身）的 W 倍？考虑到实际应用中往往有 $W \gg 1$ ，这将使计算量陡增。幸运的是，利用误差反向传播算法计算所有偏导数的总计算量，与正向计算激活与函数值所需要的计算量，大体是相当的！并不需要 W 倍计算量。

^{jj} David E. Rumelhart, Geoffrey E. Hinton* and Ronald J. Williams. Learning representations by back-propagating errors. *Nature* **323**, 533 (1986).

2.3 神经网络的训练

有了梯度 $\frac{\partial E_n}{\partial w_{ij}^{(k)}}$, 就可以利用梯度下降算法或其它优化方法来优化参数 $w_{ij}^{(k)}$ 的值。参数的初值一般取零附近的随机值。由于神经网络多层结构的存在, 误差函数是非凸的, 与线性回归和逻辑回归不同, 参数优化过程的收敛更加困难。一般可以从不同初值出发, 做多次独立训练, 得到的结果很可能不同, 即收敛到不同的地方。

神经网络训练经常采用随机梯度下降法 (SGD, 参见第二章第 4.2.1 节), 不过不是原始意义下的单实例 SGD (每次只考虑一个 $(\mathbf{x}_n, t_n)$ 样本), 而是小批量随机梯度下降 (mini-batch SGD)。假设训练数据集 $\mathcal{D}_{\text{train}}$ 包含 N 个样本, mini-batch SGD 把这些样本分成 N/b 个 mini-batch (批次), 每个批次包含 b 个样本, b 被称为 batch size。每一步 (step) 使用其中的一个 mini-batch 用于计算误差梯度并利用梯度下降法的公式更新一次参数的值; 跑完 N/b 步则所有 mini-batch 都被使用过一次, 整个 $\mathcal{D}_{\text{train}}$ 完成一轮训练, 称为一个 Epoch; 完成一个 Epoch 后, 训练样本重新随机分配到各个 mini-batch, 重复上述步骤, 进行下一个 Epoch 的训练。模型的完整训练由多轮 Epoch 构成。其中批次数目 b 、Epoch 总数目都是超参数, 会影响最终模型的性能。

当参数增多时, 模型容量上升, 神经网络也会发生过拟合 (图 5-6)。防止过拟合的一个基本方法是采用正则化, 例如在误差函数中加上对参数大小的惩罚 ($\frac{\lambda}{2} \mathbf{w}^T \mathbf{w}$)。另一种在神经网络训练中经常采用的技巧是早停法 (early stopping), 即 Epoch 总数目不是越大越好, 而是在误差降低到一定程度后就提前终止优化。此外, 还可以采用类似于贝叶斯线性回归的那种彻底的贝叶斯学派观点, 认为 $\mathbf{w}$ 不是单一的最佳值, 而是具有一定的分布, 由先验概率与似然函数所决定, 这被称为贝叶斯神经网络, 此处不再赘述。

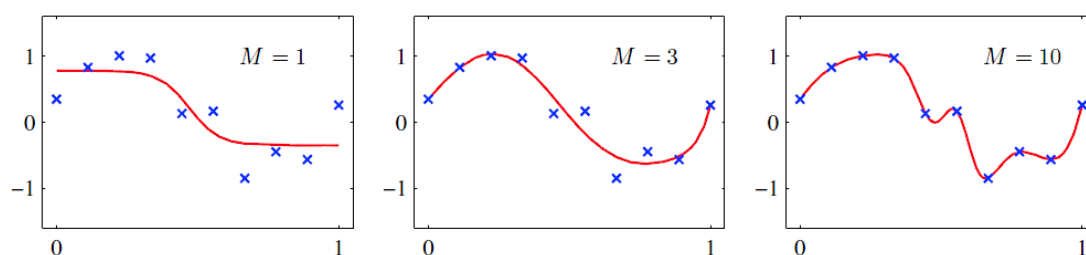


图 5.6. 神经网络的过拟合。图中离散点是训练数据, 是在正弦函数的基础上加上噪声得到的。实线是神经网络的训练结果。(a-c) 的神经网络模型分别使用了 1, 3, 10 个隐藏层节点。

第 3 节 卷积神经网络

3.1 想法：对称性的应用

利用对称性来简化模型，是物理与化学中的一种重要思想。卷积神经网络（Convolutional Neural Network, CNN）可看做是类似思想的应用。

上一节介绍的是神经网络的普适框架。如果中间层的节点数目较多，直接套用普适框架就会带来一个致命的缺点：参数太多。如果每层的节点数目为 M ，则两层之间的节点间连线（ $w_{ij}^{(k)}, i, j = 1, \dots, M$ ）数目为 M^2 。对于一个像素分辨率为 50×50 的图片，在输入层有 2500 个节点，如果隐藏层也有类似的节点数目，那么最终的参数（ $w_{ij}^{(k)}$ ）数目将有几百万之多。

卷积神经网络是针对图像识别问题提出来的，可以大大减少神经网络中独立参数的数量，其想法大致解释如下。

前面介绍过，神经网络是一种分层的模型，先从输入信息识别一些简单的模式，然后把已识别的模式组合起来构建越来越复杂的概念。例如，模型要识别图片中的一只猫，只需要看是否有猫尾、是否有猫嘴、是否有猫眼；如果都有，那就预测说图片是一只猫；而其中识别猫尾，又是由更简单的斑点、短线、斑纹等要素组成。由于猫尾（或斑点、短线）可以出现在不同位置，我们可以想象，也许神经网络隐藏层中就有很多对应的节点，来识别“这个位置有个小横线”“那个位置有个小横线”，或者说，每个位置都有节点来识别所在之处的“小横线”。此时，我们会发现，这些不同节点所要完成的任务其实是类似的，唯一差别就是位置不同。因此，这里就有个很强的对称性：平移对称性。如果有个节点 i （及其输入侧的线性权重 $w_{ij}^{(k)}$ ）能通过公式（5.5）（5.6）识别此处的“小横线”，那只要把它移到另一个地点 i' （两者间位移记为 $\Delta = i' - i$ ），就可以识别 i' 处的“小横线”，即有 $w_{i',j'}^{(k)} = w_{i'-\Delta, j'-\Delta}^{(k)} = w_{i,j-\Delta}^{(k)}$ ，参数 $w_{i',*}^{(k)}$ 可由 $w_{i,*}^{(k)}$ 推断得到，独立的参数减少了。

这种模式识别还有另一个特点，就是信息的局域性。例如，要识别 i 处是否有“小横线”，只需要 i 附近的像素信息，而不需要整张图片，即 $w_{ij}^{(k)}$ 只有在 i 与 j 比较接近时才有非零值。

这两个重要性质，平移不变性（等变性）与局域性，正是构建卷积神经网络的根据。

3.2 卷积与滤镜

我们先以图像识别的一个简化例子（图 5.7）来直观地看一下什么是卷积。

假设我们有一张大小为 100×100 的黑白图片。为了检测图片中物体或图案的边缘，设计了下面的大小为 3×3 的矩阵（卷积核，convolution kernel）：

$$K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} \quad (5.12)$$

我们从图片中取出一个大小与卷积核相同的区块，将其与卷积核相同位置的元素执行乘法后求和（类似于向量点乘，只是现在是两个相同大小的矩阵的“点乘”），则当区块来自于图案的竖直边缘（左边亮右边暗，或左边暗右边亮）的时候求和结果的绝对值会比较大，而当区块来自于亮度均匀的区域时求和结果则接近于 0。因此结算结果可以作为此区块是否处于边缘的定量指标。我们可以移动（扫描）图片，提取不同区块进行相同的运算，从而得到图片每个位置是否处于边缘的结果，这些结果合起来（形成与原图片大小近似相同^{kk}——严格讲是 98×98 ——的矩阵）就得到总的卷积结果（图 5.8）。

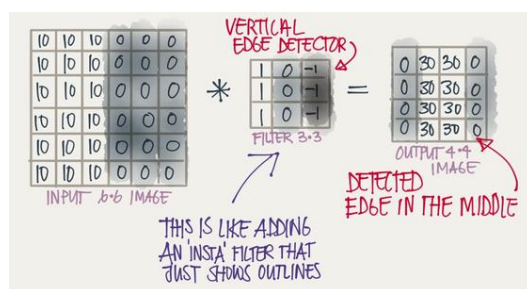


图 5.7. 图像卷积操作示意图。卷积核（小方块）在图片（大方块）上移动扫描。每在一个位置，把卷积核矩阵和其下的图片像素进行逐点相乘后求和，求和结果作为所在位置的卷积结果。所有位置的卷积结果构成一个大小与原图像大致相等的矩阵。

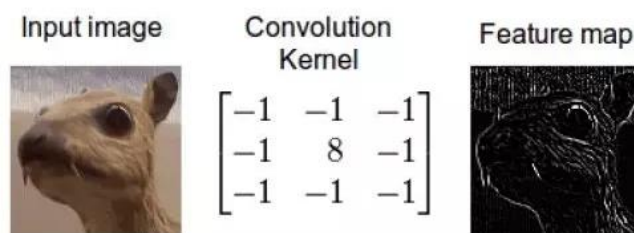


图 5.8. 利用卷积操作进行图像的边缘检测。

在数学上，二维（矩阵）数据 I 与 K （相当于上面例子中的图片与卷积核）的卷积 $S = I * K$ 可以定义如下^{ll}：

$$S(i, j) = \sum_{m, n} I(m, n) K(i - m, j - n) \quad (5.13)$$

^{kk} 卷积神经网络中有时使用填充（padding）的技巧来使卷积矩阵的大小与原图片严格相同。

^{ll} 卷积有多种不同定义的形式。这里采用的形式满足交换律： $I * K = K * I$ 。

其中 $S(i, j)$ 是矩阵 S 的第 i 行第 j 列的元素。更高维或更低维（一维）数据的卷积也可类似定义。数学上，卷积^{mm}还可以定义在两个函数 $f(x)$ 与 $g(x)$ 上：

$$(f * g)(x) = \int f(\tau)g(x - \tau)d\tau \quad (5.14)$$

手机与电脑上的很多图像处理软件中都提供了各种滤镜效果，看起来很炫酷。但其实它们的基本原理都很简单，就是公式（5.13）所示的卷积变换。利用不同的卷积核，就可以实现各种不同效果。例如，公式（5.12）所示的卷积核可以实现

边缘检测， $K = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$ 可实现图像的锐化， $K = \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$ 可实现图

像的模糊化，等等。

3.3 卷积神经网络的具体架构

比较神经网络中的线性组合公式（5.5）与上面的卷积公式（5.13），可以发现它们形式上非常相似。卷积神经网络最重要的变动就是用公式（5.13）代替（5.5），相当于是把（5.13）中的 $I(m, n)$ 看做原来的 $a_j^{(k)}$ ， $S(i, j)$ 看做原来的 $z_i^{(k+1)}$ ，而卷

积核 $K(i - m, j - n)$ 看做原来的参数 $w_{ij}^{(k)}$ 。卷积核大小是可设置的超参数，通常比图片小得多，以反映信息的局域性。由于平移不变性与信息局域性，卷积核所带来的独立参数数目（在相同隐藏层节点的条件下）远远少于通常的神经网络。卷积神经网络的节点激活函数（公式（5.6））通常不做改变，每层隐藏层节点（其上的非线性激活）及其左边的卷积线性变换合起来被称为一个卷积层。

除了卷积层以外，卷积神经网络通常还包含另一种特殊的模块，被称为池化（pooling）（也叫下采样 sub-sampling）层。用卷积核在前一层的矩阵数据上进行滑动扫描的过程中，实际上“重叠”计算了很多冗余的信息。例如，考虑图片上一段规则的边缘（左边像素取值都为 10，右边像素都为 0），则卷积核公式（5.12）将在边缘左右两列位置上都给出结果为 30 的“边缘存在”的信号。池化操作的目的是去除这些冗余信息，并加快运算的速度。具体地，池化就是一种特殊的映射，把矩阵的多个相邻矩阵元映射成一个矩阵元，从而降低矩阵大小（降低分辨率，去除冗余信息）。具体的映射有各种方案，如“最大池化”（max-pooling）就是取这些矩阵元中的最大值，而“平均池化”（average-pooling）则是取其平均值。池化层通常不包含自由参数。一个 100×100 的矩阵，通过 2×2 的池化运算后，将变成一个 50×50 的矩阵。

^{mm}卷积在科学与工程领域具有广泛应用，其中一个重要原因在于卷积定理（convolution theorem）：函数卷积的傅立叶变换是函数傅立叶变换的乘积，即 $\widehat{(f * g)}(k) = \hat{f}(k)\hat{g}(k)$ ，卷积与乘积具有对偶关系。在数值计算上，有高效的算法（快速傅里叶变换 FFT）可以实现傅立叶变换，因此卷积定理也经常被利用来（结合 FFT）计算卷积 $f * g$ 。

在卷积神经网络中，还有另一个概念，通道（channel）。在输入层（原始图片），通道可理解为不同的颜色分量，例如通常的彩色图片有 3 个通道。而在隐藏层，通道数目是可自由设置的超参数，相当于允许每个隐藏层有多个矩阵数据（类似于图层的概念），从而增加了特征的数量。

把这些模块灵活组合起来，就可构建一个完整的卷积神经网络。它通常是由多个卷积层加池化层和最后一个或多个完整连接层（full connected）构成。例如，以 LeCun 在上世纪八十年代发展的用于手写数字识别的 LeNet-5 网络为例，它的输入是 $32 \times 32 \times 3$ （分辨率 32×32 的彩色图片，通道数 3），经过 5×5 卷积核作用后变成 $28 \times 28 \times 6$ 大小（通道数 6），经过 2×2 池化后变成 $14 \times 14 \times 6$ 大小；再经过 5×5 卷积（变成 $10 \times 10 \times 16$ ）、 2×2 池化（变成 $5 \times 5 \times 16$ ）；然后把结果重新排列成 400×1 的向量（或者说，视为通常神经网络的隐藏层节点），随后接两个通常神经网络的隐藏层（节点数分别为 120 和 84），层间采用全连接（即不考虑平移对称性，不使用卷积）；最后加一个包含 10 个节点的输出层（分别对应识别结果“0” - “9”），激活函数采用 softmax 函数。

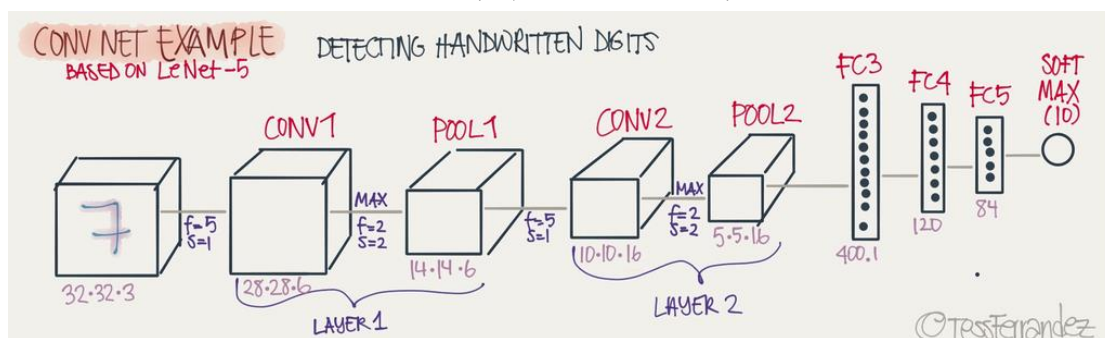


图 5.9. 卷积神经网络的完整架构：以 LeCun 用于手写数字识别的 LeNet-5 网络为例。

人脸识别（刷脸）技术，主要就是由卷积神经网络所驱动的。

3.4 循环神经网络

对于视频、语音、文本等序列数据，与图片识别的例子类似，如果用神经网络来描述，其参数应该具有某种时间平移不变性。利用这种对称性的重要模型，就是循环神经网络（Recurrent Neural Network, RNN）。

循环神经网络是一类具有短期记忆能力的神经网络。给定一个输入序列 $\mathbf{x}_{1:T} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T\}$ ，循环神经网络通过如下公式计算带反馈的信号：

$$\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1}) \quad (5.15)$$

其中 f 用一个前面介绍的神经网络来描述。而输出则是 $\mathbf{h}$ 的函数，即 $\mathbf{y}_t = g(\mathbf{h}_t)$ 。循环神经网络能够处理任意时间长度的时序数据，就如卷积神经网络能够处理任意尺寸的图片数据。

循环神经网络被广泛应用在语音识别、语言模型以及自然语言生成等任务上，此处不再赘述。

第 4 节 神经网络的应用例子

神经网络现在是机器学习中应用最广泛的模型。下面简单介绍文献中的几个例子。

4.1 机器学习力场

神经网络在化学领域中的一个重要应用方向是机器学习力场。在这方面，一个较早的代表性例子是 Parrinelloⁿⁿ等人的工作：

Jörg Behler and Michele Parrinello.

Generalized neural-network representation of high-dimensional potential-energy surfaces.

Phys. Rev. Lett. **98**, 146401 (2007).

在化学学科中，分子力学（molecular mechanics）是利用经典力学的相关概念和方法来研究分子性质的重要方向。在玻恩-奥本海默近似下，分子势能仅是原子核的位置的函数，分子体系具有势能面，可以用经典力学描述与求解。体系势能 V 随原子核位置（分子结构） $\{\mathbf{R}_i\}$ 而变化的函数关系 $V\{\mathbf{R}_i\}$ 被称为力场或力场函数，需通过与量子力学结果或实验结果的比较来确定。在 $V\{\mathbf{R}_i\}$ 的基础上，就可运用分子动力学、蒙特卡罗模拟、构象分析等方法来求解分子体系的性质。与量子力学计算相比，分子力学方法的计算量要小得多，被广泛应用于药物、团簇体系和生物大分子等体系的研究。

传统的力场模型假设 $V\{\mathbf{R}_i\}$ 可以表示为一些简单模式的贡献之和：键伸缩能（化学键在键轴方向上的伸缩运动所引起的能量变化，用弹簧振子来描述，即能量为键长的二次函数）；键角弯曲能（表示为键角的二次函数）；二面角扭转能（表示为二面角的三角函数形式）；非键相互作用（范德华力、静电相互作用）。这类模型形式比较简单，计算相对比较容易，但模型的容量较小，不容易达到很高的精度（特别是和化学精度比）。因此，另外一个思路是把力场的确定看做一个机器学习问题，利用神经网络等模型来描述 $V\{\mathbf{R}_i\}$ 的复杂函数依赖关系，被称为机器学习力场。

Parrinello 等人在这个工作中研究的对象是硅材料。他们利用量子化学 DFT 方法计算了 64 个 Si 原子组成的体系（采用周期性边界条件）在 9000 个不同结构/构象（ $\{\mathbf{R}_i\}$ ）下的能量 E （原文用的能量符号，它包含了电子的动能。在分子力学中，它被视为原子核自由度下的势能，即上述力场 V ），并把得到的数据集分

ⁿⁿ Parrinello 最著名的工作是理论与计算化学领域里的 Car-Parrinello 动力学。

成训练集（8200 个数据）与测试集（800 个数据）。所用的神经网络具有两个隐藏层，每层包含约 40 个节点；隐藏层激活函数使用 $\tanh$ ，输出层使用恒等函数；层间使用全连接。

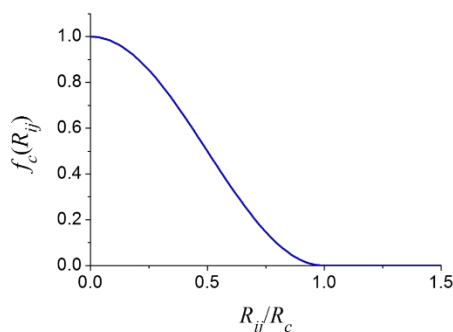


图 5.10. 距离函数 f_c ，具体表达式见公式 (5.17)。

该工作并没有直接把原子位置 $\{\mathbf{R}_i\}$ 作为神经网络的输入（可能是函数 $V\{\mathbf{R}_i\}$ 还是太复杂，基于几千个数据很难得到好的效果），而是结合领域的专家知识，把体系的总能量写成

$$E = \sum_i E_i(\{G_i^\mu\}) \quad (5.16)$$

其中 E_i 代表第 i 个原子在周围环境的影响下对总能量的贡献，通过神经网络来预测。 $\{G_i^\mu\}$ 是基于原子位置 $\{\mathbf{R}_i\}$ 计算出来的分子结构描述符 (structural descriptor)，文章中称为 symmetry function，用来作为神经网络的直接输入（特征）。先定义如下的距离函数

$$f_c(R_{ij}) = \begin{cases} \frac{1}{2} \left[\cos\left(\frac{\pi R_{ij}}{R_c}\right) + 1 \right] & (\text{for } R_{ij} \leq R_c) \\ 0 & (\text{for } R_{ij} > R_c) \end{cases} \quad (5.17)$$

其中 R_{ij} 是原子 i 和 j 之间的距离。 $R_c = 6 \text{ \AA}$ 是固定参数。 f_c 取值在 0 和 1 之间，描述了两个原子是否靠近（成键）。这种形式相对于直接使用 R_{ij} 的好处是它在两个原子距离大于 R_c 时 $f_c = 0$ ，输入到神经网络后对结果没什么贡献，即距离远的原子之间没有相互作用。这相当于利用专家知识来进行特征工程设计，使神经网络更容易学习到数据中的规律。在距离函数的基础上，进一步定义两种分子结构描述符：描述键长/距离依赖的 radial symmetry functions

$$G_i^1 = \sum_{j \neq i} e^{-\eta(R_{ij}-R_s)^2} f_c(R_{ij}) \quad (5.18)$$

与描述键角/角度依赖的 angular symmetry functions

$$G_i^2 = e^{1-\zeta} \sum_{j,k \neq i} (1 + \lambda \cos \theta_{ijk})^\zeta e^{-\eta(R_{ij}^2 + R_{ik}^2 + R_{jk}^2)} f_c(R_{ij}) f_c(R_{ik}) f_c(R_{jk}) \quad (5.19)$$

其中 θ_{ijk} 是原子 i, j, k 之间的夹角。 $\eta, R_s, \lambda (= +1, -1), \zeta$ 是超参数（特征工程的参数，在神经网络中并不能对这些参数进行梯度下降优化）。计算中采用了 48 个具有不同 $\eta, R_s, \lambda, \zeta$ 值的 symmetry functions 作为神经网络的输入（即公式（5.16）中的 $\{G_i^\mu\}$ ）。

神经网络力场模型的最终效果如下：训练误差约 4-5 meV/atom，测试误差约 5-6 meV/atom；力的误差约 0.2 eV/Å。对硅在 3000K 温度下的分子动力学模拟结果（原文图 3）表明，利用神经网络力场进行模拟得到的径向分布函数与 DFT 结果很接近，符合程度好于其它几种传统力场。

Parrinello 等人的上述工作是在深度学习兴起之前做的。在深度学习驱动人工智能进入目前这波最新的繁荣期以后，机器学习力场成为 AI for Science 领域的热门方向之一，有大量的新工作出现。下面是几个代表性的相关工作：

- J. S. Smith, O. Isayev and A. E. Roitberg. ANI-1: an extensible neural network potential with DFT accuracy at force field computational cost. *Chem. Sci.* **8**, 3192 (2017).

沿用 Parrinello 等人的大致架构。考虑了最多包含 8 个重原子（C, N, O）的 5.8 万种有机小分子，共 1.7 千万个构象。神经网络具有 3-4 个隐藏层，每层 32-128 个节点；使用了多达 768 个由公式（5.18）（5.19）所描述的 G_i^μ 。最终的测试误差大约为 1.8 kcal/mol。

- Si-Da Huang, Cheng Shang, Pei-Lin Kang and Zhi-Pan Liu（刘智攀，复旦大学）。Atomic structure of boron resolved using machine learning and global sampling. *Chem. Sci.* **9**, 8644 (2018).

研究了结构多变的硼体系。使用了涵盖体相、层状、团簇体系的 16 万个构象-能量数据。改进了分子结构描述符的形式。神经网络包含 2 个隐藏层，每层 110 个节点；使用了 173 个描述符，即输入层有 173 个节点。最终的测试集能量误差 12.4 meV/atom，力的误差 0.28 eV/Å。

- Linfeng Zhang（张林峰），Jiequn Han, Han Wang（王涵），Roberto Car, Weinan E（鄂维南）。Deep potential molecular dynamics: a scalable model with the accuracy of quantum mechanics. *Phys. Rev. Lett.* **120**, 143001 (2018).

研究了水、苯 Benzene、尿嘧啶 Uracil、萘 Naphthalene、阿司匹林 Aspirin、水

杨酸 Salicylic acid、丙二醛 Malonaldehyde、乙醇 Ethanol、甲苯 Toluene 等化学小分子体系的性质。以 $\mathbf{D}_{ij} = \left\{ \frac{1}{R_{ij}}, \frac{x_{ij}}{R_{ij}^2}, \frac{y_{ij}}{R_{ij}^2}, \frac{z_{ij}}{R_{ij}^2} \right\}$ 为分子结构描述符，其中 $\{x_{ij}, y_{ij}, z_{ij}\}$ 是原子 i, j 间的位移矢量在原子 i 的局域坐标系（由原子 i 及其两个最近的键连原子所决定）里的分量。与 Parrinello 等人的工作不同的是，这里没有对 j 求和，因此保留了更多的原始信息。对每个原子 i ，考虑其周围 $R_c = 6 \text{ \AA}$ 内所有近邻原子的影响，并将 $\mathbf{D}_{ij}$ 按一定顺序有序排列，作为神经网络 $E_i(\{\mathbf{D}_{ij}\})$ 的输入。神经网络有 5 个隐藏层，每层节点数分别为 240, 120, 60, 30, 10。最终的能量误差从 2.4 到 8.7 meV/atom 不等，大部分情况下小于 5 meV/atom；力的误差大约为 0.01 eV/Å 左右。

4.2 基于结构式的分子性质预测

在前面的例子中，分子中原子的三维坐标（分子结构）是已知的，基于此来预测分子性质。在更常见的情况下，我们可能只知道是哪种分子（即原子间的键连情况），只基于分子的这种简化描述，也可以对分子的性质进行预测。比如下面这个例子：

Alessandro Lusci, Gianluca Pollastri, and Pierre Baldi.

Deep architectures and deep learning in chemoinformatics: the prediction of aqueous solubility for drug-like molecules.

J. Chem. Inf. Model. **53**, 1563 (2013).

这个工作的目的是预测分子的水溶性。已知的数据集只包含约 3000 个数据，因此面临的主要困难是数据量不多。前面讲过，利用卷积神经网络与循环神经网络，可以大大地减少模型中独立参数的数量。这个工作就是采用了循环神经网络的思路。

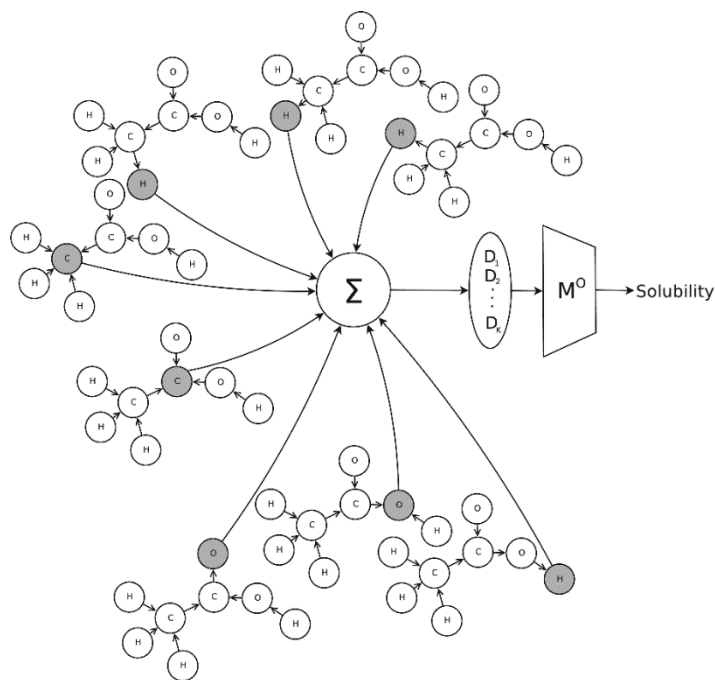


图 5.11. 无向图循环神经网络 UGRNN 示意图。图来自文献。

该工作发展了一种无向图循环神经网络 (Undirected Graph Recursive Neural Networks, UGRNN)。它的主要想法是分子中的某个原子的性质会受到周围 (第一圈) 键连原子的影响, 而后者又会受到其键连的更外围 (第二圈) 原子的影响, 然后再往外一圈圈地扩展施加影响的区域, 直到组成分子的所有原子都被涵盖在内。然后模型把上述过程反过来 (图 5.11), 先考虑最外面一圈原子性质的影响 (利用一个神经网络 M^G) 来计算次外层原子的性质, 然后用得到的这个次外层原子的性质来影响往里一层原子的性质 (利用同一个神经网络 M^G , 这就是公式 (5.15) 的主要思想), 以此类推, 一直算到最开始考虑的原子 (中心原子)。注意分子中的所有原子需要依次作为中心原子用上述流程 (被称为一个子图) 算一遍。 M^G 的作用可以使用数学公式表示成:

$$G_{v,k} = M^G \left(i_v, G_{pa_{[v,k]}^1}, \dots, G_{pa_{[v,k]}^n} \right) \quad (5.20)$$

其中 v 是原子编号, k 是子图编号, i_v 代表原子 v 的原始信息 (如原子类型)。 $G_{pa_{[v,k]}^1}, \dots, G_{pa_{[v,k]}^n}$ 是与原子 v 键连的、在子图 k 中位于原子 v 外面一层的原子的性质。接下来, 所有子图的中心原子的性质被简单加和起来:

$$G_{\text{structure}} = \sum_k G_{r_k,k} \quad (5.21)$$

并作为输入送到第二个神经网络 M^O 中:

$$p = M^O(G_{\text{structure}}) \quad (5.22)$$

从而得到最终想要预测的性质 p 。而前面的 $G_{v,k}$ 可看做广义的隐藏层信号，不能直接在实验中测量。

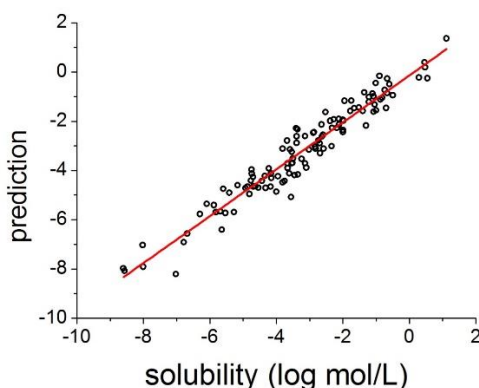


图 5.12. UGRNN 对分子水溶性的预测效果（测试集）。直线为线性拟合结果，关联系数 $R^2 = 0.92$ 。

两个神经网络 M^G 与 M^O 都只有一个隐藏层。 M^O 的隐藏层有 5 个节点； M^G 的隐藏层节点数目与输出层节点数目在 3-12 范围。通过这种类似于循环神经网络的架构，独立参数数量得到了有效的压缩。训练时采用十折交叉验证。最后得到的模型性能很不错：对分子水溶性的预测误差约为 0.58，预测值与真实值之间的关联系数为 $R^2 = 0.92$ （图 5.12）。

UGRNN 除了能应用于回归问题外，也能应用于分类问题。例如，下面这个例子：

Youjun Xu (徐优俊), Ziwei Dai, Fangjin Chen, Shuaishi Gao, Jianfeng Pei, and Luhua Lai (来鲁华, 北京大学).

Deep learning for drug-induced liver injury.

J. Chem. Inf. Model. **53**, 1563 (2013).

研究了药物肝损伤预测这个分类问题，利用 UGRNN，只使用了约 1000 个已知实验数据，就达到了 86.9% 的预测准确率。

4.3 神经网络作为万能函数应用于量子化学计算

神经网络的万能近似函数性质可以被用于求解薛定谔方程，为量子化学计算提供新手段。例如，下述工作就是一个例子：

David Pfau, James S. Spencer, and Alexander G. de G. Matthews.

Ab-initio solution of the many-electron Schrödinger equation with deep neural networks.

Phys. Rev. Res. **2**, 033429 (2020).

薛定谔方程是量子化学最重要的理论基础，它形式上很简单：

$$\hat{H}\psi(\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_N) = E\psi(\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_N) \quad (5.21)$$

$$\hat{H} = -\frac{\hbar^2}{2m} \sum_i \nabla_i^2 + \sum_{i < j} \frac{e^2}{4\pi\epsilon_0 |\mathbf{r}_i - \mathbf{r}_j|} - \sum_{i, I} \frac{Z_I e^2}{4\pi\epsilon_0 |\mathbf{r}_i - \mathbf{R}_I|} + \sum_{I < J} \frac{Z_I Z_J e^2}{4\pi\epsilon_0 |\mathbf{R}_I - \mathbf{R}_J|} \quad (5.22)$$

其中 $\mathbf{r}_i$ 是电子 i 的位置矢量， $\mathbf{R}_I$ 是原子核 I （对应的原子序数为 Z_I ）的位置矢量， $\psi(\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_N)$ 是电子波函数。这里应用了玻恩-奥本海默近似，求解本征方程（5.21）时假设原子核位置 $\mathbf{R}_I$ 是不变的，因此解出来的本征能量 E 依赖于 $\mathbf{R}_I$ ，就是前述的分子体系的势能面或力场。

从理论上讲，如果给定薛定谔方程的准确解，几乎所有的化学反应都可以通过第一原理推导出来。但在实践中，这几乎是不可能的，原因是薛定谔方程很难求解：波函数 $\psi(\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_N)$ 是个高维函数，求解过程会碰到维度诅咒问题；它还必须具有交换反对称性的怪异性质（这来自于费米-狄拉克统计的要求）。因此，实际所使用的求解方法都需要对波函数的形式做某种近似假设（ansatz），以方便求解。例如，在量子化学中，经常采用单电子波函数的 Slater 行列式来作为满足交换反对称性的多电子波函数的基：

$$\det[\Phi^k] = \det[\phi_i^k(\mathbf{r}_j)] = \begin{vmatrix} \phi_1^k(\mathbf{r}_1) & \cdots & \phi_1^k(\mathbf{r}_N) \\ \vdots & \ddots & \vdots \\ \phi_N^k(\mathbf{r}_1) & \cdots & \phi_N^k(\mathbf{r}_N) \end{vmatrix} \quad (5.23)$$

$$\psi(\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_N) = \sum_k w_k \det[\Phi^k] \quad (5.24)$$

其中 ϕ_i^k 是单电子波函数，通过某些自洽方法近似求解。公式（5.24）与线性回归方法非常类似，就是把一些固定的基函数 $\det[\Phi^k]$ 线性组合起来。与本章开头的分析类似，这种固定基方案的最大缺点就是不够灵活，需要非常多个基函数才能达到良好的效果，也就是说，求解时需要大量的 Slater 行列式。那么，遵循神经网络的思路，一种很自然的想法就是采用适应基来代替固定基，从而大大减少所需要基函数的数量。这是将机器学习与量子化学结合时的一种指导思想。上述 Pfau 等人的工作就是一个例子。他们建议将公式（5.23）中的单电子波函数 $\phi_i(\mathbf{r}_j)$ 替换成如下的多电子波函数：

$$\phi_i(\mathbf{r}_j) \rightarrow \phi_i(\mathbf{r}_j; \mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_{j-1}, \mathbf{r}_{j+1}, \dots, \mathbf{r}_n) = \phi_i(\mathbf{r}_j; \{\mathbf{r}_{/j}\}) \quad (5.25)$$

它要求对 j 以外的坐标（ $\{\mathbf{r}_{/j}\}$ ）满足交换对称性。容易验证，新的行列式基函数 $\det[\phi_i(\mathbf{r}_j; \{\mathbf{r}_{/j}\})]$ 仍然是交换反对称的。现在的 $\phi_i(\mathbf{r}_j; \{\mathbf{r}_{/j}\})$ 包含了其它原子的影响，因此更为灵活。它是一个多变量函数，进一步使用神经网络来描述。

网络的输入直接采用位移坐标信息：

$$\mathbf{h}_i^0 = \{\mathbf{r}_i - \mathbf{R}_I, |\mathbf{r}_i - \mathbf{R}_I|\} \quad (5.26)$$

$$\mathbf{h}_{ij}^0 = \{\mathbf{r}_i - \mathbf{r}_j, |\mathbf{r}_i - \mathbf{r}_j|\} \quad (5.27)$$

其中上标“0”表示这是第0层（即输入层）的输出信号。为了有效混合各个电子的信息，将其它电子的输出求平均后连接（concatenate）到电子*i*的输出上，

$$\left(\frac{1}{N}\sum_{j=1}^N \mathbf{h}_j^l, \frac{1}{N}\sum_{j=1}^N \mathbf{h}_{ij}^l, \mathbf{h}_i^l\right) = (\mathbf{g}^l, \mathbf{g}_i^l, \mathbf{h}_i^l) \rightarrow \mathbf{f}_i^l \quad (5.28)$$

然后才作为下一层的输入：

$$\mathbf{h}_i^{l+1} = \tanh(\mathbf{W}^l \cdot \mathbf{f}_i^l) + \mathbf{h}_i^l \quad (5.29)$$

$$\mathbf{h}_{ij}^{l+1} = \tanh(\mathbf{V}^l \cdot \mathbf{h}_{ij}^l) + \mathbf{h}_{ij}^l \quad (5.30)$$

得到下一层的输出 $\mathbf{h}_i^{l+1}$ 与 $\mathbf{h}_{ij}^{l+1}$ 。其中 $\mathbf{W}^l$ 与 $\mathbf{V}^l$ 是参数。式子右边最后一项是残差连接，是深度网络中的常用技巧（参见第15章）。最后一层采用恒等激活函数，并再乘上一个指数，得到需要的 Slater 行列式矩阵元：

$$\phi_i^k(\mathbf{r}_j; \{\mathbf{r}_{/j}\}) = (\mathbf{w}_i^k \cdot \mathbf{h}_j^l) \sum_m \pi_{im}^k \exp(-|S_{im}^k(\mathbf{r}_j - \mathbf{R}_m)|) \quad (5.31)$$

其中 π_{im}^k 与 S_{im}^k 是 3×3 旋转矩阵，也属于待优化的参数。最后利用变分蒙特卡罗方法来优化模型参数。

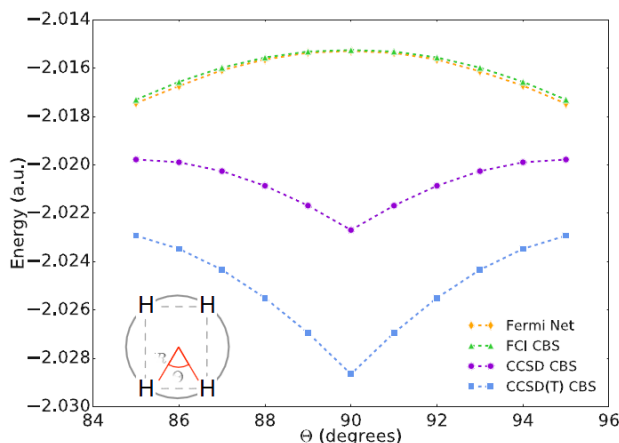


图 5.13. H_4 氢链的解离曲线。耦合聚类方法（CCSD）错误地预测 H_4 在四方构型下最稳定。而 FermiNet 则正确预测出 H_4 将解离为两个 H_2 分子。（图来自文献***）

整个架构被称为费米神经网络（Fermionic Neural Network，简称 FermiNet）。

具体地，网络使用了 4 个隐藏层，每层 256 个（单电子）或 32 个（双电子）节点。公式 (5.24) 采用了 16 个行列式。模型共有 70 万个参数。工作中研究了几个具体的小分子体系，结果发现 FermiNet 能够正确预测出氮分子和 H_4 氢链的解离曲线（图 5.13）——而这两个体系是很有难度的强关联系统。相比于之前被广泛视为量子化学黄金标准的耦合聚类方法（coupled cluster method），这种新方法得到的准确度显著更高。

4.4 贝叶斯网络在核裂变产物预测中的应用

最后一个例子是原子核在中子轰击下的裂变产物预测：

Zi-Ao Wang, Junchen Pei*（裴俊琛，北京大学）, Yue Liu, and Yu Qiang.

Bayesian evaluation of incomplete fission yields.

Phys. Rev. Lett. **123**, 122501 (2019).

新闻报导：<https://news.pku.edu.cn/jxky/75be264392a6479482ed3f395b5b509d.htm>

原子核裂变数据的精确测量是很多核物理/核化学应用的先决条件，比如核能、国防、辐射防护、核废料处理、稀有同位素制备等。特别是对于新一代核能-快中子反应堆，急需铀系核区不同能量中子诱发裂变的精确数据，但目前实验和理论很难提供完整可靠的能量相关裂变数据。国际上主要的核数据库只有有限能量点的裂变数据；理论上基于微观模型去描述裂变过程非常复杂，对裂变可观测量的描述还有较大误差；而基于唯象裂变模型对未知核区和未知能区的外推结果往往不可靠。因此，该工作利用神经网络方法对已有裂变数据库进行学习，以便能够对完整的裂变产物分布进行预测。

在裂变实验中，具有一定动能（被称为核裂变的激发能量）的中子束入射轰击（铀系）样品，有一定几率把样品中的原子核打成两半，这个几率类似于化学反应中的产率，就是模型要预测的输出。因此，这是一个典型的回归问题：

产率 $\sim$ （核质子数、核中子数、激发能量、裂片质量）

从机器学习的角度讲，这并不是一个很困难的问题（只有四个输入变量，不会面临维度诅咒）。不过，这里问题是已有的数据不是很多。为了充分挖掘数据中的信息，该工作采用了贝叶斯神经网络。与贝叶斯线性回归类似，贝叶斯神经网络认为模型中的参数 $\mathbf{w}$ 不是单一的最佳值，而是具有一定的分布，有一定的先验概率，然后通过与实验数据的对比来推测似然函数，最后得到 $\mathbf{w}$ 在已知数据下的后验概率。进行预测时，它也不是直接预测出一个产率，而是给出产率的概率分布（因为有很多可能的 $\mathbf{w}$ ）。贝叶斯神经网络不容易过拟合，但计算量会增加很多。不过在这个例子中，问题本身比较简单，所用神经网络模型也不会很大，因此增加的计算量不会造成实际困难。

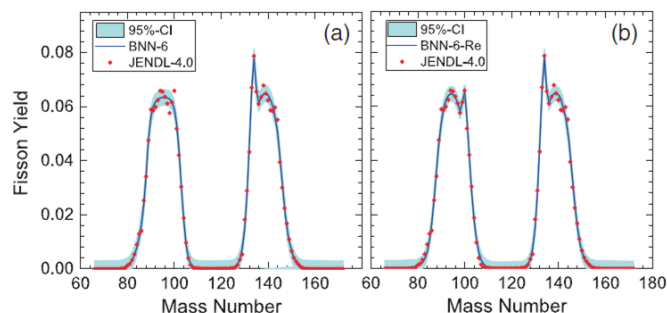


图 5.14. 贝叶斯神经网络对 ^{235}U 在 0.5 MeV 中子轰击下的产物的学习结果。离散点是实验数据，实线是预测的平均值，阴影是预测的 95% 确信度区间。

该工作先对数据最多的 ^{235}U 在能量为 0.5 MeV 的中子轰击下的产物进行简化模型的研究。此时，输入变量只有一个（裂片质量）。只用包含一个隐藏层（6 个节点）的简单贝叶斯神经网络，就能得到很好的效果（图 5.14）。

最后，该工作对 30 种不同的原子核（ $^{227,229,232}\text{Th}$, ^{231}Pa , $^{232,233,234,235,236,237,238}\text{U}$, $^{237,238}\text{Np}$, $^{238,239,240,241,242}\text{Pu}$, $^{241,243}\text{Am}$, $^{242,243,244,245,246,248}\text{Cm}$, $^{249,251}\text{Cf}$, ^{254}Es , ^{255}Fm ）进行系统的研究。使用节点数为 20-40 不等的单隐藏层模型，可以得到良好的效果。结果表明，随着中子能量增加，裂变模式逐渐从不对称裂变演化成对称裂变。这说明神经网络能成功抓住部分物理图像。当实验数据点很少时，以前的唯象模型失效，但贝叶斯神经网络仍然可以给出有效的评估和合理的置信区间。

思考题：逻辑回归能否实现逻辑运算 and, or, not? 但不能实现 XOR

扩展阅读：

- <https://www.cnblogs.com/subconscious/p/5058741.html>
神经网络浅讲：从神经元到深度学习.mht
- <https://www.huxiu.com/article/387018.html>
我是一个平平无奇的 AI 神经元
- <http://neuralnetworksanddeeplearning.com/chap4.html>
A visual proof that neural nets can compute any function.mht
- <http://colah.github.io/posts/2014-03-NN-Manifolds-Topology/>
Neural Networks, Manifolds, and Topology.mht
- <https://www.jiqizhixin.com/articles/2022-01-09>

相较神经网络，大名鼎鼎的傅里叶变换，为何没有一统函数逼近器？答案在这

- <https://www.zhihu.com/question/22553761>
如何简单形象又有趣地讲解神经网络是什么？
 - <http://www.dataguru.cn/article-13355-1.html>
直白介绍卷积神经网络（CNN）.mht
 - <https://www.jiqizhixin.com/articles/2019-05-27-4>
一文带你了解卷积神经网络 CNN 的发展史.mht
 - <https://www.jiqizhixin.com/articles/2019-09-17-8>
用神经网络求解薛定谔方程，DeepMind 开启量子化学新道路
 - <http://news.pku.edu.cn/jxky/75be264392a6479482ed3f395b5b509d.htm>
物理学院裴俊琛课题组在《物理评论快报》发表应用神经网络对核裂变产物的研究进展
 - <https://baijiahao.baidu.com/s?id=1594101736158277117>
「少数派」的深度学习成长史.mhtml
- 在 1984 年上映的《终结者 1》里，机器人终结者的扮演者阿诺德·施瓦辛格有一句台词：「我的 CPU 是一个神经网络处理器，一个会学习的计算机。」

第六章 核方法：近邻法与支持向量机

刘志荣

机器学习的工作机制基本上是相同的：

将临近的样例归类到同一个类别中^{oo}。

第 1 节 密度估计的非参数法

如果我们知道随机变量 $\mathbf{x}$ 的概率密度 $p(\mathbf{x})$ ，就可以据此通过抽样得到 $\mathbf{x}$ 的一些值（样本），这就是采样（sampling）的问题^{pp}。反过来，如果我们有 $\mathbf{x}$ 的一些样本（抽样值） $\{\mathbf{x}_n\} (n = 1, 2, \dots, N)$ ，那么我们能不能据此估计 $\mathbf{x}$ 的概率密度 $p(\mathbf{x})$ 呢？这就是密度估计或分布估计（distribution estimation）的问题，它比采样问题要困难得多。密度估计是机器学习中一个非常重要的问题^{qq}。如果能够知道数据的分布密度，就可以解决各种问题。例如，在第四章第 4.1 节中，我们知道分类器具具有理论意义上的最小误差概率，即由两条类别样本分布曲线（ $p(\mathbf{x}, C)$ ，其中 $C = 0, 1$ ）最小值下的面积给出。如果能从已知训练集中推断出每个分类的分布曲线（属于无监督学习），就可据此进行分类预测（属于监督学习）。

需要注意的是，在线性回归与逻辑回归的章节中，我们是把 $\mathbf{x}$ 看做确定性的、完全可控的变量，把 y 或 t 看做是非确定性的随机量（可以是由于噪声导致的，也可以是本身就是随机量）。而在这里，我们把 $\mathbf{x}$ 与 y 都看做是随机量。

密度估计有两类方法。一类是参数法，即假设 $p(\mathbf{x})$ 的公式形式是已知（带未知参数），比如高斯分布 $\mathcal{N}(x|\mu, \sigma^2)$ ，然后应用最大似然法等方法来确定其中参数的值。另一类是非参数法，不需假设分布曲线的形状。下面介绍的直方图方法、核密度估计法与近邻法都属于非参数法。

^{oo} 引自 Pedro Domingos 《机器学习那些事》（刘知远 翻译）。

^{pp} 例如，我们可以通过抛硬币的方法来对允许取值为 0（正面朝上）与 1（反面朝上）而概率各为 1/2 的离散随机变量进行采样。当然，更容易的是编写一个计算机程序来实现采样。

^{qq} 密度估计也是统计学中的普适问题。统计学中最重要的三大方向是：估计、推断、检验。

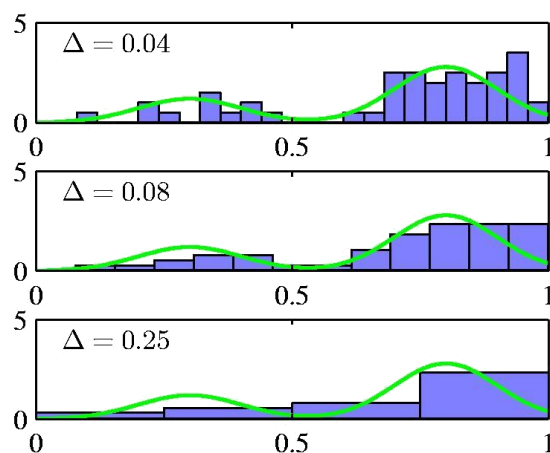


图 6.1. 直方图例子。其中绿色曲线代表真实的概率密度 $p(x)$ 。

1.1 直方图方法

直方图 (histogram) 是大家非常熟悉的一种用于分析数据分布的方法。它一般用横轴表示数据 (x) 的取值, 纵轴用一系列高度不等的长条矩形来表示不同 x 范围的分布情况。具体地, 将 x 轴分成一些隔间 (bin), 宽度各为 Δ_i , 并统计落在其中的数据个数 n_i 。第 i 个隔间内的概率密度 $p(x)$ 估计为

$$p_i = \frac{n_i}{N\Delta_i} \quad (6.1)$$

以隔间为宽度、 p_i 为高度画出长条矩形, 就得到通常的直方图 (图 6.1)。隔间宽度通常取相同的大小, 即 $\Delta_i = \Delta$, 作为方法的唯一 (超) 参数。

直方图方法简单易行, 可以直观地看出数据的分布情况。不过, 它也有明显的缺点: 首先是得到的结果曲线不光滑; 其次是在高维空间下面临维度灾难 (如果 $\mathbf{x}$ 的每个维度分量的取值范围分割成 M 个区间, 那对于 D 维数据, 这些区间所组合成的高维立方体隔间的数目将为 M^D , 随维度 D 指数上升)。

接下来, 我们从理论上仔细分析一下直方图方法背后的原理, 以及它为什么能奏效? 怎样才能更奏效?

我们把 $\mathbf{x}$ 理解成随机变量, 那么它落在 (高维 $\mathbf{x}$ 空间中的) 一个小区域 $\mathcal{R}$ 内的概率将等于概率密度 $p(\mathbf{x})$ 在 $\mathcal{R}$ 内的积分:

$$P = \int_{\mathcal{R}} p(\mathbf{x}) d\mathbf{x} \quad (6.2)$$

如果我们根据 $p(\mathbf{x})$ 进行随机采样, 得到 N 个点 $\{\mathbf{x}_n\}$ ($n = 1, 2, \dots, N$), 那其中有 K 个点落入区域 $\mathcal{R}$ 的概率服从二项式分布:

$$p(K|N, P) = C_N^K P^K (1 - P)^{N-K} \quad (6.3)$$

其中 $C_N^K = \frac{N!}{K!(N-K)!}$ 是从 N 个可分辨元素中取出 K 个的不同取法（组合）。把区域 $\mathcal{R}$ 想象成直方图方法中的某个隔间，则 n_i 就是某次实验（采样）得到的 K 的结果。也就是说， N 与 K 是已知的（或者说，可直接测量的），而 P 是未知的。从贝叶斯的观点，我们是试图利用上式从 K （以及 N ）反推 P （严格地讲，是想反推 $p(\mathbf{x})$ ）。如果 N 与 K 都很大，则分布 $p(K|N, P)$ 是一个很窄的峰（这就是统计热力学中所遇到的大数定律），有

$$K \approx NP \quad (6.4)$$

（或者说， $P \approx \frac{K}{N}$ ，即概率 P 约等于频率 $\frac{K}{N}$ ）。如果进一步地，区域 $\mathcal{R}$ 的体积（ V ，可看做直方图方法中的 Δ_i ）很小，则其内部 $p(\mathbf{x})$ 近似是个常数，此时有

$$P \approx p(\mathbf{x})V \quad (6.5)$$

结合公式（6.4）与（6.5），得到

$$p(\mathbf{x}) \approx \frac{K}{NV} \quad (6.6)$$

这就是直方图方法的理论根据。在这些假设成立的情况下，我们就可以利用直方图方法来进行可靠的密度估计。

基于以上分析，我们就可以知道区间 $\mathcal{R}$ 的选择比较矛盾：需较小以便其中的 $p(\mathbf{x})$ 基本是个常数，又需较大以使 K 较大以便获得好的统计特性。

虽然直方图方法存在不足，但它教给我们一些重要的东西：某点 $\mathbf{x}$ 处的概率密度 $p(\mathbf{x})$ 是与其近邻的数据点 $\mathbf{x}_n$ 有关的；区间宽度参数 Δ 不能太大，也不能太小。

1.2 核密度估计法

基于对直方图方法的理论分析，特别是公式（6.6），就可以进一步发展其它方法，以克服直方图方法的不足。

直方图方法对公式（6.6）的处理思路是固定区域 $\mathcal{R}$ 的体积 V ，然后从已知数据中估计 K 。类似地，为了计算某一 $\mathbf{x}$ 处的概率密度 $p(\mathbf{x})$ ，我们可以将 $\mathcal{R}$ 选择为中心位于 $\mathbf{x}$ 、边长为 h 的小立方体，并定义如下核函数（由 Emanuel Parzen 最早提出，因此也被称为 Parzen window）

$$k\left(\frac{\mathbf{x}-\mathbf{x}_n}{h}\right) = \begin{cases} 1, & \text{if } \left|\frac{x_i-x_{n,i}}{h}\right| < \frac{1}{2} \\ 0, & \text{otherwise} \end{cases} \quad (6.7)$$

其中 x_i 是 $\mathbf{x}$ 的第 i 个分量。则

$$K = \sum_n k\left(\frac{\mathbf{x}-\mathbf{x}_n}{h}\right) \quad (6.8)$$

将其代入公式 (6.6)，得到

$$p(\mathbf{x}) = \frac{1}{N} \sum_n \frac{1}{h^D} k\left(\frac{\mathbf{x}-\mathbf{x}_n}{h}\right) \quad (6.9)$$

这被称为核密度估计法 (kernel density estimation)。注意它与直方图方法不太一样：直方图方法中 $\mathcal{R}$ 是固定的，即有一系列相互紧挨着的但并不重叠的立方体填满整个空间；而核密度估计法中的 $\mathcal{R}$ 的中心总位于 $\mathbf{x}$ ，实际是随 $\mathbf{x}$ 而不断移动。直方图方法在利用公式 (6.1) 计算 p_i 后就不再需要保存训练数据 $\{\mathbf{x}_n\}$ 了，后续在预测 $p(\mathbf{x})$ 阶段需要做的是判断 $\mathbf{x}$ 落在哪个立方体 i 里面（注意此时 $\mathbf{x}$ 并不一定位于立方体 i 的中心），然后将对应的 p_i 作为 $p(\mathbf{x})$ 的预测值。而核密度估计法在训练阶段需要做的是保存所有的训练数据 $\{\mathbf{x}_n\}$ ，然后在预测 $p(\mathbf{x})$ 阶段将 $\{\mathbf{x}_n\}$ 代入公式 (6.9) 计算出预测值 $p(\mathbf{x})$ （注意这个 $\mathbf{x}$ 的值只有在预测阶段才知道，在之前的训练阶段里并不知道）。这种在训练后仍需要保存（全部或部分）训练数据 $\{\mathbf{x}_n\}$ 的做法是各种核方法的一个共同特点。

公式 (6.9) 原本计算的是落到以 $\mathbf{x}$ 为中心的小立方体里有多少个 $\mathbf{x}_n$ 数据点，但我们可以将其进一步理解成在计算 $\mathbf{x}$ 落到多少个以数据点 $\mathbf{x}_n$ 为中心的小立方体里。后一角度更容易推广到下面即将介绍的高斯核或其它核函数。

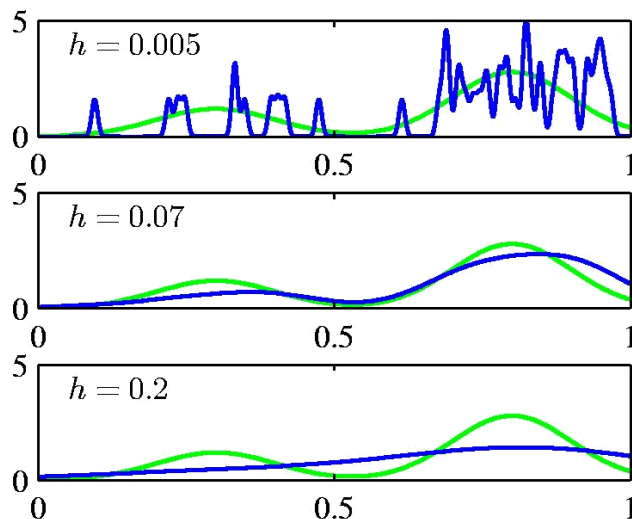


图 6.2. 采用高斯核函数的核密度估计法例子。其中绿色曲线代表真实的概率密度 $p(x)$ 。高斯核的宽度参数 h 起着光滑（超）参数的作用，既不能太小，也不能太大。

与直方图方法类似，采用 Parzen window 核函数[公式 (6.7)]的核密度估计法所给出的密度曲线 $p(\mathbf{x})$ 是不光滑的。不过，这一缺点很容易克服：我们可以采用光滑的核函数 $k(\mathbf{x}, \mathbf{x}_n)$ ，例如高斯核函数：

$$k(\mathbf{x}, \mathbf{x}_n) = \frac{1}{(2\pi h^2)^{D/2}} \exp\left(-\frac{|\mathbf{x}-\mathbf{x}_n|^2}{2h^2}\right) \quad (6.10)$$

其中 D 是 $\mathbf{x}$ 的维度。此时核密度估计法的结果可以写成如下更普适的形式：

$$p(\mathbf{x}) = \frac{1}{N} \sum_n k(\mathbf{x}, \mathbf{x}_n) \quad (6.11)$$

一个例子如图 6.2 所示。

其实，公式（6.11）相当于是把一个分立的（采样）点源 $\mathbf{x}_n$ 扩散弥散成一定范围内的密度分布 $k(\mathbf{x}|\mathbf{x}_n)$ ，然后把不同点源 $\{\mathbf{x}_n\}$ 的贡献叠加起来，就给出点 $\mathbf{x}$ 处的密度值 $p(\mathbf{x})$ 。基于这一理解，任何满足

$$k(\mathbf{x}, \mathbf{x}_n) \geq 0, \quad \int k(\mathbf{x}, \mathbf{x}_n) d^D \mathbf{x} = 1 \quad (6.12)$$

的 $k(\mathbf{x}, \mathbf{x}_n)$ 都可以作为核函数用于核密度估计法，非常灵活。除了前面的 Parzen window 与高斯分布这些双向扩散的核函数以外，还可以采用指数分布与伽马分布（ $\frac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\beta x}$ ）等单向扩散核函数。文献中常见的小提琴图就利用了核密度估计法来计算密度分布（例子见图 6.3），画图软件通常会提供多种核函数供用户选择。

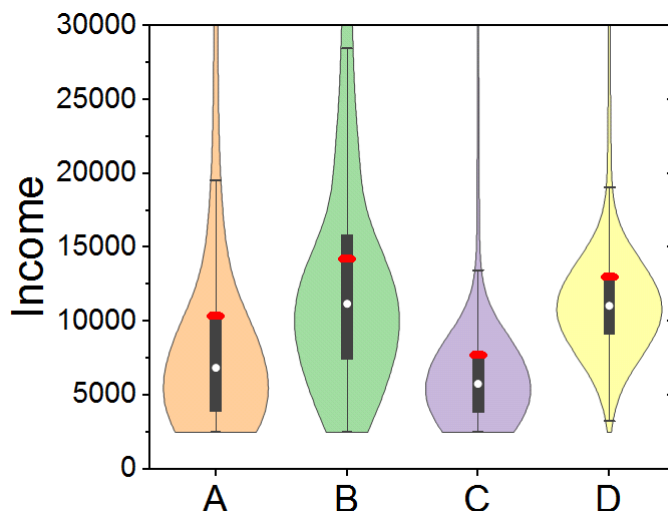


图 6.3. 小提琴图示意图。小提琴图是箱线图与核密度图的结合，因其形态类似小提琴而得名。有各种核函数可以选用。对于个人收入等取值非负的数据，可采用指数分布等单向扩散核函数来保证在 $x < 0$ 时得到 $p(x) = 0$ 。

1.3 近邻法

核密度估计法的做法是将公式（6.6）中体积 V 固定，此时密度 $p(\mathbf{x})$ 主要由落在区域 $\mathcal{R}$ 里的数据个数 K 所决定。这种做法的一个缺点是不容易兼顾密度大的地方（只需要比较小的 V 就能得到合适的 K ，从而获得好的精度）与密度小的地方

(需要比较大的 V 才能得到合适 K ，否则会得到过小的 K ，影响精度)。

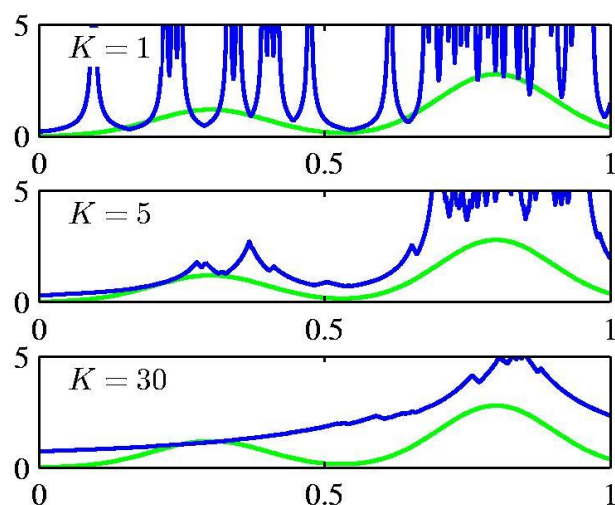


图 6.4. 采用近邻法估计分布密度 $p(x)$ 的例子。其中绿色曲线代表真实的概率。

近邻法(nearest neighbor method)的做法则刚好相反: 固定公式(6.6)中的 K , 然后利用已知数据确定其对应的 V , 进而得到密度 $p(\mathbf{x})$ 。具体实现上有各种不同做法, 例如, 可以将区域 $\mathcal{R}$ 取为中心位于 $\mathbf{x}$ 的圆球, 让其不断膨胀(半径变大), 直到包含进 K 个数据点(这些数据点被称为 $\mathbf{x}$ 的近邻)。这样就能实现密度大的地方 V 比较小、密度小的地方 V 比较大的效果。近邻法的一个例子如图 6.4 所示。

近邻法可用于密度估计, 此时属于无监督学习。但在更多的时候, 近邻法是被用于监督学习, 此时一般被称为 K 近邻算法(K -Nearest Neighbor, 简称 KNN)。 $K = 1$ 时, 被称为最近邻算法。

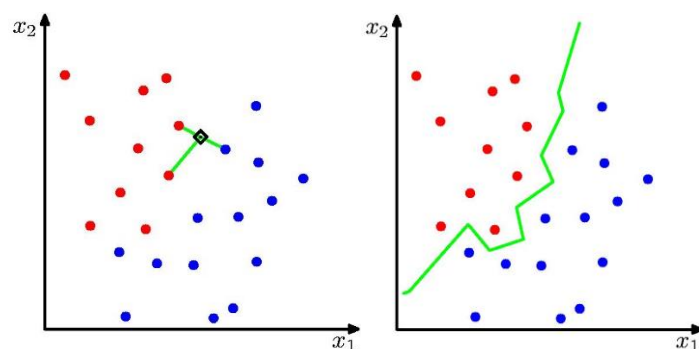


图 6.5. K 近邻分类算法示意图。(a) $K = 3$, 图中空心菱形符号标示的点 $\mathbf{x}$ 具有 3 个邻居, 两个红色, 1 个蓝色, 因此算法预测 $\mathbf{x}$ 红色的概率为 $2/3$, 蓝色概率为 $1/3$ 。(b) $K = 1$, 绿色折线是算法给出的决策面(线), 即概率为 $1/2$ 的点 $\mathbf{x}$ 所组成的曲线。

K 近邻分类算法的原理基于贝叶斯公式, 推导如下。记需要预测类别的点为 $\mathbf{x}$; 对于点 $\mathbf{x}$ 的 K 个近邻数据点(来自训练集, 因此其类别已知), 属于第 k 类的个

数记为 K_k ，则第 k 类在 $\mathbf{x}$ 处的分布概率密度为

$$p(\mathbf{x}|\mathcal{C}_k) = \frac{K_k}{N_k V} \quad (6.13)$$

其中 N_k 是训练集中第 k 类的数据总个数。第 k 类的先验概率则为

$$p(\mathcal{C}_k) = \frac{N_k}{N} \quad (6.14)$$

利用贝叶斯公式，得到后验概率

$$p(\mathcal{C}_k|\mathbf{x}) = \frac{p(\mathbf{x}|\mathcal{C}_k)p(\mathcal{C}_k)}{p(\mathbf{x})} = \frac{\frac{K_k}{N_k V} \frac{N_k}{N}}{\frac{K}{NV}} = \frac{K_k}{K} \quad (6.15)$$

这就是利用 K 近邻分类算法进行预测所使用的实际公式。对这个公式，可以撇开数学推导，采取一个直观的理解：在 $\mathbf{x}$ 附近的 K 个已知样本中，如果有 K_k 个属于第 k 类，那么属于第 k 类的概率（频率）自然为 K_k/K 。

K 近邻分类算法只有一个参数（严格讲是超参数，因为它并不能在训练阶段基于训练集进行优化，而只能结合验证集或利用交叉验证来进行优化），就是 K 。 K 与模型复杂性成反比。 K 越大，偏差越大，方差越小。 K 近邻分类算法的一个例子如图 6.5 所示。可以看出，决策面（线）犬牙交错，并不光滑。

K 近邻分类算法的核心思想可以简单描述成：像什么就是什么！也就是说，如果 $\mathbf{x}$ 像某些 $\mathbf{x}_n$ （距离很近），那它的类别就与这些 $\mathbf{x}_n$ 的类别一样。事实上，这也是大多数机器学习算法的底层逻辑。

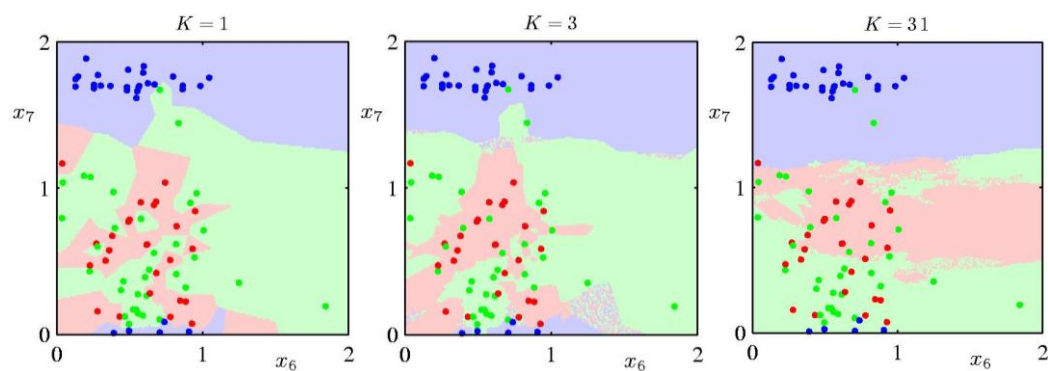


图 6.6. K 近邻分类算法能给出非常复杂的决策面。

K 近邻法在机器学习具有重要的地位。例如，Pedro Domingos 在《终极算法》中如此评价： K 近邻算法是人类有史以来发明的最简单、最快速的学习算法，是史上第一个能够利用不限数量的数据来掌握任意复杂概念的算法。确实， K 近邻法在训练时是非常快的：只需要把训练集保存下来就行。因此， K 近邻法也被

称为最懒惰的算法^π。另一方面，它的容量很大。不管真实的 $p(\mathbf{x})$ 与决策面有多复杂，只要有足够的已知数据， K 近邻法就能以任意指定精度给出符合真实值的预测结果。在直观表现上，就是 K 近邻法虽然很简单，却能给出非常复杂的决策面（见图 6.6）。

K 近邻法不仅可以用于分类，还可以用于回归。最简单的做法是将 $\mathbf{x}$ 的 K 个近邻的属性的平均值作为对 $\mathbf{x}$ 的属性的预测：

$$f(\mathbf{x}) = \frac{1}{K} \sum_{n \in NN} t_n \quad (6.16)$$

其中的求和只对 $\mathbf{x}$ 的 K 个近邻进行。当然，也可以进一步借鉴核密度估计法中的思想，采用核函数将不同距离的邻居的影响给予不同的权重。图 6.7 中给出了一个 K 近邻回归算法的例子。

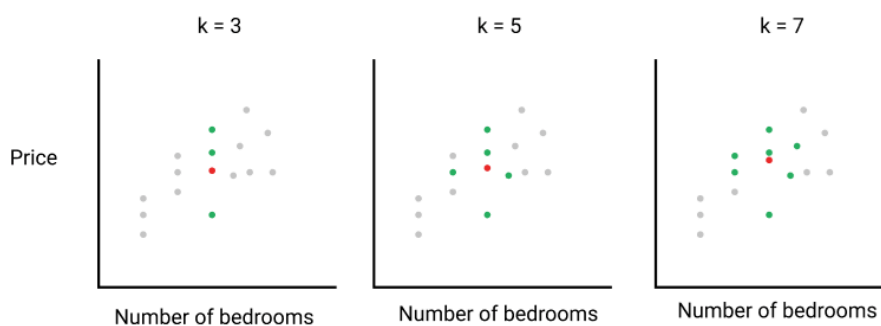


图 6.7. K 近邻回归算法例子：房子的价格预测问题。

1.4 非参数法的优缺点

非参数法的最大优点是它不需要假设分布曲线的形状。它采用简单的“局部构造法”，然后让数据告诉我们分布是什么样子的（“Let the data tell us what the function looks like!”），完备的理论保证了这种估计的无偏性与一致性。通过交叉验证来选择邻域的大小，往往可以得到非常令人满意效果（特别是对于低维数据）。在有无穷多数据（ $N \rightarrow \infty$ ）时能拟合任何函数曲线。

但是，非参数法也有缺点。首先，它需要保存所有数据用于预测，而参数法则只需要保存训练得到的参数。当数据非常多时，这个缺点尤为严重。其次，它需要大量数据才能保证良好的效果。非参数法的无偏性只有在局部邻域中的已知数据点很多的时候才能得到保障。与参数法相比，非参数法对数据的胃口非常大，如果没有足够数量的数据，那它就是典型的“巧妇难为无米之炊”。

^π 以课程学习做类比，这相当于平时上课把课程内容机械地记录下来（或直接拷贝课件），并不过脑子，考试时（开卷）通过临时查找记录内容来作答。注意这不是死记硬背，因为还需要判断数据的距离远近。这是一种“懒惰学习算法”，但在机器学习中，拖延真的可以带来成功。

第 2 节 核方法：主要想法

2.1 核方法的主要思想

前面介绍的几种方法（直方图方法、核密度估计法、近邻法）里蕴含着一个共同的核心概念：两个点之间的距离（相似性）。例如，核密度估计法通常是把一个分立的点源 $\mathbf{x}_n$ 弥散成在 $\mathbf{x}_n$ “附近”一定范围内的密度分布 $k(\mathbf{x}|\mathbf{x}_n)$ ，然后把不同点源 $\{\mathbf{x}_n\}$ 的贡献叠加起来给出任意点 $\mathbf{x}$ 处的密度估计 $p(\mathbf{x})$ ；而 K 近邻法则是利用离 $\mathbf{x}$ “最近”的 K 个已知（训练集中的）数据点的性质进行平均来预测点 $\mathbf{x}$ 处的性质。事实上，这种思想也被蕴含在大多数其它机器学习方法里。或者，如 Pedro Domingos 在《机器学习那些事》中所说的：本质上所有的学习器都是将临近的样本归类到同一个类别中；关键的不同之处在于“临近”的意义。

当我们在机器学习中引入基函数 $\{\phi_j(\mathbf{x})\}$ 以后，就可以基于 $\{\phi_j(\mathbf{x})\}$ 重新定义两个 $\mathbf{x}$ 点之间的距离。这相当于先变换坐标（通常是非线性变换）， $\mathbf{x} \rightarrow \boldsymbol{\phi}(\mathbf{x}) = \{\phi_j(\mathbf{x})\}$ ，再在新坐标系（即特征空间）下讨论点之间的相似性（距离）。既然在机器学习方法中起核心作用的是距离，那我们也可以不显式讨论 $\{\phi_j(\mathbf{x})\}$ ，而是直接引入某种形式的距离（内积）的定义，它的信息由核函数给出。这就是核方法的主要思想。

下面我们以线性回归方法为例，来看核函数是怎样起作用的。

根据第二章第 4.2.2 节，线性回归方法在正则化下的最小二乘误差为

$$\tilde{E}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N [t_n - \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n)]^2 + \frac{\lambda}{2} \mathbf{w}^T \mathbf{w} \quad (6.17)$$

对 $\tilde{E}(\mathbf{w}) = \frac{1}{2} \sum_{n=1}^N [t_n - \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n)]^2 + \frac{\lambda}{2} \mathbf{w}^T \mathbf{w}$ 误差函数求极小值， $\frac{\partial}{\partial \mathbf{w}} \tilde{E}(\mathbf{w}) = \mathbf{0}$ ，整理得

$$\mathbf{w} = -\frac{1}{\lambda} \sum_{n=1}^N [\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n) - t_n] \boldsymbol{\phi}(\mathbf{x}_n) \quad (6.18)$$

引入辅助变量

$$a_n = -\frac{1}{\lambda} [\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n) - t_n] \quad (6.19)$$

则公式（6.18）可写成

$$\mathbf{w} = \sum_{n=1}^N a_n \boldsymbol{\phi}(\mathbf{x}_n) = \boldsymbol{\Phi}^T \mathbf{a} \quad (6.20)$$

其中 Φ 是第二章第 4.2.1 节中讨论过的设计矩阵，大小为 $N \times M$ ；而 $\mathbf{a}$ 则是一个 N 维矢量。我们本来是在讨论自变量为 $\mathbf{w}$ 的误差函数 $\tilde{E}(\mathbf{w})$ ，现在通过辅助变量 $\mathbf{a}$ 以及关系式 (6.20)，将 (6.20) 代入 (6.17)，可以转而讨论自变量为 $\mathbf{a}$ 的误差函数（这被称为 dual representation，对偶表示，参见“扩展阅读：关于 dual representation 的理解”）：

$$\tilde{E}(\mathbf{a}) = \tilde{E}(\mathbf{w} = \Phi^T \mathbf{a}) = \frac{1}{2} \mathbf{a}^T \Phi \Phi^T \Phi \Phi^T \mathbf{a} - \mathbf{a}^T \Phi \Phi^T \mathbf{t} + \frac{1}{2} \mathbf{t}^T \mathbf{t} + \frac{\lambda}{2} \mathbf{a}^T \Phi \Phi^T \mathbf{a} \quad (6.21)$$

定义格拉姆矩阵（Gram matrix）

$$\mathbf{K} = \Phi \Phi^T \quad (6.22)$$

它是一个大小为 $N \times N$ 的对称矩阵，其矩阵元

$$t_{mn} = \phi(\mathbf{x}_m) \cdot \phi(\mathbf{x}_n) = k(\mathbf{x}_m, \mathbf{x}_n) \quad (6.23)$$

是新坐标系下两个点之间的内积，在核方法中被称为核函数 $k(\mathbf{x}_m, \mathbf{x}_n)$ ，可用来表征两个点之间的距离^{ss}。因此误差函数可重写成：

$$\tilde{E}(\mathbf{a}) = \frac{1}{2} \mathbf{a}^T \mathbf{K} \mathbf{K} \mathbf{a} - \mathbf{a}^T \mathbf{K} \mathbf{t} + \frac{1}{2} \mathbf{t}^T \mathbf{t} + \frac{\lambda}{2} \mathbf{a}^T \mathbf{K} \mathbf{a} \quad (6.24)$$

对其求极小值， $\frac{\partial}{\partial \mathbf{a}} \tilde{E}(\mathbf{a}) = \mathbf{0}$ ，得优化参数

$$\mathbf{a} = (\mathbf{K} + \lambda \mathbf{I}_N)^{-1} \mathbf{t} \quad (6.25)$$

代回 (6.20)，得 $\mathbf{w}$ 的优化值：

$$\mathbf{w} = \Phi^T (\mathbf{K} + \lambda \mathbf{I}_N)^{-1} \mathbf{t} \quad (6.26)$$

因此，模型对任一 $\mathbf{x}$ 的预测结果为

$$y(\mathbf{x}) = \mathbf{w}^T \phi(\mathbf{x}) = \phi(\mathbf{x})^T \mathbf{w} = \phi(\mathbf{x})^T \Phi^T (\mathbf{K} + \lambda \mathbf{I}_N)^{-1} \mathbf{t} = \mathbf{k}(\mathbf{x})^T (\mathbf{K} + \lambda \mathbf{I}_N)^{-1} \mathbf{t} \quad (6.27)$$

其中 $\mathbf{k}(\mathbf{x})$ 是 N 维矢量，分量为 $k_n(\mathbf{x}) = k(\mathbf{x}, \mathbf{x}_n) = \phi(\mathbf{x})^T \phi(\mathbf{x}_n)$ 。从这个结果可以看出，因此， $y(\mathbf{x})$ 是由表征距离性质的核函数所决定的，而且是已知函数值 $\mathbf{t}$ 的某种（线性）组合。

在一般情况下，我们可以把核方法的原理简述如下。我们本来是通过非线性

^{ss} 由内积可以计算距离， $|\phi_1 - \phi_2|^2 = \phi_1 \cdot \phi_1 + \phi_2 \cdot \phi_2 - 2\phi_1 \cdot \phi_2$ ，因此，内积函数确实包含了距离的信息。在上一节中，我们把核函数简单理解成距离的表征，但数学上的严格定义应该是新坐标系（特征空间）下的内积。

变换基函数 $\{\phi_j(\mathbf{x})\}$ 将输入空间 $\mathbf{x}$ 映射到高维特征空间 $\phi(\mathbf{x})$ 。特征空间的维数可能非常高（甚至可以是无穷维）。如果原来的机器学习方法（如上述的线性回归，或后述的支持向量机方法）的求解只用到内积运算 $\phi(\mathbf{x}) \cdot \phi(\mathbf{x}')$ ，而在低维输入空间又存在某个函数 $k(\mathbf{x}, \mathbf{x}')$ ，它恰好等于在高维空间中这个内积，即 $k(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x}) \cdot \phi(\mathbf{x}')$ 。那么原来的方法就不用计算复杂的非线性变换，而由这个函数 $k(\mathbf{x}, \mathbf{x}')$ 直接得到非线性变换的内积，从而大大简化了计算。这里的 $k(\mathbf{x}, \mathbf{x}')$ 就被称为核函数。

扩展阅读：关于 dual representation 的理解

将参数记成 x ，代价函数记为 $L(x)$ 。假设 $L(x)$ 是凸的，只有一个极小值点 x^* ，用它以来做预测。现在我们引入一个映射 $y = f(x)$ 及另一个映射 $x = g(y)$ ，它们并不一定是逆映射的关系。我们可以讨论 $L(g(y))$ 的极小值 $L(g(y^*))$ ，此时总有 $L(g(y^*)) \geq L(x^*)$ ，因为在这个函数里 y 总是通过转化成 x 再起作用的。注意 x 与 y 的维度可以不同，所以可以有很多个 x 映射到同一个 y ，或很多个 y 映射到同一个 x ，而且 y 映射到 x 后不一定能填满所有的 x 空间。如果我们把 x^* 映射到 $y = f(x^*)$ 再映射回 $x = g(y) = g(f(x^*))$ ，它不一定是 x^* 。但如果我们在选择 $f(x)$ 与 $g(y)$ 时使 $x^* = g(f(x^*))$ ，则有 $y^* = f(x^*)$ （严格讲，是 y^* 中的一个等于 $f(x^*)$ ）及 $x^* = g(y^*)$ （这个对所有 y^* 都成立，因为只有一个 x^* 极小点，肯定会有 $L(g(y^*)) \geq L(x^*)$ ）。这样 $L(x)$ 的极值问题就会等价变成 $L(g(y))$ 的极值问题，而且由于 $x^* = g(y^*)$ ，不管 y^* 有多少个，都会得到同一个 x^* （我们开始时假设只有一个 x^* ），不同 y^* 的预测都是相同的，只有一个结果。这就是我们正文中采用的方案，即根据 x^* 的方程来定义 $f(x)$ 与 $g(y)$ ，使其满足 $x^* = g(f(x^*))$ ，具体方案就是公式（6.19）与（6.20）。

核函数的选择要求满足 Mercer 定理 (Mercer's theorem)，即核函数在任意样本集上的 Gram 矩阵是半正定 (semi-positive) 的。常用的核函数有：线性核函数 ($k(\mathbf{x}, \mathbf{x}') = \mathbf{x} \cdot \mathbf{x}'$)、多项式核函数 ($(\mathbf{x} \cdot \mathbf{x}' + 1)^d$ ， d 为正整数)、高斯核函数 ($\exp(-r|\mathbf{x} - \mathbf{x}'|^2)$ ， $r > 0$)、周期核 ($\exp\left(-\frac{2\sin^2(\pi|\mathbf{x}-\mathbf{x}'|/p)}{l^2}\right)$ ，其中 p 是周期长度，而 l 决定了函数的“平滑度”或者说是特征的尺度)，等等。另外，还可以利用已知的核函数来构建新的核函数，例如，如果 $k(\mathbf{x}, \mathbf{x}')$ 是一个核函数，则 $ck(\mathbf{x}, \mathbf{x}')$ （其中 $c > 0$ ）、 $q(k(\mathbf{x}, \mathbf{x}'))$ （其中 q 表示多项式函数）、 $e^{k(\mathbf{x}, \mathbf{x}')}$ 、 $f(\mathbf{x})k(\mathbf{x}, \mathbf{x}')f(\mathbf{x}')$ 也是核函数，两个核函数相加或相乘的结果也是核函数，等等。

2.2 核方法的灵活性

核函数方法的强大之处在于它可以和不同的算法相结合，形成多种不同的基于核函数技术的方法，而且这两部分的设计可以单独进行，并可以为不同的应用

选择不同的核函数和算法。核函数使得专家可以把某个领域的知识引入到模型中，以更有效地捕捉训练数据中的趋势。下面以一个气温预测的例子来简单说明一下。

数学背景：多元高斯分布

一元高斯分布为 $p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(x-\mu)^2} = \mathcal{N}(x|\mu, \sigma^2)$ ，其中均值 $\mu = \langle x \rangle$ ，方差 $\sigma^2 = \langle (x - \mu)^2 \rangle$ ，而 $\langle \dots \rangle$ 代表求加权平均值。一元高斯分布的对数是 x 的二次函数。

多元高斯分布的对数则是 $\mathbf{x}$ 的二次函数，一般写成如下形式：

$$\mathcal{N}(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{D/2}} \frac{1}{|\boldsymbol{\Sigma}|^{1/2}} \exp \left[-\frac{1}{2} (\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}) \right]$$

其中上标 T 代表转置， $\boldsymbol{\mu}$ 是均值向量， $\boldsymbol{\Sigma}$ 是 $D \times D$ 维的协方差矩阵，它们满足

$$\boldsymbol{\mu} = \langle \mathbf{x} \rangle, \quad \boldsymbol{\Sigma} = \langle (\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^T \rangle$$

或分量形式：

$$\mu_i = \langle x_i \rangle, \quad \Sigma_{ij} = \langle (x_i - \mu_i)(x_j - \mu_j) \rangle$$

而 $|\boldsymbol{\Sigma}|$ 则是 $\boldsymbol{\Sigma}$ 的行列式。二元高斯分布的等高线是椭圆。

高斯分布有一个很赞的代数性质：它在条件作用和边缘化情况下是封闭的：把变量 $\mathbf{x}$ 分成两部分， $\mathbf{x}_a$ 与 $\mathbf{x}_b$ ，则 $p(\mathbf{x}_a|\mathbf{x}_b)$ 与 $p(\mathbf{x}_a)$ 仍然是高斯分布。做线性变换 $\mathbf{x}' = \mathbf{A}\mathbf{x} + \mathbf{b}$ ，则 $\mathbf{x}'$ 也服从高斯分布。这些良好性质使得很多统计学和机器学习中的问题变得容易求解。

这里采用的机器学习算法是高斯过程（Gaussian Processes）。在贝叶斯线性回归模型中， $y(\mathbf{x}, \mathbf{w}) = \sum_{j=0}^M w_j \phi_j(\mathbf{x})$ ， $p(\mathbf{w}|\alpha) = \mathcal{N}(\mathbf{w}|\alpha^{-1}\mathbf{I})$ ，对于固定的 $\mathbf{x}_m, \mathbf{x}_n$ ，其函数值 (y_m, y_n) 满足多元高斯分布（参见“数学背景：多元高斯分布”），其参数满足

$$\langle y_m \rangle = \langle y_n \rangle = 0 \quad (6.28)$$

$$\Sigma_{mn} \equiv \langle y_m y_n \rangle = \frac{1}{\alpha} \sum_{j=0}^M \phi_j(\mathbf{x}_m) \phi_j(\mathbf{x}_n) \equiv k(\mathbf{x}_m, \mathbf{x}_n) = K_{mn} \quad (6.29)$$

由核函数 $k(\mathbf{x}_m, \mathbf{x}_n)$ 所决定。它包含了点之间关联的信息，回归就是利用其来做预测的。在高斯过程中，是假设 $y(\mathbf{x})$ 在任何点 $\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N$ 处的值 $\{y_1, y_2, \dots, y_N\}$ （以及任一 $\mathbf{x}$ 处的 $y(\mathbf{x})$ ）的联合分布都是高斯分布，由它们的协方差矩阵 $\boldsymbol{\Sigma}$ 完全确定，其矩阵元就是核函数，即 $\Sigma_{mn} = k(\mathbf{x}_m, \mathbf{x}_n)$ 。这个多元高斯分布的条件概率分布

$p(y(\mathbf{x})|y_1, y_2, \dots, y_N)$ 就提供了对 $y(\mathbf{x})$ 的预测。

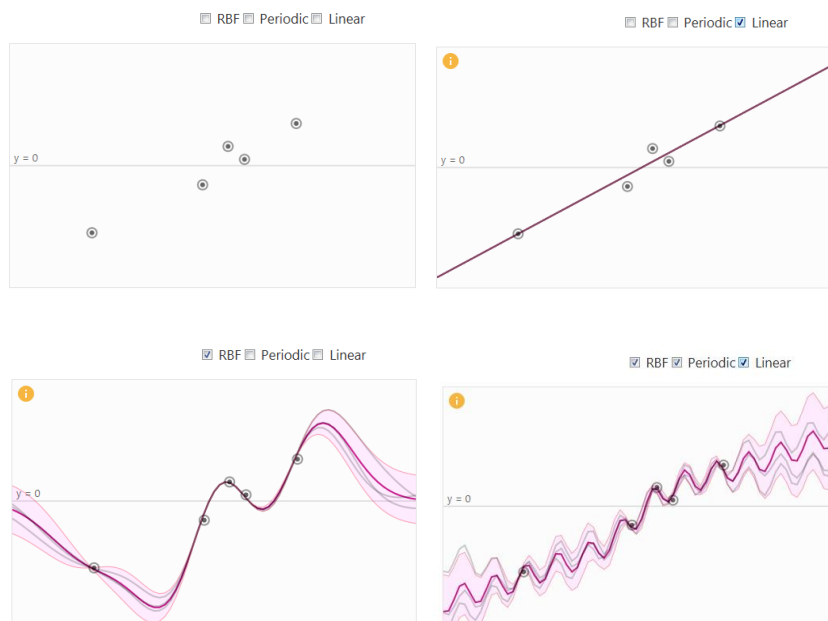


图 6.8. 不同核函数对气温预测效果的影响。采用的机器学习方法为高斯过程。(a) 原始数据, x 轴为时间, y 轴为气温。(b) 使用了线性核函数。(c) 在 (b) 的基础上增加了高斯核。(d) 在 (c) 的基础上增加了周期核。

采用的核函数包括线性核、高斯核与周期核函数三种, 可通过它们之间的乘积实现组合。具体的效果如图 6.8 所示。在这个例子中, 只有时间间隔不等的几个气温数据[图 6.8(a)]。如果采用线性核, 效果相当于一个普通线性回归, 只能得到总体的变化趋势, 即温度缓慢升高[图 6.8(b)]。如果再加上高斯核函数, 就可以更准确地逼近这些已知数据点[图 6.8(c)]。但由于高斯核是平稳的, 在远离已知训练数据的地方, 总是会回到均值, 结果可能不太准确。如果这时候有专家指出气温近似具有 24 小时的周期性, 也就是说, 两个时间点的相似程度并不是随着它们之间的时间差而单调变换。与相隔 12 小时的两个时间相比, 相隔 48 小时或 72 小时的两个时间点可能是更相似的 (因此气温更为接近)。这很容易通过引入周期核来反映这种专家认识。通过结合所有的三个核函数, 就能在保持数据的总体趋势 (温度缓慢升高) 的情况下刻画更多的变化细节 (周期性), 从而达到更好的预测效果[图 6.8(d)]。

第 3 节 支持向量机

支持向量机 (support vector machine, SVM) 是一种清晰、强大的机器学习方法, 可以通过核方法进行非线性分类。它是由模式识别中的广义肖像算法 (generalized portrait algorithm) 发展而来的分类器, 其早期工作来自前苏联学者

Vladimir N. Vapnik 和 Alexander Y. Lerner 在 1963 年发表的研究^{tt}。1964 年, Vapnik 和 Alexey Y. Chervonenkis 对广义肖像算法进行了发展并建立了硬边距的线性 SVM。此后在二十世纪 70-80 年代, 随着模式识别中最大边距决策边界的理论研究、基于松弛变量 (slack variable) 的规划问题求解技术的出现, 以及 VC 维度 (Vapnik-Chervonenkis dimension, VC dimension) 的提出, SVM 被逐步理论化并成为统计学习理论的重要内容。1992 年, Bernhard E. Boser、Isabelle M. Guyon 和 Vapnik 通过核方法得到了非线性 SVM。1995 年, Corinna Cortes 和 Vapnik 提出了软边距的非线性 SVM 并将其应用于手写字符识别问题^{uu}, 很快就在若干个方面表现出了相对于神经网络的优势: 无需调参; 高效; 全局最优解。基于这些原因, SVM 在当时迅速打败了神经网络算法成为主流。当然, 在约 10 年后, 神经网络重新崛起, 并在后来成为人工智能第三次热潮的主角, 这是后话。

3.1 支持向量机的主要想法: 类间间隔的最大化

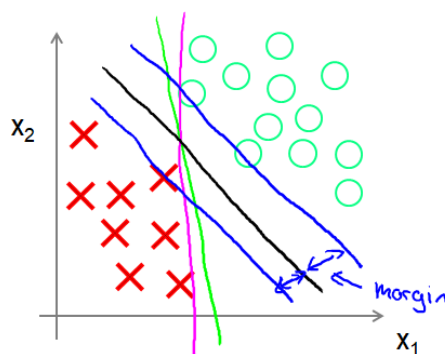


图 6.9. 支持向量机原理的示意图: 在能够把两类数据完全分开的前提下, 尽量使决策面 (图中的黑色直线) 远离两边的数据点, 或者说, 保持最大的类间间隔。

支持向量机的主要想法如图 6.9 所示。想象我们有两类数据点, 它们是线性可分的, 即可以用直线 (平面) 完全分隔开。但是, 满足这个分隔要求的直线有很多 (如图所示), 都能在训练集上实现 100% 的准确率。那我们应该选择哪一条来作为我们的训练结果 (决策面) 呢? 支持向量机的想法是选择那一条尽量远离两边数据点的直线 (图中黑色直线)。或者, 把这些数据点想象成电子游戏中来自红蓝双方所埋的地雷, 然后有一条蛇要从双方阵地中间爬过, 尽量不要触动地雷, 最佳 (安全) 的路线是什么? 显然, 最安全的路线是最肥的蛇爬过去也不会触动地雷的那一条路线, 即与两边的地雷具有最大的安全间距, 或者叫间隔 (margin)。

^{tt} V. N. Vapnik and A. Y. Lerner. Recognition of patterns with help of generalized portraits. *Avtomat. i Telemekh* **24** (6), 774-780 (1963).

^{uu} C. Cortes and V. Vapnik. Support-vector networks. *Machine Learning* **20**, 273-297 (1995). (SCI 引用超过 3 万次)

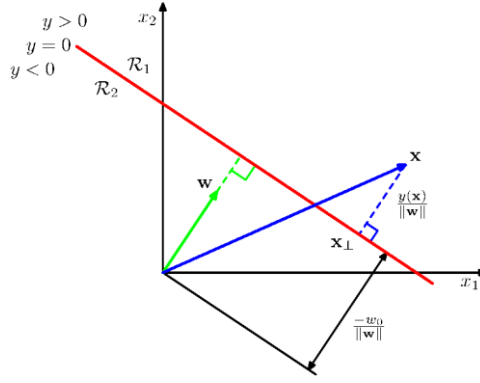


图 6.10. 任一点 $\mathbf{x}$ 与决策面 ($y(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b = 0$) 的有向距离。

支持向量机的这个想法可以进一步用数学语言表示出来。考虑如下的线性分类模型：

$$y(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b \quad (6.30)$$

或使用基函数：

$$y(\mathbf{x}) = \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}) + b \quad (6.31)$$

决策面为 $y(\mathbf{x}) = 0$ 。注意与线性回归或逻辑回归模型不同，这里的 $\mathbf{w}$ 并不包含截距参数 b （以前的 w_0 ）。如果把 $\mathbf{w}$ 看做 $\mathbf{x}$ 空间中的一个矢量，它与决策面垂直，决策面的垂直方向可写成 $\hat{\mathbf{w}} = \mathbf{w}/\|\mathbf{w}\|$ 。因此，任一点 $\mathbf{x}$ 在方向 $\hat{\mathbf{w}}$ 上的投影为 $\hat{\mathbf{w}} \cdot \mathbf{x}$ ， $\mathbf{x}$ 与决策面的（有向）距离为（参见图 6.10）

$$d(\mathbf{x}) = \hat{\mathbf{w}} \cdot \mathbf{x} - d_0 = \frac{\mathbf{w} \cdot \mathbf{x}}{\|\mathbf{w}\|} - d_0 \quad (6.32)$$

其中 d_0 是当 $\mathbf{x}$ 落在决策面上时所得到的 $d(\mathbf{x})$ 值，即 $d_0 = -\frac{b}{\|\mathbf{w}\|}$ ，因此有

$$d(\mathbf{x}) = \frac{\mathbf{w} \cdot \mathbf{x}}{\|\mathbf{w}\|} + \frac{b}{\|\mathbf{w}\|} = \frac{y(\mathbf{x})}{\|\mathbf{w}\|} \quad (6.33)$$

这个公式对使用基函数的模型（6.31）也成立（此时距离是在 $\boldsymbol{\phi}$ 空间定义的，而非 $\mathbf{x}$ 空间）。对于线性可分体系，训练集中的一类数据全部在决策面的一边（ $d(\mathbf{x}) > 0$ ），另一类数据全部在决策面的另一边（ $d(\mathbf{x}) < 0$ ）。因此，对训练集中任一数据点 $\mathbf{x}_n$ ，与决策面的距离可写成

$$\frac{t_n y(\mathbf{x})}{\|\mathbf{w}\|} = \frac{t_n [\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n) + b]}{\|\mathbf{w}\|} \quad (6.34)$$

这里我们假设标签取值为 $t_n = \pm 1$ 。在模型的分类预测全部正确时，上式总为正值。错误预测时上式则会给出负值。训练集中所有数据点离决策面的最小距离，被称为间隔/边距（margin），由下式给出：

$$\frac{1}{\|\mathbf{w}\|} \min_n \{t_n [\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n) + b]\} \quad (6.35)$$

其中 $\min_n\{*\}$ 表示变化 n 使括号内的函数取得最小值并返回这个最小值。支持向量机追求的目标是使这个间隔最大化，即

$$\arg \max_{\mathbf{w}, b} \left\{ \frac{1}{\|\mathbf{w}\|} \min_n \{t_n [\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n) + b]\} \right\} \quad (6.36)$$

其中 $\arg \max_{\mathbf{w}, b}\{*\}$ 表示变化参数（变量） $\mathbf{w}, b$ 使括号内的函数取得最大值并返回此时对应的参数 $\mathbf{w}, b$ ，即可据此得到间隔最大化时的模型。

公式（6.36）既要求最大值又要求最小值，比较复杂，但它可以进一步化简。一个可资利用的性质是：当 $\mathbf{w}$ 与 b 同时增大 κ 倍时（此时决策面不会发生改变）， $t_n [\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n) + b]$ 也增大 κ 倍，但距离公式（6.34）的值 $\frac{t_n [\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n) + b]}{\|\mathbf{w}\|}$ 保持不变。利用这个性质，不失一般性，总可以在不改变决策面位置的前提下适当选取 $\mathbf{w}, b$ 使离决策面最近的数据点 n （边界点）满足 $t_n [\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n) + b] = 1$ 。则对任何数据点都有

$$t_n [\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n) + b] \geq 1 \quad (6.37)$$

这被称为决策平面的规范表示（canonical representation）。此时， $\min_n \{t_n [\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n) + b]\} = 1$ ，目标（6.36）简化成：

$$\arg \max_{\mathbf{w}, b} \left\{ \frac{1}{\|\mathbf{w}\|} \right\} \quad (6.38)$$

或写成更方便的形式：

$$\arg \min_{\mathbf{w}, b} \left\{ \frac{1}{2} \|\mathbf{w}\|^2 \right\} \quad (6.39)$$

因此，支持向量机的目标就是在条件（6.37）的约束下求解（6.39）。这里目标函数是二次的，约束条件是线性的，在数学上属于凸优化问题，具有良好的性质。

数学背景：约束条件下函数极值求解的拉格朗日方法与 KKT 条件

函数 $f(\mathbf{x})$ 的极值(极大值、极小值)满足 $\frac{\partial}{\partial \mathbf{x}} f(\mathbf{x}) = 0$, 可通过这个方程(组)来求解。如果需要满足等式约束条件, 如 $g(\mathbf{x}) = 0$, 则可通过引入拉格朗日因子 λ , 定义 $L(\mathbf{x}, \lambda) = f(\mathbf{x}) + \lambda g(\mathbf{x})$, 从而把约束优化问题变成无约束优化问题, 求解 $\frac{\partial}{\partial (\mathbf{x}, \lambda)} L(\mathbf{x}, \lambda) = 0$ 即可。但是, 如果需要满足的是不等式约束条件, 如 $g(\mathbf{x}) \geq 0$, 那么此时 $f(\mathbf{x})$ 的极值应该如何求解呢? 很显然, 此时求解得到的 $f(\mathbf{x})$ 的极值只能有两种可能:

- 落在 $g(\mathbf{x}) > 0$ 的区域内。此时应该满足 $\frac{\partial}{\partial \mathbf{x}} f(\mathbf{x}) = 0$, 与无约束优化下的方程相同。
- 落在 $g(\mathbf{x}) \geq 0$ 的区域边界上, 即 $g(\mathbf{x}) = 0$ 。此时相当于等式约束, 满足 $\frac{\partial}{\partial (\mathbf{x}, \lambda)} L(\mathbf{x}, \lambda) = 0$, 即 $\frac{\partial}{\partial \mathbf{x}} L(\mathbf{x}, \lambda) = 0$ 与 $g(\mathbf{x}) = 0$ 。进一步的分析表明如果求极大值, 则要求 $\lambda > 0$; 如果求极小值, 则要求 $\lambda < 0$ 。

综合这两种情况, 可以给出不等式约束下的极值点需要满足如下性质:

$$\begin{cases} \lambda g(\mathbf{x}) = 0 \\ \frac{\partial}{\partial \mathbf{x}} L(\mathbf{x}, \lambda) = 0 \\ g(\mathbf{x}) \geq 0 \end{cases}$$

这被称为 KKT 条件, 可用于不等式约束下的极值问题求解。 $\lambda g(\mathbf{x}) = 0$ 的条件要求 $\lambda = 0$ (对应上述的第一种情况)或 $g(\mathbf{x}) = 0$ (对应上述的第二种情况)。前两个条件是等式, 在求解时尤其有用。需要注意的是, 在通常情况下 KKT 条件只是极值点的必要条件, 不是充分条件。

3.2 线性可分体系的求解：支持向量的存在

不等式约束条件下的函数极值求解, 可以利用拉格朗日方法与 KKT 条件(参见“数学背景：约束条件下函数极值求解的拉格朗日方法与 KKT 条件”)。我们为每个训练集数据点的约束条件(6.37)引入拉格朗日因子 λ_n , 定义如下函数

$$L(\mathbf{w}, b, \lambda) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_n \lambda_n \{t_n [\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n) + b] - 1\} \quad (6.40)$$

则(6.39)极值问题的 KKT 条件给出:

$$\frac{\partial}{\partial (\mathbf{w}, b)} L(\mathbf{w}, b, \lambda) = 0 \quad (6.41)$$

$$\lambda_n \{t_n [\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n) + b] - 1\} = 0 \quad (6.42)$$

其中 (6.41) 对 $\mathbf{w}$ 的偏导数给出

$$\mathbf{w} = \sum_n \lambda_n t_n \boldsymbol{\phi}(\mathbf{x}_n) \quad (6.43)$$

把它代回 $y(\mathbf{x}) = \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}) + b$, 得到

$$y(\mathbf{x}) = \sum_{n,i} \lambda_n t_n \phi_i(\mathbf{x}_n) \phi_i(\mathbf{x}) + b = \sum_n \lambda_n t_n k(\mathbf{x}, \mathbf{x}_n) + b \quad (6.44)$$

可以看到, 模型的结果与核函数 $k(\mathbf{x}, \mathbf{x}_n) = \sum_i \phi_i(\mathbf{x}_n) \phi_i(\mathbf{x})$ 有关。虽然上式中 λ_n 的值需进一步求解, 但由于条件 (6.42) 的存在, 所有不在间隔边缘 $t_n [\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n) + b] = 1$ 上的数据点都有 $\lambda_n = 0$, 不出现在上式的求和中。只有间隔边缘的数据点 (被称为支持向量) 才对 $y(\mathbf{x})$ 的计算有贡献! 这在某种意义上是可以理解的: 边距处的点的移动自然会影响到边距的位置, 但边距外的点稍微动一动是不会对边距造成影响的。

接下来求解 b 。间隔边缘上的数据点满足 $t_m y(\mathbf{x}_m) = 1$, 利用 (6.44), 得

$$t_m [\sum_{n \in \mathcal{S}} \lambda_n t_n k(\mathbf{x}_m, \mathbf{x}_n) + b] = 1 \quad (6.45)$$

其中 $n \in \mathcal{S}$ 表示只考虑落在间隔边缘上的点。上式两边同乘以 t_m 并对 m 求和, 得

$$b = \frac{1}{N_{\mathcal{S}}} \sum_{m \in \mathcal{S}} [t_m - \sum_{n \in \mathcal{S}} \lambda_n t_n k(\mathbf{x}_m, \mathbf{x}_n)] \quad (6.46)$$

其中 $N_{\mathcal{S}}$ 代表落在间隔边缘上的点的数目。上式表明 b 也与核函数有关。

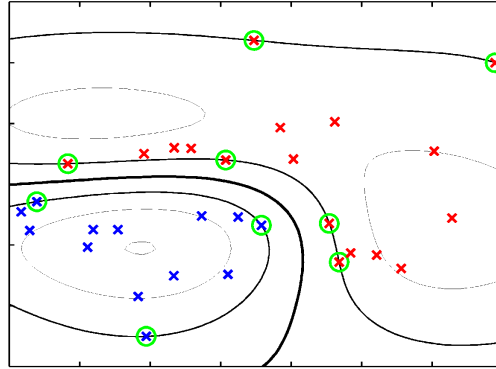


图 6.11. 支持向量机在完全可分体系中的应用例子。使用了高斯核函数。粗实线是决策面, 细实线是由 $\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}) + b = \pm 1$ 所决定的间隔边缘。绿色圆圈所圈出的就是落在间隔边缘上的数据点, 即支持向量。

在实际应用中, 通常采用序列最小最优化 (SMO) 算法来进行高效的求解 (分析比较复杂, 略)。

支持向量机的一个例子如图 6.11 所示, 使用了高斯核函数。可以看出, 决策

面把两类数据完全分隔在两边，落在间隔边缘上的数据点（支持向量，图中由圆圈所圈出）在某种意义上讲是离决策面最近的数据点，只占了所有数据点的一部分。模型训练好以后只需要这些支持向量的信息（以及其 λ_n 与 t_n ），其它数据点在预测阶段不再需要。另外，决策面“小心翼翼”地从两类数据中间穿过，尽量保持距离。这正是支持向量机具有鲁棒性的原因：努力用一个最大间距来分离样本。

3.3 线性不可分体系：软间隔的使用

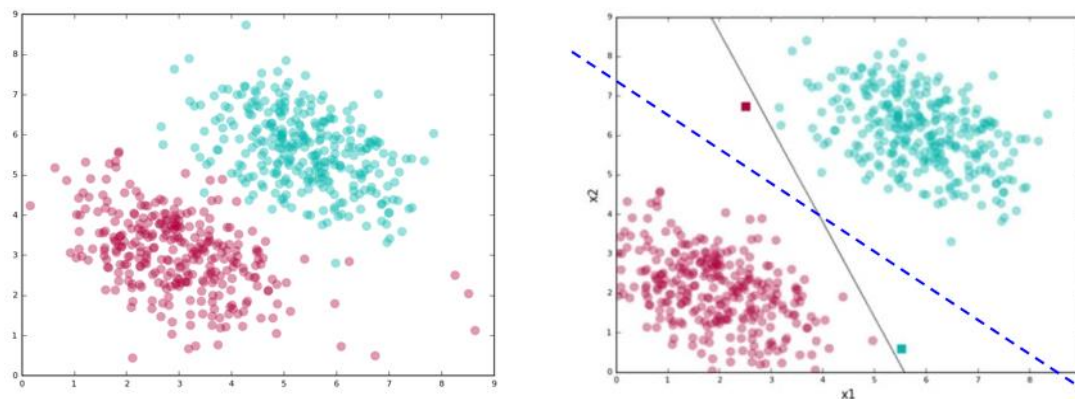


图 6.12. 支持向量机中硬间隔的可能困难。(a) 没法用一条直线（决策面）把两类数据完全分开。(b) 虽然可以用一条直线（黑色实线）把两类数据完全分开，但结果的安全距离（间隔）过小；如果允许分类错误（两个方形的数据点），则所得结果（蓝色虚线）具有更大的间隔，可以防止过拟合，效果更好。

在上一节中，我们讨论的是两类数据可以完美线性分离的简单情况，在训练集上能够实现 100% 的准确率。然而，真实世界是混乱的，经常会碰到硬间隔没法很好发挥作用的情况（图 6.12）。例如，图 6.12 (a) 中的数据是线性不可分的，不管怎样划直线，都不能把两类数据完全分开。而图 6.12 (b) 中的数据虽然是线性可分的，但按上一节方法得到的结果（黑色实线）离数据点太近，安全距离（间隔）很小。或者说，只有很瘦的蛇才能爬过去。需要注意的是，训练数据里是可能存在错误的，比如标签给错了。例如，左上角靠近黑色实线的那个红色点离蓝色点很近而离其它红色点很远，说不定它其实是蓝色的；而右下角靠近黑色实线的那个蓝色点说不定其实是红色的。如果是这样的话，那我们就应该得到虚线所示的决策面，具有很大的安全距离（间隔），蛇变得很肥，看起来合理多了。另外还需要注意，分类器的价值并不在于如何很好地将训练数据分开，而在于能够正确分类尚未看到的数据点（测试集）。因此，即使图 6.12 (b) 中的数据是没有错误的，虚线决策面很可能仍然是比实线的更好。支持向量机如何解决这些问题呢？

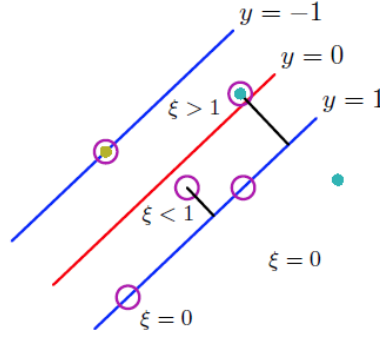


图 6.13. 松弛变量 ξ_n 衡量了数据点越过间隔边界的距离。

支持向量机采用的对策是：允许分类错误，以增加间隔。或者说，允许蛇在爬行时触发一定数量的地雷，以换取比较肥的蛇能够通过。具体地，是采用软间隔的思想。为了允许误差的存在，引入松弛变量（slack variables） $\xi_n \geq 0$ （参见图 6.13），使约束条件放宽为

$$t_n y(\mathbf{x}_n) \geq 1 - \xi_n \quad (6.47)$$

ξ_n 可看做是数据点越过间隔边界的距离，即它造成了某种误差，需给予相应的惩罚（但不是硬性禁止）。求解时可将其视为独立变量。加上这个惩罚，模型的误差（代价）函数写成

$$\tilde{E}(\mathbf{w}, b, \xi_n; C) = C \sum_n \xi_n + \frac{1}{2} \|\mathbf{w}\|^2 \quad (6.48)$$

其中 C 是超参数，用于调整对越界行为的惩罚力度。数据点没有越过边界时， $\xi_n = 0$ 。因此，这相当于惩罚错误（越界越多，惩罚越重）但不额外奖励正确（只要不越界，不管离边界多远，都是无功无过）。因此， ξ_n 所给出的误差有时候被称为铰链损失函数（hinge loss）。

现在，我们的目标是

$$\arg \min_{\mathbf{w}, b, \{\xi_n\}} \left\{ C \sum_n \xi_n + \frac{1}{2} \|\mathbf{w}\|^2 \right\} \quad (6.49)$$

同时必须满足如下约束条件：

$$t_n [\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n) + b] \geq 1 - \xi_n \quad (6.50)$$

$$\xi_n \geq 0 \quad (6.51)$$

利用拉格朗日乘子法，引入

$$L(\mathbf{w}, b, \xi_n, \lambda_n, \mu_n) = C \sum_n \xi_n + \frac{1}{2} \|\mathbf{w}\|^2 - \sum_n \lambda_n \{t_n [\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n) + b] - 1 + \xi_n\} - \sum_n \mu_n \xi_n \quad (6.52)$$

KKL 条件给出

$$\frac{\partial}{\partial(\mathbf{w}, b, \xi_n)} L(\mathbf{w}, b, \xi_n, \lambda_n, \mu_n) = 0 \quad (6.53)$$

$$\lambda_n \{t_n [\mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n) + b] - 1 + \xi_n\} = 0 \quad (6.54)$$

$$\mu_n \xi_n = 0 \quad (6.55)$$

与线性可分体系类似，由 $\frac{\partial}{\partial \mathbf{w}} L = 0$ 可解出 $\mathbf{w}$ ，代回 $y(\mathbf{x}) = \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}) + b$ ，可得到公式（6.44），结果与核函数 $k(\mathbf{x}, \mathbf{x}_n)$ 有关。由于条件（6.52）的存在，所有在间隔外面的数据点都有 $\lambda_n = 0$ ，不出现在 $y(\mathbf{x})$ 的表达式中。在实际应用中，通常采用 SMO 算法来进行求解，不再介绍。

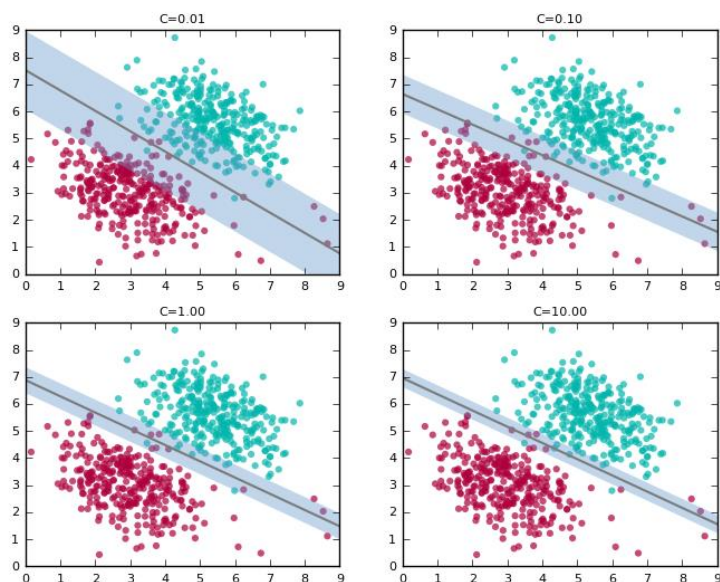


图 6.14. 支持向量机与软间隔的例子。使用了线性核函数。随着 C 的增加，间隔（图中阴影）逐渐减小，分类错误的点减少。

使用了软间隔的一个支持向量机例子如图 6.14 所示。 C 是个超参数，它决定了两个不同目标之间的权衡关系：尽量大的间隔；尽量少的分类错误。 C 越大，对分类错误的惩罚就越强烈，模型会追求更少的训练集分类错误，在图中会表现为试图容纳位于图的右下角的大多数红色点。同时，这会导致间隔减小，决策面与数据挨得太近。在前面的硬间隔例子中，间隔是一个“无人地带”，没有数据点出现在其中。而现在，总会有一些点越过边缘进入到间隔之内，这就是软间隔的含义。在实际应用中，通常使用交叉验证等技术来选择合适的 C 值。

扩展内容：逻辑回归的误差公式分析

根据逻辑回归模型的内容，当 $t_n = 1$ （第一类）时，预测结果与 t_n 一致的
 概率是 $y_n \equiv y(\mathbf{x}_n) = \sigma(\mathbf{w}^T \mathbf{x}_n) = \frac{1}{1+e^{-z}}$ 。当 $t_n = -1$ （第二类）时，预测结果与
 t_n 一致的概率是 $1 - y_n = \frac{1}{1+e^z}$ 。综合起来，可写成： $\frac{1}{1+e^{-y_n t_n}}$ 。逻辑回归模型
 认为误差是概率对数的负值，即： $-\ln \frac{1}{1+e^{-y_n t_n}} = \ln(1 + e^{-y_n t_n})$ 。

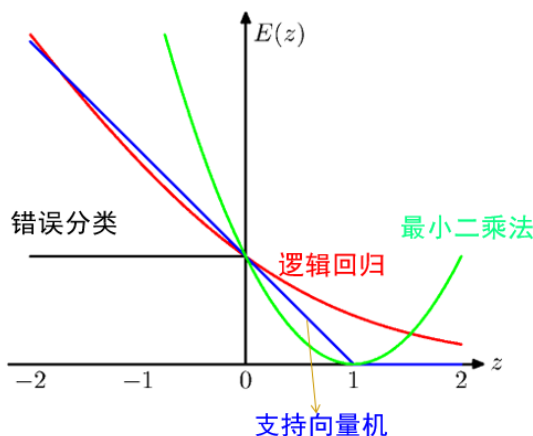


图 6.15. 支持向量机与逻辑回归的误差函数比较。 $z = y_n t_n$ 。第五章介绍的感知器模型所用的误差函数与本图中的“错误分类”曲线等价。

支持向量机的误差函数可以重新写成如下等价形式：

$$\sum_n E_{SV}(y_n t_n) + \frac{1}{2} \lambda \|\mathbf{w}\|^2 \quad (6.56)$$

其中第二项可看做是正则化项，而第一项为

$$E_{SV}(y_n t_n) = [1 - y_n t_n]_+ \quad (6.57)$$

其中 $[*]_+$ 表示只取其正值，否则为零。这一项反映了模型怎样看待不同数据点的误差，是模型的核心特性。对于逻辑回归，对应地有（参见“扩展内容：逻辑回归的误差公式分析”）

$$E_{LR}(y_n t_n) = \ln(1 + e^{-y_n t_n}) \quad (6.58)$$

它们的比较如图 6.15 所示。图中的 z 代表上面几个式子中的 $y_n t_n$ 。 $z = 0$ （等价于 $y = 0$ ）是决策面。 $z > 0$ 表示数据点在决策面的正确一边，取值越大（正值），离决策面越远，效果越好； $z < 0$ 表示数据点在错误的一边， z 越小（越负），错误越过决策面越远，效果越差。模型训练时追求所有数据点的总误差最小。可以看

出, 当 $z > 1$ 时, 支持向量机的误差函数为零, 不再发生变化。也就是说, 只要数据点没有越过间隔的边界, 误差都是相同的 (等于零)。而在 $z < 1$ 时, 误差随 z 线性变化, 所受惩罚等于越过边界的距离。整条曲线呈铰链状, 这也是支持向量机的误差函数有时候被称为铰链损失函数 (hinge loss) 的原因。与此形成对比, 逻辑回归的误差函数是随 z 增加而不断减小的。

3.4 支持向量机用于回归问题

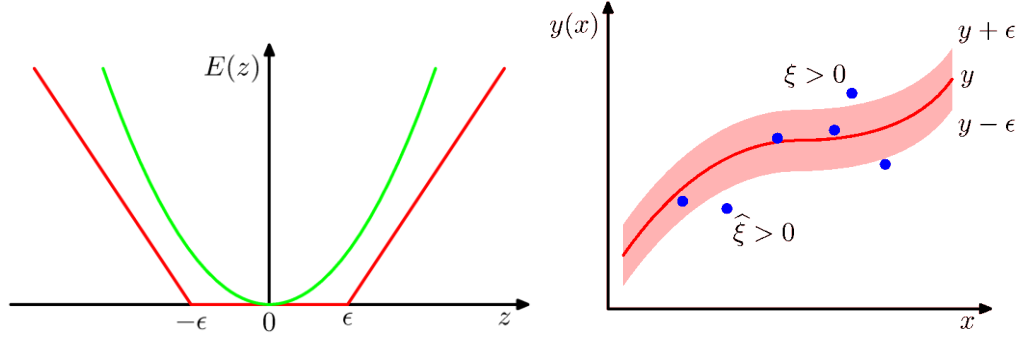


图 6.16. 支持向量机用于回归问题的原理。(a) 用于回归问题时支持向量机的误差函数 (红线), 以及与二乘误差 (绿线) 的比较。其中 $z = y(\mathbf{x}) - t$ 。(b) 对越过间隔边界的数据点施加松弛变量 $\xi_n, \hat{\xi}_n$ 。

支持向量机的发展主要是为了解决分类问题的, 不过也可以推广应用于回归问题。回归问题的二乘误差为

$$E = \frac{1}{2} \sum_n (y_n - t_n)^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2 \quad (6.59)$$

借鉴支持向量机分类算法中的思想, 将误差定义为 (参见图 6.16 (a))

$$E_\epsilon(y(\mathbf{x}) - t) = \begin{cases} 0, & \text{if } |y(\mathbf{x}) - t| < \epsilon \\ |y(\mathbf{x}) - t| - \epsilon, & \text{otherwise} \end{cases} \quad (6.60)$$

其中 ϵ 是超参数。或者等价地, 引入松弛变量 $\xi_n, \hat{\xi}_n \geq 0$, 要求 (参见图 6.16 (b))

$$\begin{cases} t_n \leq y(\mathbf{x}_n) + \epsilon + \xi_n \\ t_n \geq y(\mathbf{x}_n) - \epsilon - \hat{\xi}_n \end{cases} \quad (6.61)$$

则误差函数变成:

$$C \sum_n (\xi_n + \hat{\xi}_n) + \frac{1}{2} \|\mathbf{w}\|^2 \quad (6.62)$$

因此, 这相当于是在 $y - \mathbf{x}$ 平面让一条尽可能瘦的蛇爬过所有的数据点, 没被爬过的点按其于蛇身距离的远近给予额外惩罚 (参见图 6.16 (b))。

进一步, 可利用拉格朗日方法与 KKT 条件来求解上述问题。结果表明数据点是通过核函数起作用的, 而且只有那些处于间隔边缘或以外的点 (支持向量)

才有贡献（略）。一个例子见图 6.17。

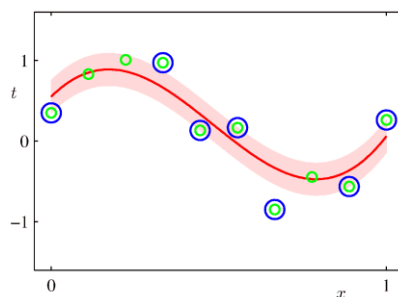


图 6.17. 支持向量机用于回归问题的例子。用了高斯核函数。红线代表所预测的 $y(\mathbf{x})$ ，阴影代表间隔。支持向量用蓝色圆圈突出显示。

3.5 支持向量机与其它方法的比较

支持向量机与逻辑回归同为凸优化，只有一个优化解，这是一个优点。不过，逻辑回归能够给出概率信息，但支持向量机不能给出概率信息。另一方面，支持向量机有个很大的优点，就是它不容易过拟合。支持向量机可以有比数据还多的参数，只要它有足够大的间隔，就不容易过拟合。那么，实际应用中应该选用支持向量机还是逻辑回归呢？吴恩达在《机器学习公开课》中给出的经验建议是（特征数 M ，样本数 N ）：

- N 较小（允许比 M 小），训练集数据量不够支持我们训练一个复杂的非线性模型，可选用逻辑回归模型或者不带核函数的支持向量机。
- 如果 M 较小（例如在 1-1000 之间），而且 N 大小中等（10-10000 之间），可使用高斯核函数的支持向量机。
- 如果 M 较小，而 N 较大（例如大于 50000），则使用支持向量机会非常慢，解决方案是创造、增加更多的特征，然后使用逻辑回归或不带核函数的支持向量机。

神经网络适用性很广，在以上三种情况下都可能会有较好的表现。但它的优化问题是非凸的，训练一般比较慢，而且从不同的初值出发可能得到不同的结果。它的好处是训练结束后不需要保存训练数据点，一般得到的模型比较紧凑（参数较少，可以用更简洁的方式来表达许多函数——这可能获益于其分层处理信息的架构），预测阶段计算更快，因此在一些对速度要求很高的场合（如股票高频交易）中很有优势。

支持向量机与近邻法都是典型的核方法。不同的地方在于前者只需要记住那些用于确定边界的关键数据点，因此在数据非常多的情况下会比近邻法计算更快。近邻法需要记住并比较所有的数据，因此在高频交易事物（如股票）中并不适用。另外，近邻算法的边界是锯齿状的，但支持向量机的边界是光滑的。

第 4 节 应用例子

4.1 MOFs 合成预测

第一个例子是关于 MOFs 水热合成法的预测：

Paul Raccuglia, Katherine C. Elbert, Philip D. F. Adler, Casey Falk, Malia B. Wenny, Aurelio Mollo, Matthias Zeller, Sorelle A. Friedler, Joshua Schrier & Alexander J. Norquist.

Machine-learning-assisted materials discovery using failed experiments.

Nature **533**, 73 (2016).

（在科研中，通常只重视成功的例子，失败的例子价值不大。但是在机器学习中，负样本与正样本是同等重要的，对于训练机器学习模型是必不可少的。这篇文章在标题中特别强调了“failed experiments”的重要性。）

MOFs 是金属有机骨架化合物（Metal organic Framework）的简称。是由无机金属中心（如金属离子或金属簇）与桥连的有机配体通过自组装相互连接而形成的一类具有周期性网络结构的晶态多孔材料。MOFs 是一种有机-无机杂化材料，兼具无机材料的刚性和有机材料的柔性特征，并具有多孔、大比表面积和多金属位点等优良性能，因此在化学化工领域（例如气体贮存、分子分离、催化、药物缓释等）呈现出巨大的发展潜力。

水热合成法是一种在密闭体系中，以水为溶剂（或其它有机溶剂），通过高温高压使常温下不溶的材料变得可溶，从而进行化学反应和结晶的方法。根据加热温度，可以分为亚临界水热合成法和超临界水热合成法。对于 MOFs 的水热合成，重要的影响因素包括：加入哪些种类的无机物与有机物作为反应物？浓度是多少？溶剂类型？反应条件（温度、时间、pH 值等）如何？机器学习模型的任务就是根据这些输入来预测输出，即产物结果，包括是否能得到晶体？纯度如何（多相产物还是单相产物）？

机器学习所需要的数据来自于文章作者所在的实验室。通过对以往的实验记录进行整理、复查（有 1.89% 的记录错误）与筛选，最后得到了 3955 个完整的样本数据（包括成功的、能得到产物的实验，也包括失败的、什么都没得到的实验）。产物结果按晶体（1-无固态物；2-无定形物；3-多晶；4-单晶（> 0.01 mm））与纯度（1-多相；2-单相）的不同情况进行分类。从机器学习的角度讲这是个分类问题。

利用专业知识对输入进行了特征设计。对于有机反应物，主要选取它们的物化性质（分子量，pH 值下的氢键受体与给体数目，极化表面积，等等），共 19 个性质。如果有多个有机反应物，会进一步计算这些性质的平均值、最小值、最大

值。对于无机反应物，有 12 个特征来反映原子性质（如离子化能、电子亲和力和电负性、硬度、原子半径，等等），22 个逻辑值（即独热编码）来反映各种不同金属，28 个逻辑值来反映其在周期表中的位置，8 个逻辑值来反映金属价态。组分信息由 6 个特征来反映：有机物/水的摩尔比，无机物/水的摩尔比，有机物/无机物的摩尔比，有机物上氢键受体与水的氢键给体的摩尔比，有机物上氢键给体与水的氢键受体的摩尔比，所有非水分子（即无机+有机+草酸盐类）与水的摩尔比。反应条件有 5 个（温度、时间、pH 值、是否缓慢降温，等等）。最后一共得到 273 个特征（输入分量）。

数据按 1:2 的比例分成测试集与训练集。测试集与训练集的分配不是完全随机的，包含同一组无机反应物-有机反应物组合的所有反应都要么全部放入测试集要么全部放入训练集，而不会有些放到测试集，有些放到训练集。目的是防止测试集与训练集的数据过于相像（例如只有反应温度不同）。采用二分类标签：原始数据的“1-无固态物”与“2-无定形物”被合并成一类（阴性），“3-多晶”与“4-单晶”被合并成另一类（阳性）。最后，有 2782 个阳性数据，1173 个阴性数据。

表 6-1 不同方法的测试集准确性

方法	准确性 Accuracy
决策树 (J48)	67.5
随机森林 (100 或 1000 棵树)	69.8, 70.5
逻辑回归	69.2
K 近邻法 ($K = 1, 2, 3$)	69.1, 66.9, 68.4
支持向量机	74.1

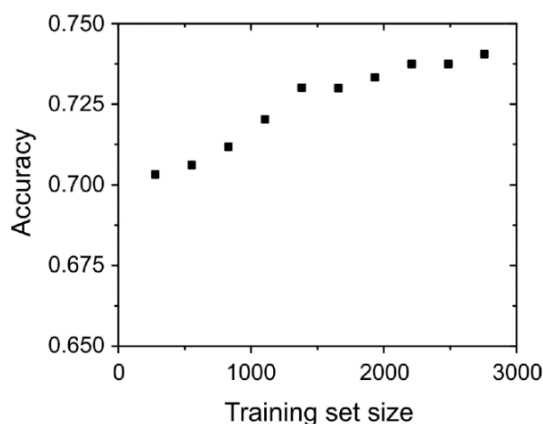


图 6.18. 支持向量机的学习曲线。横轴为训练集大小，纵轴为测试集的准确性。

文章作者使用了支持向量机、逻辑回归、K 近邻法、决策树与随机森林（在第十章将介绍这两种方法）对这些数据进行学习。最后的效果比较如表 6-1 所示。可以看出，支持向量机的效果最好，准确率达到 74.1%，比其它方法要高 4 到 8 个百分点。支持向量机的学习曲线如图 6.18 所示。准确性随训练集数据的增加而上升。即使只用 10% 的训练集（276 个数据），仍能有 70.3% 的准确率，效果仍然相当不错，没有发生过拟合。这表明高维数据对支持向量机来说也不会有问题。

作者还对模型进行了特征选择分析，以识别对分类效果影响最大的特征。最后发现有机胺的三个特征（范德华表面积、阳离子的溶剂可及表面积，氢键给体的数量）以及无机组分的三个特征（金属的鲍林电负性平均值、摩尔加权硬度和平均摩尔加权原子半径）与分类成功率的贡献最大。如果只使用这六个特征，模型的精度大概为 70.7%。这些特征也在决策树模型的结果中发挥重要作用。

为了进一步检验机器学习模型的效果，作者还做了新的实验。它们利用数据库检索从 1680 种有机二胺中找出了数据集中的有机胺相似度比较高的 34 种新的、数据集中没有的二胺，并利用前述训练好的支持向量机预测了合适的反应体系。作为比较，根据以往的专家经验，使用人工方式也进行了预测。然后根据这些预测开展实验，共进行了 276 个实验。结果是支持向量机的成功率有 89%，高于人工预测的结果（78%）。

最后，作者还对机器学习模型进行了进一步的机制分析，以便揭示 MOFs 水热合成的可能机制。发现大部分成功例子可以归结为三种可能类型：（1）小尺寸、低极化率的胺需要更长的反应时间，同时排除竞争性的 Na^+ 离子，以避免无机构筑单元的沉淀；（2）球形、低投影尺寸的胺需要含 V^{4+} 的试剂，如 VO_2 ，因为它们无法直接从典型的 V^{5+} 前体生成 V^{4+} ；（3）比较长的三胺和四胺需要草酸盐反应物来改变无机二级构筑单元的电荷密度。

4.2 有机反应预测

第二个例子是对有机反应能否进行以及产率高低的预测：

Jarosław M. Granda, Liva Donina, Vincenza Dragone, De-Liang Long & Leroy Cronin. Controlling an organic synthesis robot with machine learning to search for new reactivity. *Nature* **559**, 377 (2018).

这个工作结合了有机合成机器人，能够同时做 6 个实验，一天可以完成 36 个有机实验。与多数人印象中有手有脚的机器人不同，这个有机合成机器人应该被称为自动化系统，大部分是以零件形式被摆放在一个通风橱中。它的核心部件是一组装有化学品的原料罐和压力泵，这些泵负责将反应物送入可并行操作的 6 个反应瓶中，并待反应结束后将混合物依次送入红外光谱仪、质谱仪、核磁共振

仪中进行检测。通过支持向量机对比反应前后的指纹图谱来判断反应是否进行以及进行的程度。自动向量机是事先在 72 个人工标注的数据（即人工解读指纹图谱来判断反应是否进行）上利用留一法交叉验证进行训练的，使用了线性核函数，预测精度为 86%。

项目的目标是让这个 AI 系统（机器人）自主安排实验，以便从 18 个化合物中快速地找出可以产生化学反应的原料组合。为了减小工作量，反应使用统一的条件，并且只专注于两组分和三组分反应——可能的组合为 $C_{18}^2 + C_{18}^3 = 969$ 种。机器人首先随机地选择了 100 个组合进行第一轮实验，通过 SVM 模型判定实验结果后，使用线性判别分析（linear discriminant analysis, LDA）算法对数据进行归纳和总结（即模型训练）。随后，AI 会根据训练得到的模型对剩余组合全部预测一遍，在此基础之上，挑出反应概率最大的 100 个组合开始第二轮实验（“机器人会优先选择它认为最有可能产生化学反应的 100 个实验开始第二次尝试”）。每完成 100 个实验，机器人都会将结果增加到训练集里，并对机器学习模型重新进行训练（“使用算法重新学习”），然后重新预测剩余的组合并挑出最有希望的 100 个进行下一轮的实验，直到完成指定数量的实验或是它判断剩余实验不会产生化学反应为止。这就是所谓的能够像人一样“三思而后行”的 AI 机器人。

这个流程非常重要，它实际上结合了监督学习与强化学习，在 AI 训练/预测与实验之间实现信息的来回流动与反馈，形成完整的闭环。这是一种典型的 AI 赋能实验的范式。

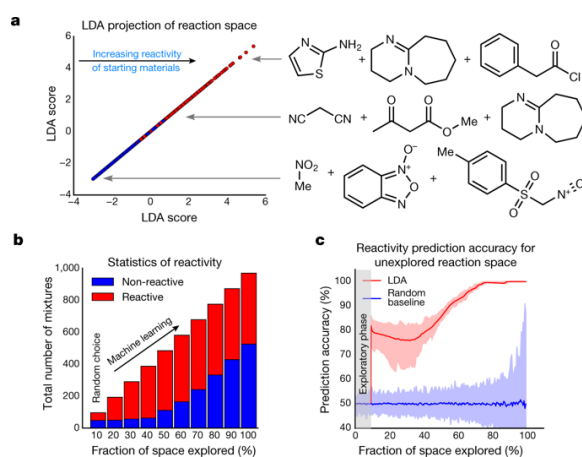


图 6.19. AI 机器人自主探索化学空间的效果。(a) 所有组合的 LDA 投影，其中红色符号代表反应组合，蓝色符号代表非反应性组合。(b) 随着实验轮次的进行，所探索到的化学组合空间（红色与蓝色分别代表反应与非反应组合）。(c) 预测准确性随实验进程的变化（模拟结果）。

不过，落实到论文所采用的具体研究体系上，其实难度比较低。原因是总的状态空间也就 969 个可能的数据点，因此机器学习模型的训练数据其实是很“稠密”的，或者说，比较容易学习。反映到论文的具体模型方案上，就是甚至不需

要利用分子的化学信息（领域知识），只是对 18 种化合物进行独热编码来作为机器学习 LDA 模型的输入。结果如图 6.19 所示。在所有的 969 种组合中，共有 444 种最终证明能发生反应，比例约为 46%。在第一轮实验中，因为是随机挑选的组合，因此找到的反应体系比例约 46%，但在此基础上经过机器学习模型的训练与预测，挑选出来的进行第二轮实验的 100 个组合就大部分是反应性的了（图 6.19(b)），接下来的几轮也保持很高的成功率。一直到第五轮开始（这时候大部分反应体系已经被挑完了），由于剩下的可供挑选的组合中能发生反应的已经不多了，非反应性的组合才开始变多。AI 的介入，确实（相对于随机猜测）大大提高了总体的准确率（图 6.19(c)）。

这篇文章的另一部分工作是让 AI 系统来预测化学反应的产率（“除了能像人类化学家一样探索新反应，机器人还可以干一些化学家们力不能及的活——预测产率”）。作者收集了文章中 5760 个 Suzuki-Miyaura 偶联反应的数据，将其分成训练集（含 3456 个数据）与测试集（含 3456 个数据），反应物仍采用独热编码。机器学习模型换成神经网络，最后得到的测试误差仅为 11%。利用前述流程并结合这些数据进行仿真，在随机挑选了 576 个数据进行学习后，模型在余下反应中选择了前 100 名预测能获得高产率的体系，结果这些体系的真实产率并不高，平均只有 39%。尽管如此，如果将这 100 个新数据添加到原来的包含 576 个数据的训练集后对模型重新进行训练，则预测水平有了大幅度提升，它在剩余反应中挑选的前 100 名准高产反应的真实平均产率达到了 85%。

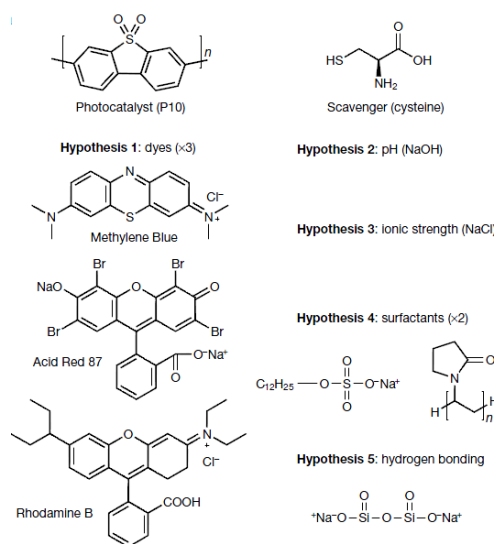


图 6.20. 水光解产氢的催化剂及配方需要优化的 10 种组分。

类似流程在其它工作中也经常采用。例如下面的工作：

Benjamin Burger, Phillip M. Maffettone, Vladimir V. Gusev, Catherine M. Aitchison, Yang Bai, Xiaoyan Wang, Xiaobo Li, Ben M. Alston, Buyi Li, Rob Clowes, Nicola Rankin, Brandon Harris, Reiner Sebastian Sprick & Andrew I. Cooper.

A mobile robotic chemist.

Nature **583**, 237 (2020).

这个工作利用了更加智慧的能自主移动的机械手，能在一个操作台上取料（称量固体分子，调配液体试剂），跑到另一个操作台上把料加到反应器里，给容器抽真空，进行化学催化实验，等反应结束后把样品送到测试表征仪器里，看起来更符合普通人对机器人的想象。工作的目标比上一论文难度更高，是想对水光解产氢的催化剂及配方进行优化，需要探索的状态空间更大。它需要优化的是 10 种化学组分比例（图 6.20）（这也是机器学习模型的输入）。通过实验-学习/预测-实验-...的不断循环，这个 AI 系统在 8 天时间里做了 688 个实验，将光催化剂的活性提高了 6 倍。

4.3 超硬材料预测

第三个例子是关于超硬材料的预测：

Aria Mansouri Tehrani, Anton O. Oliynyk, Marcus Parry, Zeshan Rizvi, Samantha Couper, Feng Lin, Lowell Miyagi, Taylor D. Sparks, and Jakoah Brgoch.

Machine learning directed search for ultraincompressible, superhard materials.

J. Am. Chem. Soc **140**, 9844 (2018).

机器学习的一个潜在应用是帮助寻找/发展具有优异性能的材料。这篇文章工作的目的就是寻找具有超高硬度的材料。传统上，超硬材料的发展主要依赖于实验试错法或简单的设计规则（很多其它材料也是）。这些研究主要集中在由轻主族元素形成强共价键的材料，如金刚石、*c*-BN、 B_6O 和 *c*- BC_2N ，或将轻元素与价电子密度比较高的过渡金属互相结合的化合物，如 ReB_2 和 WB_4 。另外，也有人通过 DFT 计算等量子化学方法来计算材料的硬度或弹性常数，但通常没法对超大规模数量的不同体系进行计算。在这方面，机器学习就提供了一个计算量较小的有用手段。

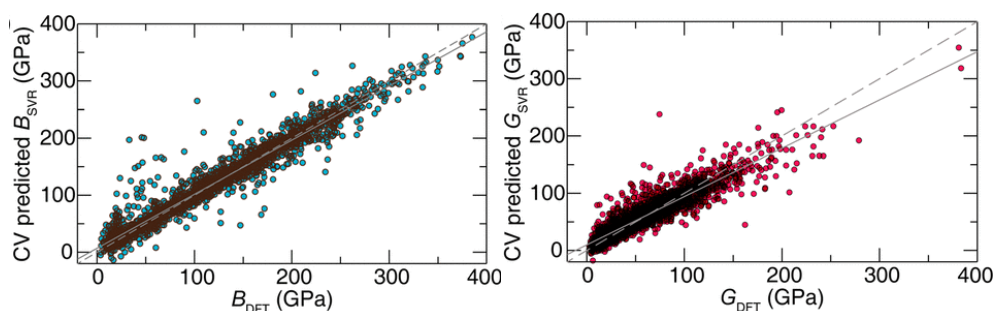


图 6.21. 支持向量机对材料弹性模量的交叉验证结果。横轴是 DFT 计算结果，即训练集数据，纵轴是交叉验证的预测结果。(a) 体模量 B ；(b) 剪切模量 G 。

文章工作利用了来自材料数据库 Materials Project database 的数据，这个数据

库提供了很多材料用 DFT 计算得到的弹性模量（体模量 B 与剪切模量 G ）。经过一些筛选（例如，要求材料在常温常压下稳定），最后得到了 2572 个数据。

用于描述不同材料的特征包括 34 种与原子组成相关的变量，例如在元素周期表上的位置、电子结构和其它物化性质（原子大小、电负性、价态，等等），以及它们相关的统计值（平均值、最大值和最小值、差值）。此外，还有 14 个与晶格、空间群和晶胞体积等变量相关的结构性质。一共有 $34 \times 4 + 14 = 150$ 个特征。

所使用的机器学习模型就是支持向量机，采用径向基函数（RBF）作为核函数，并采用十倍交叉验证方案进行训练。效果如图 6.21 所示。机器学习模型对体模量的预测精度很高，与所用数据集的关联系数高达 $R^2 = 0.97$ ；对剪切模型的预测精度稍有下降，但关联系数仍能达到 $R^2 = 0.88$ 。

有了训练好的模型，作者对一个超大数据库 Pearson's Crystal Database (PCD) 里的近 12 万种材料（它们的弹性模量与硬度未知）的性质进行了预测，并结合合成难度等考量从预测具有超高模量的体系中挑选出两种， $\text{ReWC}_{0.8}$ 与 $\text{Mo}_{0.9}\text{W}_{1.1}\text{BC}$ ，进行了实验合成与性能表征。最后测量得到的体模量分别为 380 GPa 与 373 GPa，与支持向量机的预测结果（398 与 370 GPa）很接近。另外，硬度测量表明这两种材料的维氏硬度分别为 40 ± 3 GPa 与 42 ± 2 GPa，都达到了超硬材料的标准（40 GPa）。

扩展阅读：

- <https://baike.baidu.com/item/%E6%A0%B8%E5%87%BD%E6%95%B0>
百度百科：核函数
- <https://www.cnblogs.com/liaohuiqiang/p/7805954.html>
拉格朗日乘子法和 KKT 条件
- <https://www.jiqizhixin.com/articles/2019-02-12-3>
- <https://distill.pub/2019/visual-exploration-gaussian-processes/>
看得见的高斯过程：这是一份直观的入门解读
- <https://cloud.tencent.com/developer/article/1033696>
支持向量机入门简介
- https://blog.csdn.net/v_july_v/article/details/7624837
支持向量机通俗导论（理解 SVM 的三层境界）

- http://m.sohu.com/a/244701109_610519
三思而后行！Nature 报道像化学家一样探索新反应的“AI”机器人
- <https://www.huxiu.com/article/367828.html>
无情的科研机器来了：8 天不停，做了近 700 个实验
- <https://www.yantuo.com.cn/8010.html>
《Nature》封面：化学家失业在即？不需要休息！无情的科研机器人横空出世！
- <https://baijiahao.baidu.com/s?id=1594514807888322634>
机器学习萌新必学的 Top10 算法

参考资料与文献：

[1] Vladimir N. Vapnik. The Nature of Statistical Learning Theory. 2nd edition. Springer, 1999.

[2]

第七章 概率图模型

刘志荣

第 1 节 有向图：贝叶斯网络

1.1 贝叶斯网络：想法与内容

前一章我们讲过，利用概率分布来讨论体系性质，是机器学习的一种重要思路。当变量较少时，我们可以用贝叶斯公式与直方图等方法来处理。但当变量很多时，用这些方法处理起来就很繁琐。这时候，一种更方便的做法是使用概率图模型来描述

简单地说，概率图模型（probabilistic graphical model）是用图来表示变量概率依赖关系的模型，通过结合概率论与图论的知识，有效地处理与多变量联合概率分布有关的问题。在概率图中，每个节点代表一个随机变量，连接两个节点的边代表它们之间的依赖或相关关系，其中有向边表示单向的依赖，无向边表示相互依赖关系。这种图形化的方式，优点是容易可视化，很直观，有助于模型的设计；另外，也容易从中得到有用的见解（例如后面介绍的条件依赖性）；还有就是有助于简化相关的运算^w。

贝叶斯网络（Bayesian network），又称信念网络（Belief Network），是一种概率图模型，于 1985 年由 Judea Pearl 首先提出。它是一种模拟人类推理过程中因果关系的不确定性处理模型，其网络拓扑结构是一个有向无环图（directed acyclic graph, DAG）。

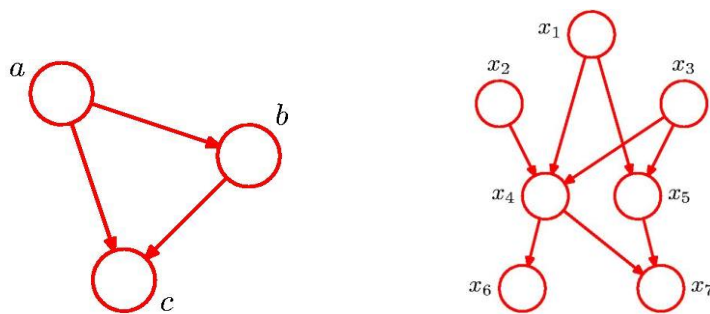


图 7.1. 贝叶斯网络：用有向无环图刻画联合概率分布中的条件概率依赖关系。

贝叶斯网络用有向边表示单向的依赖关系，即概率论里的条件概率。严格讲，这种依赖关系并不代表因果关系，但有时候可以不严格地看做因果关系，以方便理解。例如，图 7.1 (a) 的例子中， a 影响 b （或者说， b 受 a 的影响），然后 a 和 b

^w 在量子化学与量子物理领域，一个将图形化用于简化运算的成功例子是费曼图。

一起影响 c ，总的联合概率分布写成：

$$p(a, b, c) = p(c|a, b)p(b|a)p(a) \quad (7.1)$$

一般情况下，贝叶斯网络使用因子化形式把总的联合概率写成每个随机变量的局部条件概率分布的乘积：

$$p(\mathbf{x}) = \prod_{k=1}^K p(x_k|\mathbf{pa}_k) \quad (7.2)$$

其中 $\mathbf{pa}_k$ 代表节点 k 的所有父节点（在图中，箭头是从父节点指向子节点）。因此，可以很容易写出图 7.1（b）所示贝叶斯网络的联合概率：

$$p(x_1, \dots, x_7) = p(x_1)p(x_2)p(x_3)p(x_4|x_1, x_2, x_3)p(x_5|x_1, x_3)p(x_6|x_4)p(x_7|x_4, x_5) \quad (7.3)$$

图 7.1（a）是一个全连接图（在任意两个节点之间都有连线），任何一个三变量联合分布都可以用这个图来代表： $p(a, b, c) = p(c|a, b)p(a, b) = p(c|a, b)p(b|a)p(a)$ ，所以这个图本身并没有给我们带来多少信息。更有价值的其实是有某些边缺失的情况，如图 7.1（b），那会带来信息。

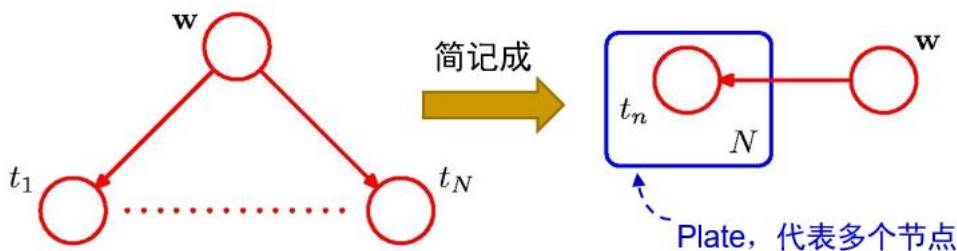


图 7.2. 贝叶斯线性回归模型所对应的贝叶斯网络。

前面各章的一些模型也可以重新用贝叶斯网络来阐述。例如，在贝叶斯线性回归中， $\mathbf{w}$ 被看做具有某种分布的随机量，而 x_n 处的输出观察值受 $\mathbf{w}$ 的影响： $t_n = y(x_n, \mathbf{w}) + \epsilon_n$ ，此处 $y(x, \mathbf{w}) = \sum_j w_j x^j$ 的函数形式是已知的（以多项式基为例）。因此，它们之间的关系可以用图 7.2 所示的贝叶斯网络来描述^{ww}，对应的联合概率为

$$p(\mathbf{t}, \mathbf{w}) = p(\mathbf{w}) \prod_n p(t_n|y(x_n, \mathbf{w})) \quad (7.4)$$

其中 $\mathbf{t}$ 代表 $\{t_n\}$ 。这里可以看出边的缺失：各个 t_n 节点之间是没有连线的。因此，在 $\mathbf{w}$ 固定的情况下，各个 t_n 之间是独立的，不会互相影响。

贝叶斯网络经常采用一些简洁的记号。例如，图 7.2 中本来有多个 t_n 节点，

^{ww} 这里 ϵ_n 的影响反映在那些边里面，即条件概率 $p(t_n|y(x_n, \mathbf{w}))$ ，本身不是节点。当然，如有需要， ϵ_n 也可以用节点表示出来，那样 t_n 就完全由 $\mathbf{w}$ 与 ϵ_n 决定，没有任何随机分布。

但我们可以用一个被方框围住的节点来代表这多个类似节点（方框右下角的“ N ”代表这里有 N 个节点），称为“盘”（plate）。又如，用圆点来显式地表示固定的值或量，例如输入自变量 x_n 与超参数 α 在贝叶斯回归中的影响（图 7.3（a））：

$$p(\mathbf{t}, \mathbf{w} | \mathbf{x}, \alpha, \sigma^2) = p(\mathbf{w} | \alpha) \prod_n p(t_n | x_n, \mathbf{w}, \sigma^2) \quad (7.5)$$

用带阴影的节点表示已观测量，例如训练集里的 t_n 就是已经被观测而知道的量（图 7.3（b））。假设 $\hat{t}$ 是 $\hat{x}$ 处待预测的（未知）值，则完整的贝叶斯回归可以用图 7.3（b）的贝叶斯网络描述，最后有^{xx}：

$$p(\hat{t} | \hat{x}, \mathbf{x}, \mathbf{t}, \alpha, \sigma^2) \propto \int p(\hat{t}, \mathbf{t}, \mathbf{w} | \hat{x}, \mathbf{x}, \alpha, \sigma^2) d\mathbf{w} \quad (7.6)$$

其中

$$p(\hat{t}, \mathbf{t}, \mathbf{w} | \hat{x}, \mathbf{x}, \alpha, \sigma^2) = p(\mathbf{w} | \alpha) [\prod_n p(t_n | x_n, \mathbf{w}, \sigma^2)] p(\hat{t} | \hat{x}, \mathbf{w}, \sigma^2) \quad (7.7)$$

这里 $\hat{t}, \mathbf{t}, \mathbf{w}$ 都是随机变量。虽然 $\hat{t}$ 与 $\mathbf{t}$ 之间没有直接的依赖关系（连线），但由于它们与 $\mathbf{w}$ （以及那些圆点所表示的固定值）之间的如图所示的依赖关系，一旦 $\mathbf{t}$ 的值被观测到， $\hat{t}$ 的可能值（分布）也会受影响。这就是我们在贝叶斯公式时所介绍过的“证据的出现改变信念”。只不过现在借助贝叶斯网络，我们的推断可以更加高效。

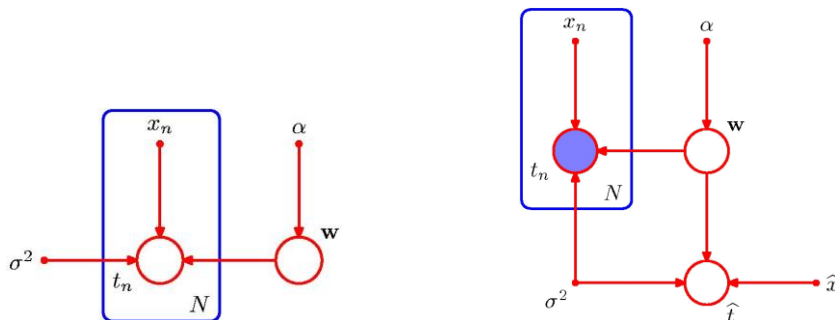


图 7.3. 贝叶斯网络的一些简化记号。(a) 圆点表示固定的值，而非随机变量。(b) 带阴影的节点表示已观测量。

与直方图方法和核密度估计法类似，贝叶斯网络既是一种无监督学习（根据数据集估计其密度分布），也是一种监督学习（有了 $p(\mathbf{x}, t)$ ，就可以进行推断，完成回归或分类的任务）。这类通过估计联合分布 $p(\mathbf{x}, t)$ 来预测 $t(\mathbf{x})$ 的方法^{yy}，被称为生成式方法或生成式模型（generative models）。与此相对的，另一大类方法则是通过估计 $p(t | \mathbf{x})$ 来预测 $t(\mathbf{x})$ ，被称为判别式模型（discriminative models），前面介绍过的线性回归、逻辑回归、支持向量机以及通常的神经网络，都属于判别式模型。

^{xx} $p(\hat{t} | \mathbf{t}) = \frac{p(\hat{t}, \mathbf{t})}{p(\mathbf{t})}$ ，既然此时 $\mathbf{t}$ 是定值，因此有 $p(\hat{t} | \mathbf{t}) \propto p(\hat{t}, \mathbf{t}) = \int p(\hat{t}, \mathbf{t}, \mathbf{w}) d\mathbf{w}$ 。展开后即公式（7.6）。

^{yy} 图 7.3（b）用于解决回归问题的贝叶斯网络假定 $\mathbf{x}$ 是固定的值（圆点），因此它其实不是完整的生成式模型。但我们可以通过改造把它变成完整的生成式模型：引入 $\mathbf{x}$ 的分布（把圆点变成圆圈）。

表面上看，生成式模型有些绕弯路的感觉：完成回归或分类任务其实只需要 $p(t|\mathbf{x})$ ；从 $p(\mathbf{x}, t)$ 可以利用 $p(t|\mathbf{x}) = \frac{p(\mathbf{x}, t)}{p(\mathbf{x})} = \frac{p(\mathbf{x}, t)}{\int p(\mathbf{x}, t) dt}$ 得到 $p(t|\mathbf{x})$ ，反之则不行，这表明 $p(\mathbf{x}, t)$ 包含更多（对目标没有帮助的）信息，肯定是干了多余的活。但是，生成式模型也有额外的好处：它们可以用来产生新的数据点。举例来说，你给 AI 一堆带标签的猫的照片和狗的照片，那么学完以后，判别式 AI 能判断一张新的不带标签的照片到底是猫还是狗，而生成式 AI 除此以外还能给你创造猫或狗的新照片。AI 画图和 ChatGPT 就都属于生成式模型。

贝叶斯网络等概率图模型经常包含两类节点：可观测节点（如线性回归中的 $\mathbf{x}$ 与 t ）与不可观测的、隐藏的节点（hidden nodes）（如线性回归中的 $\mathbf{w}$ ）。对于一个实际问题，我们希望能够挖掘隐含在数据中的规律与知识。概率图模型用观测结点表示观测到的数据，用隐藏结点表示潜在的认识，用边来描述它们之间的相互关系，最后基于这样的关系图获得一个概率分布。在学习与推断的过程中，就涉及怎样从观测数据反推潜在认识，会频繁用到贝叶斯公式，这也是贝叶斯网络名字由来的（部分）原因。

接下来我们简要讨论一下描述概率分布所需要的参数数目，以便理解概率图网络中边缺失的好处。

为了讨论方便，我们假设所有随机变量都是离散的。如果一个变量 x 有 K 个可能取值，则在最普适情况下其分布 $p(x) = \{p_1, p_2, \dots, p_K\}$ （其中 p_i 代表 x 取其第 i 个可能取值的概率）在满足归一化条件（ $\sum_i p_i = 1$ ）下需要 $K - 1$ 个参数来描述。那么，两个随机变量的联合分布需要 $K^2 - 1$ 个参数， M 个变量需要 $K^M - 1$ 个参数。所需要的参数数目随变量数目（贝叶斯网络中的节点数目）指数上升，出现维度灾难。这就是全连接的贝叶斯网络用处不大的原因。

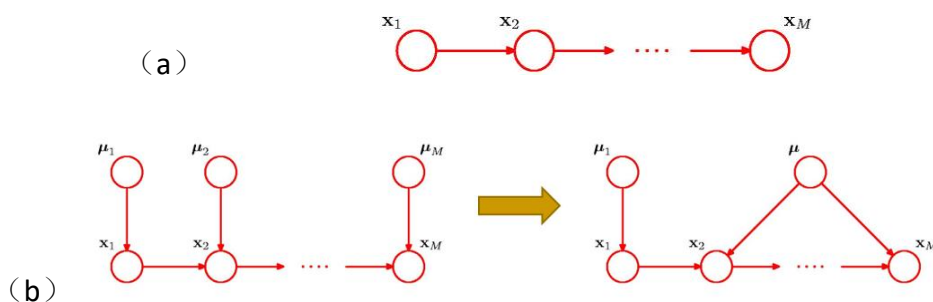


图 7.4. 贝叶斯网络的一些减少参数数目的简化情形。(a) 串联依赖。
(b) 参数共享。

在现实里，一个因素往往只跟少数近邻因素有直接的影响。因此，经常引入简化假设以减少参数个数。例如：朴素贝叶斯模型可以看做是一种特殊的贝叶斯网络，所有节点之间都没有连线，变量之间是独立的， M 个变量的概率分布只需要需要 $M(K - 1)$ 个参数，只随 M 线性增加。如果假设节点之间是通过串联的方式

互相影响的（图 7.4 (a)），则需要的参数个数^{zz}为 $K - 1 + (M - 1)K(K - 1)$ 。另外，还可以采取参数共享的形式（图 7.4 (b)）。前面第五章所介绍的循环神经网络与卷积神经网络采取的就是类似思路。

另一种减少参数数目的思路是概率的函数（公式）化，这样只需要确定其参数即可。例如，逻辑回归值只需要 $M + 1$ 个参数：

$$p(y = 1|x_1, x_2, \dots, x_M) = \sigma(w_0 + \sum_{i=1}^M w_i x_i) \quad (7.8)$$

但这种策略的代价是对函数形式的假设，没法描述“不服从这个函数形式”的分布。

1.2 重要性质：条件独立性

现在来看概率图模型中与缺失边密切相关的重要性质：条件独立性。

设 a 与 b 是两个随机事件，如果它们同时发生的概率等于各自发生的概率的乘积，

$$p(a, b) = p(a)p(b) \quad (7.9)$$

则称 a 与 b 相互独立。上式也等价于

$$p(a|b) = p(a) \quad (7.10)$$

或

$$p(b|a) = p(b) \quad (7.11)$$

即 a 的发生 b 无关，反之亦然。

如果 a 和 b 与第三个随机变量 c 满足如下条件

$$p(a, b|c) = p(a|c)p(b|c) \quad (7.12)$$

则称 a 与 b 在给定 c 的条件下是独立的 (a is conditionally independent of b given c)，记成 $a \perp\!\!\!\perp b | c$ 。上式也等价于

$$p(a|b, c) = p(a|c) \quad \text{或} \quad p(b|a, c) = p(b|c) \quad (7.13)$$

这表示：当知道（固定） c 时， a 和 b 的关系不再直接相关。换句话说， c 屏蔽了 a 和 b 之间的依赖关系。

对于贝叶斯网络，讨论三个节点之间的独立性时，经常会碰到三种情形：tail-to-tail, head-to-tail, 以及 head-to-head。

^{zz} 此时有 $p(\mathbf{x}) = p(x_1) \prod_{k=2}^K p(x_k|x_{k-1})$ ，描述 $p(x_1)$ 需要 $K - 1$ 个参数，描述 $p(x_k|x_{k-1})$ 需要 $K(K - 1)$ 个参数（注意 x_{k-1} 有 K 个取值）。因此共需要 $K - 1 + (M - 1)K(K - 1)$ 个参数。

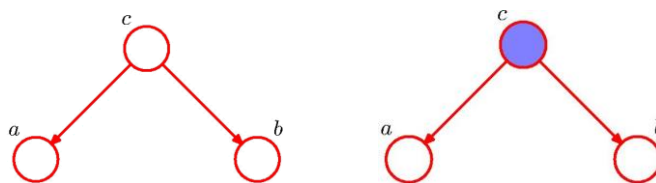


图 7.5. 三节点的 tail-to-tail 情形。(a) c 不固定时, a 和 b 是不独立的, $a \not\perp b \mid \emptyset$ 。(b) c 固定时, a 和 b 是独立的, $a \perp b \mid c$ 。

第一种情形是图 7.5 所示的“尾对尾”(tail-to-tail), 即两根箭头的尾部相连。这可以理解成 a 与 b 有共同的起因 c (严格讲并没有因果的含义, 但近似看做因果关系确实可以方便理解), 例如 a 与 b 是兄弟, c 是他们的父母。根据贝叶斯网络的性质, 此时的联合概率为

$$p(a, b, c) = p(a|c)p(b|c)p(c) \quad (7.14)$$

则

$$p(a, b) = \sum_c p(a|c)p(b|c)p(c) \quad (7.15)$$

一般情况下它并不等于 $p(a)p(b) = [\sum_c p(a|c)p(c)][\sum_c p(b|c)p(c)]$ 。因此, a 与 b 并不独立, 记成

$$a \not\perp b \mid \emptyset \quad (7.16)$$

这里 $\emptyset$ 表示空集, 即没有前提条件。当知道 (固定) c 时, 则利用 (7.14), 有

$$p(a, b|c) = \frac{p(a, b, c)}{p(c)} = p(a|c)p(b|c) \quad (7.17)$$

即 a 与 b 在 c 固定的条件下是独立的:

$$a \perp b \mid c \quad (7.18)$$

为什么 c 未知时 a 与 b 并不独立呢? 我们可以用一个生活中的例子来说明。假设我们对一个小学的所有学生进行调查并研究几个指标之间的关系: c -年龄, a -鞋子号码, b -阅读能力。如果不看孩子年龄 (即考虑所有年龄/年级的学生), 我们就会发现鞋子号码比较大的孩子阅读能力一般也比较强, 即 a 与 b 之间具有正的相关性。那这是不是表明脚大是造成阅读能力强的原因, 或者阅读能力强导致脚变大? 其实不是的, 真正的原因是年龄, 年龄大的孩子脚会比较大, 阅读能力也比较强。如果固定年龄 (挑出相同年龄的孩子来做比较), 就会发现脚的大小与阅读能力之间没有什么关系。这就是 tail-to-tail (图 7.5) 所描述的事情。

贝叶斯回归 (图 7.2) 其实也属于这种 tail-to-tail。 w 是 t 的原因。如果 w 是已知 (固定) 的, 那么我们就可以根据 w 与 x 计算 (预测) t , 此时 t 与训练集中的 t_n

之间是独立的， t_n 的值对 t 没什么影响，因此我们并不需要这些 t_n （训练好后得到 $\mathbf{w}$ ，预测时并不需要训练集数据）。但如果 $\mathbf{w}$ 是未知的，虽然 t 与 t_n 之间并没有直接影响（贝叶斯网络中的连线），但表现出来的结果却是 t 与 t_n 之间不是独立的，知道其中 t_n 的值将有助于对 t 值的推断（注意它们之间只有相关，没有因果），这就是我们能利用训练集进行机器学习的原因。



图 7.6. 三节点的 head-to-tail 情形。(a) c 不固定时， a 和 b 是不独立的， $a \not\perp b | \emptyset$ 。(b) c 固定时， a 和 b 是独立的， $a \perp b | c$ 。

第二种情形是图 7.6 所示的“头对尾”（head-to-tail），即一根箭头的头部与另一根箭头的尾部相连。此时 a 影响 c ， c 影响 b ，或者说， a 通过 c 影响 b 。例如，房间里的火灾（ a ）发出烟雾（ c ），烟雾使报警器（ b ）发出警报。根据贝叶斯网络的性质， a, b, c 的联合概率为

$$p(a, b, c) = p(a)p(c|a)p(b|c) \quad (7.19)$$

如果 c 不固定，则

$$p(a, b) = p(a) \sum_c p(c|a)p(b|c) \quad (7.20)$$

一般情况下它并不等于 $p(a)p(b) = p(a) \sum_a \{p(a)[\sum_c p(c|a)p(b|c)]\}$ 。因此， a 与 b 并不独立， $a \not\perp b | \emptyset$ 。原因是 a 能通过 c 影响 b ，就如火灾通过烟雾影响报警器。此时表面看（相关性的角度） a 会影响 b ，但其实只是一种间接的影响。当 c 固定时，

$$p(a, b|c) = \frac{p(a, b, c)}{p(c)} = \frac{p(a)p(c|a)p(b|c)}{p(c)} = \frac{[p(a)p(c|a)]p(b|c)}{p(c)} = \frac{p(a, c)p(b|c)}{p(c)} = p(a|c)p(b|c) \quad (7.21)$$

即 a 与 b 在 c 固定的条件下是独立的， $a \perp b | c$ 。某种意义上讲，固定 c ，就切断了 a 对 b 的影响。以火灾与报警器为例，考虑不确定性（随机性），假设一些火灾不产生烟雾，一些烟雾不由火灾产生（如可由吸烟导致），报警器也有误差（偶尔会误报）。那么，对有烟雾产生的所有事件进行分析，就会发现警报与火灾之间没有关联（即此时警报信号对我们预测火灾没有任何帮助）。

第三种情形是图 7.7 所示的“头对头”（head-to-head），即两根箭头的头部相连。这可以理解成 a 与 b 有共同的结果 c 。根据贝叶斯网络的性质，此时的联合概率为

$$p(a, b, c) = p(a)p(b)p(c|a, b) \quad (7.22)$$

如果 c 不固定，有

$$p(a, b) = p(a)p(b) \sum_c p(c|a, b) = p(a)p(b) \quad (7.23)$$

a 和 b 是独立的，即 $a \perp b | \emptyset$ 。但如果 c 固定，则有

$$p(a, b|c) = \frac{p(a, b, c)}{p(c)} = \frac{p(a)p(b)p(c|a, b)}{\sum_{a, b} p(a)p(b)p(c|a, b)} \quad (7.24)$$

一般情况下并不等于 $p(a|c)p(b|c) = \frac{p(a)p(b)[\sum_a p(a)p(c|a, b)][\sum_b p(b)p(c|a, b)]}{[\sum_{a, b} p(a)p(b)p(c|a, b)]^2}$ 。即 a 和 b

在 c 固定时是不独立的， $a \not\perp b | c$ 。这是一个很奇怪（有趣）的性质，有点反直觉：

a 与 b 是 c 的原因，固定结果，为什么会导致两个原因互相影响？其实关键在于：这里的“影响”其实是相关，而非因果。生活里有不少这方面的例子。例如，明星经常被分成偶像派与实力派，即外貌出众或才能（演技、唱功等）出众。实力派通常被默认为外貌不够出众，即外貌与才能是负相关的。比如，黄渤很有才能，但确实比较丑。但是，外貌与才能其实没有什么关系，只是它们都是明星的条件（如 head-to-head 所描述的），如果只把明星挑出来统计（固定结果 c ），自然就排除掉外貌与才能都不出众的人，从而导致外貌与才能之间的负相关。另一个例子是：大学里有特招运动员，会给人留下“体育好的文化成绩差、文化成绩好的体育差”的感觉。但实际上可能是体育好坏与文化成绩好坏之间并没有关系，只是大学是按“文化成绩好或体育好”的标准来招生的，把两者都差的人排除出去，才导致了两者表面上的负相关。

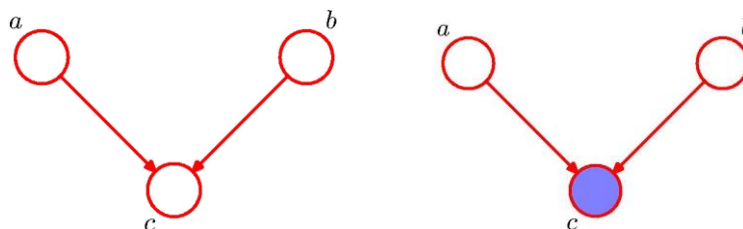


图 7.7. 三节点的 head-to-head 情形。(a) c 不固定时， a 和 b 是独立的， $a \perp b | \emptyset$ 。(b) c 固定时， a 和 b 是独立的， $a \not\perp b | c$ 。

三种情形总结一下，就是：知道（固定）共同原因、知道中间原因、或不知道共同结果，将是 a 与 b 相互独立。反之亦然。

在三种情形中， c 固不固定，对 a 与 b 之间的关系有重要影响。这其实是同一数据集的不同分组所引起的结论变化。统计学中的“辛普森悖论”与此密切相关（参见：“题外：辛普森悖论”）。

题外：辛普森悖论

当人们探究两个变量的相关性时，有时候会先分组研究，再将分组结果汇总起来。然而，在每个分组中都占优势的一方，在总评中有时反而处于劣势。这一“荒谬”现象在 20 世纪初就有人讨论，但一直到 1951 年，E. H. Simpson 在他发表的论文中阐述此一现象后，该现象才算正式被描述解释，后来被称为辛普森悖论。同一个数据集是如何证明两个完全相反的观点的？我们用一个虚拟例子来具体展示。

某校人文学院包含两个系，外语系与哲学系。长期以来，人文学院的学生男女不平衡，女多男少。

因此，今年院长私下吩咐各系，在研究生面试中，在类似条件下多照顾男生。

结果，面试结果出来后：

（秘书，慌张跑进来）“院长大人，大事不好了，有很多男生在院楼门口抗议。他们说今年女生录取率 42% 是男生 21% 的两倍，我们遴选学生有性别歧视！”

（院长，满脸疑惑）“我不是特别交代，今年要尽量提升男生录取率吗？”

（秘书，赶紧回答）“确实有交代下去。我刚刚也查过，今年外语系录取率是男生 75%，女生只有 49%；而哲学系录取率是男生 10%，女生为 5%。都是男生录取率比较高。这是我作的调查报告。”

（院长一理科素质很高的院长，一脸释然）“哦，你知道为什么个别录取率男皆大于女，但是总体录取率男却远小于女吗？这就是辛普森悖论啊！”

学生人数与录取比例

	男生	女生
外语系	$\frac{15}{20} = 75\%$	$\frac{49}{100} = 49\%$
哲学系	$\frac{10}{100} = 10\%$	$\frac{1}{20} = 5\%$
总计	$\frac{15 + 10}{20 + 100} = 21\%$	$\frac{49 + 1}{100 + 20} = 42\%$

1.3 学习与预测*

前面介绍了贝叶斯网络的基本组成与性质。那么，怎样进行训练与预测呢？

首先，需要确定图结构。这可以根据我们对世界的知识进行创建，即需要领域知识。例如，因素 a 对因素 b 是否有直接影响（相互作用）。也可以结合训练数据寻找可能的图结构。比如，从图结构所得到的独立性或条件独立性是否与训练集数据一致。在这方面，可以把前一小节所介绍的三节点性质推广到多节点，得到所谓的 D-分离（D-Separation）的概念。具体地，记 A, B, C 是没有重合的节点集合，对从 A 到 B 的一条路径，如果它包含了至少一个满足如下任一条件的节点，则称路径是被阻断（blocked）的：

- （1）路径在节点上的箭头是 head-to-tail 或者 tail-to-tail，而且节点属于 C ；
- （2）路径在节点上的箭头是 head-to-head，且节点及其后代都不属于 C 。

如果从 A 到 B 的每一条路径都被阻断，则称 A 与 B 是被 C 所 D-分离。此时，其上的所有变量满足

$$A \perp\!\!\!\perp B \mid C \quad (7.25)$$

图 7.8 给出了两个例子。在图 7.8（a）中，从 a 到 b 的路径 $a - e - f - b$ 上的节点都不属于 c ，不满足条件（1），其上只有节点 e 是 head-to-tail，但其后代包含了 c ，也不满足条件（2），因此 $a \not\perp\!\!\!\perp b \mid c$ 。而图 7.8（b）的分析则给出 $a \perp\!\!\!\perp b \mid f$ 。

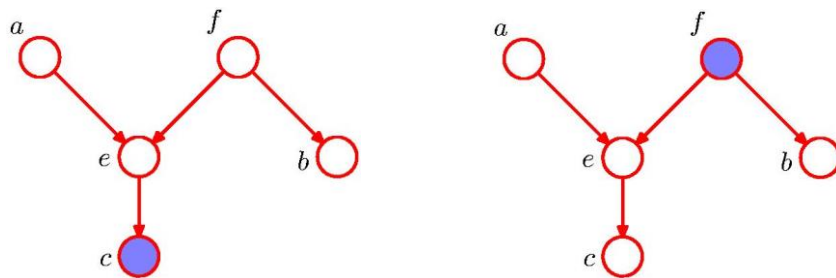


图 7.8. 利用 D-分离判断任节点之间的独立性。(a) $a \not\perp\!\!\!\perp b \mid c$ 。(b) $a \perp\!\!\!\perp b \mid f$ 。

基于 D-分离，就可以利用图论算法快速地判断出两个节点或节点集合之间是否是条件独立的。数学上可以证明，如果一个联合分布满足一个概率图用 D-分离所得到的所有条件独立性，则这个联合分布可以用这个概率图来描述。判断一个分布是否由某个图描述，只需要检验其是否满足图的所有条件独立性。这就使图模型很有威力了，可以从观察数据中进行图结构的构建^{aaa}，并在一定程度上减小维度诅咒的影响。

^{aaa} 其实还是很难，因为可能的图太多了，是个 NP 问题。一般需要结合领域知识，以及采取一些近似解法。

其次，在有了图结构以后，就需要进一步确定联合概率公式(7.2)中的因子分布，即每个随机变量的局部条件概率分布 $p(x_k|\text{pa}_k)$ 。每个因子分布只涉及一个节点及其所有父节点，一般而言只占整个图所有节点的一部分，分布估计的难度小于直接对联合概率进行估计。这也可看出边缺失的重要性。分布估计可采用上一章介绍过的直方图方法或核密度估计法，也可以采用概率的函数化的思路，以减少参数数目，然后使用梯度下降法来寻找能够最大化被观察数据的可能性的参数值（参见下一章中高斯混合模型的求解）。这个训练步骤也被称为参数估计。

最后，在完成训练以后，就可以进行预测了，即在知道一些节点的值（信息）的情况下，推断其它一些节点的值。这被称为推理（inference）。推理任务一般可分成几种：（1）边际推理（marginal inference）：即求解一个或一组特定变量的边缘（边际）概率分布，比如 $p(a)$ ，此时需要把其它变量积分掉。（2）后验推理（posterior inference）：给定某些显变量 v_E （ E 表示证据 evidence）的具体取值 e ，求某些隐藏变量 v_H 的后验分布（条件概率） $p(v_H|v_E = E)$ 。（3）最大后验（MAP）推理（maximum-a-posteriori inference）：给定某些显变量 v_E 的具体取值，求使其它变量 v_H 有最高概率的配置，即 $\max[p(v_H|v_E = E)]$ 。需要指出的是，推理在计算上很困难！在某些特定类型的图中我们可以相当高效地执行推理，但一般而言图的计算都很难，经常需要使用近似算法来在准确度和效率之间进行权衡。具体的推理方法包括变量消除（variable elimination）、信念传播或置信度传播（belief propagation）、近似推理（采样法、变分法等等），不再赘述。

1.4 概率推断例子

我们来看两个利用贝叶斯网络进行概率推断的例子。

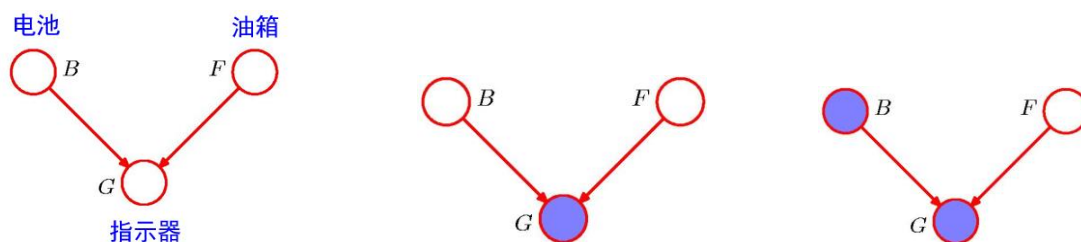


图 7.9. 电动燃油表的贝叶斯网络。(a) 仪表盘上的指示器 G 状态主要受电池电量 B 与油箱油量 F 的影响。(b) 观察到指示器状态。(b) 接着看看电池情况。

第一个是电动燃油表的例子。

燃油表是一种用于显示汽车、摩托车或其他内燃机车辆油箱中剩余油量的仪表。通过检测油箱内的油量传感器，燃油表可以通过安装在车辆仪表盘上的指示器来反映油箱中的油量情况。我们假设指示器 G 的结果主要受两个因素的影响：电池电量 B 与油箱油量 F （图 7.9）。变量的取值按 0（代表电量/油量不足）与 1

（电量/油量充足）处理。根据贝叶斯网络的性质，联合概率可写成 $p(B, F, G) = p(B)p(F)p(G|B, F)$ 。假设电动燃油表指示结果具有一定的随机性，由下面的条件概率描述：

$$\begin{cases} p(G = 1|B = 1, F = 1) = 0.8 \\ p(G = 1|B = 1, F = 0) = 0.2 \\ p(G = 1|B = 0, F = 1) = 0.2 \\ p(G = 1|B = 0, F = 0) = 0.1 \end{cases} \quad (7.26)$$

$p(G = 0|B, F)$ 的值可以由归一化条件 $p(G = 0|B, F) + p(G = 1|B, F) = 1$ 计算出来。根据这些数据，可知这个燃油表的可靠性不是很好，例如，在油箱有油（ $F = 1$ ）、电池电量（ $B = 1$ ）也充足的情况下，指示器只有 80% 几率正确显示“有油”（ $G = 1$ ），20% 几率错误显示为“缺油”（ $G = 0$ ）。另外，我们还需要 $p(B)$ 与 $p(F)$ 的信息，假设某车主的这辆车的油箱与电池的先验概率如下

$$\begin{cases} p(B = 1) = 0.9 \\ p(F = 1) = 0.9 \end{cases} \quad (7.27)$$

即一般情况下电池有 10% 的概率电量不足，油箱有 10% 的概率油量不足。好，现在是某天早上，车主走到车旁，还没拉开车门，这时候我们对此时这辆车的油量情况有什么认识呢？此时什么都还没看到，没有任何观测结果，或者说所有节点的值都没有固定（图 7.9（a）），因此我们判断的结果是 $p(F = 0) = 0.1$ ，即油量不足的概率是 10%。然后，车主打开车门坐到驾驶座上，瞥了一眼燃油表指示器，发现其指示“缺油”（ $G = 0$ ）。现在油箱油量不足的概率是多少？此时节点 G 被固定（图 7.9（b）），因此结果为：

$$p(F = 0|G = 0) = \frac{p(G=0|F=0)p(F=0)}{p(G=0)} \approx 0.257 \quad (7.28)$$

其中

$$p(G = 0|F = 0) = \sum_B p(B)p(G = 0|B, F = 0) = 0.81 \quad (7.29)$$

$$p(G = 0) = \sum_{B,F} p(B)p(F)p(G = 0|B, F) = 0.315 \quad (7.30)$$

这里公式（7.28）利用了贝叶斯公式。因此，观察到指示器指示“缺油”以后，油箱油量不足的概率从 10% 上升到 25.7%。为了减小不确定性，车主检查了电池（电池在驾驶室，因此不用下车），发现电池电量充足（ $B = 1$ ，被固定，见图 7.9（c））。此时，油箱油量不足的概率变成

$$p(F = 0|G = 0, B = 1) = \frac{p(G=0|B=1,F=0)p(F=0)}{\sum_F p(G=0|B=1,F)p(F)} = \frac{0.8 \times 0.1}{0.8 \times 0.1 + 0.2 \times 0.9} \approx 0.307 \quad (7.31)$$

概率进一步升高。不过，此时仍没有很大把握认为油箱真的油量不足，车主可能需要下车去检查油箱，进一步确认。如果检查电池的结果是电量不足（ $B = 0$ ），

则油箱油量不足的概率为

$$p(F=0|G=0, B=0) = \frac{p(G=0|B=0, F=0)p(F=0)}{\sum_F p(G=0|B=0, F)p(F)} = \frac{0.9 \times 0.1}{0.9 \times 0.1 + 0.8 \times 0.9} \approx 0.111 \quad (7.32)$$

概率变小了。此时可以有比较大的把握认为油箱其实有油，只是因为电池电量不足导致燃油表误报，车主可以直接开车上班了。

在上面这个例子中，不但可以根据观察到的结果（ G ）对原因（ F ）进行推断（利用贝叶斯公式），而且与推断目标（ F ）没有直接因果关系的变量（ B ）也对推断有帮助。这就是贝叶斯网络强大的地方。另外，还可以看到，随着观测结果的变化，推断的概率结果也在变化，这就是我们以前强调过的“证据的出现改变信念”。

第二个例子是肺病医院的贝叶斯网络辅助诊断系统^{bbb}。

有个专攻肺部疾病的医院，根据肺癌、肺结核和支气管炎的发生比例以及这些疾病典型的临床症状、病因、检查病理等信息，利用贝叶斯网络构建了辅助诊断系统。数据方面，有些是关于社会整体情况的信息：

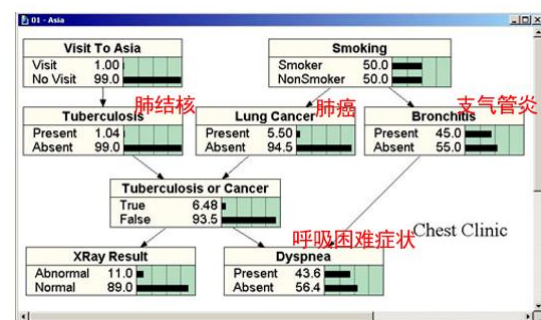
- 有 30% 的人吸烟。
- 每 10 万人中有 70 人患有肺癌。
- 每 10 万人中有 10 人患有肺结核。
- 每 10 万人中有 800 人患有支气管炎。
- 有 10% 人存在呼吸困难症状（主观上有空气不足或呼吸费力的感觉，客观上表现为呼吸频率、深度和节律的改变），可由哮喘、支气管炎和其他非肺结核、非肺癌性疾病所引起。

不过，这些数据其实作用有限，因为它们反映的是社会上所有人的情况，而非来医院看病的人（病人）的情况。更有用的是医院所积累的病人数据，其中一些统计结果如下：

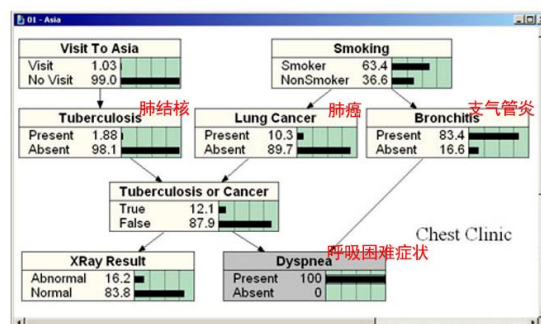
- 50% 的病人吸烟。
- 1% 患有肺结核。
- 5.5% 得了肺癌。
- 45% 患有不同程度支气管炎。

根据具体的数据，结合领域知识，对贝叶斯网络进行建模与训练，就得到实际可用的辅助诊断系统。图结构及相关变量的边缘分布如下：

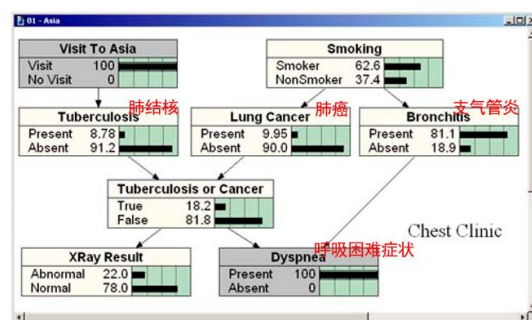
^{bbb} 参考资料：https://www.norsys.com/tutorials/netica/secA/tut_A1.htm, Introduction to Bayes Nets;
<https://www.zhihu.com/question/28006799>, 怎么通俗易懂地解释贝叶斯网络和它的应用？



这也是当一个病人走进诊室，还没开始问诊时，医生对病人情况的了解^{ccc}。接下来开始问诊，病人自述有呼吸困难症状。把这个信息输入到贝叶斯网络中，“呼吸困难症状”节点被固定为“1”（“是”）。如我们前面分析过的，节点固定与否对其它节点的性质是有很大影响的。每当有新的信息输入，网络中的概率就会自动进行调整，这就是贝叶斯推理最完美、强大之处。只要我们了解了患者的足够信息，根据这些信息获得统计知识，贝叶斯网络就会告诉我们合理的推断。此时，网络状态变成：

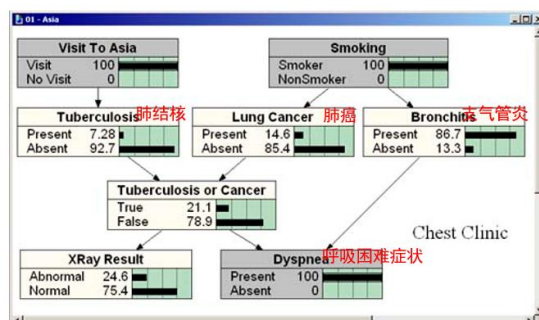


可以观察到，一旦病人有呼吸困难症状，三种疾病的概率都增大了，因为这些疾病都有呼吸困难的症状。我们可以从结果反推原因。具体地，概率增加最大的是支气管炎，从45%上升到83.4%。为什么会有如此大的增长呢？因为支气管炎比癌症和肺结核更常见（先验概率）。另外，还会影响其它与“呼吸困难症状”节点没有直接影响的其它因素，如病人是抽烟者的几率也会随之增大，从50%增加63.4%；X光照片不正常的几率也会上涨，从11%到16%。接下来，按照流程依此询问病人更多问题，如她最近是不是去过亚洲国家，吃惊的是她回答了“是”。这个信息影响了网络结果：

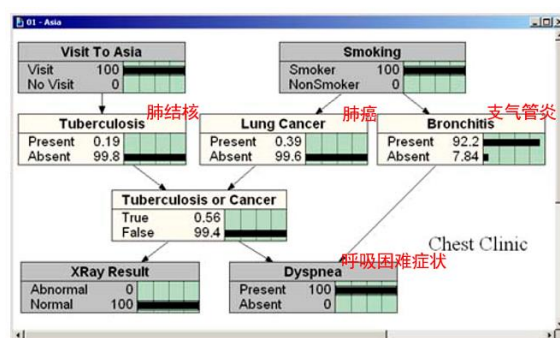


^{ccc} 什么，还没开始问就有这些认识？是的，这就是某种先验知识（先见、偏见），是基于以往病人情况积累的认识。校医院的医生第一句话往往是“是不是感冒了”，也是类似的道理。如果校医院混进去一个憨豆冒牌医生，不管什么病人都一律开维C银翘片，估计也有不低的准确性。

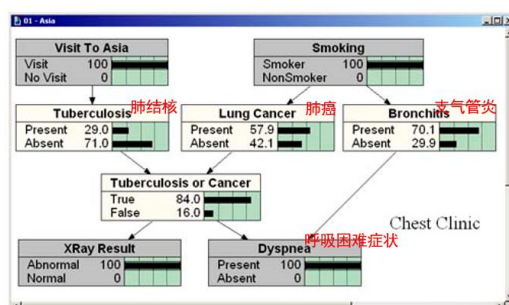
现在，病人患肺结核的几率显然增大，从 2% 变成 9%。而患有癌症、支气管炎以及该患者是吸烟者的几率都有所减少。为什么呢？它们与肺结核具有共同的结果“呼吸困难症状”。根据 head-to-head 的性质，固定共同的果，各个因之间是负相关的。继续问患者问题，知道患者是个吸烟者，则网络变为：



此时患者最大的可能仍然是患有支气管炎。该问的都问了，接下来开单子做检查。做了一个 X 光透视，结果显示正常。网络变成：



肺结核与肺癌的概率一下子变得很小（因为大部分肺结核与肺癌的 X 光检查都会显示为不正常），而支气管炎的概率进一步增加到 92.2%（X 光检查其实与支气管炎没有直接关系，但可通过 head-to-head 机制互相影响）。在这种情况下，可以先按支气管炎治疗了。但如果 X 光显示不正常的话，则结果将有很大不同：



最大的影响是肺结核与肺癌的概率增加很多。支气管炎仍然是三个疾病中最可能的一个，但它小于“结核或肺癌”这一组合的假设。此时，不能简单诊断为某个具体疾病，需要进行更多的检查，例如血液测试、CT、肺组织活检，等等。当前的贝叶斯网络没有包括这些测试，但它很容易扩展，只需添加额外的节点并训练

更新相关的因子分布。我们不需要扔掉以前的任何部分。这是贝叶斯网络的另一个强大的地方。它们很容易扩展（或删减、简化），以适应不断变化的需求和变化的知识。

贝叶斯网络还可以应用到其它领域的自动诊断/分析系统，例如，用于分析电力系统中的潜在故障（扩展阅读：“9800 万美元卖掉公司后，他用贝叶斯网络分析数据中的因与果”）。

另外需要指出的是，相关不是因果。严格地讲，贝叶斯网络描述的是相关，而非因果。为了从纷纭复杂的数据中寻找这些因果问题的答案，需要对贝叶斯网络进行重大的理论改造与升级。参见扩展阅读“贝叶斯网络发明人珀尔的《为什么》”“2021 年诺贝尔经济学奖——因果推断”。

扩展阅读：9800 万美元卖掉公司后，他用贝叶斯网络分析数据中的因与果

来源：<https://baike.baidu.com/tashuo/browse/content?id=02c9b57396229adc831a9955>

- 对事件做出预测相对容易，但分析因果关系则是一件很困难的事。就像路上很多人带着雨伞，代表可能会下雨，但雨伞不是下雨的原因。
- 新公司希望能通过因果分析技术，帮助电力、能源等公司找到电力系统中的风险所在。
- 公司的技术核心是因果分析平台 Focus64，它应用了 Judea Pearl 提出的贝叶斯网络，用于理解数据之间的关系。
- 简单来说，贝叶斯网络会引入更多变量来创建一个图（Graph），从而看这些变量之间的相互关系以及如何改变。比如展示雨、雨伞与天色的关系，会看到灰色的天空与下雨有关系，也会看到下雨时没有雨伞这个变量出现，但仍会出现灰色的天空。然后比较两种场景就会发现，灰沉的天空与下雨的关系更强，也就是它出现的原因。
- 贝叶斯网络的优势是用到了图论，相比于神经网络，其运作过程有透明性，不是黑盒子。
- 贝叶斯网络两个特点：一是能让人类了解到底发生了什么；二是由于知道发生了什么，就可以做出改变。而深度学习网络可以重新训练，但不能改变网络模型。
- 可以在虚拟中模拟可能发生的情况，模拟某个变量可能产生什么样的结果。比如，如果稍微改变电路，某片楼宇中的电力状况会发生什么变化？在现实中不能真的这么做，而在计算机中可以做这种模拟。

扩展阅读：贝叶斯网络发明人珀尔的《为什么》

（来源：<https://www.huxiu.com/article/323544.html>，克服寻找因果关系的思维定式：图灵奖得主珀尔的《为什么》）

- 大数据的许诺，让科学界偷懒地用相关性代替为了因果关联。但最终我们还是要问因果相关的问题。由图灵奖得主、计算机科学家朱迪亚·珀尔和科学作家达纳·麦肯齐合著的《为什么》一书，为这个古老的问题提供一个新的答案。
- 相关不是因果！
- 我们发现吸烟导致肺癌的唯一原因是，我们在特殊情况下观察到了两者之间的相关性。因此，就产生了一个难题：如果因果关系不能归结为相关性，那么相关性又如何能作为因果关系的证据呢？
- 但即使今天最先进的机器学习技术也不能解释数据，告诉我们为什么数据能预测。对于珀尔来说，我们现在缺少一个“现实模型”，而这主要取决于“因”。他与数据狂热分子争辩说，现代的计算机跟我们的大脑完全不一样。
- 如果世界在某种程度上不是原来的样子，反事实推理就会断言会发生什么。

扩展阅读：2021 年诺贝尔经济学奖——因果推断

（来源：<https://baijiahao.baidu.com/s?id=1713380314221700759>，因果推断：利剑和 2021 诺贝尔经济学奖三剑客的故事）

- 2021 年诺贝尔经济学奖颁给 David Card、Joshua D. Angrist 与 Guido W. Imbens，他们的工作与因果关系分析有关。
- 作为万物的灵长，人类天性当中就包含了对因果关系的好奇。当看到一桩新事物的时候，人们总是会不禁地问：“这东西为什么会这样？它背后的原因到底是什么？”比如，在战国时期著名诗人屈原的长诗《天问》中，就围绕天地万物运行的因果一口气提出了 170 多个问题。
- 工资提高会让工人失业吗？
- 教育会带来更高的回报率吗？
- 我们怎样从纷纭复杂的数据中寻找这些因果问题的答案？贝叶斯网络中只有相关，没有严格的因果。因此，我们需要对贝叶斯网络进行改造与升级。

最后小结一下，贝叶斯网络为什么有用？因为贝叶斯网络只需要分别考虑每个节点与其父母节点的联合分布，而不需所有变量的联合分布。还因为它很灵活，便于扩展，容易加入新知识。最后，对于贝叶斯网络而言，不精确的、不完整的（概率）信息也能有用（Probabilities need not be exact to be useful!）。

第2节 朴素贝叶斯

因为对险恶世界的无知从而采用了一些天真的假设。

这也算大道至简，大智若愚了。

2.1 朴素贝叶斯分类器：含义、参数估计与推断

网上一篇文章^{ddd}对朴素贝叶斯分类器（naive Bayes classifier）的诗意介绍：

“每次看到朴素这个词，我就仿佛看到贝叶斯穿着一身打满补丁衣服的样子。naive 意思是缺乏经验的、幼稚的、无知的、轻信的。从公式推导过程来看，朴素贝叶斯分类器采用了一些简化条件的假设。不过，朴素还有一层含义是专一、纯粹，在这个意义上，贝叶斯分类也算大道至简，大智若愚了。”

其实，贝叶斯分类器就是一种用于分类任务的特殊的贝叶斯网络。它假设在类别 $\mathcal{C}$ 固定时，输入 $\mathbf{x}$ 的各个分量（特征）的分布之间是独立的，可由图 7.10（a）所示的贝叶斯网络所描述。“朴素”指的就是这种算法假定各个分量的分布独立，真实的数据可能不满足这个简化假设。尽管如此，这个方法对很多复杂的问题还是很管用的。

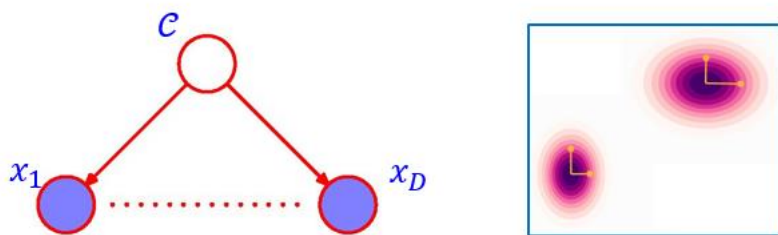


图 7.10. 朴素贝叶斯分类器示意图。(a) 所对应的贝叶斯网络。(b) 在类别 $\mathcal{C}$ 不固定的条件下，输入 $\mathbf{x}$ 的分量的分布并不是独立的。

在朴素贝叶斯模型中，类别 $\mathcal{C}$ 是父节点，输入 $\mathbf{x}$ 的各分量（特征） $x_1, x_2, \dots, x_D$ 是子节点，子节点之间没有连线，参见图 7.10（a）。联合概率分布可写成：

$$p(\mathbf{x}, \mathcal{C}) = p(\mathcal{C}) \prod_{i=1}^D p_i(x_i | \mathcal{C}) \quad (7.33)$$

这里 p_i 特意写出下标，以强调不同分量 x_i 的分布一般是不同的。上式右侧由一些

^{ddd} <https://segmentfault.com/a/1190000016563386>，朴素贝叶斯分类——大道至简

一元分布 ($p(\mathcal{C}), p_i(x_i|\mathcal{C})$) 组成, 每一个分布都比较容易训练 (根据训练数据进行分布估计), 因此很容易处理高维数据, 避免了维度灾难的问题。网络中的箭头之间属于 tail-to-tail 结构, 在 $\mathcal{C}$ 固定时, 特征 x_i 之间是独立的:

$$p(\mathbf{x}|\mathcal{C}) = \frac{p(\mathbf{x}, \mathcal{C})}{p(\mathcal{C})} = \prod_{i=1}^D p_i(x_i|\mathcal{C}) \quad (7.34)$$

但当 $\mathcal{C}$ 不固定 (未知) 时, 特征之间是不独立的,

$$p(\mathbf{x}) = \sum_{\mathcal{C}} \prod_{i=1}^D p_i(x_i|\mathcal{C}) p(\mathcal{C}) \neq \prod_{i=1}^D p_i(x_i) = \prod_{i=1}^D \sum_{\mathcal{C}} p_i(x_i|\mathcal{C}) p(\mathcal{C}) \quad (7.35)$$

一个直观的例子如图 7.10(b) 所示: $p(x_i|\mathcal{C})$ 是简单的高斯分布, $\mathcal{C} = 0$ 时 $p(x_1|\mathcal{C} = 0)$ 与 $p(x_2|\mathcal{C} = 0)$ 是独立的, $\mathcal{C} = 1$ 时 $p(x_1|\mathcal{C} = 1)$ 与 $p(x_2|\mathcal{C} = 1)$ 也是独立的; 如果不固定 $\mathcal{C}$, 则两个类别的 (x_1, x_2) 数据放在一起, 就不是独立的了。

朴素贝叶斯的推断可利用贝叶斯公式, 有

$$p(\mathcal{C}|\mathbf{x}) = \frac{p(\mathbf{x}|\mathcal{C})p(\mathcal{C})}{p(\mathbf{x})} = \frac{p(\mathcal{C}) \prod_{i=1}^D p_i(x_i|\mathcal{C})}{p(\mathbf{x})} \quad (7.36)$$

即可根据输入 $\mathbf{x}$, 预测输出 $\mathcal{C}$ 。

至于朴素贝叶斯的训练 (参数估计), 类别先验概率可用频率估算:

$$p(\mathcal{C}_k) = \frac{N_k}{N} \quad (7.37)$$

其中 N_k 是训练集中第 k 类的数据个数, N 是所有类的数据总个数。如果特征 x_i 的取值是离散的 (假设有 s_i 个可能的分立值, 记为 x_{is}), 则其条件概率可用频率估算,

$$p_i(x_{is}|\mathcal{C}_k) = \frac{N_{kis}}{N_k} \quad (7.38)$$

其中 N_{kis} 是第 k 类中 x_i 取值为 x_{is} 的数据个数。或者, 为了防止 $N_{kis} = 0$ 所导致的 $p(\mathbf{x}|\mathcal{C}_k) = 0$, 采用拉普拉斯平滑 (参考第一章中拉普拉斯对彩票问题的解答):

$$p_i(x_{is}|\mathcal{C}_k) = \frac{N_{kis} + 1}{N_k + s_i} \quad (7.39)$$

如果特征 x_i 的取值是连续的, 可采用上一章所介绍的核函数估计法来估计 $p_i(x_i|\mathcal{C}_k)$, 也可以采用概率函数化的思路, 假设 $p_i(x_i|\mathcal{C}_k)$ 具有某种简单的形式, 例如高斯分布, 然后基于训练数据估计其参数, 如高斯分布的均值与方差。

朴素贝叶斯的主要优点有: (1) 发源于古典数学理论, 算法简单, 有稳定的分类效率。(2) 对小规模的数据表现很好, 能处理多分类任务, 也适合增量式训练, 尤其是数据量超出内存时, 可以分批次进行训练。(3) 对缺失数据不太敏感

(这也是所有贝叶斯网络模型都具有的优点)。

朴素贝叶斯也有缺点, 主要包括: (1) “朴素” 的假设如果与实际情况不符, 比如在属性个数比较多或者属性之间相关性较大时, 会影响模型效果。(2) 严格讲需要知道先验概率 (参数的先验, 参见第二章), 且先验概率很多时候取决于假设, 假设的模型可以有很多种, 因此可能会因为先验模型的原因导致预测效果不佳。

2.2 小例子: 出来打网球?

假设某网球集训基地, 著名球星老 A 最近一段时间在做训练。能记录到的天气与训练情况如表 7-1 所示。今天的天气是(sunny, cool, high, strong), 请问老 A 会出来打网球吗?

表 7-1 天气与打球情况的数据记录

	天气	温度	湿度	风速	
Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

这个问题中输入 $\mathbf{x}$ 具有四个分量: 天气、温度、湿度、风速, 可能取值数目分别为3,3,2,2, 因此特征取值的总组合数为 $3 \times 3 \times 2 \times 2 = 36$, 但表中数据只有14组, 如果不做任何近似, 直接统计 $p(\mathbf{x})$, 会面临数据不足的困难。因此, 采用朴素贝叶斯模型, 对每个分量 i 单独统计 $p_i(x_i|\mathcal{C})$, 可能组合数大大减少。采用拉普拉斯平滑, 最后得到

$$\begin{aligned}
 & p(\text{打球}|\text{sunny, cool, high, strong}): p(\text{不打球}|\text{sunny, cool, high, strong}) \\
 &= \frac{p(\text{打球})p(\text{sunny}|\text{打球})p(\text{cool}|\text{打球})p(\text{high}|\text{打球})p(\text{strong}|\text{打球})}{p(\text{不打球})p(\text{sunny}|\text{不打球})p(\text{cool}|\text{不打球})p(\text{high}|\text{不打球})p(\text{strong}|\text{不打球})}
 \end{aligned}$$

$$\begin{aligned}
&= \frac{9}{14} \left(\frac{2+1}{9+3} \right) \left(\frac{3+1}{9+3} \right) \left(\frac{3+1}{9+2} \right) \left(\frac{3+1}{9+2} \right) : \frac{5}{14} \left(\frac{3+1}{5+3} \right) \left(\frac{1+1}{5+3} \right) \left(\frac{4+1}{5+2} \right) \left(\frac{3+1}{5+2} \right) \\
&= 0.007084 : 0.01822 \approx 28\% : 72\%
\end{aligned} \tag{7.40}$$

因此预测老 A 今天出来打球的概率是 28%。

第 3 节 马尔科夫随机场与玻尔兹曼机

3.1 马尔科夫随机场

马尔科夫随机场(Markov random field), 又称马尔科夫网络(Markov network), 也是一类概率图模型。不过, 它属于无向图模型 (undirected graphical model), 连接结点的边是没有方向的, 如图 7.11 所示。

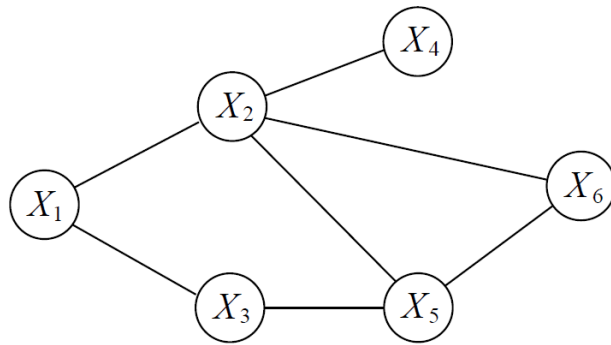


图 7.11. 马尔科夫随机场示意图。

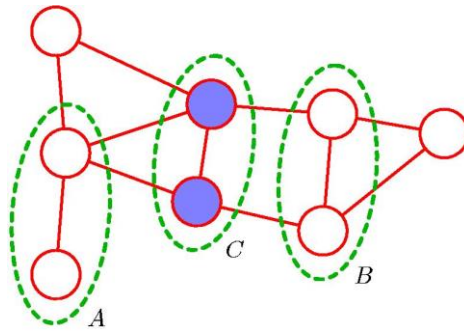


图 7.12. 马尔科夫随机场中节点连通性与条件独立性之间的对应。节点集团 A 与 B 被 C 所分离, 则有条件独立性 $A \perp\!\!\!\perp B \mid C$ 。

与贝叶斯网络类似, 马尔科夫随机场的图结构也表征了随机变量之间的条件独立性。具体地, 记 A, B, C 是没有重合的节点集合, 如果从 A 到 B 的任一条路径都会通过 C , 则称 A 与 B 是被 C 所分离, 我们要求其上的所有变量满足如下的条件独立性:

$$A \perp\!\!\!\perp B \mid C \tag{7.41}$$

换句话说，就是如果把 C 从图里删去，则 A 与 B 之间就不再连通了，如图 7.12 所示。因此，任一节点在其相连节点给定的情况下与其它节点是独立的，而任两个没有相连的节点在其它所有节点给定的情况下也是独立的。

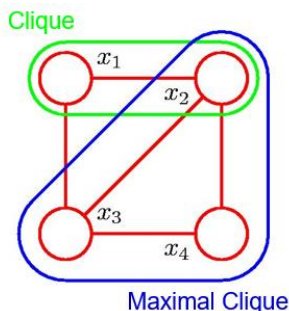


图 7.13. 马尔科夫随机场中的团与极大团。

在马尔科夫随机场中，全连接的节点集合叫团（clique）。没法在增加新节点时仍能成团的团，被称为极大团（maximal clique）。例如，图 7.13 中 $\{x_2, x_3, x_4\}$ 就是一个极大团，因为如果增加节点 x_1 以后就不能成团（ x_1 与 x_4 之间没有边连接）。而 $\{x_1, x_2\}$ 虽然是一个团，但不是一个极大团，因为它增加节点 x_3 以后仍然是一个团。马尔科夫随机场的联合概率分布可以写成所有极大团的非负函数的乘积：

$$p(\mathbf{x}) = \frac{1}{Z} \prod_c \psi_c(\mathbf{x}_c) \quad (7.42)$$

其中 Z 是归一化常数。例如，对于图 7.13，有 $p(\mathbf{x}) = \frac{1}{Z} \psi_{c1}(x_1, x_2, x_3) \psi_{c2}(x_2, x_3, x_4)$ 。如果两个节点之间没有连线，那么它们在其它所有节点固定的情况下是独立的，因此它们不会同时出现在因式分解（7.42）中的任一项里。数学上可以证明，公式（7.42）的因式分解形式与（7.41）所描述的条件独立性完全是等价的。这就给马尔科夫随机场的分析带来很大的方便。

公式（7.42）中 $\psi_c(\mathbf{x}_c)$ 是非负的，使用起来不是很方便，因此常常利用指数函数写成如下形式：

$$\psi_c(\mathbf{x}_c) = \exp[-E(\mathbf{x}_c)] \quad (7.43)$$

$E(\mathbf{x}_c)$ 是实数。上式类似于玻尔兹曼分布，可以把 $E(\mathbf{x}_c)$ 理解成 $\mathbf{x}_c$ 的相互作用能量。此时 Z 对应于统计热力学中的配分函数。注意 $\psi_c(\mathbf{x}_c)$ 不能直接解释成概率 $p(\mathbf{x}_c)$ ，因为边缘化时（ $p(\mathbf{x}_c) = \sum_{\mathbf{x}_{\setminus c}} p(\mathbf{x})$ ，其中 $\mathbf{x}_{\setminus c}$ 表示对 $\mathbf{x}_c$ 以外的变量求和或积分）需对其它极大团积分，积分结果与 $\mathbf{x}_c$ 有关（不同极大团之间有共同节点）。例如，对图 7.13，有

$$p(x_1, x_2, x_3) = \frac{1}{Z} \psi_{c1}(x_1, x_2, x_3) \int \psi_{c2}(x_2, x_3, x_4) dx_4 \quad (7.44)$$

右边积分的结果依赖于 x_2, x_3 的值，因此并不能满足 $p(x_1, x_2, x_3) = \psi_{C1}(x_1, x_2, x_3)$ 或 $p(x_1, x_2, x_3) \propto \psi_{C1}(x_1, x_2, x_3)$ 。

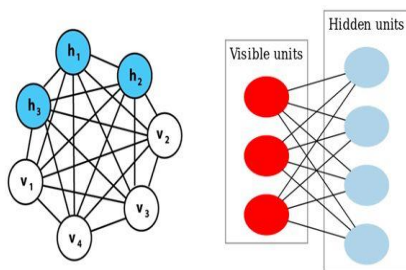
3.2 玻尔兹曼机与受限玻尔兹曼机

玻尔兹曼机 (Boltzmann machine) 最初由 Geoffrey Hinton、Terrence Sejnowski 和 David Ackley 在 1985 年提出，也是 Hinton 获得 2024 年诺贝尔物理学奖的重要原因之一。

玻尔兹曼机可以看做是一类特殊的马尔科夫随机场，它假设 x_i 是二值变量（允许取值为+1, -1，或0,1），而且能量函数可以写成如下简单形式：

$$E(\mathbf{x}) = -\sum_i b_i x_i - \sum_{i,j} w_{ij} x_i x_j \quad (7.45)$$

可近似理解为把 $E(\mathbf{x})$ 做泰勒展开并保留至二阶项（谐振子近似），或者，只考虑二体相互作用。玻尔兹曼机可以用来解决两类问题。一类是搜索问题，当给定参数 $\{b_i, w_{ij}\}$ 时，通过模拟退火等方法找到一组二值向量 $\mathbf{x}$ ，使得整个网络的能量最低（概率最大化）。有时候也通过采样的方法（如吉布斯采样）来产生服从 $p(\mathbf{x})$ 分布的 $\mathbf{x}$ 样本。另一类是学习问题（训练、参数推断），当给定一组部分变量的观测值（训练集）时，计算最优的参数。虽然（7.45）只保留至二阶项，但玻尔兹曼机通常既包含可被观察到的节点（如训练集对应的 $\mathbf{x}$ 与 $\mathbf{t}$ ），又包含不可见节点（隐藏节点），所以能够表达更加复杂的模式。



玻尔兹曼机(左)与受限玻尔兹曼机(右)的区别

图 7.14. 玻尔兹曼机与受限玻尔兹曼机。

全连接的玻尔兹曼机仍然比较复杂，学习效率并不高。在实际应用中，经常使用一种带限制的版本，即受限玻尔兹曼机 (restricted Boltzmann machine)。它是二分图 (bipartite graph)，由一个可见层（包含可见节点 $\{v_i\}$ ）和一个隐藏层（包含隐藏节点 $\{h_j\}$ ）组成，这两个层之间完全连接，但同一层内的节点之间没有任何连接（参见图 7.14）。也就是说，不存在层内通信，这就是名字中的“限制”二字的由来。这种结构限制使得受限玻尔兹曼机的训练算法比普通玻尔兹曼机更高效（参见“扩展阅读：受限玻尔兹曼机的训练”）。

受限玻尔兹曼机可以用于降维、分类、回归、协同过滤、特征学习以及主题建模的算法。例如在推荐系统中，它可以用来预测用户对物品的评分，通过编码用户的历史评分到隐藏层，然后解码预测用户对未评分物品的评分。

扩展阅读：受限玻尔兹曼机的训练

系统的能量函数为

$$E(\mathbf{v}, \mathbf{h}) = -\sum_i a_i v_i - \sum_i b_i h_i - \sum_{i,j} w_{ij} v_i h_j$$

如果要算 $p(\mathbf{h})$:

$$p(\mathbf{h}) = \frac{\sum_{\mathbf{v}} \exp(-E(\mathbf{v}, \mathbf{h}))}{\sum_{\mathbf{v}, \mathbf{h}} \exp(-E(\mathbf{v}, \mathbf{h}))} = \frac{\exp[\sum_i b_i h_i] \prod_i [\exp(a_i + \sum_j w_{ij} h_j) + \exp(-b_i - \sum_j w_{ij} h_j)]}{\sum_{i,j} \exp[\sum_i (a_i + \sum_j w_{ij} h_j) v_i + \sum_i b_i h_i]}$$

分子容易求，但分母（配分函数 Z ）不容易求。从另一角度看，有环体系很难求。

进一步地， $p(h_j)$ 更加难求，分子都不好求。

但是， $p(\mathbf{h}|\mathbf{v})$ 却很容易求。此时有些点被固定住，剩下的点无环：

$$p(\mathbf{h}|\mathbf{v}) = \frac{\exp(-E(\mathbf{v}, \mathbf{h}))}{\sum_{\mathbf{h}} \exp(-E(\mathbf{v}, \mathbf{h}))} = \prod_j \frac{\exp[(b_j + \sum_i w_{ij} v_i) h_j]}{\sum_j \exp[(b_j + \sum_i w_{ij} v_i) h_j]} = \prod_j p(h_j|\mathbf{v})$$

此时 h_j 之间是独立的，有

$$p(h_j|\mathbf{v}) = \frac{\exp[(b_j + \sum_i w_{ij} v_i) h_j]}{\sum_j \exp[(b_j + \sum_i w_{ij} v_i) h_j]}$$

如果二值变量取值0,1，则

$$p(h_j = 1|\mathbf{v}) = \frac{1}{1 + \exp[-(b_j + \sum_i w_{ij} v_i)]} = \sigma\left(b_j + \sum_i w_{ij} v_i\right)$$

反过来从 $\mathbf{h}$ 固定计算 $p(v_i = 1|\mathbf{h})$ 是类似的。进一步，如果采用平均场近似，将有

$$\langle h_j \rangle = p(h_j = 1|\mathbf{v}) = \sigma\left(b_j + \sum_i w_{ij} \langle v_i \rangle\right)$$

这就是受限玻尔兹曼机学习阶段所采用的公式。即以训练集数据作为可见层 v_i ，利用上式计算 $\langle h_j \rangle$ ，再利用类似式子计算回可见层 $\langle v_i \rangle$ ，此时与最开始的训练集输入 v_i 可能不同，两者的二乘误差就可以用来优化学习 b_j 与 w_{ij} 。

受限玻尔兹曼机还可以进一步采用多个隐藏层，变成深度玻尔兹曼机（deep Boltzmann machine）。

第 4 节 应用例子：快速发现酶的底物肽链

下面介绍一个通过结合朴素贝叶斯方法与和生物化学实验来快速优化酶的底物肽链的工作：

Lorillee Tallorin, JiaLei Wang, Woojoo E. Kim, Swagat Sahu, Nicolas M. Kosa, Pu Yang, Matthew Thompson, Michael K. Gilson, Peter I. Frazier, Michael D. Burkart & Nathan C. Gianneschi.

Discovering de novo peptide substrates for enzymes using machine learning.

Nat. Commun. **9**, 5253 (2018).

这个工作的目标是利用机器学习方法来加快酶的多肽底物的发现。采用的酶是一种翻译后修饰酶，4'-phosphopantetheinyl transferase (PPTase)（磷酸泛酰巯基乙胺基转移酶，简称磷酰转移酶）。磷酰转移酶 PPTase 能够在各种运输蛋白上的保守的丝氨酸残基上共价修饰上磷酸泛乙烯基团。人们在以前的工作中已经发现了 PPTase 的一个多肽底物——长度为 11 个残基的多肽 YbbR。YbbR 可以作为全长底物蛋白的替代品被 PPTase 所修饰，也可用作生化蛋白质标记和亲和纯化等应用的短肽标签。这个工作是试图寻找 PPTase 的更多选择性多肽底物，而且是希望找到正交底物，即能与两种类型的 PPTase（来自枯草芽孢杆菌的 Sfp 型以及来自于天蓝色链霉菌的 AcpS 型）中的一种发生反应而不能与另一种发生反应。

由于在所有的多肽（整个多肽空间）中能被酶所催化的多肽的比例是非常低的（这就是酶的特异性），因此，为了提高筛选实验的成功率，作者结合了机器学习的方法。先利用机器学习模型预测哪些多肽能被 PPTase 所催化，然后进行实验验证；再把实验验证的结果作为新数据加入到训练集中，提高机器学习模型的准确率；然后再进行下一轮的预测-实验验证。经过多轮迭代，就有望以比较高的成功率筛选得到 PPTase 的选择性正交底物。

初始的数据来自于天然蛋白底物的多肽片段，以及文献中已知的其它筛选得到的多肽底物。已知的一些没有活性的蛋白或多肽片段被用作阴性数据。所有截取后的多肽的长度不多于 38 个残基，Ser（丝氨酸）居于序列中央。如果考虑所有的 20 种氨基酸，则输入 $\mathbf{x}$ 的状态空间大小为 20^{38} ，非常庞大。为了减少过拟合的风险，作者将残基种类根据化学性质分成 8 大类，acidic (包括 D, E), amidic (N, Q), aromatic (F, W, Y), basic (H, K, R), nonpolar (A, I, L, M, V), aliphatic (G, P), hydroxylic (S, T), sulfur (C)。这样，状态空间就缩小为 8^{38} 。

采用的机器学习模型为朴素贝叶斯分类器。这一方面是因为朴素贝叶斯假设在固定类别下输入各分量的分布是独立的，因此只需要对每个分量进行统计即可，避免了维度灾难的发生；另一方面则是因为贝叶斯分类器提供了每个类别下的条件概率 $p(\mathbf{x}|\mathcal{C})$ 信息，可以直接用来采样，即直接预测阳性底物用于实验验证（这

在逻辑回归、神经网络等判别式模型中是做不到的)。贝叶斯分类器的参数先验概率采用狄利克雷分布(拉普拉斯平滑使用的则是均匀分布)。利用贝叶斯公式计算出后验概率以后对后验概率做平均,以进一步提高准确性。

实验上每轮实验能测试 600 种多肽。因此,每轮实验之前,机器学习模型需要提供 600 种候选多肽的建议。为了防止预测的多肽过于相像,在预测出一个最佳多肽后,把它作为负样本加入训练集重新训练并预测第二个多肽,以实现预测序列的多样性。

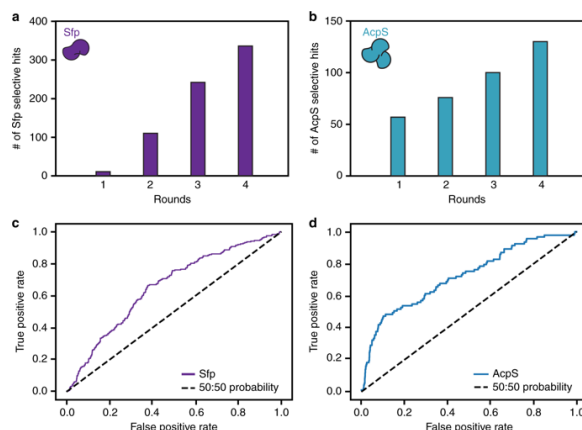


图 7.15. 磷酸转移酶 PPTase 的多肽底物筛选结果。(a,b) 两种类型 (Sfp 型与 AcpS 型) 的 PPTase 的四轮底物筛选实验的阳性多肽数量。每轮实验包含 600 种候选多肽。(c,d) ROC 曲线。

实验验证的结果如图 7.15 所示。共进行了四轮实验。在机器学习模型的帮助下,实验筛选到的阳性多肽底物的数量每一轮都在提高,对 Sfp 型 PPTase 的效果尤其明显,最后一轮的阳性多肽数量比最初一轮提高了几十倍(图 7.15(a))。

相比于传统的基于变异进化的搜索方案,这种机器学习模型与实验结合的方案具有更强的导向性,整体命中率达 30%,远高于变异进化方案的 3%和随机搜索的 0.001%,大幅提高了特异性多肽底物的发现速度。这种机器学习模型与实验相结合进行多轮迭代的思路具有很强的通用性,在其它的实验优化问题中具有广泛的应用潜力。

$$a \perp\!\!\!\perp b \mid c$$

$$a \not\perp\!\!\!\perp b \mid c$$

⌋ ⌋

扩展阅读:

- <https://juejin.im/post/5d29509fe51d455070227030>
贝叶斯网络，看完这篇我终于理解了(附代码)!
- <https://baike.baidu.com/tashuo/browse/content?id=02c9b57396229adc831a9955>
9800 万美元卖掉公司后，他用贝叶斯网络分析数据中的因与果
- <https://www.jiqizhixin.com/articles/2018-10-26-6>
迷人又诡异的辛普森悖论：同一个数据集是如何证明两个完全相反的观点的？
- <https://blog.csdn.net/xixiaoyaoww/article/details/105683407>
逻辑回归与朴素贝叶斯的战争
 - 机器学习模型基本上兵分两路：像朴素贝叶斯这种，通过计算样本 x 与类别 y 的联合分布来进行分类的机器学习模型被称为生成式模型；像逻辑回归这种，在固定住特定样本 x 的情况下，计算该样本 x 与类别 y 的条件分布来进行分类的机器学习模型被称为判别式模型。
- <https://zhuanlan.zhihu.com/p/74586507>
机器学习中的判别式模型和生成式模型
- <https://baijiahao.baidu.com/s?id=1713380314221700759>
因果推断：利剑和 2021 诺贝尔经济学奖三剑客的故事
- <https://finance.sina.cn/2024-07-14/detail-inceanfx2867140.d.html>
6700 万参数比肩万亿巨兽 GPT-4！微软 MIT 等联手破解 Transformer 推理密码
 - 通过因果模型构建数据集，直接教模型学习公理，结果只有 67M 参数的微型 Transformer 竟能媲美 GPT-4 的推理能力。
- <https://www.jiqizhixin.com/articles/2021-01-22-4>
因果推理、正则化上榜：权威专家盘点过去 50 年最重要的统计学思想
- <https://www.huxiu.com/article/323544.html>
贝叶斯网络发明人珀尔的《为什么》
- <https://baijiahao.baidu.com/s?id=1713380314221700759>
因果推断：利剑和 2021 诺贝尔经济学奖三剑客的故事
- <https://www.huxiu.com/article/508964.html>
帅哥和顶级程序员
 - 谷歌的工程师将机器学习技术应用到他们自己的招聘过程中，试图找出最高效的员工。他们发现了一个令人惊讶的结果：工作表现与之前在编程竞赛中的成绩呈负相关。
 - 在约会场所非常活跃的朋友们有时抱怨说，当和帅哥出去约会时，会发现这个人原来是个浑蛋；而当感觉某个人品质很好时，却发现这个人长得不帅。

■ 这是怎么回事？

- <https://segmentfault.com/a/1190000016563386>
朴素贝叶斯分类——大道至简
- <https://blog.csdn.net/serryuer/article/details/89350019>
玻尔兹曼机和受限玻尔兹曼机
- <https://baijiahao.baidu.com/s?id=1599798281463567369>
一起读懂传说中的经典：受限玻尔兹曼机

参考资料与文献：

- [] Michael I. Jordan. Probabilistic graphical models。讲义，未出版，参见 <https://people.eecs.berkeley.edu/~jordan/prelims/>
- [] Daphne Koller and Nir Friedman. Probabilistic Graphical Models: Principles and Techniques. The MIT Press, 2009。1千多页的教科书。
Daphne Koller, Nir Friedman 著。概率图模型：原理与技术。清华大学出版社，2015。
- [] William L. Hamilton. Graph Representation Learning. Springer, 2020。电子版参考 https://www.cs.mcgill.ca/~wlh/grl_book/
- [] David Easley and Jon Kleinberg. Networks, Crowds, and Markets: Reasoning about a Highly Connected World. Cambridge University Press, 2010。电子版参考 <http://www.cs.cornell.edu/home/kleinber/networks-book/>
- [] Albert-László Barabási. Network Science. Cambridge University Press, 2016。
- [] Yao Ma and Jiliang Tang. Deep Learning on Graphs. Cambridge University Press, 2021。
- [] Marloes Maathuis, Mathias Drton, Steffen Lauritzen and Martin Wainwright (ed.). Handbook of Graphical Models. CRC Press, 2018。由多位世界知名的统计学家合作完成，整理了图模型自上世纪 80 年代诞生以来的发展脉络，可以为传统数据科学工作者学习因果建模提供一份有价值的入门材料。500 页图模型巨著，从图、概率图、统计和因果推理带你纵览神奇的图模型。电子版书籍链接：<https://stat.ethz.ch/~maathuis/papers/Handbook.pdf>
- [] Judea Pearl and Dana Mackenzie. The Book of Why: The New Science of Cause and Effect. Penguin Books Ltd., 2019。

朱迪亚·珀尔, 达纳·麦肯齐 著, 为什么: 关于因果关系的新科学。中信出版社, 2019。

- [] Judea Pearl, Madelyn Glymour, Nicholas P. Jewell. Causal Inference in Statistics - A Primer. Wiley, 2016.
- [] Judea Pearl. Causality: Models, Reasoning, and Inference. Cambridge University Press, 2009.
- [] Jonas Peters, Dominik Janzing, and Bernhard Scholkopf. Elements of Causal Inference: Foundations and Learning Algorithms. The MIT Press, 2017.
- [] Miguel A. Hernán, James M. Robins. Causal Inference: What If. CRC Press, 2025.
- [] Guido W. Imbens, Donald B. Rubin. Causal Inference for Statistics, Social, and Biomedical Sciences: An Introduction. Cambridge University Press, 2015.
- [] Paul R. Rosenbaum. Observation and Experiment: An Introduction to Causal Inference. Harvard University Press, 2019
- [] Scott Cunningham. Causal Inference: the Mixtape. Yale University Press, 2021.